



Verilog HDL Design and Applications
of Digital System

Verilog HDL

数字系统设计及其应用

2022 2022 2022 2022



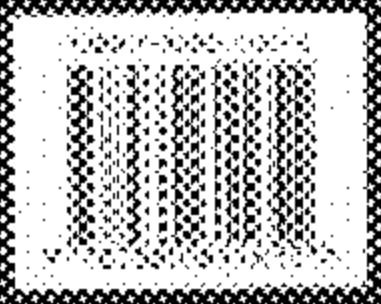
© 2022 2022 2022 2022

1234567890



- Comprehensive coverage of Verilog HDL syntax and semantics
- Detailed examples of Verilog HDL code for various digital systems
- In-depth discussion of Verilog HDL simulation and synthesis
- Practical applications of Verilog HDL in digital system design

Verilog HDL Design and Applications of Digital System



[illegible][illegible]

.00
.00
.00
.00
.00
.00
.00
.50
.30
.50
.00
.00
.00
.00
.30
.30
.30
.50
.00
.50
.00
.00
.00
.50
.00
.00
.00
.00
.00
.00
.00
.00
.00
.80
.00
.30
.00
.00
.50
.00
.00
.00
.00
.00
.00

10
10
10
10
10
10
10
50
30
50
00
00
00
00
00
30
30
30
30
00
50
00
00
00
50
00
00
00
00
00
.00
.00
.00
.00
.00
.00
.00
.00
.00
.80
.00
.30
.00
.00
.50
.00
.00
.00
.00

内 容 简 介

本书系统地介绍了一种在专用集成电路设计领域具有广泛应用前景的硬件描述语言——Verilog HDL 语言。利用 Verilog HDL 语言设计数字逻辑电路和数字系统的新方法，是电子电路设计方法的一次革命性的变化，也是 21 世纪的电子工程师所必须掌握的专门知识。

本书共分 12 章。第 1 章对硬件描述语言进行了概述，并给出了 EDA 的典型设计流程与有关硬件描述语言的最新发展；第 2 章对采用 Verilog HDL 设计数字系统的方法以及 Verilog HDL 程序的基本结构进行了简单的阐述；第 3~8 章主要介绍 Verilog HDL 的基本知识、用户自定义元件以及 Verilog HDL 的两种描述方式；第 9 章详述了有关 Verilog HDL 程序测试与仿真的内容；第 10 章与第 11 章分别给出了使用 Verilog HDL 设计简单逻辑电路与复杂电路的实例；第 12 章对 Verilog HDL 的开发工具进行了简单的介绍。

本书简明扼要，易读易懂，并列举了众多的实例，便于读者学习与参考。本书可作为本科生和研究生的教科书，也可作为一般从事电子电路设计工程师的自学参考书。

图书在版编目 (CIP) 数据

Verilog HDL 数字系统设计及其应用 / 袁俊泉等编著.

—西安: 西安电子科技大学出版社, 2002. 11

ISBN 7-5606-1165-6

I. V… II. 袁… III. ①数字系统—系统设计②硬件描述语言,

Verilog HDL—计算机辅助设计—数字电路 IV. ①TP271②TN790.2

中国版本图书馆 CIP 数据核字 (2002) 第 054097 号

责任编辑 毛红兵 龙晖

出版发行 西安电子科技大学出版社(西安市太白南路 2 号)

电 话 (029)8227828 邮 编 710071

<http://www.xduph.com> E-mail: xdupfxb@pub.xaonline.com

经 销 新华书店

印 刷 西安兰翔印刷厂

版 次 2002 年 11 月第 1 版 2002 年 11 月第 1 次印刷

开 本 787 毫米×1092 毫米 1/16 印张 18.75

字 数 441 千字

印 数 1~4 000 册

定 价 25.00 元

ISBN 7-5606-1165-6/TP·0597

XDUP 1436001-1

如有印装问题可调换

本书封面贴有西安电子科技大学出版社的激光防伪标志，无标志者不得销售。

前 言

随着电子技术的发展, 芯片的复杂程度越来越高, 人们对数万门乃至数百万门的电路设计的需求也越来越多。仅依靠原理图输入方式已不能满足要求, 采用硬件描述语言 HDL 的设计方式就应运而生。设计工作从行为、功能级开始, 并向着设计的高层次发展。这样就出现了第三代 EDA 系统, 其特点是高层次设计的自动化(HLDA, High Level Design Automation)。

第三代 EDA 系统中除了引入硬件描述语言(一般采用两种语言, 即 VHDL 语言和 Verilog HDL 语言), 还引入了行为综合和逻辑综合工具。采用较高的抽象层次进行设计, 并按层次式方法进行管理, 可大大提高处理复杂设计的能力, 缩短设计周期。综合优化工具的采用使芯片的品质如面积、速度和功耗等获得了优化, 因而第三代 EDA 系统迅速得到了推广应用。

熟悉并掌握这些现代设计工具, 已成为电子系统设计人员所必备的一门技术。然而, 面对这样的一个客观现实, 国内出版了多本介绍 VHDL 的书籍, 有关 Verilog HDL 的书籍却很少。Verilog HDL 提供了非常精练和易读的语法, 普及程度远远高于 VHDL。许多大规模的电路设计都是用 Verilog HDL 来完成的。美国的许多著名高校如斯坦福大学、南加州大学等, 都以 Verilog HDL 为主要授课内容, 这与我国高校多偏重 VHDL 语言教学的现实形成了明显反差。作者编写此书的目的在于向广大电子设计人员介绍 Verilog HDL 的基本知识和使用它来设计数字系统硬件电路的方法, 从而使读者摆脱传统的人工设计方法的束缚, 使数字系统设计的水平上升到一个新的阶段。

本书共分 12 章。第 1 章在简单回顾了电路设计的演变过程后, 对硬件描述语言进行了概述, 并给出了 EDA 的典型设计流程以及有关硬件描述语言的新发展。第 2 章对采用 Verilog HDL 设计数字系统的方法以及 Verilog HDL 程序的基本结构进行了简单的阐述。第 3 章至第 8 章主要介绍 Verilog HDL 的基本知识、用户自定义元件以及 Verilog HDL 的两种描述方式。第 9 章详述了有关 Verilog HDL 程序的测试与仿真的内容。第 10 章与第 11 章分别给出了使用 Verilog HDL 设计简单逻辑电路与复杂电路的实例。第 12 章对 Verilog HDL 的开发工具进行了简单的介绍。

书中通过大量的实例介绍了该语言的基本内容和结构, 这些实例不仅对读者掌握语言本身和建模方法有很大的帮助, 而且对实际数字系统设计也极有帮助。其中, 袁俊泉同志负责第 3、9、10、11、12 章的编写工作, 孙敏琪同志负责第 4、5、6、7、8 章的编写工作, 曹瑞同志负责第 1、2 章的编写工作。

7.53791.7

本书在编写过程中引用了诸多学者和专家的著作，在这里向他们表示衷心的感谢。同时，也向一贯热情支持和关心作者的西安电子科技大学出版社的领导、编辑及工作人员表示深深的谢意。

作者希望通过本书与读者交流数字系统设计方面的体会，并期望为我国数字系统设计人才的培养与信息产业的发展贡献微薄之力。

由于作者水平有限，错误和不当之处在所难免，敬请各位读者批评指正。

编著者
2002 年 5 月

目 录

第 1 章 概述	1
1.1 电子系统设计方法的演变过程	1
1.2 硬件描述语言	2
1.2.1 硬件描述语言(HDL)	2
1.2.2 为什么要用 HDL	3
1.2.3 HDL 的发展历史	3
1.2.4 Verilog HDL 与 VHDL 的比较	4
1.3 EDA 典型流程	4
1.4 硬件描述语言的新发展	6
1.4.1 OO VHDL	6
1.4.2 DE VHDL	6
1.4.3 VITAL	7
1.4.4 系统级描述语言	7
1.4.5 IEEE Std 1364—2000	8
第 2 章 初识 Verilog HDL	9
2.1 Verilog HDL 的设计方法	9
2.1.1 自下而上(Bottom-Up)的设计方法	9
2.1.2 自上而下(Top-Down)的设计方法	9
2.1.3 综合设计方法	10
2.2 Verilog HDL 中的模块及其描述方式	10
2.2.1 模块的概念及结构	10
2.2.2 模块的描述方式	12
2.2.3 设计的仿真与测试	14
2.3 Verilog HDL 设计流程	16
第 3 章 Verilog HDL 基础知识	17
3.1 词法	17
3.1.1 间隔符与注释符	17
3.1.2 数值	19
3.1.3 字符串	20
3.1.4 关键字	21
3.2 数据类型	22
3.2.1 物理数据类型	22
3.2.2 抽象数据类型	26
3.3 运算符	27
3.3.1 算术运算符	28

3.3.2	逻辑运算符	28
3.3.3	关系运算符	29
3.3.4	相等关系运算符	29
3.3.5	按位运算符	31
3.3.6	归约运算符	32
3.3.7	移位运算符	33
3.3.8	条件运算符	33
3.3.9	连接与复制操作	34
3.3.10	运算符的优先级	35
3.4	系统任务与系统函数	35
3.4.1	标准输出任务	36
3.4.2	文件管理任务	37
3.4.3	仿真控制任务	38
3.4.4	时间函数	39
3.4.5	其他	40
3.5	编译指令	41
3.5.1	宏编译指令	41
3.5.2	文件包含指令	41
3.5.3	条件编译指令	42
3.5.4	时间定标指令	42
3.5.5	工作库定义指令	43
第 4 章	用户自定义元件(UDP)	44
4.1	UDP 的定义	44
4.2	组合逻辑电路 UDP	47
4.3	时序逻辑电路 UDP	51
4.3.1	初始化状态寄存器	52
4.3.2	电平触发时序电路 UDP	52
4.3.3	边沿触发时序电路 UDP	53
4.3.4	电平触发和边沿触发混合的时序电路 UDP	55
第 5 章	行为描述(一): 模块基本结构	58
5.1	行为描述的结构	58
5.1.1	过程块	59
5.1.2	initial 过程块	60
5.1.3	always 过程块	62
5.2	语句块	65
5.2.1	串行块(begin-end 块)	66
5.2.2	并行块(fork-join 块)	68
5.2.3	串行块和并行块的混合使用	69
第 6 章	行为描述(二): 时间控制和赋值语句	73

6.1 时间控制	73
6.1.1 延时控制	73
6.1.2 边沿触发事件控制	77
6.1.3 电平敏感事件控制(wait 语句).....	85
6.2 赋值语句	87
6.2.1 过程赋值语句的基本格式	87
6.2.2 过程赋值的两种延时方式	89
6.2.3 阻塞型过程赋值	93
6.2.4 非阻塞型过程赋值	94
6.2.5 连续赋值语句	97
6.2.6 过程连续赋值语句	102
第 7 章 行为描述(三): 高级程序语句、函数和任务	109
7.1 分支语句	109
7.1.1 if-else 条件分支语句	109
7.1.2 case 分支控制语句	113
7.2 循环控制语句	119
7.2.1 forever 循环语句	119
7.2.2 repeat 循环语句	121
7.2.3 while 循环语句	123
7.2.4 for 循环语句	124
7.3 任务(task)与函数(function)	126
7.3.1 任务(task)	126
7.3.2 函数(function)	131
第 8 章 结构描述	138
8.1 结构描述方式	138
8.2 模块级建模	139
8.2.1 模块的定义	139
8.2.2 模块的端口	140
8.2.3 模块的调用	143
8.2.4 在模块调用时对参数值的更改	150
8.2.5 举例	154
8.3 门级建模	156
8.3.1 内置基本门级元件	156
8.3.2 门级建模的例子	167
8.4 specify 说明块和时序检验	170
8.4.1 延时参数的定义: specparam 语句	172
8.4.2 对模块输入输出端口之间的路径延时进行说明	172
8.4.3 借助时序检验系统任务对模块输入输出时序进行时序检验	175
第 9 章 测试与仿真	177

9.1 测试与仿真的流程	177
9.1.1 产生输入向量	177
9.1.2 测试模块	178
9.2 测试举例	180
第 10 章 设计举例与设计技巧	188
10.1 加法器	188
10.1.1 带进位输入的 8 位加法器	188
10.1.2 带进位的通用加法器	191
10.1.3 长度为 N 的向量加法器	192
10.2 向量乘法器	193
10.3 比较器	195
10.4 多路选择器与译码器	196
10.4.1 8 选 1 多路选择器	196
10.4.2 3-8 译码器	199
10.5 寄存器	201
10.5.1 带同步复位的边沿触发器	201
10.5.2 带异步复位和置位的边沿触发器	203
10.5.3 带使能和异步复位的 8 位寄存器	205
10.6 边沿控制的脉冲发生器	208
10.7 计数器	210
10.7.1 带使能和进位输出的 4 位计数器	210
10.7.2 并行加载的通用增 1/减 1 计数器	216
10.8 移位寄存器	219
10.8.1 串行输入/并行输出的移位寄存器	219
10.8.2 并行输入/串行输出的移位寄存器	221
10.9 分频器	223
10.10 FIR 滤波器	225
第 11 章 综合设计实例	228
11.1 有限状态机的概念及其设计实例	228
11.1.1 有限状态机的概念	228
11.1.2 有限状态机的设计实例	230
11.2 RISC 中央处理单元(CPU)的顶层设计	247
11.2.1 累加器用寄存器	247
11.2.2 RISC 算术运算单元	248
11.2.3 数据控制器	249
11.2.4 指令寄存器	249
11.2.5 状态控制器	250
11.2.6 动态存储器	252
11.2.7 程序计数器	253

11.2.8 地址多路器	253
11.2.9 时钟发生器	254
11.2.10 顶层设计模块	255
第 12 章 开发工具介绍	256
12.1 EDA 基本工具	256
12.1.1 编辑器	256
12.1.2 仿真器	257
12.1.3 检查/分析工具	257
12.1.4 优化/综合工具	257
12.2 Verilog HDL 开发工具	257
12.2.1 综合工具	257
12.2.2 仿真器	258
12.3 VeriLogger Pro 概况	258
12.3.1 VeriLogger Pro 适用平台	258
12.3.2 VeriLogger Pro 支持的标准	258
12.3.3 VeriLogger Pro 进行仿真的基本步骤	259
12.3.4 VeriLogger Pro 的窗口构成	259
12.4 VeriLogger Pro 使用指南	259
12.4.1 创建与编辑一个 Verilog 语言的文件与工程	260
12.4.2 Verilog 语言工程的编译	265
12.4.3 Verilog 语言工程的调试	268
12.4.4 Verilog 语言工程的仿真	271
附录 Verilog HDL 形式化语法	275
参考文献	290



第 1 章 概 述



本章简单介绍电子系统设计方法的演变过程, 以及什么是 HDL, 为什么使用 HDL, 并且给出了 EDA 设计的典型流程, 最后讲述了 HDL 的最新发展。

1.1 电子系统设计方法的演变过程

自 1959 年第一片集成电路问世以来, 至今已有 40 多年了。随着科学技术的不断进步, 人类社会已进入高度发达的信息社会。信息社会的发展以信息产业的进步为动力, 而信息产业的核心是集成电路。现代集成电路在性能提高、复杂度增加的同时, 不仅价格呈下降趋势, 而且产品更新换代的频率也越来越快。实现这种进步的主要原因是集成电路的生产制造技术和设计技术的发展。前者以微细加工技术为代表, 目前已进展到深亚微米阶段, 现在集成电路制造厂商可以在几平方厘米的硅圆晶片上集成数千万乃至上亿只晶体管, 当前的微型计算机处理器的制造工艺已经达到了 $0.18\mu\text{m}$, 并正向 $0.13\mu\text{m}$ 过渡; 后者的核心是 EDA 技术, EDA 是指以计算机为工作平台, 把应用电子技术、计算机技术、智能化技术等融合在一个电子 CAD 通用软件包中, 辅助进行三方面的电子设计工作: 集成电路设计、电子电路设计以及 PCB 设计。

制作工艺的进步和 EDA 技术的发展是相辅相成的。没有 EDA 技术的支持, 要完成上述超大规模集成电路的设计制造是不可想象的; 反之, 生产制造技术的不断进步又必将对 EDA 技术提出更高的要求。

回顾 40 多年来电子系统(集成电路)设计自动化的发展, 可将 EDA 技术分为三个阶段:

1) CAD 阶段(20 世纪 60 年代~80 年代初期)

CAD 阶段分别研制了一些单独的软件工具, 主要有 PCB(Printed Circuit Board)布线设计、电路模拟、逻辑模拟及版图的绘制等, 即此时人们已经开始用计算机辅助进行 IC 版图编辑、PCB 布局布线, 取代了手工操作, 产生了计算机辅助设计的概念。例如, 目前常用的 PCB 布线软件 Tango 以及用于电路模拟的 Spice 软件和后来产品化的 IC 版图编辑与设计规则检查等软件都是这个时期的产品。

20 世纪 80 年代初, 由于集成电路规模越来越大, 制作也趋于复杂, EDA 技术有了较快的发展, 许多软件公司如 Mentor、Daisy System 等进入市场, 软件工具的产品开始增多。这个时期的软件主要针对产品开发, 分为设计、分析、生产、测试等多个独立的软件包, 每个软件只能完成其中的一项工作。但如果通过顺序循环使用这些软件完成设计软件的全过程, 则存在两个方面的问题: 第一, 各软件工具是由多个公司开发的, 只解决一个领域中的问题, 如果将某个软件输出作为另一个软件的输入, 需要手工处理, 这往往很烦琐, 极大影响了设计速度; 第二, 对于复杂电子系统的设计, 当时的 EDA 工具不能够提



供系统级的仿真与综合。由于缺乏系统级的设计考虑，常常在产品开发后期才发现设计中存在的错误，而此时再进行弥补将十分困难或已经贻误了宝贵的商机。

2) CAE 阶段(20 世纪 80 年代初期~90 年代初期)

CAE 阶段在集成电路与电子系统设计方法学以及设计工具集成化方面取得了许多成果。各种设计工具，如原理图输入、编译与链接、逻辑模拟、测试码生成、版图自动布局以及各种单元库均已经齐全。由于采用了统一的数据管理技术，因而能够将各个工具集成为一个 CAE(Computer Aided Engineering)系统。运用这种系统，按照设计方法学制定的某种设计流程，可以由 RTL 级开始，实现从设计输入到版图输出的全过程设计自动化。CAE 阶段中主要采用基于单元库的半定制设计方法。采用门阵列和标准单元设计方法设计的 ASIC 得到了极大的发展，将集成电路工业推进到了 ASIC 时代。多数 CAE 系统中还集成了 PCB 自动布局布线软件以及热特性、噪声、可靠性等分析软件，进而可以实现电子系统设计自动化。这个阶段典型的 CAE 系统有 Mentor Graphics、Valid Daisy 等公司的产品。

3) EDA 阶段(20 世纪 90 年代以来)

20 世纪 90 年代以来，微电子技术以惊人的速度发展，其工艺水平已经达到了深亚微米级，在一个芯片上已经可集成数百万乃至上千万只晶体管，工作速度可以达到 Gb/s，这为制造出规模更大、速度和信息容量更高的芯片系统提供了基础条件，同时也对 EDA 系统提出了更高的要求。尽管 CAD/CAE 技术取得了巨大的成功，但并没有把人从繁重的设计工作中彻底解放出来。在整个设计过程中，自动化和智能化程度还不高，各种 EDA 软件界面千差万别，学习使用困难，并且互不兼容，直接影响到设计环节间的衔接。基于以上不足，人们开始追求贯彻整个设计过程的自动化，这就是 ESDA(Electronic System Design Automation，电子系统设计自动化)。它代表了当今电子设计技术的最新发展方向，设计人员按照“自顶向下”的设计方法，对整个系统进行方案设计和功能划分，系统的关键电路用一片或几片专用集成电路(ASIC)实现，然后采用硬件描述语言(HDL)完成系统行为级设计，最后通过综合器和适配器生成最终的目标器件。这不仅极大地提高了系统的设计效率，而且使设计者摆脱了大量的辅助性工作，使他们能将精力集中于创造性的方案与概念的构思上。

在第三代 EDA 系统中，除了引入硬件描述语言，还引入了行为综合和逻辑综合工具，并采用较高的抽象层次进行设计。按层次式方法进行管理，大大提高了处理复杂设计的能力，并且大幅度缩短了设计所需的周期。另外，采用专用的综合优化工具，使芯片的品质如面积、速度和功耗等获得了优化。因而第三代 EDA 系统一面世就迅速得到了广泛的应用。

1.2 硬件描述语言

本节简单介绍硬件描述语言的基本含义，采用硬件描述语言来进行数字电路设计的原因，以及硬件描述语言的发展历史，并对目前通用的两种硬件描述语言作了简单的比较。

1.2.1 硬件描述语言(HDL)

硬件描述语言(Hardware Description Language)是硬件设计人员和 EDA 工具之间的界



面,它主要用于从算法级、门级到开关级的多种抽象设计层次的数字系统建模。被建模的数字系统对象既可以是简单的门,也可以是完整的电子数字系统。硬件描述语言的主要功能是编写设计文件,建立电子系统行为级的仿真模型,然后利用高性能的计算机对用 Verilog HDL 或 VHDL 建模的复杂数字逻辑进行仿真,之后再对它进行自动综合以生成符合要求且在电路结构上可以实现的数字逻辑网表(Netlist),然后根据网表和适合某种工艺的器件自动生成具体电路,最后生成该工艺条件下具体电路的延时模型。仿真验证无误后用于制造 ASIC 芯片或写入 FPGA 和 CPLD 器件中。

在 EDA 领域中,一般把用 HDL 语言建立的数字模型称为软核(Soft Core),把用 HDL 建模和综合后生成的网表称为固核(Hard Core)。重复利用这些模块可以缩短开发时间,提高产品开发成功率,并提高设计效率。

1.2.2 为什么要用 HDL

目前电子设计的规模越来越大,复杂度越来越高,20 世纪 90 年代末普通设计的规模已经达到百万门的数量级,并且有继续增加的趋势。集成度达千万只晶体管以上的芯片已经屡见不鲜,为使如此复杂的芯片变得易于被人脑理解,很有必要用一种高级语言来表达其功能,隐藏其具体实现的细节。这也就是在大系统程序编写中高级程序设计语言取代汇编语言的原因。同样,在芯片设计中也不得不使用硬件描述语言,而具体实现交由逻辑综合工具完成。

另外,电子领域的竞争越来越激烈,刚刚涉入电子市场的厂商要面对巨大的压力:提高逻辑设计的效率,降低设计成本,更重要的是缩短设计周期。而多方位的仿真可以在设计完成之前检测到其错误,减少设计重复的次数。因此,有效的 HDL 语言和主计算机仿真系统在将设计错误的数目减少到最低限度方面起到不可估量的作用,并使第一次投片便能成功地实现芯片的功能成为可能。

使用硬件描述语言将使检测各种设计方案变成一件很容易、很方便的事情,因为对方案的修改只需要修改 HDL 程序就行了,这比修改原理图要容易得多。

正是基于上面这几点,传统的用原理图设计电路的方法正在逐渐被取代,硬件描述语言正被人们广泛接受。

1.2.3 HDL 的发展历史

HDL 最早是由 Iverson 公司于 1962 年提出的,迄今为止已经出现了多种 HDL。其中,绝大多数都是专有产品,如 Silvar-lisco 公司的 HHDL、Zycad 公司的 ISP、Gateway Design Automation 公司的 Verilog 以及 Mentor Graphics 公司的 BLM 等。还有一些高等院校及科研单位也开发了数百种产品,比较著名的包括 AHPL、MIMOLA 和 SCHOLAR 等。另外,一些大型的计算机制造商也都有其内部使用的各自的设计语言,如得克萨斯仪器公司的 TIHDL。

有些 HDL 是从一些已有的软件程序设计语言发展而来的,如 Silicon Compiler 公司的 M 和 Gateway 公司的 Verilog HDL 是从 C 语言发展而来的,而 BLM、MIMOLA 和 SCHOLAR 是以 PASCAL 语言为基础的。



Verilog HDL 语言最初是于 1983 年由 Gateway Design Automation 公司为其模拟器产品开发的硬件建模语言, 当时它只是一种专用语言。由于 GDA 公司的模拟、仿真器产品的广泛使用, Verilog HDL 作为一种易于使用且实用的硬件描述语言逐渐为众多设计者所接受。1989 年, GDA 公司被 Cadence 公司并购。1990 年, Cadence 公司正式发布 Verilog HDL 语言, 并成立了 Open Verilog International (OVI) 这一促进 Verilog 发展的国际性组织。1992 年, OVI 开始致力于推广 Verilog OVI 标准成为 IEEE 标准, 并于 1995 年使 Verilog HDL 语言成为 IEEE 标准, 称为 IEEE Std 1364—1995。

1.2.4 Verilog HDL 与 VHDL 的比较

Verilog HDL 与 VHDL 是目前两种最常用的硬件描述语言, 同时也都是 IEEE 标准化的 HDL 语言。归纳起来, 它们主要有以下几点不同:

(1) 从推出过程来看, VHDL 偏重于标准化的考虑, 而 Verilog HDL 与 EDA 工具的结合更为紧密。VHDL 是国际上第一个标准化的 HDL 语言(IEEE-1076), 是为了实现美国国防部 VHSIC 计划所推出的各个电子部件供应商具有统一数据交换格式的要求。相比之下, Verilog HDL 则是在全球最大的 EDA/ESDA 供应商 Cadence 公司的扶持下针对 EDA 工具开发的 HDL 语言。

(2) 与 VHDL 相比, Verilog HDL 的编程风格更加简洁明了、高效便捷。如果单纯从描述结构上考察, 两者的代码之比为 3:1。

(3) Verilog HDL 也于 1995 年实现标准化(IEEE-1364)。目前市场上所有 EDA/ESDA 工具都同时支持这两种语言, 而在 ASIC 设计领域, Verilog HDL 占有明显的优势。

1.3 EDA 典型流程

进入 20 世纪 90 年代以来, 电子信息类产品的开发明显出现两个特点: 一是产品的复杂程度加深; 二是产品的上市时限紧迫。这两点对 EDA 技术提出了新的要求, 为此业界开始使用一种高层次的电子设计方法, 也称为系统级的设计方法。

高层次设计是一种“概念驱动”形式的设计。使用高层次设计方法, 设计人员不需要再通过门级原理图来描述电路, 而只要针对设计目标进行功能描述。这样设计人员就可以摆脱电路细节的束缚, 把精力集中在创造性的方案与概念构思上, 一旦这些概念构思以高层次描述的形式输入到 EDA 系统中之后, EDA 系统就能以规则驱动的方式自动完成整个设计。

采用高层次设计方法, 可以把新概念迅速转化为产品, 使产品的研制周期大大缩短。高层次设计只定义系统的行为特性, 不涉及实现工艺。在厂家综合库的支持下, 利用综合优化工具可以将高层次描述转换成针对某种工艺优化的网表, 这样工艺转化也变得轻松容易。

高层次设计方法的典型流程如图 1.1 所示。其具体设计步骤如下:

(1) 对系统进行任务划分。

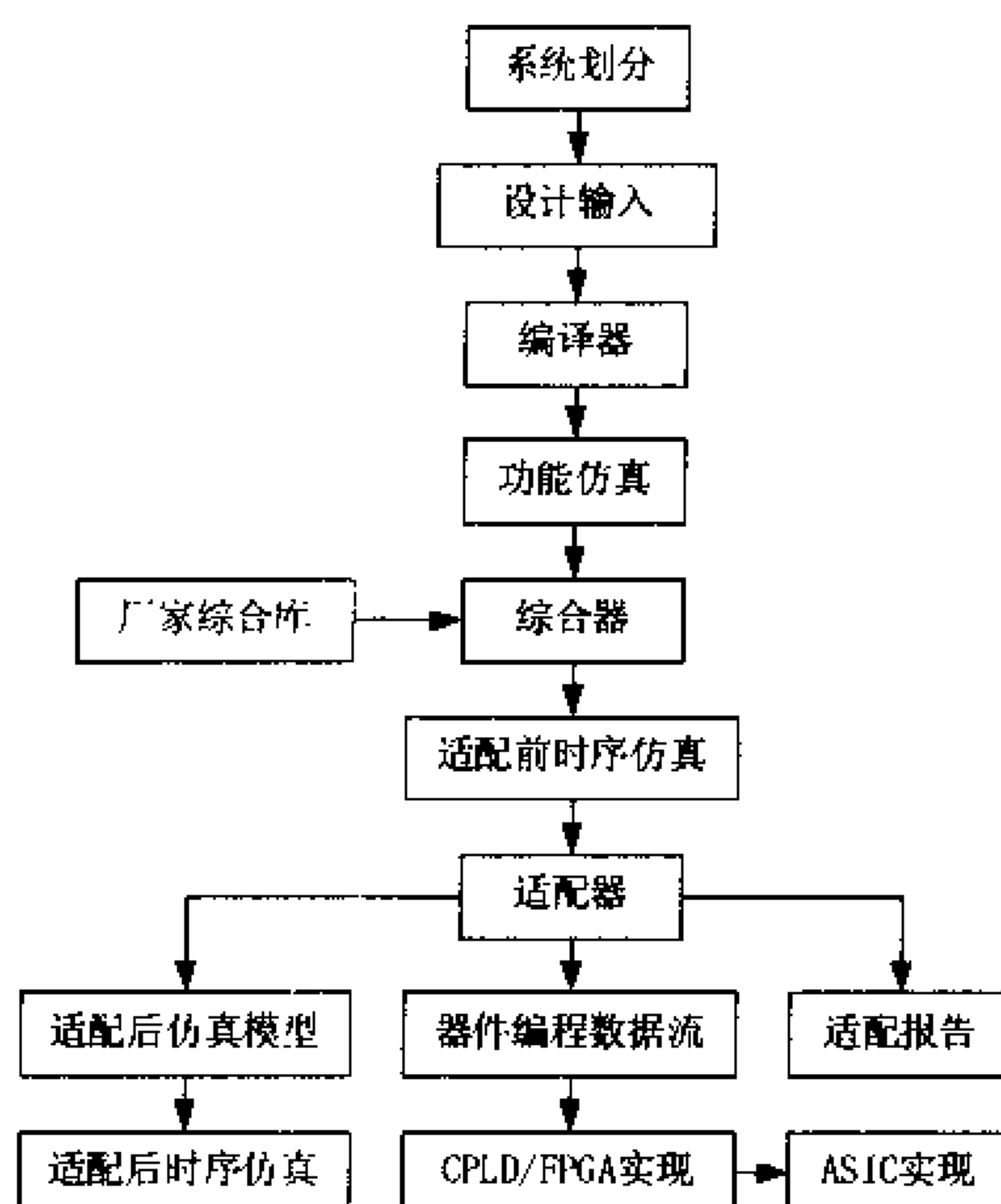


图 1.1 高层次设计方法的典型流程

(2) 进行设计输入。HDL 语言是高层次设计中最普遍的输入方式。此外还可以采用图形输入方式，如原理图、框图、状态图等。

(3) 将以上的设计输入编译成标准的 HDL 文件。对于大型设计，还要进行代码级的功能仿真，检验系统功能设计的正确性。因为对于大型设计，综合、适配要花费数小时，在综合前对源代码进行仿真，可以大大减少设计重复的次数和时间。在不是很复杂的设计中，可略去这一仿真步骤。

(4) 利用综合器对 HDL 源代码进行综合优化处理，生成门级描述的网表文件，这是将高层次描述转化为硬件电路的关键步骤。综合优化是针对 ASIC 芯片供应商的某一产品系列进行的，所以综合的过程要在相应的厂家综合库支持下才能完成。综合完成后，可利用生成的网表文件进行适配前的时序仿真。仿真过程不涉及具体器件的硬件特性，是较为粗略的。在一般情况下，这一仿真步骤也可略去。

(5) 利用适配器将综合后的网表文件针对某一确定的目标器件进行逻辑映射，这个操作包括底层器件配置、逻辑分割、逻辑优化、布局布线。适配完成后，产生多项设计结果：①适配报告，包括芯片内部资源利用情况、设计的布尔方程描述情况等；②适配后的仿真模型；③器件编程文件。根据适配后的仿真模型，可以进行适配后的时序仿真。因为此时已经得到器件的实际硬件特性(如时延特性等)，所以此仿真结果能比较精确地预期未来芯片的实际性能。如果仿真结果达不到设计要求，就需要修改 VHDL 源代码或选择不同速度品质的器件，直至满足设计要求。

(6) 将适配器产生的器件编程文件通过编程器或下载电缆载入 FPGA 或 CPLD 中。如果是大批量产品开发，通过更换相应的厂家综合库，可以很容易转由 ASIC 实现。



1.4 硬件描述语言的新发展

当前超大规模集成电路的设计面临着这样一些问题:

- (1) 设计重用、知识产权和内核插入。
- (2) 综合,特别是高层次综合和混合模型的综合。
- (3) 验证,包括仿真验证和形式验证等自动验证手段。
- (4) 深亚微米效应。

这些问题给 EDA 技术的发展提出了新的课题,为了解决这些问题,对 HDL 语言进行改进和发展是很必要的,例如 IEEE 在 1993 年就对 VHDL 语言进行了第一次修订。

目前众多研究者都认为从更高的抽象层次上开展设计,并提高元件模型的可重用性(Reusability),可以提高设计效率。这方面的工作以 OO VHDL 和 DE VHDL 为代表。另外,如何拓宽 HDL 语言的应用范围,也是研究的重点之一。这方面值得注意的有 VITAL (VHDL Initiative Towards ASID Library)等工作。此外,为解决系统级设计和软硬件协同设计的问题,EDA 工业协会的工程技术建议委员会提出了系统级描述语言(System Level Description Language)的概念。

1.4.1 OO VHDL

OO VHDL(Object Oriented VHDL),即面向对象的 VHDL。其主要概念来自美国国防部支持的 RPASSP(Rapid Prototyping of Application Specification Signal Processors,快速专用信号处理器原型)计划。目前 IEEE 有一个专门的小组对 OO VHDL 进行研究。

在软件工程界大家一致公认,面向对象的方法在处理复杂应用和增加软件的可重用性方面的能力比较强。面复杂性和可重用性正是当前集成电路设计中迫切需要解决的问题。因此,人们希望把面向对象方法应用到 HDL 语言中。OO VHDL 就是在这种背景下产生的。它的主要特点是引入了新的语言对象 Entity Object。此外 OO VHDL 中的 Entity 和 Architecture 具有继承机制,不同的 Entity Object 之间可以用消息来通信。OO VHDL 通过引入 Entity Object 作为抽象、封装和模块性的基本单元,解决了 VHDL 在抽象性方面的不足和在封装性上能力不强等问题,另外它还通过其继承机制解决了实际设计中的一些问题。OO VHDL 模型的代码比 VHDL 模型短30%~50%,有利于缩短开发时间,提高开发效率。

1.4.2 DE VHDL

现在电子系统的设计通常采用自顶向下的方法,并广泛使用硬件描述语言如 VHDL 和 Verilog HDL 等。采用自顶向下的设计方法,一般要把系统划分为若干个子系统,这些子系统有的是全新的设计,有的是已存在的元件。如果是已有的元件,那么就涉及到可重用性的问题。美国杜克大学发展的 DE VHDL(Duke Extended VHDL)通过增加 3 条语句,使设计者可以在 VHDL 描述中调用不可综合的子系统(包括连接该子系统和激活相应的功能)。此外,DE VHDL 计划提供一种抽象子系统功能的方式,使设计者能够在不熟悉子系

统实现细节的情况下了解其功能。

杜克大学用 DE VHDL 进行了一些多芯片系统的设计, 已完成的系统有水下生活研究系统的硬件形式的缓存仿真器等。结果表明, DE VHDL 可以极大地提高设计能力。

1.4.3 VITAL

长期以来, 业界一直缺乏高效可靠的VHDL语言描述的ASIC库, 这在一定程度上影响了VHDL的广泛应用。而建立ASIC库的最大困难在于VHDL中没有统一、有效的方法处理时间。对此, 业界和IEEE开展了一系列研究工作, 尝试解决这一问题。VITAL是其中最重要的结果。VITAL有以下一些主要特点:

(1) 精确描述时序关系, 其中包括描述延时模型(可能是管脚到管脚的延时模型或分布式延时模型)、时序检查(如建立/保持时间检查)、电平尖峰处理和宏单元间的互连延时等。

(2) 高效率仿真。VITAL 主要处理门级逻辑单元, 因此必须具备高的仿真效率, 否则现在 ASIC 的规模将使仿真时间长得令人难以接受。

(3) 具有反向注释能力。超大规模集成电路的设计是层次式的迭代与提炼的过程, 越到低层次越可以得到更精确的延时信息, 因此需要一种机制把低层次的延时信息反向注释到较高层次, 同时, 现在已经有描述延时信息的工业标准 SDF(Standard Delay Format), 所以 VITAL 应具有从 SDF 获得反向注释的能力。

(4) VITAL 模型适用于各种 VHDL 仿真器, 即具备通用性。

VITAL 没有对 VHDL 语言本身进行扩展, 而是通过提供一定的编程指导方针和预定义的 VHDL 库, 来实现对 ASIC 宏单元的准确建模。使用 VITAL 时, 在模型内部描述功能, 而所有延时的计算则在模型外部完成。现在业界有专门的委员会负责 VITAL 的修订工作。该委员会由来自 Synopsys、Cadence、HP、TI 及 The VHDL Technology Group 的成员组成。Synopsys 支持 VITAL 风格的 VHDL 模型, Exemplar Logic 公司的 Galileo 支持用 VITAL 进行 Xilinx FPGA 设计。

1.4.4 系统级描述语言

集成电路的设计现已进入片上系统(System On a Chip 或 System On Silicon)的时代。面对日益庞大的硬件规模, 芯片设计人员必须从更高的层次上进行设计, 并且需要有一种规范化的方式来描述系统。VHDL 和 Verilog HDL 都是为仿真而发展的语言, 它们的语法定义源于离散时间系统仿真, 并不能覆盖当前片上系统设计的全部工具和方法的要求。例如, VHDL 和 Verilog HDL 都是硬件描述语言, 而现在的片上系统通常既包含硬件, 也包含软件。另外, 由于复杂性和集成度的增加, 片上系统的设计人员应该能够以比现在的硬件描述语言更加抽象、更加形式化的方式来描述时域信息、设计属性和设计约束。

为此, EDA 工业协会的工程技术建议委员会(EDA Industry Council Project Technical Advisory Board, 简称 PTAB)建议发展下一代的“硬件”描述语言——系统级描述语言(System Level Description Language, 简称 SLDL)。PTAB 认为, SLDL 不是 VHDL/Verilog HDL 的替代者, 也不仅仅是它们的扩展, 而是比它们更高一个抽象层次的系统级描述语言。对 SLDL 的研究表明, 系统级描述通常包括抽象时间概念、非确定性控制、灵活处理、组合并行进程的技术和对层次式系统的多视角的形式约束。



现在 SLDL 委员会正在审查一系列现有的和建议中的语言，以期在几年内找到新语言的原型。

1.4.5 IEEE Std 1364—2000

为了弥补 IEEE Verilog 1364—1995 中存在的缺陷和不足，Sutherland HDL 公司提出了新的标准——IEEE Verilog 1364—2000 Standard，详细内容请参照附录。

第 2 章 初识 Verilog HDL

本章将简要介绍有关 Verilog HDL 设计的入门知识,包括 Verilog HDL 的设计方法、Verilog HDL 中的模块及其描述方式,以及 Verilog HDL 的设计流程。读者通过本章的介绍,可对利用 Verilog HDL 进行数字电路设计的过程有一初步的了解。

2.1 Verilog HDL 的设计方法

Verilog HDL 的设计方法归纳起来主要有两种:自下而上(Bottom-Up)的设计方法与自上而下(Top-Down)的设计方法。另外,还可根据实际情况,利用这两种方法的组合进行综合设计,即综合设计方法。

2.1.1 自下而上(Bottom-Up)的设计方法

自下而上的设计方法是一种传统的电子系统设计方法,它的主要步骤是:首先根据系统要求编制技术规格书,并画出系统控制流程图;然后依照技术规格书和系统控制流程图,对系统的功能进行细化,合理划分功能模块,并画出系统的功能框图;接着进行各功能模块的细化和电路设计;当各功能模块电路设计、调试完成后,将各功能模块连接起来进行全系统的调试。其具体流程如图 2.1 所示。

自下而上的设计方法常用于原理图设计中,它的优点是:

- (1) 设计人员对于用这种方法进行设计比较熟悉。
- (2) 实现各个子块电路所需的时间较短。

但这种设计方法也有许多不足之处,具体表现为:

- (1) 一般来说采用这种设计方法,容易造成对系统的整体功能把握不足。
- (2) 因为必须先完成各个小模块,所以整个系统的实现需要较长的时间。
- (3) 这种方法对设计人员之间相互进行协作有比较高的要求。



图 2.1 自下而上设计方法的步骤

2.1.2 自上而下(Top-Down)的设计方法

随着硬件技术以及 HDL 语言的发展,电子系统的设计方法发生了巨大的变化,传统的自下而上的设计方法已经不能满足复杂度日益增长的系统的要求。目前在电子系统的设计中已经越来越多地采用自上而下的设计方法了。在这种新的设计方法中,由系统用户对整个系统进行方案设计和功能划分,把系统划分为基本单元,然后再把每个基本单元划分为下一层次的基本单元,直到可以直接用元件库中的元件来实现为止。其具体流程如图



2.2 所示。

自上而下的设计方法一般采用 HDL 语言, 具有以下一些优点:

- (1) 在设计周期开始就做好了系统分析。
- (2) 由于设计的主要仿真和调试过程是在高层次完成的, 所以能够在早期发现结构设计上的错误, 避免设计工作的浪费, 同时也减少了逻辑仿真的工作量。
- (3) 自顶向下的设计方法方便了从系统划分和管理整个项目, 使得几十万门甚至几百万门规模的复杂数字电路的设计成为可能。并可减少设计人员, 避免不必要的重复设计, 提高了设计的一次成功率。

自上而下的设计方法有以下缺点:

- (1) 因采用的综合工具不一样, 得到的最小单元不标准。
- (2) 制造成本高。

2.1.3 综合设计方法

复杂数字逻辑电路和系统的设计过程通常是以上两种设计方法的结合。复杂系统的设计需要考虑多个目标的综合平衡。在高层系统采用自上而下的设计方法来实现, 而在低层系统采用自下而上的方法从库元件或数据库中调用已有的单元设计。综合设计方法兼有上述两种设计方法的优点, 而且可以使用矢量测试库进行测试。

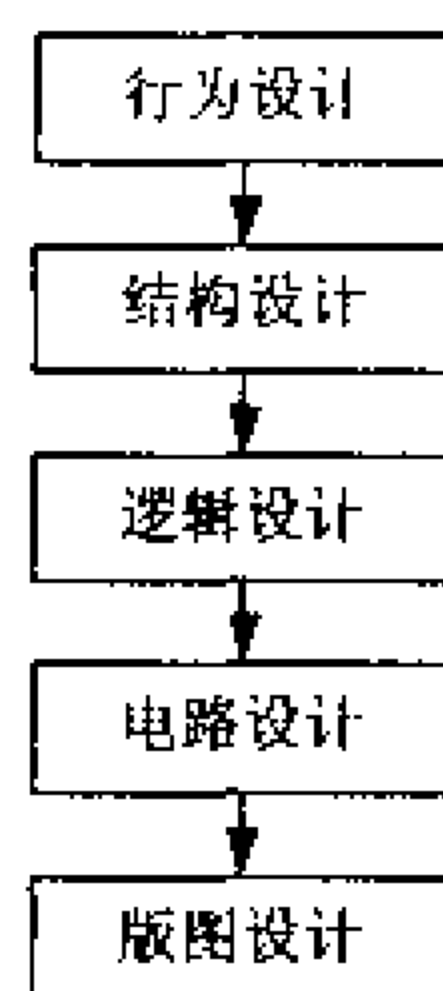


图 2.2 自上而下设计方法的步骤

2.2 Verilog HDL 中的模块及其描述方式

模块是 Verilog HDL 的基本单元, 掌握了模块的概念对学习 Verilog 语言至关重要。本节将简单介绍模块的概念、结构和它的三种不同的描述方式, 并给出了对 Verilog 语言设计进行测试与仿真的方法。

2.2.1 模块的概念及结构

和其它高级语言一样, Verilog HDL 语言也是模块化的。它以模块集合的形式来描述数字系统。模块(Module)是 Verilog HDL 语言的基本单元, 它用于描述某个设计的功能或结构及其与其他模块通信的外部端口。每一个模块都有接口部分, 用来描述与其它模块之间的连接, 通常一个文件对应一个模块。Verilog HDL 中的各个模块是并行运行的, 不过常用的做法是用一个包括测试数据和硬件描述的高层模块来定义一个封闭的系统, 并在这一模块中调用其它模块的实例。

模块的基本语法如下:

```
module <模块名> (<端口列表>)  
    端口说明(input, out, inout)  
    参数定义(可选)  
    数据类型定义
```




连续赋值语句 (assign)
过程块 (initial 和 always)
- 行为描述语句
低层模块实例
任务和函数
延时说明块

endmodule

其中<模块名>是模块唯一性的标识符；<端口列表>是由模块各个输入、输出和双向端口组成的一张端口列表，这些端口用来与其它模块进行连接；数据类型定义部分用来指定模块内用到的数据对象为寄存器型、存储器型或连线型；过程块包括 initial 过程块和 always 过程块两种，行为描述语句只能出现在这两种过程块内；延时说明块用来对模块各个输入和输出端口间的路径延时进行说明。

调用模块实例的一般形式如下：

<模块名> <参数列表> <实例名> (<端口列表>)

下面以一个 2 选 1 选择器来说明模块，图 2.3 是 2 选 1 选择器电路示意图。

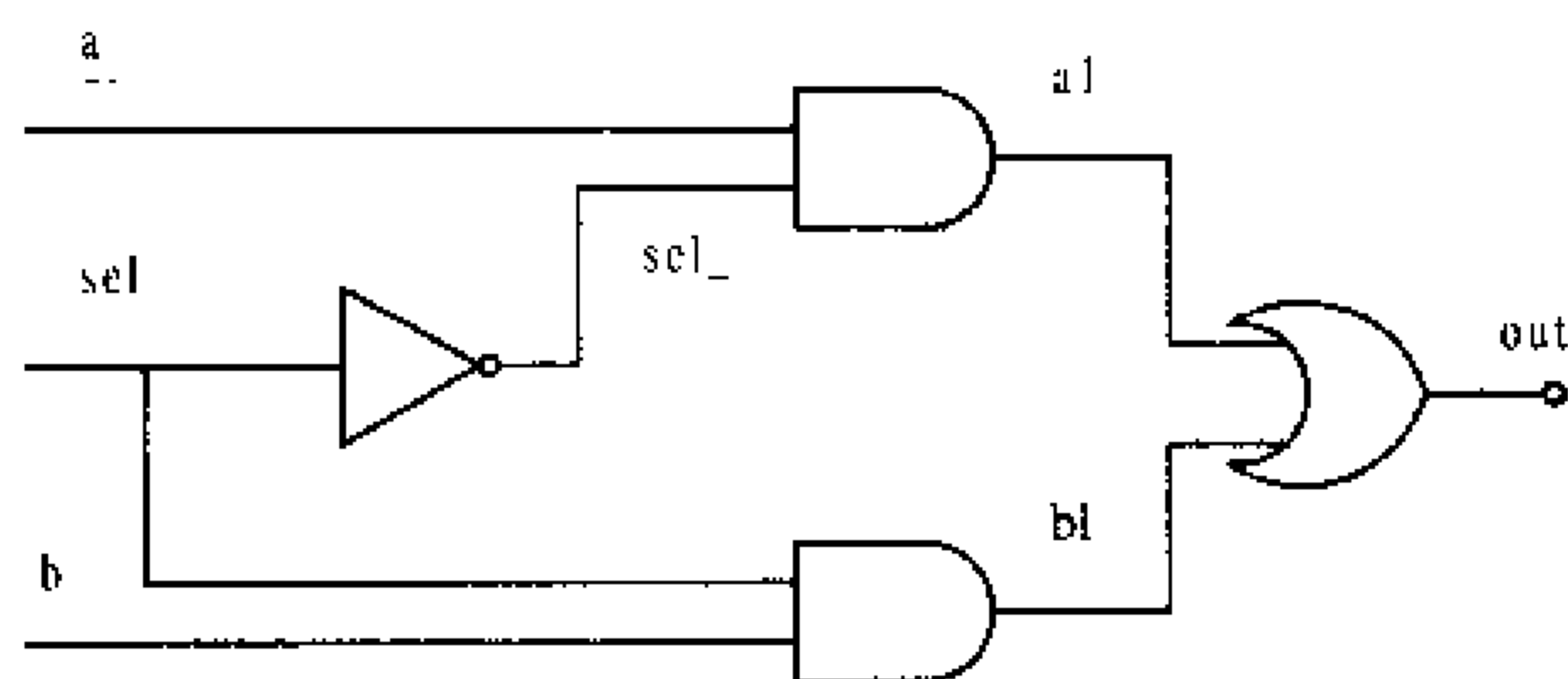


图 2.3 MUX2_1 模块电路示意图

用 Verilog HDL 语言实现上述电路的模块如下：

【例 2-1】一个 2 选 1 模块的例子。

```
module MUX2_1(out, a, b, sel); // 端口定义
    output out;
    input a, b, sel;           // 输入输出列表
    not(sel_, sel);
    and(a1, a, sel_);
    and(b1, b, sel);
    or(out, a1, b1);           // 结构描述
endmodule
```

在关键词 module 和 endmodule 之间包含着完整的 2 选 1 选择器的设计实现。a、b 和 sel 是设备的输入端口，out 是输出端口，所有信号都从这些端口输入和输出。and、or、not 是 Verilog 中预定义好的基本门级元件，可以在结构描述方式下直接使用。

当要在其它模块中调用这一选择器模块时，只需使用其模块名和所定义的端口名，不



需要知道其内部的具体实现。因为对一个模块的定义可以是行为级描述、门级结构描述或混合描述，而采用什么描述方式对调用它的高层模块来说是没有什么区别的，采用的描述方式对高层模块不产生任何影响。这种模块设计方法是自上而下设计方法的一个主要特征。

Verilog HDL 程序是由模块构成的，每个模块的内容都包含在“module”和“endmodule”两个语句之间。每个模块要进行端口定义，说明输入、输出端口。Verilog HDL 程序的书写与 C 语言类似，一行可以写多条语句，也可以一条语句分成多行书写，每条语句以分号结束，endmodule 语句后面不必写分号。用“/*……*/”可以对多行程序进行注释，用“//”可以对一行程序进行注释。

2.2.2 模块的描述方式

Verilog 模块代表硬件上的逻辑实体，其范围可以从简单的门到整个电路系统，例如计数器、存储子系统和微处理器等等。模块可以根据所采用的不同描述方法而分成行为描述模块、结构描述模块两类(也有的文献将其分成三类：行为描述模块、数据流描述模块和结构描述模块)，也可以采用以上几种描述方式的组合。下面分别介绍模块的这几种描述方式。

1. 数据流描述方式

数据流型描述主要用来描述组合功能，具体由“assign”连续赋值语句来实现。assign 连续赋值语句可以分成“显式连续赋值语句”和“隐式连续赋值语句”两种类型。

● 显式连续赋值语句

连线型变量类型 [连线型变量位宽] 连线型变量名;

assign #(延时量) 连线型变量名 = 赋值表达式;

“显式连续赋值语句”包含了两条语句：第一条语句是对连线型变量进行类型说明的说明语句；第二条语句是对这个已得到声明的连线型变量进行连续赋值的赋值语句。

● 隐式连续赋值语句

连线型变量类型 (赋值驱动强度) [连线型变量位宽] #(延时量) 连线型变量名 = 赋值表达式;

“隐式连续赋值语句”把连线型变量说明语句和连线型变量连续赋值语句结合在一条语句内。它可以在对连线型变量进行类型说明的同时进行连续赋值。

assign 连续赋值语句的功能是：当赋值表达式中变量的值发生变化时，重新计算赋值表达式的值，并在指定的时延后将得到的结果赋给等式左端的连线型变量。时延定义了右边表达式操作数变化时刻与对左边变量进行赋值时刻之间的延迟时间，如果没有定义延时量，缺省时延为 0。assign 语句只能用来实现组合功能。

下面是一个 NAND 与非逻辑模块的数据流型描述，输出 out 是输入 in1 和 in2 相与后求反的结果。

【例 2-2】数据流描述的例子。

//与非门的数据流型描述

```
module NAND(in1, in2, out);
```

```
    input in1, in2;
```



```
output out;  
assign out=~(in1&in2); //连续赋值语句  
endmodule
```

in1、in2 和 out 端口都被定义为连线型。在程序中，一旦 in1 或者 in2 的值发生变化，assign 连续赋值语句将立刻重新计算右端表达式并把得到的结果传给等式左端进行输出。assign 连续赋值语句用来描述组合电路，一旦其输入发生变动，输出也随之而改变。

由于数据 assign 语句描述了连线型变量的取值逻辑，所以从本质上说它是对连线型变量的行为进行了描述，因此本书将这种数据流描述方式归类于行为描述方式，我们将在行为建模这一章中对 assign 连续赋值语句作进一步的说明。

2. 行为描述方式

行为描述是一种高级语言使用的方法。Verilog 中的行为描述方式和软件编程语言的描述方式类似，具有很强的通用性和有效性。行为型描述是通过行为语句来实现的，行为功能可使用下述过程语句结构描述：

(1) initial 语句：此语句只执行一次。

(2) always 语句：此语句循环执行。

只有寄存器类型数据能够在这两种语句中被赋值。寄存器类型数据在被赋新值前保持原有值不变。所有的 initial 语句和 always 语句在 0 时刻并发执行。

下例所示为一个行为描述的 1 位加法器。

【例 2-3】行为描述的例子。

```
module ADDER (A, B, Cin, Sum, Cout);  
    input A, B, Cin;  
    output Sum, Cout;  
    reg Sum, Cout;  
    reg T1, T2, T3;  
    always @ ( A or B or Cin ) begin  
        Sum = (A ^ B) ^ Cin;  
        T1 = A & Cin;  
        T2 = B & Cin;  
        T3 = A & B;  
        Cout = (T1 | T2) | T3;  
    end  
endmodule
```

模块 ADDER 有 3 个输入 A、B、Cin 和 2 个输出 Sum、Cout。其中 Sum、Cout、T1、T2 和 T3 要在 always 语句中被赋值，所以它们被定义为 reg 类型(reg 是寄存器数据类型的一种)。always 语句中包含一个或事件控制(紧跟在字符@后面的表达式)，以及相关顺序过程(begin-end 对)。此或事件控制的作用是当 A、B 或 Cin 上发生事件，即 A、B 或 Cin 中某一个的值发生变化时，就执行下面的顺序过程。在顺序过程中的语句顺序执行，并且在顺序过程执行结束后被挂起。顺序过程执行完成后，程序指针返回到或事件控制语句，always 语句再次等待 A、B 或 Cin 的值发生变化。这种机制有点类似 C++中的消



息循环。

3. 结构描述方式

结构描述是通过实例进行描述的方法。它将 Verilog 中预定义的基本元件实例嵌入到语言中，监控实例的输入，一旦其中任何一个发生变化，便重新运算并输出。

在 Verilog HDL 中可使用如下结构描述部件：

- (1) 用户自己定义的模块。
- (2) 用户自定义元件 UDP。
- (3) 内置门级元件。
- (4) 内置开关级元件。

下例所示是一 AND 模块的结构型描述。这一模块是通过将一个 NAND 的输出连接到另一个 NAND 的两个输入上得到的。

【例 2-4】结构描述的例子。

//由两个 NAND 生成的与门的结构型描述

```
module AND(in1, in2, out);  
    input in1, in2;  
    output out;  
    wire w1;  
    NAND NAND1(in1, in2, w1); // NAND 模块实例 1  
    NAND NAND2(w1, w1, out);  // NAND 模块实例 2  
endmodule
```

这个模块含有 NAND1 和 NAND2 两个 NAND 模块实例，它们通过内部连线 w1 连接起来。

4. 混合型描述

在模块中，用户可以混合使用结构描述和行为描述方式，也就是说，模块中可以包含门的实例、模块实例语句、连续赋值语句、always 语句和 initial 语句以及它们的混合语句，并且它们之间可以相互包含。always 语句和 initial 语句(只有寄存器类型数据可以在这两种语句中被赋值)中的值能够驱动门或开关，而来自于门或连续赋值语句(只能驱动连线类型)的值能够反过来用于触发 always 语句和 initial 语句。

2.2.3 设计的仿真与测试

在编写完 Verilog HDL 程序之后，需要对它进行验证。在这方面 Verilog HDL 语言同样可以胜任，它不仅提供描述设计的能力，而且提供对激励、控制、存储响应和设计验证的建模能力。验证中使用的激励和控制可用初始化语句产生，验证运行过程中的响应可以作为“变化时保存”或作为选通的数据存储。另外，还可以在初始化过程中写入相应的语句，把设计验证的结果与期望的响应值进行比较。

下面举一个高层模块测试的例子。我们通过在这一模块中设置测试数据对前面的 NAND 和 AND 两个模块中的变量进行监测。

【例 2-5】测试模块的例子。

//测试以上两个模块的高层模块



```
module TEST_AND;
    reg a, b;
    wire out1, out2;
    initial begin //产生测试数据
        a=0; b=0;
        #1 a=1;
        #1 b=1;
        #1 a=0;
    end

    initial begin //设置监测功能
        $monitor("Time=%0 a=%b b=%b out1=%b out2=%b", $time, a, b, out1, out2);
    end

    //模块 AND 和 NAND 实例调用
    AND gate1(a, b, out2);
    NAND gate2(a, b, out1);
endmodule
```

应注意的是, *a* 和 *b* 都要被定义为 *reg* 类型, 这是因为它们的值需要保持, 而寄存器型变量可以存储过程赋值的最终结果(同传统的命令式编程语言中的变量相类似)。连线型变量则没有存储能力, 它们需要被连续驱动, 例如用连续赋值语句或由一个模块的输出进行驱动, 若连线型输入的左端悬空, 则其值为未知态“x”。

下面介绍一下连续赋值和过程赋值的区别。Verilog HDL 语言中过程赋值和连续赋值之间区别很大。连续赋值使用关键词“*assign*”, 而过程赋值的形式是

<寄存器变量>=<赋值表达式>;

其中<寄存器变量>必须是寄存器型或存储器型变量。过程赋值只允许出现在 *initial* 和 *always* 过程块中, 过程赋值改变一个寄存器的状态, 即时序逻辑; 而连续赋值用来描述组合逻辑, 连续赋值语句驱动连线型变量, 输入操作数的值一发生变化, 就重新计算并更新它所驱动的变量。掌握这一区别很重要。

模块中前一个 *initial* 结构块中的语句是顺序执行的, 并且都设定了延迟, *#1* 表示一个仿真时间单位的延迟。*always* 结构块与 *initial* 结构块功能相同, 但它是无限循环的过程(直到仿真停止), 而 *initial* 结构块只执行一次。*initial* 和 *always* 结构多用来描述时序逻辑, 即有限状态自动控制。

第二个 *initial* 结构块包含一条 *\$monitor* 语句, 它的功能是在参数表中指定的变量值发生变化时打印指定的字符串。

TEST_AND 模块运行后的结果如下:

```
Time=0 a=0 b=0 out1=1 out2=0
Time=1 a=1 b=0 out1=1 out2=0
Time=2 a=1 b=1 out1=0 out2=1
```




Time=3 a=0 b=1 out1=1 out2=0

此处无循环操作，所以仿真器执行所有的事件后自行停止，不需要指定仿真结束时间。

本节介绍了 Verilog HDL 语言中的模块以及模块的仿真与测试，后面的章节将会对其进行更详细的描述。

2.3 Verilog HDL 设计流程

在用 Verilog HDL 进行硬件设计的过程中，开发人员通常是将设计分成三个层次进行设计，如图 2.4 所示。

第一层次是行为描述。所谓行为描述，实质上就是使用数学模型对整个系统进行的描述。一般来说，对系统进行行为描述的目的是为了在系统设计的初始阶段，通过对系统行为的仿真来发现设计中存在的问题。在行为描述阶段并不真正考虑实际的算法和操作用什么方法来实现，注意力主要集中在系统的结构和工作过程能否达到设计要求方面。在进行完行为描述之后，通常要把它转换为 RTL 级的描述。之所以这么做，主要是因为现有的 EDA 工具只能接受 RTL 级描述的 HDL 文件进行自动逻辑综合。

第二层次是 RTL 方式描述，又称为寄存器传输描述。用行为方式描述系统结构的程序抽象程度很高，很难直接映射到具体逻辑元件的实现。要想得到硬件的具体实现，必须将行为方式的 Verilog HDL 程序改写为 RTL 方式的程序。在编写完 RTL 方式的程序之后，就可以利用仿真工具对程序进行仿真了。如果仿真通过，就可以利用逻辑综合工具进行综合了。

第三层次是逻辑综合。在这一阶段主要是利用逻辑综合工具，将 RTL 级的程序转换成用基本逻辑元件表示的文件(门级网表)，并且综合结果也可以以原理图的方式输出。得到网表之后，还需要进行门级仿真和定时检查。如果一切正常，则设计工作到此结束。

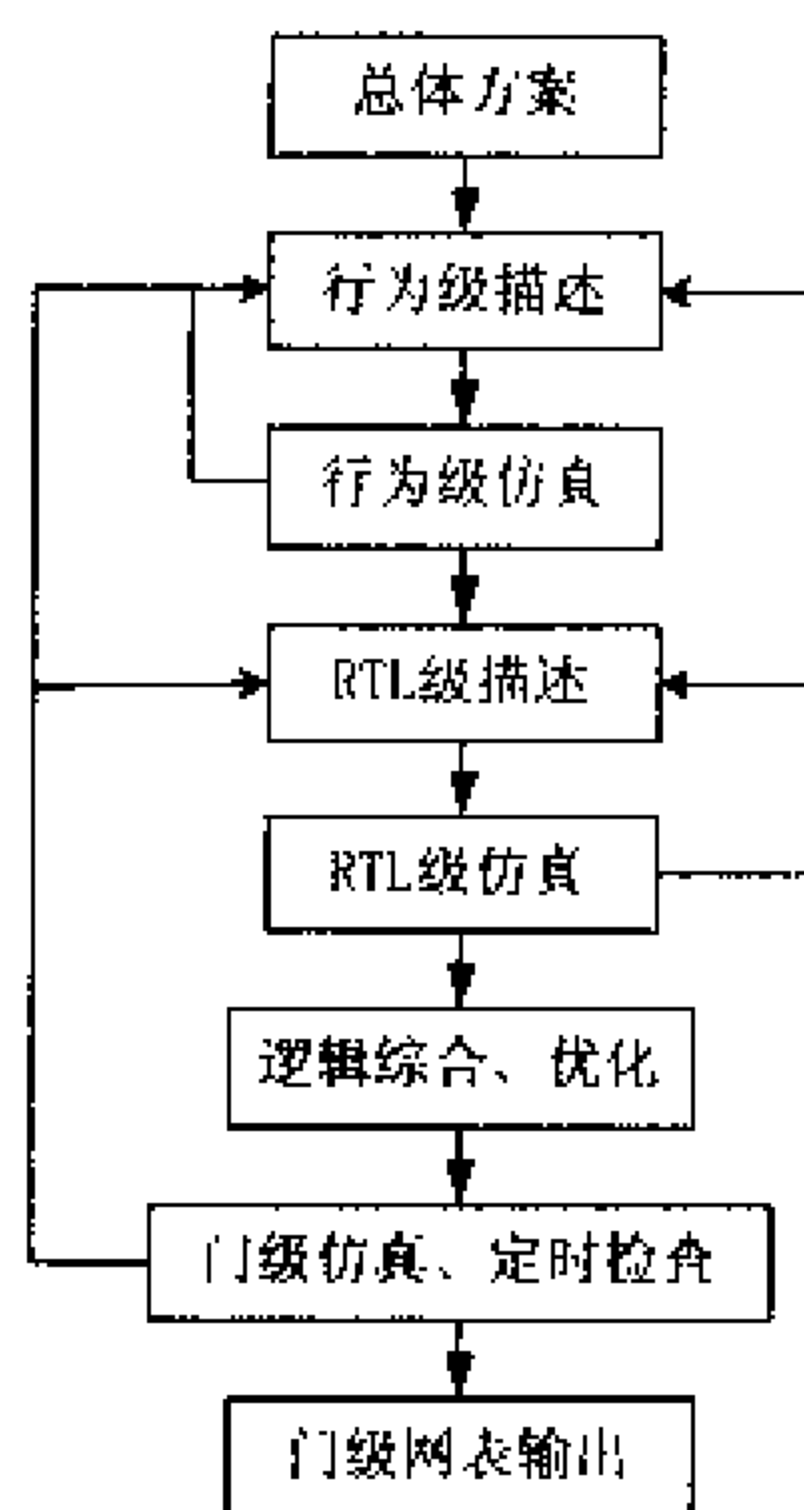


图 2.4 Verilog HDL 设计流程



第3章 Verilog HDL 基础知识

同其他高级语言一样，Verilog 硬件描述语言具有自身固有的语法说明与定义格式。本章从以下 5 个方面，对 Verilog HDL 的基础知识进行详细说明。希望通过本章的介绍，读者能够对 Verilog HDL 有初步的了解。

- (1) 词法：包括间隔符与注释符、数值、字符串与关键字。
- (2) 数据类型：包括物理数据类型与抽象数据类型。
- (3) 运算符：包括算术运算符、逻辑运算符、关系运算符、相等关系运算符、按位运算符、归约运算符、移位运算符、条件运算符、连接与复制操作及其优先级。
- (4) 系统任务与系统函数：包括标准输出任务、文件管理任务、仿真控制任务、时间函数及其他常用函数。
- (5) 编译指令：宏编译指令、文件包含指令、条件编译指令、时间定标指令与工作库定义指令。

3.1 词 法

Verilog HDL 的源文本文件是由一串词法标识符构成的，一个词法标识符包含一个或若干个字符。源文件中这些标识符之间的间隔可以是任意的格式，也就是说，选择间隔符来分隔词法标识符，对编译器来说并不具有重要意义，只是提高描述的可读性。Verilog HDL 中的词法类型包括：

- (1) 间隔符与注释符。
- (2) 数值。
- (3) 字符串。
- (4) 关键字。

3.1.1 间隔符与注释符

Verilog HDL 的间隔符包括空格(\b)、tab(\t)、换行符(\n)及换页符。如果间隔符并非出现在字符串中，则该间隔符被忽略。

间隔符除起到分割的作用外，在必要的地方插入相应的空格或换行符，使文本错落有致，可方便用户阅读与修改。

【例 3-1】间隔符的例子。

```
module addbit(a, b, ci, sum, co);  
input a, b, ci; output sum, co;  
wire a, b, ci, sum, co;
```




```
.....
```

```
endmodule
```

我们通常不会采用上面的书写形式，如果在适当的地方添加空格与换行符，写成下面的形式，则文本便于理解。

```
module addbit (a, b, ci, sum, co);
```

```
input      a;
```

```
input      b;
```

```
input      ci;
```

```
output     sum;
```

```
output     co;
```

```
wire      a;
```

```
wire      b;
```

```
wire      ci;
```

```
wire      sum;
```

```
wire      co;
```

```
.....
```

```
endmodule
```

Verilog HDL 支持两种形式的注释符：/*.....*/与//。其中，/*.....*/为多行注释符，用于对多行语句注释；//为单行注释符，只对注释符所在的行有效。下面是一个使用注释符对 1 位加法器进行说明的例子。

【例 3-2】注释符的例子。

```
/* 该例利用一位加法器来说明单行注释符与多行注释符的用法(此处为多行注释)*/
```

```
module addbit (a, b, ci, sum, co);
```

```
// 输入端口(此处为单行注释)
```

```
input      a;
```

```
input      b;
```

```
input      ci;
```

```
// 输出端口
```

```
output     sum;
```

```
output     co;
```

```
// 数据类型
```

```
wire      a;
```

```
wire      b;
```

```
wire      ci;
```

```
wire      sum;
```

```
wire      co;
```

```
.....
```

```
endmodule
```




3.1.2 数值

Verilog HDL 中的数值可取下面的四类值:

- (1) 0: 逻辑 0 或假状态。
- (2) 1: 逻辑 1 或真状态。
- (3) x: 未知状态。
- (4) z: 高阻态。

Verilog HDL 中有两类数值常量: 整型数和实型数。

下划线符号“_”除了不能放于数值的首位以外, 可以随意用在整型数与实型数中, 它们对数值大小没有任何改变, 只是为了提高可读性。

1. 整型数及其表示

Verilog HDL 中的整数可以是二进制(b 或 B)、十进制(d 或 D)、十六进制(h 或 H)与八进制(o 或 O), 有下面三种书写形式。

1) 简单的十进制格式

这种格式是直接由 0~9 的数字串组成的十进制数, 可以用符号“+”或“-”来表示数的正负。

2) 缺省位宽的基数格式

这种格式表示形式为:

`'<base_format><number>`

符号“'”为基数格式表示的固有字符, 该字符不能省略, 否则为非法表示形式; 参数<base_format>用于说明数值采用的进制格式; 参数<number>为相应进制格式下的一串数字。这种格式未指定位宽, 其缺省值至少为 32 位。

3) 指定位宽的基数格式

这种格式表示形式为:

`<size>'<base_format><number>`

参数<size>用来指定所表示数的位宽。当位宽小于数值的实际位数时, 相应的高位部分被忽略; 当位宽大于数值的实际位数, 且数值的最高位是 0 或 1 时, 相应的高位部分补 0; 而当位宽大于数值的实际位数, 但数值的最高位是 x 或 z 时, 相应的高位部分补 x 或 z。二进制的-一个 x 或 z 表示 1 位处于 x 或 z, 八进制的-一个 x 或 z 表示 3 位二进制位都处于 x 或 z, 十六进制的一个 x 或 z 表示 4 位二进制位都处于 x 或 z。另外, 数值中的 z 可以用“?”来代替。

【例 3-3】三种格式的整数表示实例。

```
659      //简单的十进制格式
'h 837FF //缺省位宽的十六进制数
'o7460   //缺省位宽的八进制数
4af      //非法的整数表示(十六进制数表示需'h)
b001     //非法的整数表示, 基数格式必须有字符“'”
4'b1001  //4 位的二进制数
5'D3     //5 位的十进制数
```




```
12'hx           // 12 位的未知数
16'hz           // 16 位的高阻态
16'b0001_1010_1111_1000 //使用下划线提高可读性
8'b_0001_1010   // 非法格式，下划线不能用于首字符
```

2. 实型数及其表示

Verilog HDL 中的实数可以用十进制与科学计数法两种格式来表示，如果采用十进制格式，小数点两边必须都有数字，否则为非法的表示形式。

【例 3-4】实数表示实例。

```
1.8             //十进制计数法
3.8e10          //科学计数法
2.1E-9          //科学计数法可用 e 或 E 表示，其结果相同
3_2387.3398_3047 //使用下划线提高可读性
3.              //非法表示
.2e6            //非法表示
```

3.1.3 字符串

字符串是用双引号括起来的字符序列，它必须包含在同一行中，不能分成多行书写。若字符串用作 Verilog HDL 表达式或赋值语句中的操作数，则字符串被看作 8 位的 ASCII 值序列，即一个字符对应 8 位的 ASCII 值。

【例 3-5】字符串实例。

```
"this is a string"
"print out a message\n"
"\t show the results"
"simulation time=%b"
"bell!\007"
```

1. 字符串变量的声明

Verilog HDL 中采用寄存器变量来存储字符串，寄存器变量的位数要大于字符串的最大长度。需要注意的是，Verilog HDL 中并不需要特殊位来存储终止符。

【例 3-6】字符串变量的声明。

```
reg [8*12: 1] stringvar;
initial
begin
    stringvar = "Hello world! ";
end
```

2. 字符串操作

可采用 Verilog HDL 的标准操作符对字符串进行诸如连接类的操作。在操作过程中，如果声明的字符串变量位数大于字符串的实际长度，则字符串变量的高位补 0。

【例 3-7】字符串操作。

```
module string_test;
```




```
reg [8*14: 1] stringvar;  
initial  
begin  
    stringvar = "Hello world";  
    $display("%s is stored as %h", stringvar, stringvar);  
    stringvar = {stringvar, "!!!"};  
    $display("%s is stored as %h", stringvar, stringvar);  
end  
endmodule
```

3. 特殊字符

特殊字符又称转意符，用“\”来说明。表 3.1 给出了特殊字符的表示及其意义。

表 3.1 特殊字符的表示及其意义

特殊字符表示	意 义
\n	换行符
\t	Tab 键
\\	符号\
*	符号*
\ddd	3 位八进制数表示的 ASCII 值
%%	符号%

3.1.4 关键字

关键字也称为保留词，它是 Verilog HDL 语言内部的专用词，所有的关键字都采用小写形式。表 3.2 给出了 Verilog HDL 中常用的关键字。

表 3.2 Verilog HDL 中常用的关键字

always	else	highz1	notif1	rtranif1	tri
and	end	if	or	scalared	tri0
assign	endattribute	initial	output	signed	tri1
attribute	endcase	inout	parameter	small	triand
begin	endfunction	input	pmos	specify	trior
buf	endmodule	integer	posedge	specpram	trireg
bufif0	endprimitive	join	primitive	strength	unsigned
bufif1	endspecify	large	pull0	strong0	vectored
case	endtable	macromodule	pull1	strong1	wait
casex	endtask	medium	pulldown	supply0	wand
casez	event	module	pullup	supply1	weak0
cmos	for	nand	rcmos	table	weak1
deassign	force	negedge	real	task	while
default	forever	nmos	realtime	time	wire
defpram	fork	nor	reg	tran	wor
disable	function	not	release	tranif0	xnor
edge	highz0	notif0	repeat	tranif1	xor



3.2 数据类型

Verilog HDL 作为一种硬件描述语言,其数据类型集合用来表示硬件数字电路中数据进行存储和传输的要素。Verilog HDL 不仅支持如整型、实型等抽象数据类型,同时也支持物理数据类型来表示真实的硬件。

3.2.1 物理数据类型

连线型和寄存器型是 Verilog HDL 中最主要的两种物理数据类型,这两种类型的变量在定义时均要设置位宽,当缺省状态时,位宽默认值为 1 位。变量的每一位可以取 0、1、x 或 z 中的任意值。x 表示一个未知初始状态的变量,或者由于多个驱动源试图将其设定为不同的值而引起的冲突型连线型变量,z 代表高阻状态或浮空量。

1. 连线型数据

连线型数据对应的是硬件电路中的物理信号连线,没有电荷保持作用(trireg 除外)。连线型数据必须有驱动源驱动,有两种方式对它进行驱动:一是在结构描述中把它连接到一个门或模块的输出端;二是用连续赋值语句 assign 对其进行赋值。当没有驱动源对其驱动时,它将保持高阻态。

1) Verilog HDL 中的连线型数据

为了能够精确地反映硬件电路中各种可能的物理信号连接特性,Verilog HDL 提供了多种连线型数据。表 3.3 给出了类型的定义及其相应的功能描述。

表 3.3 Verilog HDL 中的连线型数据及其功能描述

连线型数据	功能描述
wire, tri	两种常见类型
wor, trior	多重驱动时,具有线或特性的连线型
wand, triand	多重驱动时,具有线与特性的连线型
trireg	具有电荷保持特性的连线型
tri1	上拉电阻
tri0	下拉电阻
supply1	电源线,逻辑 1
supply0	电源线,逻辑 0

2) 连线型数据的真值表

在被一个以上驱动源驱动时,不同的连线型数据有各自决定其最终值的分辨办法。为了更好地理解不同连线型数据的功能,表 3.4 给出了多个驱动源驱动时四种连线型数据的真值表,以便读者参考。

表 3.4 多个驱动源驱动时四种连线型数据的真值表

wire/tri	0	1	x	z
0	0	x	x	0
1	x	1	x	1
x	x	x	x	x
z	0	1	x	z

wand/triand	0	1	x	z
0	0	0	0	0
1	0	1	x	1
x	0	x	x	x
z	0	1	x	z

wor/trior	0	1	x	z
0	0	1	x	0
1	1	1	1	1
x	x	1	x	x
z	0	1	x	z

tri0/tri1	0	1	x	z
0	0	x	x	0
1	x	1	x	1
x	x	x	x	x
z	0	1	x	0/1

2. 寄存器型数据

寄存器型数据对应的是具有状态保持作用的硬件电路元件，如触发器、锁存器等。若寄存器型数据未初始化，它将为未知状态 x。寄存器型数据的关键字为 reg，缺省时为 1 位数。寄存器型数据与连线型数据的区别在于：寄存器型数据保持最后一次的赋值，而连线型数据需要有持续的驱动。寄存器型数据的驱动可以通过过程赋值语句实现，过程赋值语句只能出现在过程语句后面的过程块语句中。为了更好地理解这一点，我们下面通过例 3-8 进行说明。

【例 3-8】示例模块。

```
module D_ff(q, data, enable, reset, clock);
    output q;
    input data, enable, reset, clock;
    reg q;
    always @(posedge clock) //时钟上升沿有效
        if (reset == 0)
            q = 1'b0;
        else if (enable == 1)
            q = data;
        //隐含: else q = q;
endmodule
```

在上面的例子中，寄存器 q 通过过程赋值语句进行赋值，且必须出现在过程语句中；



其值一直保持到满足相应条件为止。

3. 物理数据类型的声明

物理数据类型的声明格式如下：

```

<net_declaration>
    ::= <NETTYPE> <expandrange>? <delay>? <list_of_variables> ;
    ||= trireg <charge_strength>? <expandrange>? <delay>? <list_of_variables> ;
    ||= <NETTYPE> <drive_strength>? <expandrange>? <delay>? <list_of_assignments> ;

<reg_declaration>
    ::= reg <range>? <list_of_register_variables> ;

<list_of_variables>
    ::= <name_of_variable> <, <name_of_variable>*>

<name_of_variable>
    ::= <IDENTIFIER>

<list_of_register_variables>
    ::= <register_variable> <, <register_variable>*>

<register_variable>
    ::= <name_of_register>

<name_of_register>
    ::= <IDENTIFIER>

<expandrange>
    ::= <range>
    ||= scaled <range>
        if [the data type is not a trireg] the following syntax is available:
    ||= vectored <range>

<range>
    ::= [ <constant_expression> : <constant_expression> ]

<list_of_assignments>
    ::= <assignment> <, <assignment>*>

<charge_strength>
    ::= ( <CAPACITOR_SIZE> )

<drive_strength>
    ::= ( <STRENGTH0>, <STRENGTH1> )
    ||= ( <STRENGTH1>, <STRENGTH0> )
  
```

针对上述声明格式说明以下几点：

(1) 符号“：=”表示某项的定义；符号“?”表示该项可选；符号“*”表示该项列表至少有一项；符号“||”表示该项的另外一种定义形式。

(2) <NETTYPE> 表示连线型数据的类型。

(3) <delay>指定延迟时间，在以后的章节将作详细的论述。

(4) <CAPACITOR_SIZE>只有当连线型为 trireg 时才使用，可取关键字 small、

medium、large 中的任一个, 用来指定所保持的电荷容量的相对大小。

(5) <STRENGTH0>可取关键字 supply0、strong0、pull0、weak0、highz0 中的任何一个, <STRENGTH1>可取关键字 supply1、strong1、pull1、weak1、highz1 中的任何一个, 用来指定连线型数据赋值时的驱动强度。

(6) <range>用来指定物理数据类型为标量或矢量。若该项缺省, 表示物理数据类型为1位的标量形式; 反之, 由该项指定物理数据类型的矢量形式。

(7) 如果物理数据类型为矢量形式, 可由关键字 scalared 或 vectored(trireg 除外)来指定为标量类矢量或矢量类矢量。

【例 3-9】物理数据类型的例子。

```
reg a;                      //标量寄存器变量
reg[3: 0] v;                //4位的寄存器变量
tri[15: 0] bus;             //16位的三态总线
tri1[7: 0] data_bus;        //8位的数据线, 当处于三态时为上拉电阻
tri0[31: 0] add_bus;        //32位的连线型, 当处于三态时为下拉电阻
reg scalared[1: 4] b;       //4位的标量类寄存器矢量
reg[7: 0] RAM[0: 1023];     //8位宽存储量为1K的RAM
wire #(2.4, 1.8) carry;     //具有升降延迟的连线型
wire(strong1, pull0) sum=a+b; //指定驱动强度且连续赋值的连线型
trireg (small) storeit;     //小强度的电荷存储线
```

4. 存储器

Verilog HDL 中采用寄存器数组来表示存储器, 用来对 ROM、RAM 或寄存器文件进行建模。数组中的每一个寄存器均称为一个元素, 用不同的索引号来寻址。Verilog HDL 中没有多维数组。

存储器的语法格式如下:

```
<register_variable>
    ::= <name_of_register>
    ||= <name_of_memory> [ <constant_expression> : <constant_expression> ]
<constant_expression>
    ::= <expression>
<name_of_memory>
    ::= <IDENTIFIER>
```

【例 3-10】存储器的例子。

```
parameter // 实时参数
    wordsize = 16,
    memsize = 256;
// 声明 16 位 256 字的存储器与寄存器
reg [wordsize-1: 0]
    mem [memsize-1: 0],
    writereg,
```




```
readreg;
```

3.2.2 抽象数据类型

除了与具体的硬件电路对应的物理数据类型外, 为了描述方便, Verilog HDL 中还提供了以下几种抽象数据类型。抽象数据类型只是纯数学的抽象描述, 不与任何实际的物理硬件电路相对应。

1. integer 整型

整型数据常用于对循环控制变量的说明, 在算术运算中被视为二进制补码形式的有符号数。除了寄存器型数据被当作无符号数来处理之外, 整型数据与 32 位寄存器型数据在实际意义上相同。

整型数据的声明格式如下:

```
<integer_declaration>
```

```
::= integer <list_of_register_variables> ;
```

2. time 时间型

时间型数据与整型数据相类似, 只是它是 64 位的无符号数。时间型数据主要用于对模拟时间的存储与计算处理, 常与系统函数 \$time 一起使用。

时间型数据的声明格式如下:

```
<time_declaration>
```

```
::= time <list_of_register_variables> ;
```

3. real 实型

Verilog HDL 支持实型常量与变量。实型数据在机器码表示法中是浮点型数值, 可用于对延迟时间的计算。

实型数据的声明格式如下:

```
<real_declaration>
```

```
::= real <list_of_variables>;
```

有关实型数据的详细内容, 可参照上一节中有关实型数及其表示的说明部分。

4. parameter 参数型

参数型数据是被命名的常量, 在仿真开始前对其赋值, 在整个仿真过程中保持其值不变, 数据的具体类型是由所赋的值来决定的。可以用它来定义变量的位宽以及延迟时间等, 其方法是用一文字参数来代替一个数字量, 从而增加描述的可读性, 还可以为以后的修改带来很大的方便。

参数型数据的声明格式如下:

```
<parameter_declaration>
```

```
::= parameter <list_of_assignments> ;
```

5. 抽象数据类型的例子

为了更好的理解抽象数据类型的用法, 我们通过下面的例子进行说明。

【例 3-11】抽象数据类型的例子。

```
integer count;           //简单的 32 位有符号整数
integer k[1: 64];        //含有 64 个元素的整型数组
```




```
time start, stop;           //两个 64 位的时间变量
real stime;                 //实型数据
parameter msb = 7;          //定义 msb 为常数 7
parameter e = 25, f = 9;    //定义两个常数
parameter average_delay = (r + f) / 2;
parameter byte_size = 8, byte_mask = byte_size - 1;
parameter r = 5.7;          //定义 r 为实参
```

注：系统软件中预定义宏分为两类，即 Preprocessor 预定义宏和 Parameter 预定义宏。其中 Preprocessor 预定义宏的功能与 C 语言中的宏类似，而 Parameter 预定义宏的功能与 Verilog HDL 中的 parameter 类似。在 Verilog HDL 中，宏定义与 parameter 定义是两种不同的概念，宏定义是在编译前进行的，而 parameter 定义是在编译时进行的。宏定义通常用于定义常量表达式，而 parameter 定义通常用于定义常量值。宏定义通常用于定义常量表达式，而 parameter 定义通常用于定义常量值。

3.3 运算符

Verilog HDL 中定义了许多运算符，按功能不同可以分为算术运算符、逻辑运算符、关系运算符等九大类，同时同其它高级语言一样，各类运算符之间也有优先级之分。表 3.5 详细给出了 Verilog HDL 中定义的运算符分类及简单的功能说明。

表 3.5 Verilog HDL 中的运算符分类及功能说明

运算符类型	运算符	简单说明
算术运算符	+, -, *, / %	算术运算 取模
逻辑运算符	! && 	逻辑非 逻辑与 逻辑或
关系运算符	<, >, <=, >=	关系
相等运算符	== != === !==	相等 不等 全等 非全等
按位运算符	~ & ^ ^~or~^	按位非 按位与 按位或 按位异或 按位异或非
归约运算符	& ~& ~ ^ ~^or^~	归约与 归约与非 归约或 归约或非 归约异或 归约异或非
移位运算符	<< >>	左移位 右移位
条件运算符	?:	条件
连接运算符	{}	连接



3.3.1 算术运算符

算术运算符是二元运算符，操作数可以是物理数据类型，也可以是抽象数据类型。如果操作数为寄存器型或连线型，则为无符号数；如果为整型或实型，则可以有符号数。当用基数格式表示整数时，一定为无符号数；若两个操作数中有一个为未知数 x ，则结果为未知的。

算术运算符包括： $+$ (加法运算)、 $-$ (减法运算)、 $*$ (乘法运算)、 $/$ (除法运算)和 $\%$ (取模)。

【例 3-12】算术运算符的例子。

```
module arithTest;
  reg [3: 0] a, b;
  initial begin
    a = 4'b1100          // 12
    b = 4'b0011;         // 3
    $displayb(a * b);    // 乘法运算符，结果为 4'b0100
    // 等于 36 的低 4 位
    $display(a / b);     // 除法运算符，结果为 4
    $display(a + b);     // 加法运算符，结果为 15
    $display(a - b);     // 减法运算符，结果为 9
    $display((a + 1'b1) % b); // 取模运算符，结果为 0
  end
endmodule
```

3.3.2 逻辑运算符

逻辑运算符包括三种： $\&\&$ (逻辑与)、 $\|\|$ (逻辑或)和 $!$ (逻辑非)。其中 $\&\&$ 和 $\|\|$ 为二目运算符，要求有两个操作数，而 $!$ 为单目运算符，只需要一个操作数。当进行逻辑运算时，如果操作数是 1 位的，则用“1”来表示逻辑真状态，用“0”来表示逻辑假状态；若操作数由多位组成，则必须把操作数当作一个整体来处理，即如果操作数所有位都是 0，那么，它就具有逻辑 0；反之，只要其中有某一位为 1，就把这个操作数整体看作具有逻辑 1。如果操作数中存在未知数 x ，则结果也是未知的。表 3.6 给出了逻辑运算的真值表。

表 3.6 逻辑运算的真值表

m	n	!m	!n	m&& n	m n
0	0	1	1	0	0
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	1	1

【例 3-13】逻辑运算符的例子。

```
module logicalTest;
  reg [3: 0] a, b, c;
```




```
initial begin
    a = 2; b = 0; c = 4'hx;
    $display(a && b);    // 逻辑与, 结果为 0
    $display(a || b);    // 逻辑或, 结果为 1
    $display(!a);        // 逻辑非, 结果为 0
    $display(a || c);    // 结果为 1, 未知数 || 1 (=1)
    $display(!c);        // 结果为未知数
end
endmodule
```

3.3.3 关系运算符

关系运算符用于比较两个操作数的大小关系, 如果比较关系成立, 则返回逻辑 1; 否则返回逻辑 0。如果两个操作数中有一个为未知数 x, 则结果也是未知的。

Verilog HDL 中的关系运算符有: <(小于)、>(大于)、<=(小于等于)和>=(大于等于)。

【例 3-14】关系运算符的例子。

```
module relatTest;
    reg [3: 0] a, b, c, d;
    initial begin
        a=2;
        b=5;
        c=2;
        d=4'hx;
        $display(a < b);    // 结果为真 1
        $display(a > b);    // 结果为假 0
        $display(a >= c);   // 结果为真 1
        $display(d <= a);   // 结果为未知数
    end
endmodule
```

3.3.4 相等关系运算符

Verilog HDL 提供了四种相等关系运算符: == (相等)、!= (不等)、=== (全等)和!== (非全等)。

这四个运算符均为双目运算符, 将两个操作数的值逐位进行比较。如果两个操作数的位数不同, 则位数较小的操作数在高位以 0 补齐。

对于相等算符, 如果两操作数所有的位值均相等, 那么相等关系成立, 结果返回逻辑 1; 否则返回逻辑 0。但若任何一个操作数中的某一位为未知数 x 或处于高阻态, 则结果为未知的。两操作数的对应位分别取四个逻辑值 0、1、x、z, 则相等算符的运算规则可用表 3.7 来描述。



表 3.7 相等算符的运算规则

=	0	1	x	z
0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

如果两操作数的所有位取 0 或 1, 那么不等算符与相等算符的运算规则正好相反; 如果任何操作数中含有未知数 x 或高阻态, 则不等算符与相等算符的运算规则相同, 如表 3.8 所示。

表 3.8 不等算符的运算规则

!=	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

对于全等算符, 其比较过程与相等算符相同, 但其返回结果只有逻辑 1 或逻辑 0 两种状态, 不存在未知数, 即全等算符将未知数 x 与高阻态看作是逻辑状态的一种参与比较, 如果两操作数的相应位均为 x 或 z, 那么全等关系成立, 结果返回逻辑 1, 并非返回未知数 x。非全等算符与全等算符正好相反, 这里不再赘述。表 3.9 给出了全等算符的运算规则。

表 3.9 全等算符的运算规则

===	0	1	x	z
0	1	0	0	0
1	0	1	0	0
x	0	0	1	0
z	0	0	0	1

【例 3-15】相等关系运算符的例子。

```

module equTest;
    reg [3: 0] a, b, c, d, e, f;
    initial begin
        a = 4;   b = 7;    //缺省值为十进制数
        c = 4'b010;
        d = 4'bx10;
        e = 4'bx101;
        f = 4'bxx01;
        $displayb(c);      // 输出 0010
        $displayb(d);      // 输出 xx10
        $display(a == b);  // 相等运算符, 结果为 0
    end
endmodule

```




```
$display(c != d);    // 不等运算符, 结果为 x
$display(c != f);    // 不等运算符, 结果为 1
$display(d == e);    // 全等运算符, 结果为 0
$display(c !== d);   // 非全等运算符, 结果为 1

end
endmodule
```

3.3.5 按位运算符

Verilog HDL 提供的按位运算符有: $\sim$ (按位非)、 $\&$ (按位与)、 $|$ (按位或)、 $\wedge$ (按位异或) 和 $\wedge\sim$ 或 $\sim\wedge$ (按位异或非)。

上面的按位运算符中, 除了按位非为单目运算符外, 其余均为双目运算符。按位运算符对其操作数的每一位进行操作, 例如表达式 $a\&b$ 的结果是 a 和 b 的对应位相与的值。对具有不定值的位进行操作, 视情况而定会得到不同的结果。例如 x 和 0 相或得 x , x 和 1 相或得 1 。对于双目运算符, 如果两个操作数的长度不相等, 则较短的操作数将用 0 来补位。逐位运算将返回一个与两个操作数中位宽较大的一个等宽的值。

不管是单目按位运算符, 还是双目按位运算符, 经过按位运算后, 原来的操作数有几位, 则所得结果仍为几位。为了更好地运用按位运算符, 表 3.10 给出了五类按位操作符的运算规则。

表 3.10 五类按位操作符的运算规则

$\sim$	0	1	x	z
结果	1	0	x	x

$\&$	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

$ $	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

$\wedge$	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	1



$\sim^{\wedge}\text{or}^{\wedge}\sim$	0	1	x	z
0	1	0	x	0
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

【例 3-16】按位运算符的例子。

```

module bitTest;
    reg [3: 0] a, b, c;
    initial begin
        a = 4'b1100; b = 4'b0011; c = 4'b0101;
        $displayb(~ a);      // 按位非运算符, 结果为 4'b0011
        $displayb(a & c);    // 按位与运算符, 结果为 4'b0100
        $displayb(a | b);    // 按位或运算符, 结果为 4'b1111
        $displayb(b ^ c);    // 按位异或运算符, 结果为 4'b0110
        $displayb(a ~^ c);   // 按位异或非运算符, 结果为 4'b0110
    end
endmodule

```

值得注意的是, 不要将逻辑运算符和按位运算符相混淆。比如, “!” 是逻辑非, 而 “~” 是按位操作的非, 即按位取反。例如, 对于前者 $!(3==2)$ 结果是 1, 而后者对位进行操作, $\sim\{0, 0, 1, 0\}=1101$ 。

3.3.6 归约运算符

归约运算符是单目运算符, 包括 &(归约与)、| (归约或)、^(归约异或)及其相应的非运算: $\sim\&$ (归约与非)、 $\sim|$ (归约或非)、 $\sim^{\wedge}$ 或 $^{\wedge}\sim$ (归约异或非)。运算规则类似于按位运算符的运算规则, 但其运算过程不同。按位运算是操作数的相应位进行与、或、异或及其非运算, 操作数是几位数, 则运算结果也是几位数。而归约运算则不同, 归约运算是操作数进行归约的递推运算, 最后的运算结果是 1 位的二进制数。归约运算的具体运算过程是: 先将操作数的第 1 位与第 2 位进行归约运算, 然后将运算结果与第 3 位进行归约运算, 依次类推, 直至最后一位。

【例 3-17】归约运算符的例子。

```

module reductTest;
    reg [3: 0] a, b, c;
    initial begin
        a = 4'b1111;
        b = 4'b0101;
        c = 4'b0011;
        $displayb(& a);    // 归约与运算符, 结果为 1
        $displayb(| b);    // 归约或运算符, 结果为 1
        $displayb(^ b);    // 归约异或运算符, 结果为 0
    end
endmodule

```




```
end  
endmodule
```

3.3.7 移位运算符

移位运算符是双目运算符，将运算符左边的操作数左移或右移运算符右边的操作数指定的位数，用 0 来补充空闲位。如果右边操作数的值为 x 或 z，则移位结果为未知数 x。

Verilog HDL 中移位运算符有两种：<< (逻辑左移) 和 >> (逻辑右移)。

【例 3-18】移位运算符的例子。

```
module shiftTest;  
    reg [3: 0] a;  
    initial begin  
        a = 4'b1010;  
        $displayb(a << 1);    // 左移 1 位，结果为 4'b0100  
        $displayb(a >> 2);    // 右移 2 位，结果为 4'b0010  
    end  
endmodule
```

进行移位运算时应注意移位前后变量的位数，如下面给出的例子。

【例 3-19】移位运算前后位数的变化。

```
4'b1010 << 1 = 5'b10100;  
1 << 6 = 32b'1000000;  
4'b1010 << 4 = 4'b0000;
```

3.3.8 条件运算符

Verilog HDL 中只有一种条件运算符“?:”，也是惟一的三目运算符，即条件运算符有三个操作数。若第一个操作数是逻辑 1，则算子返回第二个操作数；若第一个操作数是逻辑 0，则返回第三个操作数；若第一个操作数为未知数或高阻态，则按表 3.11 给出的逻辑，将第二个操作数与第三个操作数按位比较得到最终结果。

表 3.11 第一个操作数为未知数或高阻态时“?:”的运算规则

?:	0	1	x	z
0	0	x	x	x
1	x	1	x	x
x	x	x	x	x
z	x	x	x	x

【例 3-20】条件运算符的例子。

```
module add_or_subtract(a, b, op, result);  
    parameter ADD=1'b0;  
    input [7: 0] a, b;  
    input op;
```




```
output [7: 0] result;  
assign result = (op == ADD) ? a+b : a-b;  
endmodule
```

3.3.9 连接与复制操作

连接运算是将多个小的表达式合并形成一个大的表达式。Verilog HDL 中，用符号“{}”实现多个表达式的连接运算，各个表达式之间用“,”隔开。除了非定长的常量外，任何表达式都可进行连接运算。同时连接运算可以对信号的某些位进行。

【例 3-21】连接运算的例子。

```
module concatTest;  
    reg    a;  
    reg [1: 0] b;  
    reg [5: 0] c;  
    initial begin  
        a = 1'b1;  
        b = 2'b00;  
        c = 6'b101001;  
        $displayb({a, b});      // 产生一个 3 位数 3'b100  
        $displayb({c[5: 3], a}); // 产生一个 4 位数 4'b1011  
    end  
endmodule
```

此外在 Verilog 语言中还有一种复制运算符“{ }”，将一个表达式放入双重花括号中，而复制因子放在第一层括号中，用来指定复制的次数。该运算符为复制一个常量或变量提供一种简便记法。

【例 3-22】复制运算的例子。

```
module replicTest;  
    reg    a;  
    reg [1: 0] b;  
    reg [5: 0] c;  
    initial begin  
        a = 1'b1;  
        b = 2'b00;  
        $displayb({4{a}}); // 结果为 1111  
        c = {4{a}};  
        $displayb(c);      // 结果为 001111  
    end  
endmodule
```


3.3.10 运算符的优先级

与其它高级语言一样，Verilog HDL 中的所有运算符也有优先级之分。表 3.12 给出了运算符从最高优先级到最低优先级的排列，同一行中的运算符具有相同的优先级。

表 3.12 运算符的优先级

运算符	描 述
[]	位选择或部分选择
()	圆括号
! ~	逻辑非与按位非
& ~& ~ ^ ~^^ ~	归约运算符
+ -	单目算术运算符
{ }	连接运算符
* / %	算术运算符
+ -	双目算术运算符
<< >>	移位运算符
> >= < <=	关系运算符
== !=	相等运算符
&	按位与
^^ ~ ~^	按位异或与异或非
	按位或
&&	逻辑与
	逻辑或
? :	条件运算符

在表 3.12 中，除了条件运算符从右向左关联外，其余所有运算符均是从左向右关联。

3.4 系统任务与系统函数

为了便于设计者对仿真过程进行控制，以及对仿真结果进行分析比较，Verilog HDL 提供了大量的系统功能调用，大致可以分成两种：一种是任务型的功能调用，称为系统任务；另一种是函数型的功能调用，称为系统函数。Verilog HDL 的系统任务与系统函数是以字符“\$”开头的标识符，它们的主要区别有两点：系统任务可以没有返回值，或多个返回值，而系统函数只有一个返回值；系统任务可以带有延迟，而系统函数不允许延迟，在 0 时刻执行。

系统任务与系统函数已内置于 Verilog HDL 中，用户可以随意调用。而且用户可以根据自己的需要，基于 Verilog 仿真系统提供的 PLI(Program Language Interface)编程接口，编制特殊的系统任务与系统函数，这将在第 7 章中进行详细阐述。根据系统任务与系统函数实现的功能不同，可将其分成以下几类：

- (1) 标准输出任务。
- (2) 文件管理任务。
- (3) 仿真控制任务。
- (4) 时间函数。
- (5) 其他。



3.4.1 标准输出任务

Verilog HDL 中有两种主要的标准输出任务：`$display` 和 `$write`。这两种标准输出任务的格式相同，区别在于 `$display` 任务在将特定信息输出到标准输出设备时，具有自动换行的功能，而 `$write` 则不带有行结束符。下面以 `$display` 任务为例进行详细说明。

`$display` 可以用来输出字符串、表达式及变量值，其语法格式与 C 语言中的 `printf` 函数相同，可表示如下：

`$display(<format_specifiers>, signal, signal, .....);`

其中，`<format_specifiers>` 用来指定输出格式。表 3.13 给出了各种不同的输出格式。

表 3.13 输出格式符的说明

输出格式符	格式说明
<code>%d</code> or <code>%D</code>	十进制格式
<code>%b</code> or <code>%B</code>	二进制格式
<code>%h</code> or <code>%H</code>	十六进制格式
<code>%m</code> or <code>%M</code>	模块的分级名
<code>%o</code> or <code>%O</code>	八进制格式
<code>%t</code> or <code>%T</code>	时间格式
<code>%e</code> or <code>%E</code>	指数格式输出实数
<code>%f</code> or <code>%F</code>	浮点格式输出实数
<code>%g</code> or <code>%G</code>	以上两种格式中较短的输出实数
<code>%c</code> or <code>%C</code>	ASCII 码格式
<code>%s</code> or <code>%S</code>	字符串格式
<code>%v</code> or <code>%V</code>	输出连线型数据的驱动强度
<code>\n</code>	换行
<code>\t</code>	Tab 键
<code>\\</code>	字符 \
<code>*</code>	字符 *
<code>\ddd</code>	3 位八进制数表示的 ASCII 值
<code>%%</code>	符号 %

【例 3-23】`$display` 任务的例子。

```

$display("Hello Dr Blair");
..... output: Hello Dr Blair

$display($time) //目前的仿真时间
..... output: 460

counter = 4'b10;

$display(" The count is %b", counter);
..... output: The count is 0010

```

在 Verilog HDL 中，除了 `$display` 与 `$write` 这两种主要的标准输出任务外，还有以下几种标准输出任务：



- (1) \$displayb 与 \$writeb(输出二进制数)。
- (2) \$displayo 与 \$writeo(输出八进制数)。
- (3) \$displayh 与 \$writeh(输出十六进制数)。

3.4.2 文件管理任务

1. 打开文件

Verilog HDL 中利用关键字 \$fopen 来打开文件，其语法格式如下：

```
<file_handle>=$fopen("<file_name>");
```

<file_name>指定被打开的文件名及其路径，如果路径与文件名正确，则返回一个 32 位的句柄描述符 <file_handle>，且其中只有一位为高电平，否则返回出错信息。因为标准输出具有自己的最低位设置，所以当第一次使用 \$fopen 时，返回的 32 位句柄描述符中将次低位设置为高电平。每一次调用 \$fopen，都会返回一个新的句柄，且高电平依次左移。为了更好地理解这一点，下面给出例子加以说明。

【例 3-24】打开文件的例子。

```
integer handleA, handleB; // 定义 2 个 32 位的整数
initial begin
    handleA = $fopen("myfile.out");
    // handleA = 0000_0000_0000_0000_0000_0000_0010
    handleB = $fopen("anotherfile.out");
    // handleB = 0000_0000_0000_0000_0000_0000_0100
end
```

2. 输出到文件

Verilog HDL 中用来将信息输出到文件的系统任务有 \$fdisplay、\$fwrite 和 \$fmonitor。它们具有如下相同的语法格式：

```
<task_name>(<file_handles>, <format_specifiers>);
```

其中 <task_name> 是上述三种系统任务中的任一种。<file_handles> 是文件句柄描述符，与打开文件所不同的是，这里可以对句柄进行多位设置(见下面的例子)。<format_specifiers> 用来指定输出的格式，格式说明符可参照表 3.13。

【例 3-25】输出到文件的例子。

```
//利用例 3-24 的句柄
integer channelsA;
initial begin
    channelsA = handleA | 1;
    $fdisplay(channelsA, "Hello");
end
```

该例中的 handleA 为例 3-24 中的句柄输出。

3. 关闭文件

系统函数 \$fclose 能够用于关闭文件，其语法格式如下：

```
$fclose(<file_handle>);
```




当使用多个文件时，为了提高速度，可以将一些不再使用的文件关闭。一旦某个文件被关闭，则不能再向它写入信息，且其它文件可以使用该文件的句柄。

4. 从文件中读出数据到存储器

Verilog HDL 中有两个系统任务能够把一个数据文件中的数据内容，读入到一个指定的存储器中。这两个系统任务是\$readmemb 和\$readmemh。其区别在于，前者要求以二进制数据格式存放数据文件，而后者要求以十六进制数据格式存放数据文件。它们具有相同的语法格式：

`<task_name>(<file_name>, <register_array>, <start>, <end>);`

其中<task_name>用来指定系统任务，可取上述任务中的一个。<file_name>是读出数据的文件名。<register_array>为要读入数据的存储器，<start>与<end>分别为存储器的起始地址与结束地址。

现有一数据文件 mem.data，可通过下面的例子将其读入到存储器中。

【例 3-26】从文件中读出数据到存储器的例子。

```
module testmemory;
    reg [7: 0] memory [9: 0];
    integer index;
    initial begin
        $readmemb("mem.dat", memory);
        for(index = 0; index < 10; index = index + 1)
            $display("memory[%d] = %b", index[4: 0], memory[index]);
    end
endmodule
```

3.4.3 仿真控制任务

1. 仿真监控任务

Verilog HDL 中有三种仿真监控任务：\$monitor、\$monitoron 和\$monitoroff。

\$monitor 的语法格式为

`$monitor(<format_specifiers>, signal, signal, .....);`

可以看出，\$monitor 与\$display 的语法格式相同，因而\$monitor 的参数说明可参照\$display 部分。该任务用来连续监控指定的信号参数，如果发现其中任何一个信号发生变化，则系统按照调用\$monitor 时所规定的格式，在时间步结束时显示整个信号表。

可以通过系统任务\$monitoron 打开监控任务，通过系统任务\$monitoroff 关闭监控任务。

【例 3-27】系统任务\$monitor 的例子。

```
module myTest;
    integer a, b;
    initial begin
        a = 2;
        b = 4;
```




```
        forever begin
            #5 a = a + b;
            #5 b = a - 1;
        end
    end
    initial #40 $finish;
    initial begin
        $monitor($time, " a = %d, b = %d", a, b);
    end
endmodule
```

输出结果为:

0	a =	2,	b =	4
5	a =	6,	b =	4
10	a =	6,	b =	5
15	a =	11,	b =	5
20	a =	11,	b =	10
25	a =	21,	b =	10
30	a =	21,	b =	20
35	a =	41,	b =	20

2. 仿真结束任务

Verilog HDL 中有两个系统任务可以用来结束仿真: \$finish 和 \$stop。

\$finish 用来终止仿真器的运行, 结束仿真过程并返回到操作系统。\$stop 暂时挂起仿真器, 进入 Verilog 界面, 可以通过输入相应命令使仿真继续运行。

【例 3-28】仿真结束任务的例子。

```
initial begin
    clock = 1'b0;
    ..... // 需要完成的任务
    #200 $stop // 暂停仿真并进入交互方式
    #500 $finish // 结束仿真任务
end
```

3.4.4 时间函数

1. 时间标度函数

Verilog HDL 提供了两种时间标度函数: \$timeformat 和 \$printtimescale。

\$timeformat 用于控制 %t 格式如何显示时间信息, 语法格式为

\$timeformat(<unit>, <precision>, <suffix>, <min_field_width>);

其中, <unit> 用于指定时间单位, 其取值范围为 0 ~ -15, 各值所代表的时间单位如表 3.14 所示。



表 3.14 <unit>的取值及其代表的时间单位

取值	时间单位	取值	时间单位
0	1s	-8	10 ns
-1	100 ms	-9	1 ns
-2	10 ms	-10	100 ps
-3	1 ms	-11	10 ps
-4	100 μs	-12	1 ps
-5	10 μs	-13	100 fs
-6	1 μs	-14	10 fs
-7	100 ns	-15	1 fs

<precision>指定所要显示时间信息的精度，<suffix>为诸如“ms”、“ns”之类的字符，<min_field_width>说明时间信息的最小字符数。

【例 3-29】\$timeformat 的例子。

```
$timeformat(-9, 2, "ns", 10);
```

```
$display("current simulation time is: %t", $time);
```

\$printtimescale 的语法格式为

```
$printtimescale(module_hierarchical_name);
```

该函数给出指定模块的时间单位与精度。如果没有指定参数，则用于输出包含该任务调用的所有模块的时间单位与精度。

2. 时间显示函数

时间显示函数包括：

- (1) \$time：返回 64 位的整数，指定当前的仿真时间。
- (2) \$stime：返回 32 位的仿真时间。
- (3) \$realtime：以实数形式返回当前的仿真时间。

3.4.5 其他

1. 随机函数

当进行模块测试时，常需要提供随机脉冲序列，该功能可以通过系统函数\$random 来实现，它的语法格式为

```
$random %<number>;
```

其中，<number>为一大于 0 的整数，用来指定所产生随机数的范围，即- <number>+1 到 <number>-1。

【例 3-30】系统函数\$random 的例子。

```
reg [7: 0] rand_data;
```

```
always
```

```
#100 rand_data=$random %50;
```

2. 变换函数

有时需要将整数变换成实数，或将实数变换成整数，或用向量形式来表示实数等等。

Verilog HDL 提供了许多变换函数可以方便地实现上述功能。



- (1) \$rtoi: 通过截断小数数值将实数变换成整数。
- (2) \$itor: 将整数变换成实数。
- (3) \$realtobits: 将实数变换为 64 位的实数向量表示法。
- (4) \$bitstoreal: 将位模式变换为实数。

3.5 编译指令

与 C 语言中的编译预处理指令类似, Verilog HDL 中提供了大量的编译指令。通过这些编译指令, EDA 工具开发商使他们的工具解释 Verilog HDL 模型变得相当容易。在详细介绍这些编译指令之前, 有必要先进行几点说明:

- (1) 编译指令以字符 “`” 开头。
- (2) 编译指令并非 Verilog HDL 的描述, 因而编译指令末尾不需要分号。
- (3) 编译指令不受模块与文件的限制。在进行 Verilog 语言编译时, 已定义的编译指令一直有效, 直到有其它的编译指令修改它或将该编译指令关闭。

3.5.1 宏编译指令

`define 指令用于宏替换, 语法格式如下:

```
`define<macro_name>(<arguments>)<text_string>(<arguments>);
```

其中, <macro_name>为用于被替换的宏名, 即在编译过程中, 一旦遇到该宏名则用 <text_string>定义的文本内容来替换。这里需要注意的是, 文本内容必须在一行中定义, 不能采用多行。

宏替换指令与参数定义的区别在于, 参数具有全局可见性, 而宏替换指令只局限于宏函数。

【例 3-31】宏替换的例子。

```
`define cycle 20//clock period
always #(cycle/4) clk= ~ clk;
`define NAND(dval) nand #(dval)
`NAND(3) i1 (y, a, b);
`NAND(3: 4: 5) i2 (o, c, d);
```

可以利用`undef 指令来取消宏定义。

3.5.2 文件包含指令

Verilog HDL 中文件包含指令`include 类似于 C 语言中的编译指令#include, 文件包含指令在模块调用时非常有用, 可以通过该指令包含诸如类型声明与函数之类的 Verilog 码。

【例 3-32】文件包含指令的例子。

//文件 file1.v 的内容

```
`define WORDSIZE 8
function [WORDSIZE-1: 0] fastadder;
```




```
.....  
    endfunction  
//文件 secondfile 的内容  
module secondfile (in1, in2, out)  
    `include file1.v  
    wire [WORDSIZE-1: 0] temp;  
    assign temp = fastadder (in1, in2);  
    .....  
endmodule
```

3.5.3 条件编译指令

Verilog HDL 中有三个条件编译指令：`ifdef、`else 和 `endif。

这三个条件编译指令通常一起使用，其语法格式如下：

```
`ifdef <macro_name>  
    verilog_source_code  
`else  
    verilog_source_code  
`endif
```

如果定义了指定的宏名，则编译第一组描述语句，否则就编译第二组描述语句。下面通过一个例子来说明。

【例 3-33】条件编译指令的例子。

```
// 条件编译  
`ifdef TEST //当且仅当测试宏 TEST 定义时编译模块 counter  
    module counter;  
        .....  
    endmodule  
`else // 否则编译模块 counter_test  
    module counter_test;  
        .....  
    endmodule  
`endif
```

3.5.4 时间定标指令

时间定标指令 `timescale 的语法格式为

```
`timescale<time_unit> <base>/<precision> <base>
```

在 Verilog HDL 模型中，所有时延都用单位时间描述，且是一个相对的概念。<time_unit>用于指定计时单位值，必须为整数，且只能取 1、10 或 100。<base>用于指定计时单位，其范围从秒级到毫微微秒级变化。<precision>为计时精度值，也只能取 1、10 或 100。这里需要注意的是，在定义 `timescale 时，计时单位必须大于等于精度单位，否则



就失去了定标的意义，而且是非法的。

【例 3-34】合法与非法的时间定标例子。

```
`timescale 1 ns / 10 ps    //合法定标
`timescale 1ps / 10 ps    //非法定标
```

3.5.5 工作库定义指令

工作库定义指令`uselib 的语法格式为

```
`uselib <lib_specifiers>
```

其中，<lib_specifiers>可以是一个带路径的库文件名，也可以是一个带路径的库文件目录。如果该项缺省，则表明取消上一次对工作库的定义。下面通过一个例子进行说明。

【例 3-35】工作库定义指令的例子。

```
`uselib file=/models/rtl_lib
ALU il(y1, a, b, op); //RTL 模型
`uselib dir=/models/gate_lib libext=.v
ALU il(y1, a, b, op); //Gate 模型
`uselib //关闭工作库 `uselib
```




第4章 用户自定义元件(UDP)

本章集中介绍了 Verilog HDL 中用户自定义元件的描述方法，无论是组合逻辑电路还是时序逻辑电路元件，用户都可以通过这种描述方法进行自定义。

Verilog HDL 语言内部已经包含了一些常用的基本元件和逻辑门。用户可以方便地将它们调用到自己的设计中，这些基本元件可以分成基本门级元件(Basic Gate-level Primitives)和基本开关级元件(Basic Switch-level Primitives)两类。但是考虑到不同用户的需要，Verilog 还为用户提供了自己进行元件定义的方法，这就是本章将介绍的用户自定义元件 UDP(UDP: User Defined Primitives)。UDP 元件的定义是通过真值表的形式来进行的，可以用真值表来描述组合逻辑 UDP 元件和时序逻辑 UDP 元件的逻辑功能。用户自定义元件一旦定义好后，就可以像调用 Verilog 内含的基本元件一样地来对它进行调用了。

4.1 UDP 的定义

UDP 的定义是由一个独立的定义模块构成的，该模块由关键词“primitive”和“endprimitive”界定。

UDP 定义模块的语法格式如下：

primitive <元件名称> (<输出端口名>, <输入端口名 1>, <输入端口名 2>, ……,
<输入端口名 n>);

 输出端口类型说明(output);

 输入端口类型说明(input);

 输出端寄存器变量说明(reg);

 元件初始状态说明(initial);

 table

 <table 表项 1>;

 <table 表项 2>;

 ……

 <table 表项 n>;

 endtable

endprimitive

对上述的语法格式，需要说明如下几点：

(1) UDP 定义模块不能出现在其它模块定义之内，UDP 定义模块和其它模块具有相同的语法结构地位，它的定义必须独立于其它模块结构成分。所以 UDP 定义模块必须出现在其它模块定义之外，同时它也可以出现在独立的文件中。比如，例 4-1 所示的结构是正



确的,而例 4-2 所示的结构是错误的。

【例 4-1】UDP 定义模块的正确位置。

```
module MODULE1;           //模块 MODULE1 定义的开始
.....                   //模块 MODULE1 描述体
endmodule                 //模块 MODULE1 定义结束
primitive MY_GATE(out, a, b); //UDP 元件 MY_GATE 定义模块的开始
.....                   //UDP 元件 MY_GATE 定义的描述体
endprimitive              //UDP 元件 MY_GATE 定义模块结束
module MODULE2;           //模块 MODULE2 定义的开始
.....                   //模块 MODULE2 描述体
endmodule                 //模块 MODULE2 定义结束
```

【例 4-2】UDP 定义模块的错误位置。

```
module MODULE1;           //模块 MODULE1 定义的开始
.....                   //模块 MODULE1 描述体
    primitive MY_GATE(out, a, b); //UDP 元件 MY_GATE 定义模块的开始
    .....                   //UDP 元件 MY_GATE 定义的描述体
    endprimitive           //UDP 元件 MY_GATE 定义模块结束
endmodule                 //模块 MODULE1 定义结束
```

在例 4-1 中,UDP 元件 MY_GATE 的定义模块与其它两个模块 MODULE1 和 MODULE2 并列出现,相互独立;而在例 4-2 中,UDP 定义模块却包含在另一个模块 MODULE1 结构之中,这种情况是不允许的。

(2) 在 UDP 定义模块的第一行,关键词“primitive”标志着定义模块的开始;其后出现的“元件名”是为被定义 UDP 元件所取的一个名字;后面出现的端口列表指明了 UDP 元件的输入输出端口,该端口列表由若干个端口组成,其中只能有一个输出端口并且该输出端口必须是端口说明表中的第一项,而 UDP 的输入端口数目可以是一个或多个,一般对时序电路的 UDP 最多允许有 9 个输入端,对组合电路的 UDP 最多允许有 10 个输入端。

必须注意在定义 UDP 元件时所有输入端口和输出端口都只能是 1 位的(标量)。

(3) 输出端口类型说明是对 UDP 元件的输出端口进行说明,这和普通模块定义中的输出端口说明一样是通过关键词“output”来进行的,比如:

```
output out;
```

就说明了端口 out 是一个输出端口。

(4) 输入端口类型说明是对 UDP 元件的输入端口进行说明,这和普通模块定义中的输入端口说明一样是通过关键词“input”来进行的,比如:

```
input a, b;
```

就说明了端口 a 和端口 b 是两个输入端口。

(5) 输出端寄存器变量说明是在对时序逻辑 UDP 元件进行定义时,为了对输出端口寄存器变量类型进行说明而引入的。寄存器变量用于描述时序电路 UDP 中的内部状态。要注意只有在被定义的元件是时序逻辑元件时才能将输出端口定义成寄存器类变量。寄存器



变量说明用关键词“reg”实现, 比如:

```
reg out;
```

就将 UDP 输出端口 out 定义为一个寄存器变量。

(6) 元件初始状态说明也是为进行时序电路 UDP 的定义而引入的, 它的作用是对时序电路上电时刻(即 0 时刻)的初始状态进行定义, 只允许有 0、1、x 三种状态。缺省为 x 态。元件初始状态说明用“initial”过程块来实现, 比如:

```
initial
```

```
out=x;
```

就将时序逻辑 UDP 元件的初始状态定义为“x”。

(7) 由关键词“table”和“endtable”界定的多个 table 表项构成了 UDP 元件定义模块内的一个输入输出真值表。在 table 表项中只允许出现 0、1、x 三种逻辑值, 不允许出现高阻态“z”。组合电路 UDP 定义模块和时序电路 UDP 定义模块内的 table 表项格式是不同的(详见下面几节的讨论)。table 表项构成了 UDP 元件输入、输出以及内部状态(时序逻辑电路情况)间的一张“逻辑真值表”。对不同逻辑功能的 UDP 元件进行定义时的差别将体现在不同的 table 表项描述上。

(8) 关键词“endprimitive”出现在定义模块的末尾, 它标志着 UDP 定义模块的结束。

下面给出对四选一 MUX 进行 UDP 定义描述的一个例子。

【例 4-3】将四选一 MUX 定义成 UDP 元件。

```
primitive mux4_1(out, in1, in2, in3, in4, ctrl1, ctrl2);
```

```
output out;
```

```
input in1, in2, in3, in4, ctrl1, ctrl2;
```

```
table
```

```
// in1 in2 in3 in4 ctrl1 ctrl2 : out 注释行, 用于保证表项中各项的正确
```

```
// 排列顺序
```

```
0 ? ? ? 0 0 : 0
```

```
1 ? ? ? 0 0 : 1
```

```
? 0 ? ? 0 1 : 0
```

```
? 1 ? ? 0 1 : 1
```

```
? ? 0 ? 1 0 : 0
```

```
? ? 1 ? 1 0 : 1
```

```
? ? ? 0 1 1 : 0
```

```
? ? ? 1 1 1 : 1
```

```
endtable
```

```
endprimitive
```

在例 4-3 中, 输入端口中的 in1~in4 为四个输入信号, ctrl1、ctrl2 为选通控制信号: 当这两个值为 00、01、10 和 11 时将分别选通 in1、in2、in3、in4 信号到输出端口 out。例中的 table 表由 8 个表项组成, 它们构成了该四选一 MUX 的逻辑真值表。表项中的?字符代表不必关心相应变量的具体取值, 即它可以是 0、1 或 x。注意不能将控制端口定义成一个二位的输入端口, 即不能采用如下定义方式:



【例 4-4】错例。

```
primitive mux4_l(out, in1, in2, in3, in4, ctrl); //错例
output out;
input in1, in2, in3, in4 ;
input [2:1] ctrl;      //错，在 UDP 定义中的端口只能是一位宽度的标量类型

table
//in1  in2  in3  in4  ctrl  :  out
0      ?    ?    ?    00   :   0
1      ?    ?    ?    00   :   1
?      0    ?    ?    01   :   0
?      1    ?    ?    01   :   1
?      ?    0    ?    10   :   0
?      ?    1    ?    10   :   1
?      ?    ?    0    11   :   0
?      ?    ?    1    11   :   1
endtable
endprimitive
```

4.2 组合逻辑电路 UDP

对组合逻辑电路 UDP 元件的描述相当于直接把电路的逻辑真值表搬到 UDP 描述的 table 表中。组合逻辑电路 UDP 定义时的 table 表项的格式为

<输入 1 逻辑值> <输入 2 逻辑值> …… <输入 n 逻辑值> : <输出逻辑值>

对于上述 table 表项格式，必须注意：

- (1) 各个输入输出逻辑值只能是 0, 1, x 和 ? 中的一个，不能取高阻态“z”。
- (2) 输出端口在 primitive 定义语句的端口表项中列第一的位置，而输出端逻辑值在 table 表项内则是位于最后一项。
- (3) 在表项中要用空格分隔开不同的输入端逻辑值，各个输入端口在 table 表项中的排列顺序必须与它们在“primitive”定义语句中端口表项内的排列顺序保持严格一致。
- (4) table 表项中的输入部分与输出部分之间要用一个冒号隔开。
- (5) 为了明确观察输入输出相互关系和各项排列顺序是否正确，通常在 table 表项的第一栏中插入一条注释语句，比如例 4-3 中的注释行：

```
// in1 in2 in3 in4 ctrl1 ctrl2 : out
```

UDP 元件的逻辑功能是由 UDP 定义模块内关键词“table”和“endtable”之间的 table 表的内容所决定的，table 表相当于构成了 UDP 元件输入输出的一个真值表。在进行模拟时，当模拟器发现 UDP 的某个输入发生改变时，将会自动进入 table 表查找相匹配的 table 表项，再把查到的输出逻辑值赋给输出端口。如果在 UDP 定义模块的 table 表中找不到与当前输入相匹配的 table 表项，则输出端将取不定态“x”。所以，在进行 UDP 元件描述时，应当将尽可能多的输入状态设置到 table 表项中。



以设计一个一位的全加器为例，表 4.1 是它的本位和与进位的逻辑真值表，由于一个 UDP 元件只允许有一个一位的输出，所以要分别定义两个 UDP 元件来实现全加器的本位和与进位。把它们的真值表分别移植到两个 UDP 描述的 table 表中，就得到了例 4-5 所示的全加器的 UDP 描述。

表 4.1 全加器本位和与进位的逻辑真值表

Cin	a	b	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

【例 4-5】全加器的 UDP 描述。

//进位的 UDP 定义

```
primitive carry(cout, cin, a, b);
```

```
output cout;
```

```
input a, b, cin;
```

```
table
```

//cout 对应的 table 表项

```
//cin a b : cout
```

```
0 0 0 : 0;
```

```
0 0 1 : 0;
```

```
0 1 0 : 0;
```

```
0 1 1 : 1;
```

```
1 0 0 : 0;
```

```
1 0 1 : 1;
```

```
1 1 0 : 1;
```

```
1 1 1 : 1;
```

```
endtable
```

```
endprimitive
```

//本位和的 UDP 定义

```
primitive summ(sum, cin, a, b);
```

```
output sum;
```

```
input a, b, cin;
```

```
table
```

//sum 对应的 table 表项

```
//cin a b : sum
```

```
0 0 0 : 0;
```

```
0 0 1 : 1;
```

```
0 1 0 : 1;
```

```
0 1 1 : 0;
```




```
1 0 0 : 1;
1 0 1 : 0;
1 1 0 : 0;
1 1 1 : 1;
```

```
endtable
```

```
endprimitive
```

在例 4-5 的 table 表没有考虑到输入为 x 时的情况, 因此在某个输入为 x 时, 模拟器找不到对应的 table 表项, 输出将为 $\text{cout} = x$, $\text{sum} = x$ 。而实际上对于进位 cout 而言, 只要三个输入项(cin, a 和 b)中有二个值为 1, 则不管第三个的值是什么(0, 1 或 x), cout 一定为 1; 反之, 如果输入中有二个为 0, 则 cout 也一定为 0。为了表示出进位的这一特征, 可以在 cout 对应的 UDP 定义 table 表内用 ? 来替代第三个值对应表项, 得到例 4-6 的定义描述。

【例 4-6】考虑了输入状态“x”后的全加器进位 cout 的 UDP 描述。

//进位的 UDP 定义

```
primitive carry(cout, cin, a, b);
```

```
output cout;
```

```
input a, b, cin;
```

```
table
```

//cout 对应的 table 表项, 较例 4-5 的情况有所精简

```
//cin a b : cout
0 0 ? : 0;
0 ? 0 : 0;
? 0 0 : 0;
1 1 ? : 1;
1 ? 1 : 1;
? 1 1 : 1;
```

```
endtable
```

```
endprimitive
```

又比如当我们要将如图 4.1 所示的组合逻辑电路描述为一个 UDP 元件时, 首先得到该电路的真值表如表 4.2 所示。

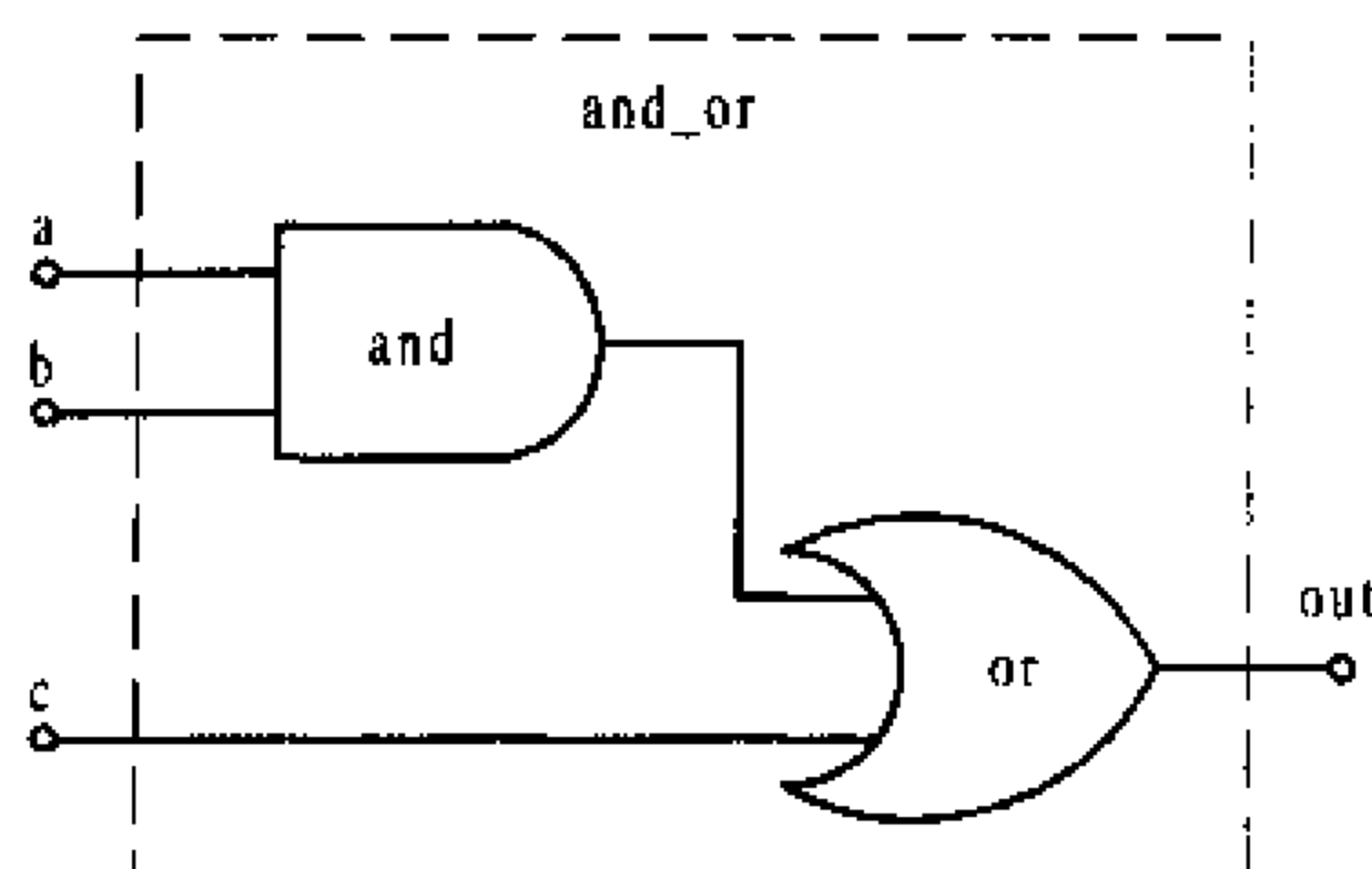


图 4.1 一个组合逻辑电路的例子

表 4.2 图 4.1 所示电路的逻辑真值表

a	b	c	out
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



将上述真值表移植到 UDP 描述的 table 表中, 就得到了例 4-7 所示的 UDP 描述。

【例 4-7】图 4.1 所示电路的 UDP 描述。

```
primitive and_or(out, a, b, c);
output out;
input a, b, c;
    table
    //a    b    c    :    out
    0      0    0    :    0;
    0      0    1    :    1;
    0      1    0    :    0;
    0      1    1    :    1;
    1      0    0    :    0;
    1      0    1    :    1;
    1      1    0    :    1;
    1      1    1    :    1;
    endtable
endprimitive
```

若再考虑到输入取 x 态的情况, 由图 4.1 可知, 当 $abc=0x0, x00$ 时输出为 0; 当 $abc=0x1, 11x, 1x1, x01, x11, xx1$ 时输出为 1; 将这些状态补充到 table 表中就得到例 4-8 的情况。

【例 4-8】图 4.1 所示电路的 UDP 描述。

```
primitive and_or(out, a, b, c);
output out;
input a, b, c;
    table
    //a    b    c    :    out
    0      0    0    :    0;
    0      0    1    :    1;
    0      1    0    :    0;
    0      1    1    :    1;
    1      0    0    :    0;
    1      0    1    :    1;
    1      1    0    :    1;
    1      1    1    :    1;
    //补充的包含有 x 输入态的表项
    0      x    0    :    0;
    x      0    0    :    0;
    0      x    1    :    1;
    1      1    x    :    1;
```




```
1      x      1      :      1;
x      0      1      :      1;
x      1      1      :      1;
x      x      0      :      1;
endtable
endprimitive
```

由上面给出的逻辑图 4.1 和例 4-8 中的 table 表可以看出, 当输入 c 取值为 1 时, 无论 a, b 取值如何, 输出 out 取值都是 1; 当输入 a b 取值为 11 时, 无论 c 取值如何输出, out 取值都是 1; 当输入 b c 取值为 00 时, 无论 a 取值如何, 输出 out 取值都是 0; 当输入 a c 取值为 00 时, 无论 b 取值如何, 输出 out 取值都是 0。这样可以对 table 表进行简化得到例 4-9 所示的 UDP 描述。

【例 4-9】图 4.1 所示电路的 UDP 描述。

```
primitive and_or(out, a, b, c);
output out;
input a, b, c;
table
//a      b      c      :      out
?      ?      1      :      1;
1      1      ?      :      1;
?      0      0      :      0;
0      ?      0      :      0;
endtable
endprimitive
```

4.3 时序逻辑电路 UDP

与组合逻辑电路相比, 时序逻辑电路元件的输出除了与当前的输入状态有关, 还与时序元件本身的内部状态有关。

对时序逻辑电路 UDP 进行定义时的 table 表项格式为:

<输入 1 逻辑值> <输入 2 逻辑值> <输入 n 逻辑值> : <内部状态> : <输出逻辑值>

与组合逻辑电路 UDP 的定义一样, 时序逻辑电路 UDP 定义模块中各个输入端口在 table 表项中的排列顺序必须与它们在“primitive”语句中端口表项内的排列顺序保持严格一致; 在表项中要用空格分隔开不同的输入逻辑值。

时序逻辑电路 UDP 定义时的 table 表项格式与组合逻辑电路 UDP 的表项格式的不同之处在于:

- (1) 表项中多了关于元件内部状态的描述。
- (2) 要用两个冒号分别将输入逻辑值部分与元件内部状态, 元件内部状态与输出逻辑值部分分隔开。



(3) 在构建 table 表项时要把元件内部状态对输出的影响考虑进去。

时序逻辑电路元件可根据触发方式分为电平触发与边沿触发两类。它们对应的 table 表项格式虽然都是上面给出的格式，但是 table 表项中输入输出信号可取的状态在两种电路类别下是不同的。这一点将分别在 4.3.2 小节和 4.3.3 小节中加以说明。

4.3.1 初始化状态寄存器

因为时序逻辑电路元件有自己的内部状态，所以必须有一个寄存器变量来保持其内部状态。在时序逻辑电路 UDP 的定义模块中必须将输出端口定义为寄存器类型，所以如 4.1 节所讲述的，时序逻辑电路 UDP 的定义模块中必须增加“输出端寄存器变量说明”这一项来将输出端口说明成寄存器变量类型。

在有些情况下必须对上电时刻(即 0 时刻)元件的初始状态值加以指定。所以在时序逻辑电路 UDP 的定义模块中还可以增加“元件初始状态说明”这一项通过 initial 过程块来指定元件上电时刻的初始状态(0、1 或 x)。如果该说明项(initial 语句)缺省，则元件的初始状态被默认为不定态“x”。

4.3.2 电平触发时序电路 UDP

电平触发时序电路 UDP 的特点是其内部状态改变是由某一输入信号电平触发的。比如一个电平触发的锁存器，其真值表示于表 4.3 中。当它的时钟输入信号 clk 为逻辑 0 时，锁存器的状态及输出时刻保持与输入信号 d_in 相同；当时钟输入信号为逻辑 1 时，锁存器进入锁存状态，其内部状态及输出保持不变。

例 4-10 给出了电平触发锁存器的 UDP 描述。

表 4.3 电平触发锁存器的逻辑真值表

d_in	clk	current_state	next state (q)
0	0	0 或 1 或 x	0
1	0	0 或 1 或 x	1
0 或 1 或 x	1	1	1
0 或 1 或 x	1	0	0

【例 4-10】电平触发锁存器的 UDP 描述。

```
primitive latch (q, clk, d_in);
output q;
reg q;
input clk, d_in;
initial q = 1'b0; // 内部状态和输出被初始化为 0 状态

table
//clk  d_in  :  current_state  :  next_state
0      1    :      ?           :  1;
0      0    :      ?           :  0;
1      ?    :      ?           :  -; // “-”表示输出状态不变
```




```
endtable  
endprimitive
```

例 4-10 中最后一个 table 项内引入了一个新的标记“—”，它的含义就是 UDP 的内部状态(也就是输出端状态)将保持原有状态不变(No-change)。表项中的“?”仍代表“任意态”。例 4-10 中第一个 table 表项对应于真值表中第二行的情况，第二个 table 表项对应于真值表中第一行的情况，第三个 table 表项对应于真值表中第三、四行的情况，此时 clk=1，不管输入数据 d_in 的值是多少，锁存器都将保持原来的状态不变，处于锁存状态。

同时需要注意到，如同 4.3.1 小节中所述，例 4-10 中的定义中增加了对说明输出 q 为寄存器类变量的 reg 语句；又增加了指定 UDP 初始状态为逻辑 0 值的 initial 语句。

4.3.3 边沿触发时序电路 UDP

边沿触发时序电路 UDP 的特点是其内部状态的改变是由输入时钟的有效边沿(上升沿或下降沿)触发的，而与时钟信号稳定时的输入状况无关。所以对边沿触发时序逻辑元件的描述中就需要对输入信号的变化和变化方式加以考虑。模拟器只有在检测到输入信号发生跳变时才会搜索 table 表得到新的内部状态值和输出逻辑值。这样 table 表项中的输入逻辑值部分需要列出输入逻辑值的变化情况即跳变情况。在 Verilog 中用一对括号括起来的两个数字“(vw)”的形式来表示从一个状态到另一个状态的转化，其中 v、w 可以是 0、1、x、? 之一，因而(01)代表的就是由 0 往 1 的上升沿正跳变，(10)则代表了下降沿负跳变，(1x)代表了由 1 往不定态的跳变，(??)代表在 0、1、x 三状态间的任意跳变……。必须注意 Verilog 规定在每一条 table 表项中最多只允许一个输入信号处于跳变状态，例如：

```
//  clk      data      :  state      :  next_state  
   (01)    (10)      :    0          :    0      ;
```

表示的这条 table 表项就是不允许的，因为其中有两个输入信号 clk 和 data 同时发生了跳变。

以设计一个上升沿 T 触发器为例，其真值表示于表 4.4 中。

表 4.4 上升沿 T 触发器的逻辑真值表

clk	t_in	current_state	next state (q)
(01)	0	0	0
(01)	0	1	1
(01)	1	0	1
(01)	1	1	0
(10)	0	0	0
(10)	0	1	1
(10)	1	0	0
(10)	1	1	1
0 或 1 或 x	(??)	0	0
0 或 1 或 x	(??)	1	1



真值表的第 1~4 行说明了在时钟信号 `clk` 发生上跳变时, 若输入 `t_in` 为 0 则触发器状态保持不变, 而若 `clk` 上跳变时 `t_in` 为 1 则触发器状态翻转; 真值表的第 5~8 行说明了在时钟信号 `clk` 发生下跳变时, 不管输入 `t_in` 为何值, 触发器状态保持不变; 真值表的第 9、10 行说明在时钟信号 `clk` 处于稳定值但 `t_in` 发生跳变时, 触发器状态保持不变; 将真值表中的各种情况加以简化并移植到 UDP 描述的 `table` 表中可以得到例 4-11 所示的上升沿 T 触发器的 UDP 描述。

【例 4-11】上升沿 T 触发器的 UDP 描述。

```
primitive t_trigger(q, clk, t_in);
output q;
reg q;
input t_in, clk;

table
//   clk   t_in   :   state   :   q
(01)  0     :   ?     :   - ;
(01)  1     :   0     :   1 ;
(01)  1     :   1     :   0 ;
(10)  ?     :   ?     :   - ;
(0x)  0     :   ?     :   - ;
(1x)  0     :   ?     :   - ;
?     (??)   :   ?     :   - ;

endtable
endprimitive
```

上例中 `table` 表中的第 1 项与真值表的第 1、2 两行对应; `table` 表中的第 2、3 项分别与真值表的第 3、4 两行对应; `table` 表中的第 4 项与真值表的第 5~8 行对应; `table` 表中的第 5、6 项内容的目的是为了把尽可能的定态输出情况考虑进去。它们在本例中的意思是, 当时钟从 0 状态变化到不定态 `x` 时或从 1 状态变化到不定态 `x` 时, 如果输入 `t_in` 为 0, 则 UDP 元件状态及输出逻辑值保持原来的取值不变; `table` 表中的最后一项与真值表的第 9、10 两行对应, 注意这项是描述一个 T 触发器所必需的, 因为只要在 `table` 表中出现了一个含有输入逻辑值跳变的表项, 模拟器就会对任何一个输入信号的跳变作出反应, 进入 `table` 表查找相应的表项, 如果上例中没有这条表项来对数据输入端 `t_in` 发生跳变时元件输出端对应的逻辑取值进行说明, 模拟器将会因为找不到对应的说明项而输出一个不定态 `x`。所以在进行边沿触发时序电路 UDP 描述时应该对每一个输入端发生的任何跳变情况加以考虑, 并将这些跳变状态以及对应的输出端逻辑状态作为一个 `table` 表项添加到 `table` 表中去。

为了简化 UDP 元件定义模块中 `table` 表项的描述, 以及为了加强 `table` 表的可读性, Verilog 在对 UDP 进行定义时引入了一些缩记符号来描述 `table` 表项, 这些符号列于表 4.5 之中, 利用这些缩记符号, 例 4-11 可以被改写成例 4-12 的形式。



表 4.5 UDP 定义时的缩记符号

table 表项缩记符号	定 义	说 明
0	逻辑 0	能用来描述输入、输出信号
1	逻辑 1	能用来描述输入、输出信号
x 或 X	为不定态	能用来描述输入、输出信号
-	输出状态保持不变	只能用来描述时序逻辑电路 UDP 的输出状态
?	0 或 1 或 x(任意态)	不能用来对输出进行描述
b 或 B	0 或 1	不能用来对输出进行描述
(vw)	输入从逻辑值 v 变化到逻辑值 w	不能用来对输出进行描述, 其中的 v, w 是 0, 1, x, ? 中的任意一个
r 或 R	等价于(01), 表示输入的上跳变沿	不能用来对输出进行描述
f 或 F	等价于(10), 表示输入的下跳变沿	不能用来对输出进行描述
p 或 P	(01)、(0x)、(x1)中的任一个, 表示输入端的包含 x 态的正跳变	不能用来对输出进行描述
n 或 N	(10)、(1x)或(x0)中的任一个, 表示输入端的包含 x 态的负跳变	不能用来对输出进行描述
*	等价于(??), 表示输入端的任意变化	不能用来对输出进行描述

【例 4-12】使用缩记符号后的上升沿 T 触发器的 UDP 元件定义模块。

```
primitive t_trigger(q, clk, t_in);
output q;
reg q;
input t_in, clk;

table
//   clk   t_in   :   state   :   q
//   r     0     :   ?       :   - ;
//   r     1     :   0       :   1 ;
//   r     1     :   1       :   0 ;
//   f     ?     :   ?       :   - ;
//   (??)  0     :   ?       :   - ;
//   ?     (??)  :   ?       :   - ;
endtable
endprimitive
```

4.3.4 电平触发和边沿触发混合的时序电路 UDP

有些时序逻辑电路中的状态改变要既受电平触发、又受边沿触发。这种电路称为电平触发和边沿触发混合的时序电路。例如一个带有异步清零输入和异步置位输入的触发器, 清零和置位操作时要求电平触发, 而触发器本身却是边沿触发的。在这种混合触发形式时



序电路类型的 UDP 定义模块中, 稳定的输入状态和输入信号的跳变状况都将对元件的输出和元件内部状态的改变产生影响。所以在进行 UDP 定义描述时需要同时对输入的稳定取值和变化状态加以考虑。在 table 表项中要同时包括有稳定输入状态和输入跳变状态。例 4-13 中所描述的就是一个具有异步清零功能的下降沿 D 触发器。

【例 4-13】具有异步清零功能的下降沿 D 触发器的 UDP 定义模块。

```
primitive dff (q, d, clk, clr);
output q;
input clk, clr, d;

reg q;                                //定义输出 q 为寄存器变量类型
initial q = 0;                         //上电时 q 值初始化为 0

table
//d   clk   clr :   state :   q
/*清零情况*/
?   ?   0   :   ?   :   0;           //clr=0, 执行触发器清零
/*正常触发下的时序逻辑*/
0   r   1   :   ?   :   0;           //时钟上升沿时输入 d=0, 触发器置 0
1   r   1   :   ?   :   1;           //时钟上升沿时输入 d=1, 触发器置 1
?   n   1   :   ?   :   -;           //时钟负跳变时, 触发器状态保持不变
/*输入 d 发生跳变情况下的逻辑*/
*   ?   1   :   ?   :   -;           //clk 处于定态时, 忽略 d 跳变, 触发器状态保持不变
/*输入 clr 发生正跳变情况下的逻辑*/
?   ?   p   :   ?   :   -;           //clr 处于定态时, 忽略 clr 正跳变,
//           触发器状态保持不变
/*减少输出不定态*/
0   (0X) 1   :   0   :   0;
1   (0X) 1   :   1   :   1;
endtable
endprimitive
```

在上面的例子中, 电平触发方式对应的 table 表项排在边沿触发方式对应的 table 表项之前, 这是因为清零操作要比正常的同步触发逻辑操作具有更高的优先级, 将电平触发清零操作对应的表项放在 table 表的最前面, 依据查表顺序, 可以保证模拟器在进行仿真运行时会对异步清零信号作出响应。

例 4-13 第 1 条 table 表项描述的就是当异步清零输入有效时(低电平有效)的处理方式。

例 4-13 第 2~4 条 table 表项描述是同步 D 触发器正常的触发操作, 其中第 2 条和第 3 条表项说明: 时钟发生上升沿跳变后, 元件的输出状态将与发生跳变时输入 d 的逻辑值相同; 而第 4 条表项说明输入时钟的负跳变不会对元件输出端状态产生影响;

例 4-13 第 5 条 table 表项说明当时钟输入处于稳定态时, d 输入端信号的跳变不会影响触发器的输出和内部状态。



例 4-13 第 6 条 table 表项说明当时钟输入处于稳定态时, clr 输入端的正跳变不会影响触发器的输出和内部状态。

例 4-13 描述中的最后两个 table 表项是为了减少输出的不定态而设计的。



本章对模块在进行行为描述时所具有的基本构成框架进行描述；下一章将对行为描述方式使用的赋值语句和时间控制方式进行讨论；第 7 章将对用于行为描述方式的高级程序语句以及任务和函数进行讨论。

5.1 行为描述的结构

函数定义(function)(可选)

endmodule

其中:

端口列表中可以有单个或多个输入、输出或双向端口; 这些端口类型要在“模块端口说明”部分进行类型说明。

参数定义用关键词 `parameter` 实现, 如果模块定义了参数, 那么该模块的每个实例可以对该参数进行重新定义, 使得该参数值对每一个具体实例是唯一的。

数据类型说明用来对模块中用到的各类变量类型进行说明, 如果某个变量没有进行数据类型说明, 则它的类型缺省为连线类型(wire)。

过程块是由过程语句 `initial` 或 `always` 开头的一个语句块, 根据这两个不同的关键词, 过程块可以被分为“`initial` 过程块”和“`always` 过程块”两种类型。过程块中包含一条或多条行为语句, 过程块是行为描述的主要组成部分。

连续赋值语句是由关键词 `assign` 来标识的一种赋值语句, 它只能对连线型类型(`net` 类型)变量进行驱动, 它和语句块一样也是一种行为描述语句。在下一章讨论过程赋值语句时我们将对连续赋值语句一并进行讨论。

任务定义和函数定义部分都是可选的, 它们引入的目的是为了描述模块中被多次执行的部分以及为了增强代码的易读性。

上面列出的各个模块组成项可以以任意次序出现, 但是端口说明和数据类型说明必须出现在这些端口和数据被引用之前。

与第 2 章中描述的模块的一般格式定义相比, 我们可以看出: 在进行行为描述时, 模块内不能出现模块实例和原语实例语句(包括用户自定义原语实例), 模块内只能存在过程块(`initial` 过程块和 `always` 过程块)、连续赋值语句、任务定义(task)和函数定义(function)这四种结构成分。对电路进行行为描述主要是通过过程块和连续赋值语句来进行的, 任务和函数的使用也需要在过程块或连续赋值语句中进行调用。本章以及第 6、7 章将对这几种结构成分进行介绍。

一个行为描述模块中可以同时包含多个过程块和多个连续赋值语句, 这些组成成分将以并行方式各自独立地执行: 行为描述模块也可以只包含过程块或者只包含连续赋值语句。

5.1.1 过程块

每个过程块是由“过程语句(`initial` 或 `always`)”和语句块所组成的, 而语句块主要是由过程性赋值语句(包括“过程赋值语句”和“过程连续赋值语句”)和高级程序语句(包括“条件分支语句”和“循环控制语句”)这两种行为语句构成的。过程块具有如下特点:

(1) 在行为描述模块中出现的每个过程块(`always` 过程块或 `initial` 过程块)都代表一个独立的进程。

(2) 在进行仿真时, 所有 `initial` 过程块和 `always` 过程块的执行都是从 0 时刻开始并行地进行的。

(3) 每一个过程块内部的多条语句的执行方式可以是顺序执行的(当块定义语句为 `begin-end` 时的情况), 也可以是并行执行的(块定义语句为 `fork-join` 时的情况)。

(4) `always` 过程块和 `initial` 过程块都是不能嵌套使用的。比如下例 5-1 就是错误的。



【例 5-1】错例：嵌套的过程块。

```
module nested_procedural_block(a,b,c):  
input a,b;  
output c;  
reg a,b,c;  
initial      // initial 过程块 1  
begin  
    c=a;  
        initial      //嵌套在 initial 过程块 1 内部的 initial 过程块 2  
        begin  
            c=b;  
        end  
    end  
endmodule
```

下面两小节将分别对 initial 和 always 这两类过程块进行介绍。

5.1.2 initial 过程块

initial 过程块是由过程语句“initial”和语句块组成的，它的格式为：

```
initial  
    语句块  
而“语句块”的格式为：  
<块定义语句 1> : <块名>  
块内局部变量说明：  
时间控制 1    行为语句 1;  
.....  
时间控制 n    行为语句 n;  
<块定义语句 2>
```

在如上所示的格式中：

(1) 过程语句关键词 initial 表明了该过程块是一个“initial 过程块”。

(2) <块定义语句 1>和<块定义语句 2>构成了一组块定义语句，它们可以是“begin, end”语句组或“fork, join”语句组。这两条块定义语句将它们之间的多条行为语句组合在一起，使之构成一个语句块，并使其在格式上更像一条语句。当语句块中只包含一条行为语句并且不需要定义块名和块内局部变量时，这两条块定义语句可以缺省。

(3) <块名>为可选项，块名可以为语句块创建一个局部作用域。定义了块名的过程块称为“有名块”，在有名块中可以定义局部变量，有名块内部语句的执行可以被 disable 语句中断。

(4) 块内局部变量说明也是可选项，只有在有名块中才能定义局部变量，并且块内局部变量只能是寄存器类数据类型。

(5) 时间控制用来对过程块内各条语句的执行时间进行控制。它可以是任何一种时间

控制方式。关于时间控制我们将在第6章进行详细讨论。

(6) 行为语句可以是如下语句中的一种:

- 过程赋值语句(阻塞型或非阻塞型过程赋值语句)。
- 过程连续赋值语句(assign/deassign 或 force/release 语句组)。
- if 条件分支语句。
- case 条件分支语句。
- 循环控制语句(forever, repeat, while, for 循环控制语句)。
- wait 等待语句。
- disable 中断语句。
- 事件触发语句(event_trigger)。
- 任务调用语句(用户自定义的任务或系统任务)

我们可以将上面的这些行为语句分成三类: 过程性赋值语句、高级程序语句和时间控制语句。其中过程性赋值语句和高级程序语句是过程块的基本组成部分。要注意上面列出的行为语句只能对寄存器类数据变量进行赋值(除 force/release 过程连续赋值语句以外)。

在进行仿真时, 一个 initial 过程块从模拟 0 时刻开始执行, 它在仿真过程中只执行一次, 在执行完一次后该 initial 过程块就被挂起, 不再执行。如果一个模块中存在多个 initial 过程块, 则每个 initial 过程块都是同时从 0 时刻开始并行执行的。initial 过程块内的多条行为语句可以是顺序执行(语句块是 begin-end 串行块时)的, 也可以是并行执行(语句块是 fork-join 并行块时)的。

initial 过程块的使用主要是面向功能模拟的, 它通常不具有可综合性。initial 过程块通常用来描述测试模块的初始化、监视、波形生成等功能行为; 而在对硬件功能模块的行为描述中, initial 过程块常常用来对只需执行一次的进程进行描述, 例如它可以用来为寄存器变量赋初值。

下面给出 initial 过程块的几个例子。

【例 5-2】 initial 过程块用于对变量和存储器进行初始化。

```
module register_initialize (memory);
  inout areg;
  inout memory;
  parameter SIZE=1024;
  parameter BYTESIZE=8;
  reg[BYTESIZE-1:0] memory [SIZE-1:0] ;
  initial                                //initial 过程语句
  begin :SEQ_BLK_A                      //块定义语句, SEQ_BLK_A 为块名
    integer:index ;                    //计数变量, 为块中的一个局部变量
    for(index=0; index<SIZE; index=index+1)
      memory[index]=0;                //对存储器 memory 进行初始化
      areg=0;                          //对寄存器 areg 进行初始化
  end                                    //块结束标志
endmodule
```




这个例子的作用是用 `initial` 语句在仿真的开始时刻对寄存器变量和存储器进行赋零值操作，从而实现各个变量的初始化。例 5-2 中的串行块被定义了一个块名“SEQ_BLK_A”，这是由于为了循环计数需要定义局部变量 `index`，而只有当串行语句块为有名块时才能在块中定义局部变量，所以上例中的块名定义不能缺省。`initial` 过程块中包含了一个 `begin-end` 串行语句块。所以对存储器的赋值操作将先于对寄存器的赋值操作进行。这个 `initial` 过程块在 0 ns 时开始执行，并且由于串行块中 `for` 循环控制语句和过程赋值语句都没有指定时延，所以对存储器和寄存器的赋值操作都是在 0 时刻开始和完成的。在完成对寄存器的赋值操作后，`initial` 过程块中所有的语句都已执行完毕，所以 `initial` 过程块将结束执行，并且将被永远挂起不再重新执行。

【例 5-3】`initial` 过程块用于为电路仿真生成激励波形。

```
initial
begin
    inputs='b000000;           // 初始时刻为 0
    #10  inputs='b011001;       // 10 个时间单位后取值改变
    #20  inputs='b011011;       // 20 个时间单位后取值改变
    #5   inputs='b011000;       // 5 个时间单位后取值改变
    #10  inputs='b001000;       // 20 个时间单位后取值改变
end
```

在例 5-3 中，我们用 `initial` 过程块来生成特定的电路仿真激励信号，和例 5-2 中的情况一样，`initial` 语句包含了一个串行块。但是在此例中 `begin-end` 串行块内的语句是带有时延的，假定 `initial` 语句从 $t=0$ 时刻开始执行，那么首先执行第一条赋值语句，在 $t=0$ 时刻 `input` 的值变为 'b000000；由于第二条赋值语句带有延时控制，所以第二条语句的执行是在第一条语句执行结束 10 个单位时间后（即在 $t=10$ 时刻）开始的，在 $t=10$ 时刻 `input` 的值变为 'b011001；然后再等待 20 个单位时间后开始第三条赋值语句的执行，依次类推可以知道 `input` 取值的变化情况将如表 5.1 所示。当第五条赋值语句执行完毕后（ $t=45$ 时刻），`initial` 过程块中所有语句已全部执行完，仿真进程跳出 `initial` 过程块，`initial` 语句被永远挂起。

表 5.1 例 5-3 中 `input` 变量的变化情况

时间	输入值
0	<code>input='b000000</code>
10	<code>input='b011001</code>
30	<code>input='b011011</code>
35	<code>input='b011000</code>
45	<code>input='b001000</code>

如上例所示，`initial` 语句块主要用于初始化和波形生成。

5.1.3 `always` 过程块

`always` 过程块是由“`always`”过程语句和语句块组成的，它的格式为：

`always @ (敏感事件列表)`

语句块

其中“语句块”的格式为:

<块定义语句 1> : <块名>

块内局部变量说明:

时间控制 1 行为语句 1;

.....

时间控制 n 行为语句 n;

<块定义语句 2>

(1) 关键词 **always** 表明了该过程块是一个“always 过程块”。

(2) @(敏感事件列表) 是可选项, 带有敏感事件列表的语句块被称为“由事件控制的语句块”, 它的执行要受敏感事件的控制。(敏感事件列表)的形式为:

(event-expression [or event-expression*])

敏感事件列表是由一个或多个事件表达式(event-expression)构成的, 当存在多个事件表达式时要用 **or** 将它们组合起来。**or** 用来说明: 只要发生了敏感事件列表所列出的多个事件中的任何一个, 就将启动后面语句块的执行。敏感事件列表实际上代表了一个事件控制类型的时间控制。我们将在第 6 章进一步讨论时间控制、事件控制和敏感事件列表。

(3) **always** 过程块中关于 <块定义语句> 的规定和 **initial** 过程块中的情况一样, 它们可以是“**begin-end**”语句组或“**fork-join**”语句组, 由这两组语句组合成的语句块分别称为串行块和并行块。当过程块中只包含一条语句且不需定义块名与块内局部变量时, 块定义语句可以缺省。

(4) <块名>为可选项, 对它的规定与 **initial** 过程块中的情况一样。定义了块名的过程块称为有名块。

(5) 块内局部变量是可选项, 对它的规定与 **initial** 过程块中的情况一样。只有在有名块中才能定义局部变量, 块内局部变量只能是寄存器类数据类型;

(6) 时间控制用来对过程块内各条语句的执行时间进行控制。其作用与 **initial** 过程块中的“时间控制”部分的作用是相同的。

(7) 对“行为语句”的规定与上一小节中 **initial** 过程块中的情况相同。

从上面的格式定义可以看出, **always** 过程块和 **initial** 过程块在格式上的区别主要在于: **always** 过程语句后面可以有一个敏感事件列表, 该敏感事件列表的作用是用来激活 **always** 过程语句的执行, 而 **initial** 过程语句的后面则不允许有敏感事件列表。

在进行仿真时, 和 **initial** 过程块一样, **always** 过程块是从模拟 0 时刻开始执行的。但是 **always** 语句在仿真过程中是不断重复执行的, 而不像 **initial** 过程块中的语句那样在执行一次后就被挂起, 不会再次被执行。并且 **always** 过程块内各条语句的真正执行必须由敏感事件列表中列出的事件进行触发才能启动。如果 **always** 过程块中的敏感事件列表缺省, 则认为触发条件始终被满足, **always** 过程块将无条件地循环执行下去, 直到遇到 **\$finish** 或 **\$stop** 系统任务为止。

敏感事件列表由一个或多个“事件表达式(event-expression)”构成, 事件表达式说明了启动块内语句执行时的触发条件。当存在多个事件表达式时要用关键词 **or** 将多个触发条件组合起来。**Verilog** 规定: 只要这些事件表达式所代表的多个触发条件中有一个成立, 就



启动块内语句的执行。例如，在语句：

```
always @ (a or b or c);  
begin  
.....  
end
```

中，always 过程块的多个事件表达式分别是信号 a、b 和 c，它们代表的触发条件分别是信号 a、b、c 发生变化，这三个事件表达式之间以关键词 or 分隔，这时只要其中一个触发条件被满足，也就是只要信号 a、b、c 中有一个的值发生变化，begin-end 语句块的执行就被触发。而如果信号 a、b、c 的值都没有发生变化，则任何一个触发条件都没有被满足，begin-end 语句块中的语句不被执行；此时由于 always 过程语句的重复执行的特点，仿真进程将不断地对触发条件进行监测，等待 a、b、c 信号发生变化。

always 过程块的使用主要是对硬件功能模块的行为进行描述，它也可以在测试模块中用来对时钟进行描述。利用 always 过程块可以实现锁存器和触发器，它也可以用来实现组合逻辑。

在用 always 过程块实现组合逻辑时要注意将所有的输入信号都列入敏感事件列表中，而在用 always 过程块实现时序逻辑时却不一定要将所有的输入信号都列入敏感事件列表。敏感事件列表中未包含所有输入信号的情况称为“不完整事件说明(Incomplete Event Specification)”。“不完整事件说明”情况在仿真时可能会引起模拟器的误解。我们可以通过如下两个例子来说明这个问题。

【例 5-4】敏感事件列表中未包含所有输入信号的情况。

```
module three_input_and(f,a,b,c);  
output f;  
input a,b,c;  
reg f;  
always @(a or b)  
begin  
    f = a & b & c;  
end  
endmodule
```

例 5-4 描述了一个 3 输入与门，由于输入 c 没有出现在敏感事件列表中，所以仿真时当输入信号 c 发生变化时不会重新计算 f 值。所以仿真得到的功能并不是一个 3 输入与门的功能行为，而正确实现 3 输入与门应该采用例 5-5 所示的描述方式。

【例 5-5】敏感事件列表中包含了所有的输入信号。

```
module three_input_and(f,a,b,c);  
output f;  
input a,b,c;  
reg f;  
always @(a or b or c)  
begin
```




```
f = a & b & c;  
end  
endmodule
```

在使用 `always` 过程块时还要注意避免引起仿真死锁状态的发生。因为 `always` 语句有重复循环执行的特性, 所以当敏感事件列表缺省时, 语句块将一直循环执行下去, 这就有可能在仿真时产生仿真死锁情况, 如例 5-6 给出的语句所示。

【例 5-6】 不恰当使用 `always` 语句而产生仿真死锁情况的例子。

```
always  
begin  
    clk=~clk;  
end
```

上例中的 `always` 语句不带有敏感事件列表, 所以 `begin-end` 语句块的执行是无条件循环进行的, 这样当仿真进程进行到该 `always` 过程块后(假设为 t_0 时刻), 将开始重复执行块中语句“`clk=~clk;`”, 由于该语句没有时间控制部分, 该语句每次执行都不需要时延, 这样仿真进程将停留在 t_0 时刻不断地循环执行这条语句。这时, 仿真就不能往下一时刻继续进行, 这就进入了一种仿真死锁状态。

如果例 5-6 中的语句能加上延时控制, 则该 `always` 语句将变为一条非常有用的描述语句(见例 5-7)。

【例 5-7】 对例 5-6 的改进。

```
always  
begin  
    #50 clk=~clk;  
end
```

例 5-7 中的赋值语句带有 50 个单位时间的时延, 这样每一次该赋值语句被执行的时刻是不同的, 彼此间隔 50 个单位时间。所以循环就不是停留在 t_0 时刻, 而是往前推进的。例 5-7 可以生成一个周期为 100 的无限延续的信号波形。

5.2 语 句 块

语句块就是在“`initial` 过程块”或“`always` 过程块”中位于过程语句(`initial` 语句或 `always` 语句)后面的, 由“块定义语句 1”和“块定义语句 2”所界定的一组行为语句。

“块定义语句 1”和“块定义语句 2”构成了一对“块定义语句”。

块定义语句分成两种:

- 一种是“`begin-end`”语句组, 它们用来组合需要顺序执行的语句, 由“`begin-end`”语句组标识的语句块称为“串行块”。

- 另一种是“`fork-join`”语句组, 它们用来组合需要并行执行的语句。由“`fork-join`”语句组标识的语句块称为“并行块”。

“块定义语句 1”(`begin` 或 `fork` 语句)标识了语句块的开始, “块定义语句 2”(`end` 或 `join` 语句)则标识着语句块的结束。当语句块内只包含一条语句时块定义语句可以缺省。



如果在定义语句块的同时还引入了一个块名, 则该语句块可以由这个块名惟一标识, 该语句块被称为一个“有名块”。在有名块内可以定义内部寄存器变量, 并且可以用“disable”中断语句来中断有名块内语句的执行。同时语句块的块名还提供了惟一标识寄存器的一种方法。

语句块只能出现在行为描述模块中, 但它不必非得出现在与过程语句 initial 或 always 的结合中, 后面我们可以看到: 在高级程序语句中以及任务和函数中都可以出现语句块结构。

下面分别介绍串行块和并行块。

5.2.1 串行块(begin-end 块)

串行块的块定义语句为“begin, end”, 串行块中的各条语句按串行方式顺序执行。串行块的格式如下:

```
begin : <块名>
    块内局部变量说明
    时间控制1  行为语句 1
    .....
    时间控制n  行为语句 n
end
```

其中, “块内局部变量说明”可以是 reg 型变量声明语句、integer 型变量声明语句及 real 型变量声明语句。

串行块执行时的特点如下:

(1) 串行块内的各条语句是按它们在块内出现的次序逐条顺序执行的, 当前面一条语句执行完毕后下一条语句才能开始执行。

(2) 块中每条语句中的延时控制都是相对于前一条语句结束时刻的延时控制。

(3) 在进行仿真时, 当遇到串行块时, 块中第一条语句随即就开始执行; 当串行块中最后一条语句执行完毕时, 程序流程控制就跳出串行块, 串行块结束执行。整个串行块的执行时间等于其内部各条语句执行时间的总和。

下面给出串行语句块的几个例子。

【例 5-8】 一个时钟上升沿控制的三位移位寄存器。

```
module Three_bit_shift_register (clk, d_in, d_out);
    input clk;
    input d_in;
    output d_out;
    reg reg_a, reg_b, d_out;
    always @(posedge clk)
    begin
        d_out=reg_b;
        reg_b=reg_a;
        reg_a=d_in;
    end
endmodule
```



```
end
endmodule
```

在例 5-8 的代码内包含了一个 begin-end 串行块, 它属于 always 过程块的一部分。时钟的上升沿跳变将触发串行块的执行: 第一条赋值语句首先执行, 将 reg_b 的值赋给寄存器 d_out; 执行完第一条语句后, 程序流程控制转到第二条语句, 开始执行第二条语句, 将 reg_a 的值赋给 reg_b; 然后执行第三条语句, 将输入 d_in 值赋给 reg_a; 在执行完第三条语句(串行块中的最后一条语句)后, 程序流程跳出串行块, 串行块结束执行。

下面是一个块内语句带有延时的情况。

【例 5-9】由带有延时控制的语句组成的串行块, 用于产生时序波形。

```
module sequential_signal_gen( d_out ):
output d_out;
reg d_out;
initial
begin
    d_out=0;
    #1 d_out=1;
    #2 d_out=0;
    #3 d_out=1;
    #4 d_out=0;
end
endmodule
```

例 5-9 中的 begin-end 串行块属于 initial 过程块的一部分。块中语句执行过程如下:

(1) 在仿真开始后($t=0$ 时刻)串行块就开始执行, 首先执行第一条语句, 第一条语句执行完时也就是在 0 时刻 d_out 值变为 0

(2) 然后开始执行第二条语句, 由于第二条赋值语句带有 1 个单位时间的延时, 所以直到第一条语句结束 1 个单位时间后($t=1$ 时刻), 第二条赋值语句才开始执行, d_out 的值变为 1。

(3) 同样, 第三条赋值语句是在第二条语句结束 2 个单位时间后($t=3$ 时刻)开始执行, d_out 的值变为 0。

(4) 第四条赋值语句是在第三条语句结束 3 个单位时间后($t=6$ 时刻)开始执行, d_out 的值变为 1。

(5) 第五条赋值语句是在第四条语句结束 4 个单位时间后($t=10$ 时刻)开始执行, d_out 的值变为 0。

(6) 第五条赋值语句结束后, 串行块结束执行。

上例中串行语句块执行过程所产生的波形如图 5.1 所示。

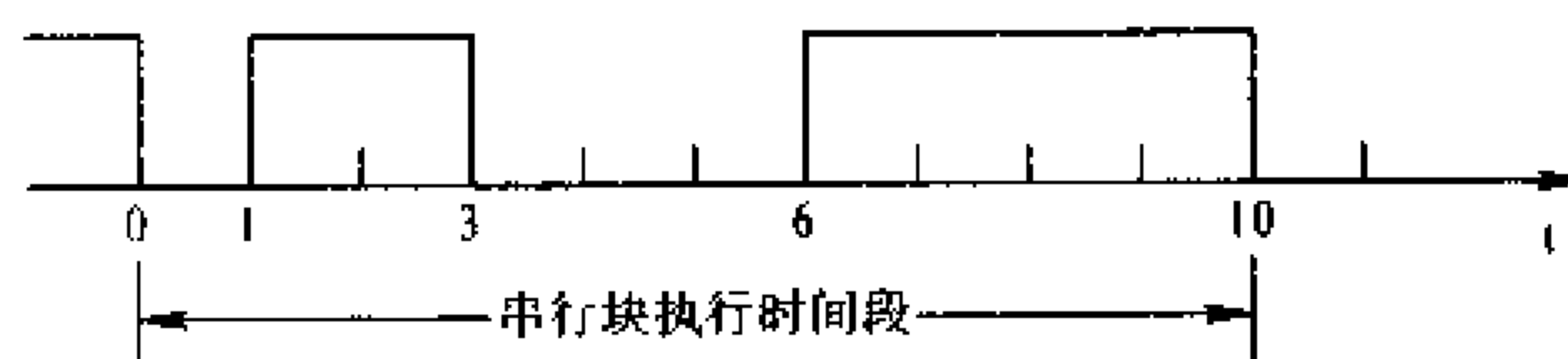


图 5.1 例 5-9 的执行产生的波形



5.2.2 并行块(fork-join 块)

并行块的块定义语句为“fork, join”，并行块中的多条语句是并行执行的。并行块的格式如下：

```
fork : <块名>
      块内局部变量说明
      时间控制1  行为语句 1;
      .....
      时间控制n  行为语句 n;
join
```

其中，“块内局部变量说明”可以是 reg 型变量声明语句、integer 型变量声明语句、real 型变量声明语句、time 型变量声明语句及事件(event)声明语句。

并行块执行时的特点如下：

(1) 并行块内各条语句是同时并行地执行的，也就是说当程序流程控制进入并行块后，块内各条语句都各自独立地同时开始执行。各条语句的起始执行时间都等于程序流程控制进入该并行块的时间。

(2) 块内各条语句中指定的延时控制都是相对于程序流程控制进入并行块的时刻的延时，也就是相对于并行块开始执行时刻的延时。

(3) 当并行块内所有的语句都已经执行完毕后，也就是当执行时间最长的那一条块内语句结束执行后，程序流程控制才跳出并行块，结束并行块的执行。整个并行块的执行时间等于执行时间最长的那条语句所需的执行时间。

下面给出并行块的几个例子。

【例 5-10】用并行块来产生时序波形。

```
module sequential_signal_gen( d_out );
output d_out;
reg d_out;
initial
fork
    d_out=0;
    #1 d_out=1;
    #2 d_out=0;
    #3 d_out=1;
    #4 d_out=0;
join
endmodule
```

从例 5-10 与例 5-9 的比较中可以看出，它们两者惟一的差别就是把块定义语句由原来的串行块定义语句“begin-end”改为这里的并行块定义语句“fork-join”。但这一改变将导致语句块的执行过程与执行结果都发生变化。

在仿真开始后(t=0 时刻)initial 过程语句随即被执行，并行块随后也开始执行(t=0 时



刻), 块内所有语句的执行被同时启动:

由于第一条赋值语句没有时延, 所以第一条语句的赋值操作也是在 0 时刻同时进行的, 使 `d_out` 取值变为 0。

由于第二条赋值语句带有 1 个单位时间的延时, 所以在并行块开始执行 1 个单位时间后($t=1$ 时刻), 第二条赋值语句对应的赋值操作才真正得到执行, 将 `d_out` 的取值改变为 1。

同样, 第三条赋值语句对应的赋值操作是在并行块开始执行 2 个单位时间后($t=2$ 时刻)才进行的, 将 `d_out` 取值变为 0; 第四条赋值语句对应的赋值操作是在并行块开始执行 3 个单位时间后($t=3$ 时刻)才进行的, 将 `d_out` 取值变为 1; 第五条赋值语句对应的赋值操作是在并行块开始执行 4 个单位时间后($t=4$ 时刻)进行的, 使 `d_out` 的值变为 0。

由于并行块中的五条语句的执行所需时间分别是 0, 1, 2, 3, 4 个单位时间, 其中第五条赋值语句所需时间最长, 所以在第五条语句执行完毕后($t=4$ 时刻), 并行块结束执行。

该并行语句块执行过程产生的波形如图 5.2 所示, 它和上一小节中用串行块实现的图 5.1 所示的波形是不同的。

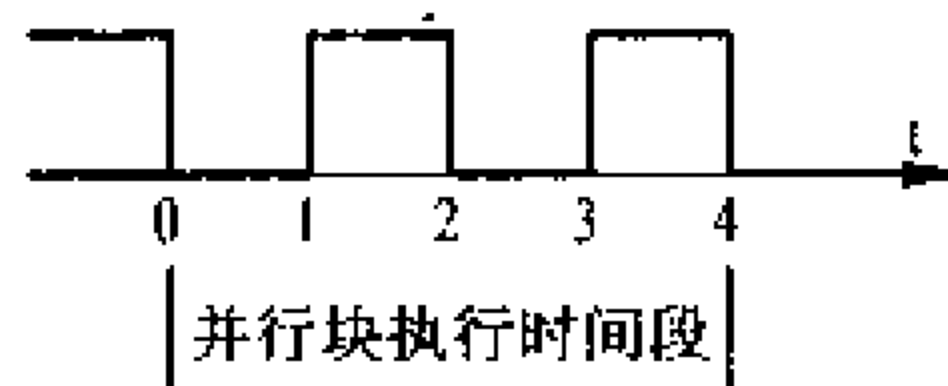


图 5.2 例 5-10 的执行产生的波形

要注意例 5-10 并行块内各条赋值语句出现的次序可以是不同的, 比如其中各条语句的次序可以是例 5-11 所示的情况。

【例 5-11】 并行块内语句的不同次序。

```
initial
fork
    #3 d_out=1;
    #2 d_out=0;
    #4 d_out=0;
    #1 d_out=1;
    d_out=0;
join
```

在例 5-11 中, 并行块内的各条语句并不是按它们被执行的先后顺序给出的, 但例 5-11 同样可以生成与例 5-10 相同的波形。

5.2.3 串行块和并行块的混合使用

在分别对串行块和并行块进行了介绍之后, 我们还必须讨论一下串行块和并行块的混合使用。它可以分为如下两种情况。

(1) 当串行块和并行块 属于不同的过程块(`initial` 或 `always` 过程块)时, 串行块和并行块是并行执行的。

【例 5-12】 串行块和并行块属于不同过程块时的情况。

```
module belong_to_different_pro_block(a,b);
output a, b;
reg a, b;
```




```

initial          // 第一个 initial 过程块
begin           // 串行块
    a=0;        // 语句 S1
    #20 a=0;    // 语句 S2
    b=1;        // 语句 S3
    #40 a=0;    // 语句 S4
    b=0;        // 语句 S5
end
initial          // 第二个 initial 过程块
fork            // 并行块
    b=1;        // 语句 P1
    #10 a=1;    // 语句 P2
    #10 b=0;    // 语句 P3
    #30 a=1;    // 语句 P4
    #50 a=1;    // 语句 P5
join
endmodule

```

例 5-12 中出现了两个 initial 过程块，这两个过程块是并行执行的，所以它们内部所包含的 begin-end 串行语句块和 fork-join 并行语句块也是并行执行的。而在串行块内部，各条语句按顺序方式执行；在并行块内部，各条语句按并行方式执行。例 5-12 中各条语句的执行时序如表 5.2 所示。

表 5.2 例 5-12 中各条语句的执行时间

时间	串行块中语句的执行	并行块中语句的执行
0	S1	P1
10		P2, P3
20	S2, S3	
30		P4
40	S4, S5	
50		P5

(2) 当串行块和并行块嵌套使用在同一过程块内时，内层语句块可以看作是外层语句块中的一条普通语句，内层语句块在什么时刻得到执行是由外层语句块的规则所决定的；而在内层语句块开始执行后，其内部各条语句的执行要遵守内层语句块的规则。

【例 5-13】 嵌套使用的串行块和并行块。

```

module para_bloc_nested_in_seri_bloc (a, b);
output a, b;
reg a, b;
initial          // initial 过程块
begin           // 外层的串行块

```



```

a=0;           // 语句 S1
b=1;           // 语句 S2
#10 a=1;       // 语句 S3
fork           //内层的并行块, 将其标记为 S_P
    b=0;       // 语句 P1
    #10 b=1;   // 语句 P2
    #20 a=0;   // 语句 P3
join
#10 b=0;       // 语句 S4
#10 a=1;       // 语句 S5
b=1;           // 语句 S6
end
endmodule

```

在例 5-13 所示模块中, initial 过程块内部包含了一个串行块, 而该串行块内部又嵌套了一个并行块。这时可以将整个并行块看作串行块中的一条语句(标记为 S_P, 用来代表整个并行块), 根据串行块内部语句的顺序执行方式, 语句 S1, S2, S3, S_P, S4, S5, S6 将按顺序方式执行, 内层并行块(由 S_P 表示)要等到语句 S1、S2、S3 依次执行完毕后才能开始执行。而一旦内层并行块开始执行, 并行块中的各条语句将按并行块规定的并行方式执行。在本例中语句 S3 完成执行的时刻是仿真开始 10 个单位时间后($t=10$ 时刻), 所以内层并行块是在 $t=10$ 时刻开始执行的, 此时($t=10$ 时刻)并行块内语句 P1、P2、P3 同时开始执行。当内层并行块的所有语句都执行完毕后, 外层串行块中排在内层并行块后面的那条语句就开始执行。在本例中, 内层并行块中最后执行完的语句是 P3, 它在内层并行块执行开始 20 个单位时间后($t=30$ 时刻)才能结束执行, 此时整个内层并行块结束执行, 仿真进程将进入下一条语句 S4。

例 5-13 内各条语句的执行时序如表 5.3 所示。

表 5.3 例 5-13 中各条语句的执行时间

时间	外层串行块中语句的执行	内层并行块中语句的执行
0	S1, S2	
10	S3	P1
20		P2
30		P3
40	S4	
50	S5, S6	

当然也可以有串行块被嵌套在并行块内部的情况发生, 这种情况下的执行过程与上面讲述的相类似, 就是将内层串行块看成是外层并行块的一条特殊语句, 这条特殊语句在并行块中与其它语句一起并行执行, 而内层串行块内的各条语句在串行块内得到顺序执行。

【例 5-14】 串行块被嵌套在并行块内部的情况。

```

module seri_bloc_nested_in_para_bloc(a,b,c,d,e);

```




```

output a,b,c,d,e;
reg a,b,c,d,e;
initial          // initial 过程语句
fork            // 外层的并行块
    #10 a = 1;    // 语句 P1
    #15 b = 1;    // 语句 P2
    begin        // 内层的串行块
        #20 c = 1; // 语句 S1
        #10 d = 1; // 语句 S2
    end
    #25 e = 1;    // 语句 P3
join
endmodule

```

例 5-14 中的内层串行块可看作是外层并行块的一条语句，它要与并行块中的其它语句 P1, P2, P3 一起并行执行。所以当仿真进程进行到 initial 过程块后，内层串行块与语句 P1, P2, P3 同时开始执行；而内层串行块中的两条语句(S1 和 S2)则是以顺序方式执行的：先执行 S1，在 S1 执行完毕后 S2 才开始执行。

例 5-14 中各条语句的执行时序如表 5.4 所示。

表 5.4 例 5-14 中各条语句的执行时间

时间	外层并行块中语句的执行	内层串行块中语句的执行
10	P1	
15	P2	
20		S1
25	P3	
30		S2



第6章 行为描述(二): 时间控制和赋值语句



行为描述模块的主要组成部分是过程块(initial 过程块和 always 过程块), 而过程块和语句块的基本构成成分是过程赋值语句和高级程序语句。本章我们将对过程赋值语句以及连续赋值语句进行讨论, 并且介绍与赋值语句的使用相关的各种时间控制方式。高级程序语句将在第7章进行讨论。

6.1 时间控制

时间控制可以用来对过程块中各条语句的执行时间(时序)进行控制。时间控制可以分为两类:

- (1) 延时控制: 为行为语句的执行指定一个延时时间的时间控制方式。
- (2) 事件控制: 为行为语句的执行指定触发事件的时间控制方式。

其中事件控制方式又可以分成两类: 边沿触发事件控制和电平敏感事件控制。

下面分别对延时控制、边沿触发事件控制和电平敏感事件控制这三种时间控制方式进行介绍。

6.1.1 延时控制

延时控制的格式为:

#<延迟时间> 行为语句;

或

#<延迟时间>;

其中: 符号“#”是延时控制的标识符; “<延迟时间>”是直接指定的延迟时间量, 它是以多少个仿真时间单位的形式给出的, 可以是一个立即数、变量或表达式。

1. 第一种形式的延时控制语句

在上面给出的第一种延时控制格式中, <延迟时间>的后面跟着一条行为语句。在这种情况下, 仿真进程在遇到这条带有延时控制的行为语句后并不立即执行行为语句指定的操作, 而是要延迟等待到“<延迟时间>”所指定的时间量过去后才真正开始执行行为语句指定的操作, 如例6-1所示的程序。

【例6-1】第一种形式的延时控制语句。

```
module clk_gen(clk);
```




```

output clk;
reg clk;
initial
begin
    clk=0;           //语句 S1
    #10 clk = 1;      //语句 S2
    #20 clk = 0;      //语句 S3
    #30 clk = 1;      //语句 S4
end
endmodule

```

例 6-1 中的串行块内共有四条赋值语句，其中语句 S1 没有时间控制部分，而语句 S2、S3 和 S4 带有延时控制。在仿真时：

当程序控制流程遇到语句 S1 时 ($t=0$ 时刻) 立即执行该语句指定的赋值操作，将 0 值赋给 clk，语句 S1 结束。

随后程序控制流程遇到语句 S2，由于语句 S2 带有 10 个单位时间的延时控制，所以程序控制流程要在等待 10 个单位时间后 ($t=10$ 时刻) 才执行语句 S2 对应的赋值操作，给 clk 赋值 1。

与语句 S2 的执行一样，随后依次执行的语句 S3 和 S4 所对应的赋值操作也是在分别等待了由两条语句中的延时控制部分指定的延时时间 (分别是 20 个单位时间和 30 个单位时间) 以后才真正执行的。

表 6.1 给出了例 6-1 在进行仿真时的时序以及输出信号 clk 的变化情况。

表 6.1 例 6-1 进行仿真时的程序控制流程和输出信号 clk 的变化情况

时 间	程序控制流程	输出 clk
0	进入串行块，进入 S1， 执行 S1 后进入 S2	0
10	退出 S2 后进入 S3	1
30	退出 S3 后进入 S4	0
60	退出 S4，退出串行块	1

2. 第二种形式的延时控制语句

延时控制的第二种格式中，在“<延迟时间>”后面没有出现任何行为语句，而只有一个语句结束符号“;”。在这种情况下，仿真进程在遇到这条延时控制语句后不执行任何操作，而是进入一种等待状态，等到过了由“<延迟时间>”所指定的时间量后仿真流程，结束这条延时控制语句的执行。这种形式的延时语句出现在串行块和并行块中时产生的作用是不同的，下面我们举例说明。

【例 6-2】第二种形式的延时控制语句使用于串行块。

```

module clk_gen(clk);
    output clk;
    reg clk;
    initial

```



```
begin          //串行块
    clk=0;      //语句 S1
    #10 clk = 1; //语句 S2
    #10 ;       //语句 S3
    #10 clk = 0; //语句 S4
    #30 clk = 1; //语句 S5
end
endmodule
```

例 6-2 中的串行块内共有四条赋值语句 S1, S2, S4 和 S5, 还有一条不带有行为语句的第二种形式的延时控制语句 S3。在仿真时, 程序控制流程先后进入语句 S1 和 S2, 语句 S1 对应的赋值操作在 $t=0$ 时刻得到执行, 语句 S2 对应的赋值操作在 $t=10$ 时刻得到执行。

在语句 S2 执行完毕后, 在 $t=10$ 时刻程序流程控制进入语句 S3。由于语句 S3 是一条不帶任何行为语句的延时控制语句, 所以程序控制流程将停留在 S3 语句处而不执行任何操作。直到指定的 10 个单位的延迟时间过去以后 ($t=20$ 时刻), 程序控制流程退出语句 S3 同时开始下一条语句 S4 的执行。表 6.2 给出了例 6-2 所示代码进行仿真时的程序控制流程和输出变化情况。

表 6.2 例 6-2 仿真时的程序控制流程和输出信号 clk 的变化情况

时 间	程序控制流程	输出 clk
0	进入串行块, 然后进入 S1, 退出 S1 后进入 S2	0
10	退出 S2, 进入 S3(进入延时)	1
20	(延时结束)退出 S3, 进入 S4	不变, 仍为 1
30	退出 S4, 进入 S5	0
60	退出 S5, 退出串行块	1

在例 6-2 的执行中我们可以看出: 第二种形式的延时控制语句在串行块中的作用是使该控制语句后的下一条语句的执行时刻延迟了指定的时间量。因此, 如下语句:

```
begin
    #10 ;
    a=b ;
end
```

将等价于下面这一条语句:

```
#10 a=b;
```

下面我们给出另一个例子来说明第二种形式的延时控制语句在并行块中使用的情况。

【例 6-3】第二种形式的延时控制语句使用于并行块。

```
module clk_gen(clk);
    output clk;
    reg clk;
```




```

initial
    fork                //并行块
        clk=0;          //语句 P1
        #10 clk = 1;     //语句 P2
        #10 ;           //语句 P3
        #20 clk = 0;     //语句 P4
        #30 clk = 1;     //语句 P5
    join
endmodule

```

例 6-3 中的并行块内共有四条赋值语句 P1, P2, P4 和 P5, 还有一条不带有行为语句的第二种形式的延时控制语句 P3。在仿真执行时, 程序控制流程同时进入并行块内各条语句, 所以语句 P1, P2, P3, P4 和 P5 都在 $t=0$ 时刻开始执行, 其中进入延时控制语句 P3 的程序控制流程在等待了指定的 10 个单位时间后 (在 $t=10$ 时刻) 退出 P3, 结束 P3 的执行; 而由于其它几条并行语句 P1, P2, P4, P5 分别是在 $t=0, 10, 20, 30$ 时刻执行赋值操作并结束语句执行的, 所以程序控制流程是在执行时间最长的那条语句 (语句 P5) 执行完毕后 ($t=30$ 时刻) 才退出 fork-join 并行块。在这种情况下延时控制语句 P3 既没有产生具体的操作, 又没有对并行块仿真执行的时序产生影响, 所以在上例中有没有延时控制语句 P3 对于要实现的功能不会产生任何影响。

但是如果我们将例 6-3 中的延时控制语句 P3 改为 “#40;”, 那么情况就不一样了。这种情况下这条延时控制语句是并行块内所有并行语句中执行结束时刻最迟的一条, 所以只有在该延时控制语句执行完毕后 ($t=40$ 时刻), 并行块的执行才能结束。因此在这种情况下延时控制语句的作用是对并行块的结束时间进行控制。

表 6.3 中分别给出了例 6-3 所示代码以及经过修改后的代码的仿真执行情况。

表 6.3 例 6-3 所示代码以及将其修改后的代码在进行仿真时的执行情况

时间	程序控制流程(修改前)	输出 clk(修改前)	程序控制流程(修改后)	输出 clk(修改后)
0	同时进入 P1~P5, 退出 P1	0	同时进入 P1~P5, 退出 P1	0
10	同时退出 P2, P3	1	退出 P2	1
20	退出 P4	0	退出 P4	0
30	退出 P5, 退出并行块	1	退出 P5	1
40	已离开并行块		退出 P3, 然后退出并行块	不变, 仍为 1

在介绍了如上两种形式的延时控制语句后, 还必须对语法定义中的 “<延迟时间>” 这一项进行说明。“<延迟时间>” 这一项还可以是一个变量或表达式, 而不必将其局限于某立即数, 比如:

```

initial
begin : SEQ_1
    parameter DELAY_TIME=50 ;
    #DELAY_TIME out_1=0;          //第一条赋值语句

```



```
#(DELAY_TIME / 2) out_2=1;    //第二条赋值语句
end
```

在上面程序的第一条赋值语句内的延时时间是由一个参数“DELAY_TIME”来指定的,它代表了50个单位时间的延迟,而在第二条赋值语句中则用了一个表达式:“DELAY_TIME/2”来指定延迟时间,它表示需要延迟 $50/2=25$ 个单位时间。

当“<延迟时间>”是由一个变量或由一个表达式表示时,有可能出现该变量或表达式的值为负值以及取值为x或z的情况。Verilog语法规定:如果代表延时时间的变量或表达式的值为x或z,那么该延时控制等效于零时延控制;如果代表延时时间的变量或表达式的值为负值,则以该负值的二进制补码值作为实际的延时量。

6.1.2 边沿触发事件控制

事件控制方式是时间控制的另一种方式。在这种时间控制方式下,行为语句的执行需要由指定事件的发生来触发,该事件被称为“触发事件”。在always语句块格式中所使用的敏感事件列表就是一种事件控制。事件控制方式不仅能用于always语句块,而且还可以用于其它的行为语句。

事件控制方式又可以进一步分成两类:边沿触发事件控制和电平敏感事件控制。

本小节只对边沿触发事件控制进行讨论,电平敏感事件控制将在下一小节进行叙述。

1. 边沿触发事件控制

这一类时间控制方式在指定的信号变化时刻,也就是在指定的信号的跳变边沿才触发语句的执行;而当信号处于稳定状态时则不会触发语句的执行。

边沿触发事件控制的语法格式可以为如下几种:

第一种: @ (<事件表达式>) 行为语句;

第二种: @ (<事件表达式>);

第三种: @ (<事件表达式1> or <事件表达式2> oror <事件表达式n>) 行为语句;

第四种: @ (<事件表达式1> or <事件表达式2> oror <事件表达式n>);

在以上四种格式中,符号“@”是边沿触发事件控制的标识符,“事件表达式”代表着触发语句执行的触发事件;而“行为语句”则指出了触发事件所要触发执行的具体操作。

上面格式中由括号界定的事件表达式部分: (<事件表达式>)和(<事件表达式1> or <事件表达式2> oror <事件表达式n>)称为“敏感事件列表”。

1) 事件表达式

事件表达式可以是如下三种形式之一:

- <信号名>。
- posedge <信号名>。
- negedge <信号名>。

其中,“<信号名>”可以是任何数据类型的标量或矢量。

(1) 第一种事件表达式形式:代表的触发事件是“<信号名>”所指定的信号发生了某种逻辑变化(不论是正跳变还是负跳变)。它是信号除了保持稳定态以外的任意一种变化过程。比如语句:



```
@ (reg_a) reg_b=reg_a;
```

就表示触发赋值语句“reg_b=reg_a;”执行的触发事件是信号“reg_a”的逻辑值发生了变化,只要 reg_a 从 0、1、x、z 四个值中的一个变化到另一个(比如 reg_a 由 1 变到 z),赋值语句就被触发执行,执行对应的赋值操作(将 reg_a 的值赋给 reg_b)。

(2) 第二种事件表达式形式:在“<信号名>”前面加上了一个关键词“posedge”。这种形式代表的触发事件是“<信号名>”所指定的信号发生了正跳变。所谓“正跳变”就是发生如下逻辑转换中的一种:

0→x

0→z

0→1

x→1

z→1

比如语句:

```
@ (posedge reg_a) reg_b=reg_a;
```

就表示触发赋值语句“reg_b=reg_a;”执行的触发事件是信号“reg_a”的逻辑值发生了正跳变,因此只要“reg_a”发生了上面给出的五种逻辑转换中的一种(比如 reg_a 由 x 变为 1),就启动赋值操作的执行。

(3) 第三种事件表达式形式:在“<信号名>”前面加上了一个关键词“negedge”。这种形式代表的触发事件是“<信号名>”所指定的信号发生了负跳变。所谓“负跳变”就是发生如下逻辑转换中的一种:

1→x

1→z

1→0

x→0

z→0

比如语句:

```
@ (negedge reg_a) reg_b=reg_a;
```

就表示触发赋值语句“reg_b=reg_a;”执行的触发事件是信号“reg_a”的逻辑值发生了负跳变,因此只要 reg_a 发生了上面给出的五种负跳变逻辑转换中的一种(比如 reg_a 由 1 变为 x),就启动赋值操作的执行。

2) 语法格式

下面我们再来讨论前面给出的边沿触发事件控制的四种语法格式:

(1) 边沿触发事件控制的第一种语法格式:

```
@ (<事件表达式>) 行为语句;
```

这种语法格式的敏感事件列表内只包含了一个触发事件,只有当这个指定的触发事件发生之后,后面的行为语句才能启动执行;在仿真进程中遇到这种带有事件控制的行为语句时,如果指定的触发事件还没有发生,则仿真进程就会停留在此处等待,直到指定触发事件发生之后再启动执行后面的行为语句,仿真进程继续向下进行。

【例 6-4】时钟脉冲计数器。


```

module clk_counter(clk, count_out);
input clk,
output count_out;
reg [3:0] count_out;
initial
count_out=0;           //给 count_out 赋初值 0
always
@( posedge clk ) count_out=count_out+1; //在 clk 的每个正跳变边沿 count_out 增加 1
endmodule

```

例 6-4 中的事件控制语句所指定的触发事件是时钟信号“clk”发生正跳变。因此在执行这条事件控制语句时，仿真进程将等待时钟“clk”正跳变的发生，然后为计数寄存器“count_out”增值。

(2) 边沿触发事件控制的第二种语法格式：

@ (<事件表达式>) ；

这种语法格式与前面的第一种语法格式一样只指定了一个触发事件，但是在这种格式中没有行为语句来指定触发事件发生时要执行的操作。这种格式的事件控制语句的执行过程与延时控制语句中没有行为语句的情况类似，仿真进程在遇到这条事件控制语句后会进入等待状态，直到指定的触发事件发生后才结束等待状态，退出该事件控制语句的执行并开始下一条语句的执行。比如例 6-5 中程序所示。

【例 6-5】用于测定输入时钟正电平，负电平持续时间以及时钟周期的模块。

```

module clk_time_mea(clk );
input clk;
time pos_edge_time, neg_edge_time;
time high_last_time, low_last_time, last_time;
initial
begin
    @( posedge clk) ;    /* 等待，直到时钟发生正跳变后退出等待状态，继续执行
    下一条语句 */
    pos_edge_time=$time;
    @( negedge clk) ;    /* 等待，直到时钟发生负跳变后退出等待状态，继续执行
    下一条语句 */
    neg_edge_time=$time;
    @( posedge clk) ;    /* 等待，直到时钟再次正跳变后退出等待状态，继续执行
    下一条语句 */
    last_time=$time- pos_edge_time ;
    high_last_time= neg_edge_time - pos_edge_time ;
    low_last_time= last_time- high_last_time;
    $display("The clk stay in High level for:%t", high_last_time );
    $display("The clk stay in Low level for:%t", low_last_time );

```




```

    $display("The clk signal Period is :%1", last_time );
end
endmodule

```

例 6-5 中的三条边沿触发事件控制语句的作用是使仿真进程在遇到这几条语句时进入等待状态，分别等待时钟信号正跳变、负跳变和再次正跳变的发生。当发生了对应的跳变之后仿真进程就退出等待状态，结束对应事件控制语句的执行并开始下一条语句的执行，保存当前的仿真时间值。

(3) 边沿触发事件控制的第三种语法格式：

@ (<事件表达式 1> or <事件表达式 2> oror <事件表达式 n>) 行为语句；

这种语法格式的“敏感事件列表”内指定了由不同“<事件表达式>”代表的多个触发事件，这些“<事件表达式>”要用关键词“or”组合起来。只要这些触发事件中的任何一个得到发生，就启动行为语句的执行。在仿真进程遇到这种格式的边沿触发事件控制语句时如果所有的触发事件都没有发生，则仿真进程就会进入等待状态，直到其中的某一个触发事件发生后才启动执行后面给出的行为语句，仿真进程继续向下进行。例如例 6-6 给出的 initial 语句块所示。

【例 6-6】边沿触发事件控制语句的例子。

```

initial
begin
    a=1;
    @(posedge clk1 or negedge clk2 or b) a=b;
end

```

在例 6-6 所示的程序中，边沿触发事件控制语句的“敏感事件列表”内包含了三个事件表达式：“posedge clk1”，“negedge clk2”和“b”。其中的前两个事件表达式分别属于前面讲过的第二种和第三种事件表达式形式，它们指定了信号跳变边沿种类。这三个事件表达式所代表的触发事件分别是：信号 clk1 发生正跳变、信号 clk2 发生负跳变和信号 b 发生任何变化。

只要发生了这三个触发事件中的任一个，赋值语句“a=b;”的执行就被启动。

所以例 6-6 所实现的功能是：给 a 赋初值 1，然后等到 clk1 发生正跳变或 clk2 发生负跳变或信号 b 发生跳变时，将 b 的逻辑值赋值给 a。

(4) 边沿触发事件控制的第四种语法格式：

@ (<事件表达式 1> or <事件表达式 2> oror <事件表达式 n>) ；

同第三种语法格式一样，这种语法格式内指定了多个触发事件。但是在这种格式中没有行为语句。在这种情况下，该语句的执行过程与第二种语法格式的执行过程类似，仿真进程在遇到这条事件控制语句后会进入等待状态，直到敏感事件列表所包含的多个触发事件中的任何一个得到发生后才结束等待状态，退出该事件控制语句并开始执行该事件控制语句后的下一条语句。比如例 6-7 模块所示。

【例 6-7】连续监测信号 a 或 b 发生变化的时间。

```

module display_information_of_change(a, b);
input a, b;

```



```
wire a, b;
always          //always 过程块, 重复进行监测
begin
    @(a or b);    /* 等待, 直到 a 或 b 发生变化后退出等待状态,
                  并开始下一条语句的执行 */
    $display ("One of a and b changed in time:%t", $time);
end
endmodule
```

例 6-7 中的模块在仿真执行时, 仿真进程遇到事件控制语句后就进入等待状态, 直到信号 a 或 b 发生变化后才退出等待状态, 结束这条事件控制语句的执行, 并开始下一条 \$display 语句的执行, 显示当前仿真时刻的时间值(也就是 a 或 b 发生变化时的时间值)。

可以看出, 例 6-7 中 begin-end 串行块内的两条语句可以用如下一条语句替换:

```
@(a or b) $display ("One of a and b changed in time:%t", $time);
```

2. 敏感事件列表在 always 过程块中的使用

我们在第 5.1.2 小节中讲过, always 过程块可以带有事件控制。带有事件控制的 always 过程块既可以实现组合逻辑又可以实现时序逻辑; 在用于实现这两种不同逻辑功能种类时, always 过程块内的敏感事件列表会具有不同的特点。下面分别加以讨论。

1) 用 always 过程块实现组合逻辑功能

(1) 事件表达式内不能包含“posedge”和“negedge”关键词, 也就是说敏感事件列表中的事件表达式只能是前面所述的第一种事件表达式形式。

(2) 组合逻辑的所有输入信号都要作为“<信号名>”出现在敏感事件列表中。

上面给出的第一个特点是因为组合逻辑电路的输出是由输入电平所决定的, 输入不同的跳变方式对输出不会产生任何影响, 因此敏感事件列表中的事件表达式不能是其余两种对跳变边沿加以指定的形式。

上面给出的第二个特点是因为组合逻辑电路的输入输出关系在每个时刻都应该是严格成立的, 也就是说, 无论什么时刻输入发生了变化, 在输出端都应该立即有反应。如果某个输入信号没有出现在敏感事件列表中, 那么过程块就没有办法检测到该信号的变化, 这个输入信号对输出的影响就不能实现, 这样就不能在任意时刻保证组合逻辑的输入输出关系。

【例 6-8】不能正确实现三输入与门的模块。

```
module three_input_and(a, b, c, out);
input a, b, c;          //三个输入 a, b, c
output out;
reg out;
always @(a or b)        //敏感事件列表中只包含了两个输入: a, b
begin
    out=a&b&c;
end
endmodule
```




例 6-8 中模块的目的是实现三个输入 (输入信号 a, b, c) 的与逻辑运算, 但是在 always 语句后面的敏感事件列表中只包含了 a 和 b 两个输入信号而没有包含输入信号 c。所以当某个时刻 c 发生逻辑变化时, 事件控制语句无法检测到信号 c 的变化, 从而不能启动 begin-end 顺序块的执行来对输出 out 进行更新。这显然没有实现三输入与门的逻辑功能。

所以要正确实现一个三输入与门的功能, 需要将输入 c 添加到上例的敏感事件列表中去, 如例 6-9 所示。

【例 6-9】 正确实现三输入与门的模块。

```
module three_input_and(a, b, c, out);
    input a, b, c;           //三个输入 a, b, c
    output out;
    reg out;
    always @(a or b or c) //敏感事件列表中包含了所有输入
    begin
        out=a&b&c;
    end
endmodule
```

下面再举一个用 always 过程块实现组合逻辑功能的例子。

【例 6-10】 利用敏感事件列表来对组合逻辑建模。

```
module selective_adder_and_multiplier(a, b, sel, y);
    input a, b, sel;
    output y;
    wire[3:0] a, b;
    reg[7:0] y;
    always @(a or b or sel)
    begin
        if (sel==0) y = a + b;
        else if(sel==1) y = a * b;
    end
endmodule
```

例 6-10 所示模块有三个输入, 分别是数据输入 a, b 和控制信号 sel。当控制信号 sel 为高电平时模块实现乘法功能, 而当 sel 为低电平时模块实现加法功能。always 过程块中的敏感事件列表内包含了所有的输入信号, 当任何一个输入发生变化都能及时地对输出进行更新。

2) 用 always 过程块实现时序逻辑功能

(1) 事件表达式可以是事件表达式三种形式中的任何一种, 也就是说所采用的事件表达式既可以是带有“posedge”或“negedge”关键词 (分别表示所描述时序电路逻辑状态的改变是发生在输入信号的正跳变边沿或负跳变边沿) 的事件表达式, 也可以是只包含“信号名” (表示输入信号的任何边沿变化都会引起时序逻辑电路逻辑状态的改变) 的事件表达式。

(2) 不要求所有的输入信号都出现在敏感事件列表的“信号名”中。这是由于时序逻辑电路逻辑状态的改变只发生在某个或某几个时钟输入信号的变化边沿,而在其它输入信号发生变化时电路逻辑状态保持不变。所以这时事件控制只需对这几个时钟输入信号进行监测就可以了,没有必要把其它输入信号也放入敏感事件列表中。

下面我们举几个例子来说明敏感事件列表在描述时序逻辑功能时的使用。

【例 6-11】时钟下降沿触发的 D 触发器。

```
module d_ff(q, clk, d);
input clk, d;
output q;
reg q;
always @(negedge clk)
begin
    q=d;
end
endmodule
```

在上例模块内, always 过程块的敏感事件列表包含了事件表达式“negedge clk”。这个事件表达式带有“negedge”关键词,它指出该 D 触发器状态的改变发生在 clk 信号的负跳变边沿,在其余时刻 D 触发器状态将保持不变。

【例 6-12】同时由两个时钟沿: clk1 上升沿和 clk2 下降沿进行控制的 8 位移位寄存器。

```
module 8bits_shift_register(d_in, d_out, clk1, clk2);
input clk1, clk2, d_in;
output d_out;
reg d_out;      //d_out 保存 1bit
reg[1:7] data;  // reg[1:7]保存其余 7bits
reg[1:4] i;     // i 用于循环计数
always @(posedge clk1 or negedge clk2)
begin
    d_out=data[1];
    for (i=1; i<7; i=i+1)
        data[i]=data[i+1];
    data[7]=d_in;
end
endmodule
```

在例 6-12 模块内, always 过程块的敏感事件列表包含了两个事件表达式“posedge clk1”和“negedge clk2”。这个敏感事件列表表示该移位寄存器的移位操作是由 clk1 信号的正跳变边沿或 clk2 信号的负跳变边沿触发的,移位寄存器的内部状态和输出只在这两个跳变边沿发生变化,而在其它任何情况下都保持不变。



3) always 过程块实现组合和时序混合的逻辑功能

要实现组合和时序混合的逻辑功能，我们可以在一个模块中建立多个并行结构来实现。用一个或多个带有事件控制的 always 过程块来实现时序逻辑部分的功能；再用连续赋值语句或另外一个(或多个) always 过程块来实现组合逻辑部分功能。在这种情况下，每一个 always 过程块只实现了一类逻辑，而我们现在要讨论的是用同一个 always 过程块来实现组合和时序混合的逻辑功能的情况。

在组合逻辑和时序逻辑混合的情况下，模块逻辑状态和输出发生变化时刻要同时受时钟信号跳变沿和某些输入控制信号的控制，典型的例子是带有异步置位(清零)控制输入的时钟边沿触发器。比如一个带有异步清零控制输入位 clr 的时钟上升沿 T 触发器，其输入输出真值表如表 6.4 所示。

表 6.4 带异步清零位的上升沿 T 触发器的输入输出真值表

clk	clr	t	current state	next state (q)
?	1	?	?	0
↑	0	0	0	0
↑	0	0	1	1
↑	0	1	0	1
↑	0	1	1	0

真值表的第 1 行说明：当清零信号 clr 为 1 时，无论其它输入信号和触发器当前状态如何(用“?”表示“任何状态”)，触发器状态(也就是输出 q)将始终为 0；真值表的第 2 行到第 5 行说明了清零信号 clr 为 0 时，T 触发器处于正常工作状态：若在时钟上升沿输入 t=0 则触发器状态保持不变，而如果在时钟上升沿输入 t=1 则触发器的状态将发生翻转。

因此在输入 clr 为 0 时，触发器的逻辑功能是普通的时序逻辑，其状态和输出在时钟上升沿发生变化；而在输入 clr 变为 1 时，T 触发器的输出立即变为 0，这时的逻辑功能就是关于控制信号 clr 的组合逻辑。我们可以用一个 always 过程块来实现这种混合的逻辑功能，如例 6-13 所示。

【例 6-13】带有异步清零位 clr(高电平有效)的时钟上升沿 T 触发器。

```

module asyn_clear_Tff(clk, clr, t, q);
input clk, clr, t;
output q;
reg q;
always @(posedge clk or posedge clr)
begin
    if(clr==1)
        q=0;
    else if(t==1)
        q=~q;
end
endmodule

```


由例 6-13 我们可以看出: 在实现组合和时序混合逻辑时, `always` 语句的敏感事件列表内要包含时序逻辑部分的时钟信号以及组合逻辑部分的输入控制信号。在上例中就是时钟信号 `clk` 和控制信号 `clr` 都要出现在敏感事件列表中。这两个信号中的任何一个发生变化都会启动 `begin-end` 串行语句块的执行: 根据控制信号 `clr` 的逻辑电平取值, 分别实现组合逻辑 (`clr` 为 1 时) 和时序逻辑 (`clr` 不为 1 时)。

虽然可以利用一个 `always` 过程块来实现组合逻辑功能或者混合的逻辑功能, 但必须注意到 `always` 过程块主要是为了对时序逻辑建模而引入的一种描述方式。所以在实际硬件设计工作中应该尽量使用结构描述方式 (连续赋值语句、模块实例、原语实例等语句) 来描述组合逻辑, 这样可以使综合后的结果更加接近实际硬件的结构。

6.1.3 电平敏感事件控制 (wait 语句)

“电平敏感事件控制”是与“边沿触发事件控制”对应的另一种事件控制类别。与边沿触发事件控制不同, 在电平敏感事件控制方式下启动语句执行的触发条件是某一个指定的条件表达式为真。

电平敏感事件控制用关键词“`wait`”来表示。它可以有如下三种形式:

- `wait` (条件表达式) 语句块;
- `wait` (条件表达式) 行为语句;
- `wait` (条件表达式);

电平敏感事件控制的第一种形式中包含了一个语句块, 它可以是串行块 (`begin-end` 块) 或并行块 (`fork-join` 块)。在这种事件控制语句形式下, 语句块启动执行的触发条件是: 条件表达式的值为“真 (逻辑 1)”。如果当仿真进程执行到这条电平敏感事件控制语句时条件表达式的值是“真”, 那么语句块立即得到执行; 否则语句块要一直等到条件表达式变为“真”时再开始执行。比如如下语句:

```
wait (enable==1)
begin
    d=a&b;
    d=d | c;
end
```

所实现的功能是: 等待使能信号 `enable` 变为高电平后进行 `a`, `b`, `c` 的与或逻辑操作。`wait` 语句的作用是对 `begin-end` 串行块的启动执行时间进行控制: 只有当 `enable` 信号为 1 时才开始串行块的执行。如果在仿真进程遇到 `wait` 语句时 `enable` 是 1, 则立即开始执行串行块中的赋值语句; 如果仿真进程遇到 `wait` 语句时 `enable` 不是 1, 那么要等到 `enable` 变为 1 后才开始串行块的执行。

电平敏感事件控制的第二种形式中包含了一条行为语句, 它可以是 6.1.2 小节中给出的任何一种行为语句。在这种事件控制语句形式下, 行为语句启动执行的触发条件与前面第一种形式下的触发条件相同。两者之间惟一不同之处在于: 这里启动的是一条行为语句的执行, 而在第一种形式下启动的是一个语句块的执行。比如:

```
wait (enable==1) d= (a&b) | c ;
```

同样实现了“等待使能信号变为 1 后执行 `a`, `b`, `c` 的与或逻辑操作”的功能。在这里 `wait`



语句触发执行的是单条赋值语句“ $d = (a \& b) | c$;”。

在电平敏感事件控制的第三种形式中没有包含行为语句或语句块。在这种电平敏感事件控制语句形式下，如果当仿真进程执行到该 `wait` 控制语句时条件表达式的值是“真”，那么立即结束该 `wait` 事件控制语句的执行，仿真进程继续往下进行；而如果当仿真进程执行到这条 `wait` 控制语句时条件表达式的值不是“真”，则仿真进程进入等待状态，一直等到条件表达式取值变为“真”时才退出等待状态同时结束该 `wait` 语句的执行，仿真进程继续往下进行。这种形式的电平敏感事件控制常常用来对串行块中各条语句的执行时序进行控制。比如如下语句：

```
begin
    wait(enable==1);
    d=a&b;
    d=d | c;
end
```

也能实现“等待使能信号 `enable` 取值变为 1 后进行 `a`, `b`, `c` 的与或逻辑操作”的功能。仿真进程遇到 `begin-end` 串行块后执行其中第一条语句也就是不带有行为语句的 `wait` 语句，这条 `wait` 语句的执行将使仿真进程在 `enable` 信号变为 1 后才能继续往下进行，从而对 `wait` 语句后面两条赋值语句的执行时序进行控制。注意这种形式的 `wait` 语句在串行结构(串行块)中才能实现时序控制功能。

在使用电平敏感事件控制语句(`wait` 语句)时，要注意体会它与上一小节所介绍的“边沿触发事件控制”的区别：以“@”开头的边沿触发事件控制只对信号的跳变边沿敏感，它给出的触发事件是指定的信号跳变边沿；以“`wait`”开头的电平敏感事件控制只对信号的电平敏感，它给出的触发事件是指定的信号稳定逻辑状态。在此举一例加以说明。

【例 6-14】 电平敏感事件控制与边沿触发事件控制的比较。

```
module level_and_edge_sensitive_compare(flag1, flag2);
    output flag1, flag2;
    reg enable, flag1, flag2;
    initial          //第一个 initial 过程块，用于产生 enable 信号
    begin
        flag1=1;
        flag2=1;
        enable=1;
        #10 enable=0;
        #10 enable=1;
        #10 $finish;
    end
    initial          //第二个 initial 过程块，用于实现电平敏感事件控制
    begin
        #10 ;
        wait(enable==1) flag1=~flag1;    //若触发条件成立，flag1 翻转
```



```

end
initial          //第三个 initial 过程块, 用于实现边沿触发事件控制
begin
    #10 ;
    @(posedge enable) flag2=~flag2; //若触发条件成立, flag2 翻转
end
endmodule

```

例 6-14 模块中包含了三个并行执行的 initial 过程块, 其中第一个 initial 过程块用来生成使能信号 enable 的波形, 并给信号 flag1 和 flag2 赋初值 1; 第二个 initial 过程块用来在 t=10 时刻引入电平敏感事件控制, 每当触发条件成立 (enable==1) 就使 flag1 取值翻转; 第三个 initial 过程块用来在 t=10 时刻引入边沿触发事件控制, 每当其触发条件成立 (enable 发生正跳变) 就使 flag2 取值翻转; 通过对输出 flag1, flag2 的变化进行观察, 我们可以对两种事件控制方式下的不同触发方式进行比较。

例 6-14 模块仿真执行产生的波形如图 6.1 所示。

由上述波形图可以看出: flag1 信号的变化发生在 t=10 时刻, 这是因为当在 t=10 时刻开始引入电平敏感事件控制时 enable 信号的值已经是 1 了, 即在 t=10 时刻触发条件得到满足, 所以立即对 flag1 进行取反操作。

而 flag2 信号的变化则发生在 t=30 时刻, 这是因为边沿触发事件控制只对 enable 信号指定的正跳变边沿敏感, 虽然在 t=10 时刻引入边沿触发事件控制时 enable 信号的值已经是 1 了, 但是这时 enable 信号是处于稳定状态而没有发生跳变, 所以在 t=10 时刻边沿触发条件没有得到满足。在 t=20 时刻虽然 enable 信号发生了跳变, 但这个跳变是一个负跳变同样没有满足正跳变边沿的触发条件。直到 t=30 时刻, enable 信号发生正跳变, 指定的边沿触发条件得到满足, 对 flag2 取反的操作被触发执行。

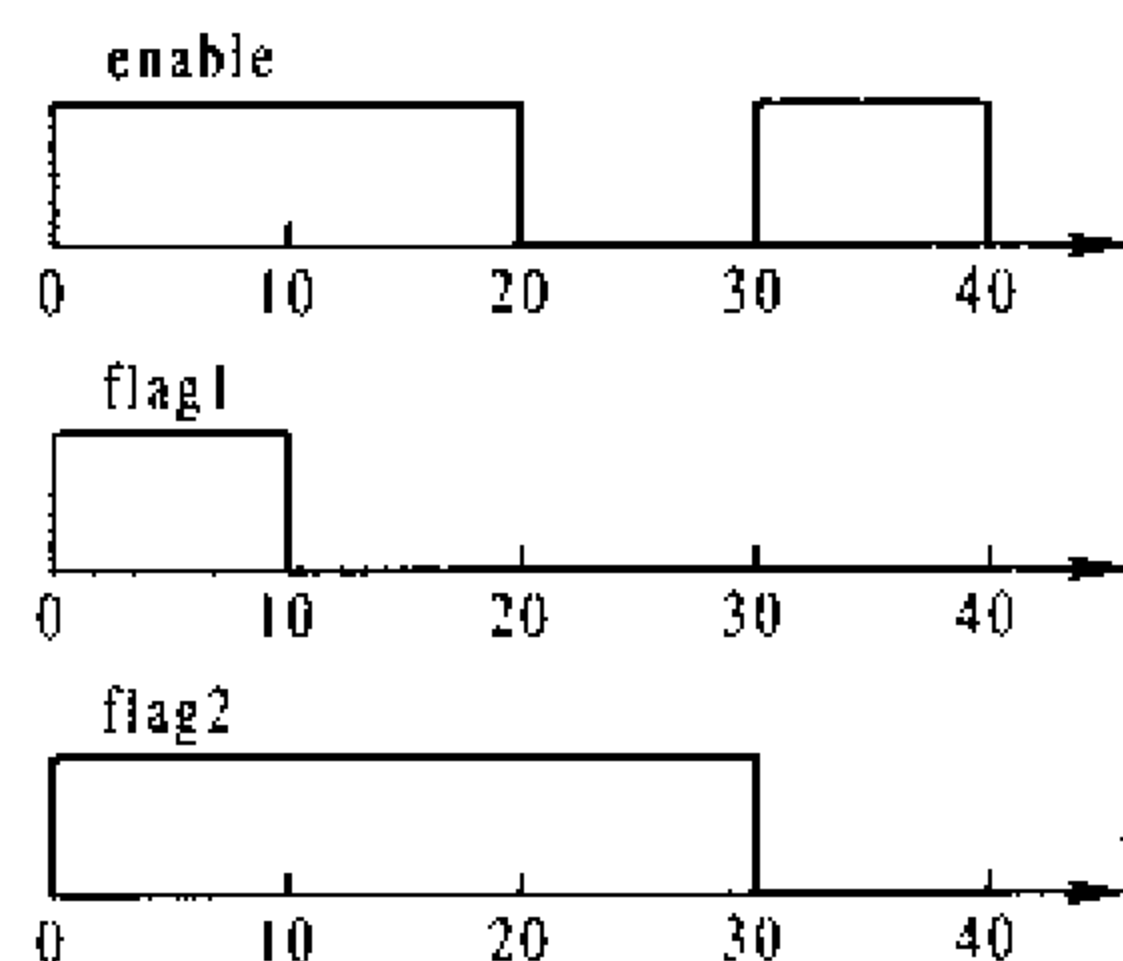


图 6.1 例 6-14 所示模块仿真执行时产生的波形

6.2 赋值语句

行为描述模块中的语句块 (begin-end 串行块或 fork-join 并行块) 是由“过程赋值语句”和“高级程序语句”这两种基本成分构成的。在本节我们将对过程赋值语句进行讨论, 并且同时介绍与过程赋值语句对应的“连续赋值语句”。

6.2.1 过程赋值语句的基本格式

过程赋值语句是用于两种结构化过程块 (initial 过程块和 always 过程块) 中的赋值语句。在过程块中只能使用过程赋值语句 (不能在过程块中出现连续赋值语句), 同时过程赋值语句也只能用在过程块中。

过程赋值语句的基本格式为:



<被赋值变量> <赋值操作符> <赋值表达式>

其中<赋值操作符>是“=”或“<=”之一，它们分别代表了阻塞型赋值和非阻塞型赋值类型。可以在上述格式中增加时间控制部分，这点将在 6.2.2 小节对过程赋值语句中的时间控制进行讨论。

过程赋值语句只能用于对寄存器类的变量(寄存器 reg、整型 integer、实型 real 或时间型 time 变量)进行赋值操作。在经过过程赋值后，上面的这些变量的取值将保持不变，直到另一条过程赋值语句对变量重新赋值为止。过程赋值操作的具体目标可以是：

- (1) reg, integer, real, time 型变量(标量或矢量)。
- (2) 上述类型变量的某一位或某几位(在矢量情况下)。
- (3) 存储器类，只能对某地址单元的整个字进行赋值，不能对其中某些位单独进行赋值。
- (4) 上述几个变量用连接运算符“{, }”拼接起来构成的寄存器整体。

【例 6-15】过程赋值语句赋值操作的各种目标变量形式。

```
reg a;
reg [0:7] b;
integer i;
reg[0:7] mem_1[0:1023] ;
initial
begin
    a=0;           //对一个 1 位寄存器 a 赋值
    i=356;         //对一个整型变量 i 赋值
    b[2]=1'b1;     //对 8 位寄存器 b 的第 3 位赋值
    b[0:3]=4'b1111; //对 8 位寄存器 b 的前 4 位赋值
    mem_1[200]=8'hfd; //对存储器 mem_1 的第 201 个存储单元进行赋值
    {a, b}=9'b101110110; //对用连接运算符构成的一个整体进行赋值
end
```

过程赋值语句右端的赋值表达式可以是任何合法的表达式。比如：

```
reg c;
#10 c=a&(~b);
```

所给出的过程赋值语句中带有 10 个单位时间的延时控制，它表明“赋值表达式”的计算和赋值操作的执行将被延迟 10 个单位时间。而上例过程赋值语句中的赋值表达式是一个位运算表达式“a&(~b)”，它的取值将被赋给变量 c，该赋值表达式的取值是：先对 b 进行“按位取反”运算，然后将其结果与 a 进行“按位与”运算后得到的结果。

根据对被赋值变量产生作用的优先级差别，可以将过程赋值语句分为如下两种类型：

- 普通过程赋值语句(简称过程赋值语句)。
- 过程连续赋值语句。

其中普通过程赋值语句又可以根据赋值操作执行时的不同时序特点分成阻塞型过程赋值语句和非阻塞型过程赋值语句。



6.2.2 过程赋值的两种延时方式

各种过程赋值语句都可以带有时间控制, 过程赋值语句中可以出现延时控制形式的时间控制和事件控制形式的时间控制。根据时间控制部分在过程赋值语句中出现的位置, 可以把过程赋值语句中的时间控制方式分为如下两类:

- 外部时间控制方式。
- 内部时间控制方式。

1. 外部时间控制方式

如果时间控制部分出现在整个过程赋值语句的最左端, 也就是出现在赋值目标变量的左边, 那么这种时间控制方式就是外部时间控制方式。

在外部时间控制方式下, 过程赋值语句在仿真时是这样执行的: 仿真进程遇到这条带有时间控制的过程赋值语句后, 首先要延迟等待由时间控制部分指定的延时时间量, 或者是要等待到指定触发事件发生后, 才开始计算右端的赋值表达式并将其取值赋给左端被赋值变量。比如语句:

```
#5 a=b;
```

在仿真执行时就相当于如下几条语句的执行:

```
initial
begin
    #5;      //先延时
    a=b;     //再求 b 值并将其赋值给 a
end
```

可见, 在外部时间控制方式下, 时间控制所“控制”的是过程赋值语句开始执行时刻到赋值表达式被计算以及赋值操作执行时刻之间的时间差。赋值表达式的计算以及对左端变量的赋值操作是发生在由时间控制部分所指定的时刻之后的。

【例 6-16】外部时间控制方式。

```
module timing_control_out_mode(b, c, d, clk, enable);
input clk, enable;
output b, c, d;
reg b, c, d;
initial          //此过程块用于对输出进行初始化
fork
    b=0;
    c=0;
    d=0;
join
initial          //此过程块用于对输出进行赋值, 其中包含了外部时间控制
fork
    #5 b=~b;      //语句 S1
    @(posedge clk) c=~c; //语句 S2
```




```

wait (enable) d=~d;      //语句 S3

join
endmodule

```

例 6-16 中的第二个 `initial` 过程块内出现了三条过程赋值语句，这三条过程赋值语句都带有外部时间控制形式的时间控制。其中第一条过程赋值语句（语句 S1）的时间控制采用了延时控制方式，第二条过程赋值语句（语句 S2）的时间控制采用了边沿触发事件控制方式，第三条过程赋值语句（语句 S3）采用了电平敏感事件控制方式。

图 6.2 给出了例 6-16 所示模块在仿真执行时产生的波形图。

在仿真进程进入第二个 `initial` 过程块后，仿真进程将同时进入三条赋值语句 S1、S2 和 S3，三条赋值语句同时开始得到执行（在 $t=0$ 时刻），而赋值操作真正得到执行的时刻是由各条语句中的时间控制部分所决定的：

对于语句 S1，仿真进程要在指定的 5 个单位的延时量过去后（ $t=5$ 时刻）才计算“ $\sim b$ ”的取值并将其结果赋给 b 。

对于语句 S2，仿真进程要等到指定的触发事件发生时，也就是 `clk` 发生正跳变时（ $t=20$ ）才计算“ $\sim c$ ”的取值并将其赋给 c 。

对于语句 S3，仿真进程要在指定的触发条件得到满足时，也就是信号 `enable` 为高电平时（ $t=35$ ）才计算“ $\sim d$ ”的取值并将其赋给 d 。

在外部时间控制方式下还有一种特殊情况，那就是当外部时间控制所指定的延时时间为零时的情况，我们称这种情况下的时间控制为“显式零延时控制”。

比如如下这条过程赋值语句内就是一条带有外部显式零延时控制的过程赋值语句：

```
#0 a=b;
```

上面这条语句的执行过程与不具有延时控制的赋值语句“`a=b;`”的执行过程是很相似的：在语句开始执行后，它们都是首先计算表达式 b 的取值，再对变量 a 进行赋值；同时两条语句的开始执行时刻、表达式计算时刻和变量赋值时刻都是相同的。

但是，上面这条语句在执行时序上还是与语句“`a=b;`”的执行时序存在着微妙的差别：在显示零延时控制下，赋值语句内赋值表达式的计算以及变量的赋值操作是在当前仿真时刻的末尾进行的，也就是说要等到这一时刻其它正常操作完成后才进行的。而赋值语句“`a=b;`”内的表达式计算和变量赋值操作则是在赋值语句开始执行后立即进行的，并不需要等待同一时刻的其它操作完成后才进行。比如如下语句：

```

initial
begin
    a = 0;      //语句 S1
    b = 0;      //语句 S2
end

```

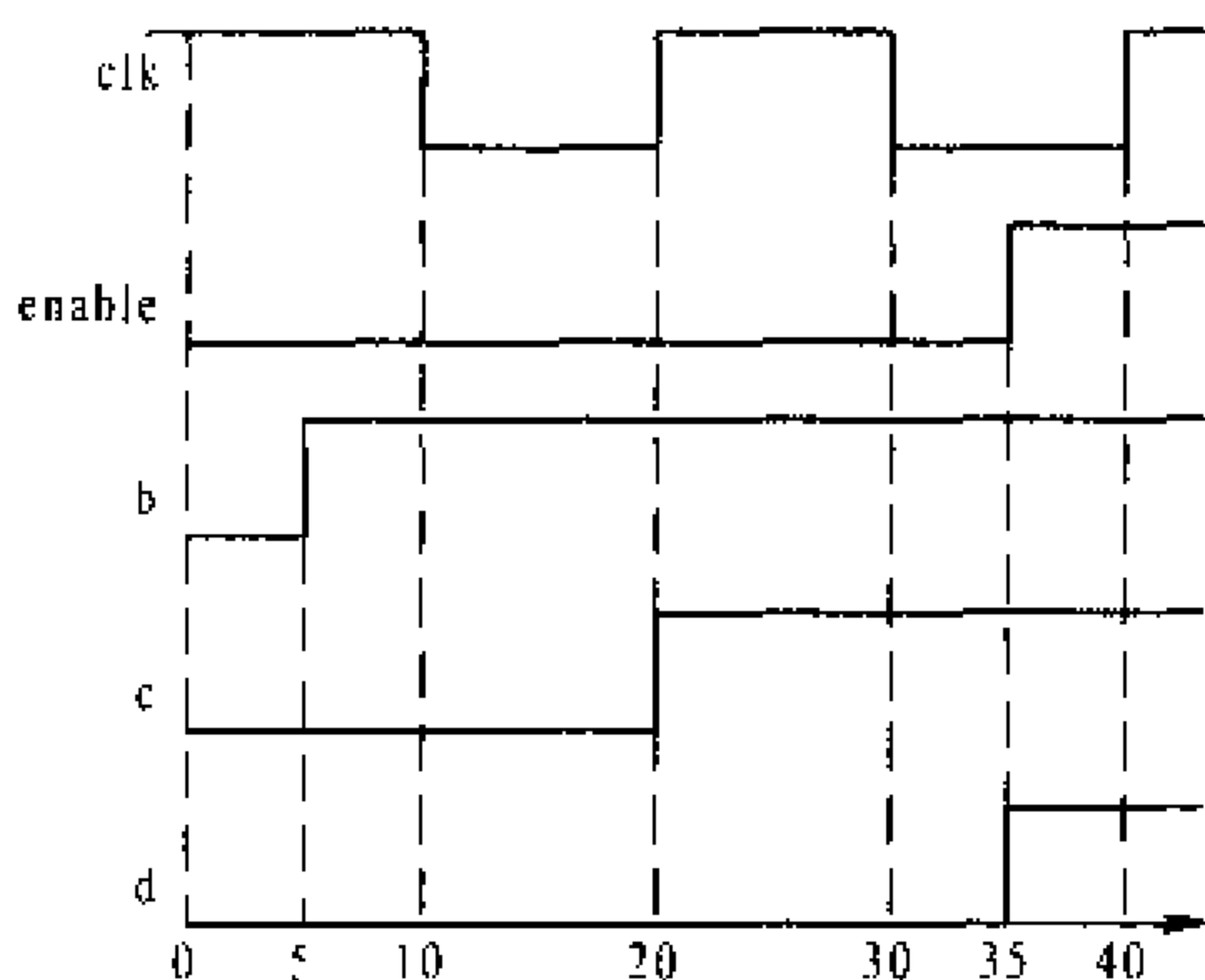


图 6.2 例 6-16 模块在进行仿真时的输入输出波形



```
initial
begin
    #0 a = 1;      //语句 S3
    #0 b = 1;      //语句 S4
end
```

其中的四个赋值语句对应的赋值操作都要在仿真时刻 0 得到执行。但是由于后两条赋值语句 S3 和 S4 的左端加上了显式零延时控制“#0”，所以 S3, S4 这两条赋值语句对应的赋值操作是在 S1, S2 这两条普通赋值语句对应的赋值操作执行完毕后才开始执行的。因此上面这段语句的执行过程将是：先给 a, b 赋值“0”，然后在仿真时刻 0 的末尾执行对 a, b 赋值“1”的操作，a, b 的最终取值将都是“1”。

由上面的例子可见，显式零延时控制提供了一种对语句执行顺序进行控制的有效的方法。

2. 内部时间控制方式

除了外部时间控制方式以外，过程赋值语句中的时间控制部分还可以出现在“赋值操作符”和“赋值表达式”之间。这种情况下的时间控制称为“内部时间控制方式”。

在内部时间控制方式下，过程赋值语句是这样执行的：仿真进程遇到带有内部时间控制的过程赋值语句后，立即计算赋值语句中“赋值表达式”的值，然后进入时间控制部分指定的等待状态，一直等到指定的延时时间量过去之后或者是指定的触发事件得到发生之后再将其赋值表达式的取值赋给左端的被赋值变量。比如语句：

```
a = #5 b;
```

中的时间控制部分“#5”就出现在赋值操作符“=”和赋值表达式“b”的中间，因此在这条过程赋值语句内带有内部时间控制方式的时间控制。它在执行时就相当于如下几条语句的执行：

```
initial
begin
    temp=b;      //先求 b 的值
    #5;          //再延时
    a=temp;      //延时结束后再进行赋值操作
end
```

可见，在内部时间控制方式下，时间控制所“控制”的是赋值表达式被计算时刻到赋值操作被执行时刻之间的时间差，赋值表达式被计算时刻和赋值变量被赋值的时刻是不同的，它们之间的时间差是由时间控制部分决定的。

【例 6-17】 过程赋值语句中的内部时间控制方式。

```
module timing_control_inner_mode(a, b, c, d, clk, enable);
input a, clk, enable;
output b, c, d;
reg b, c, d;
initial          //此过程块用于对输出进行初始化
fork
```




```

    b=0;
    c=0;
    d=0;
join
initial      //此模块用于对输出进行赋值，其中包含了内部时间控制
fork
    b=#5 a;          //语句 S1
    c=@(posedge clk) a ;    //语句 S2
    d= wait (enable) a ;    //语句 S3
join
endmodule

```

例 6-17 中第二个 `initial` 过程块内出现的三条过程赋值语句都带有内部方式的时间控制。图 6.3 给出了例 6-17 所示模块在执行时的输入输出波形。

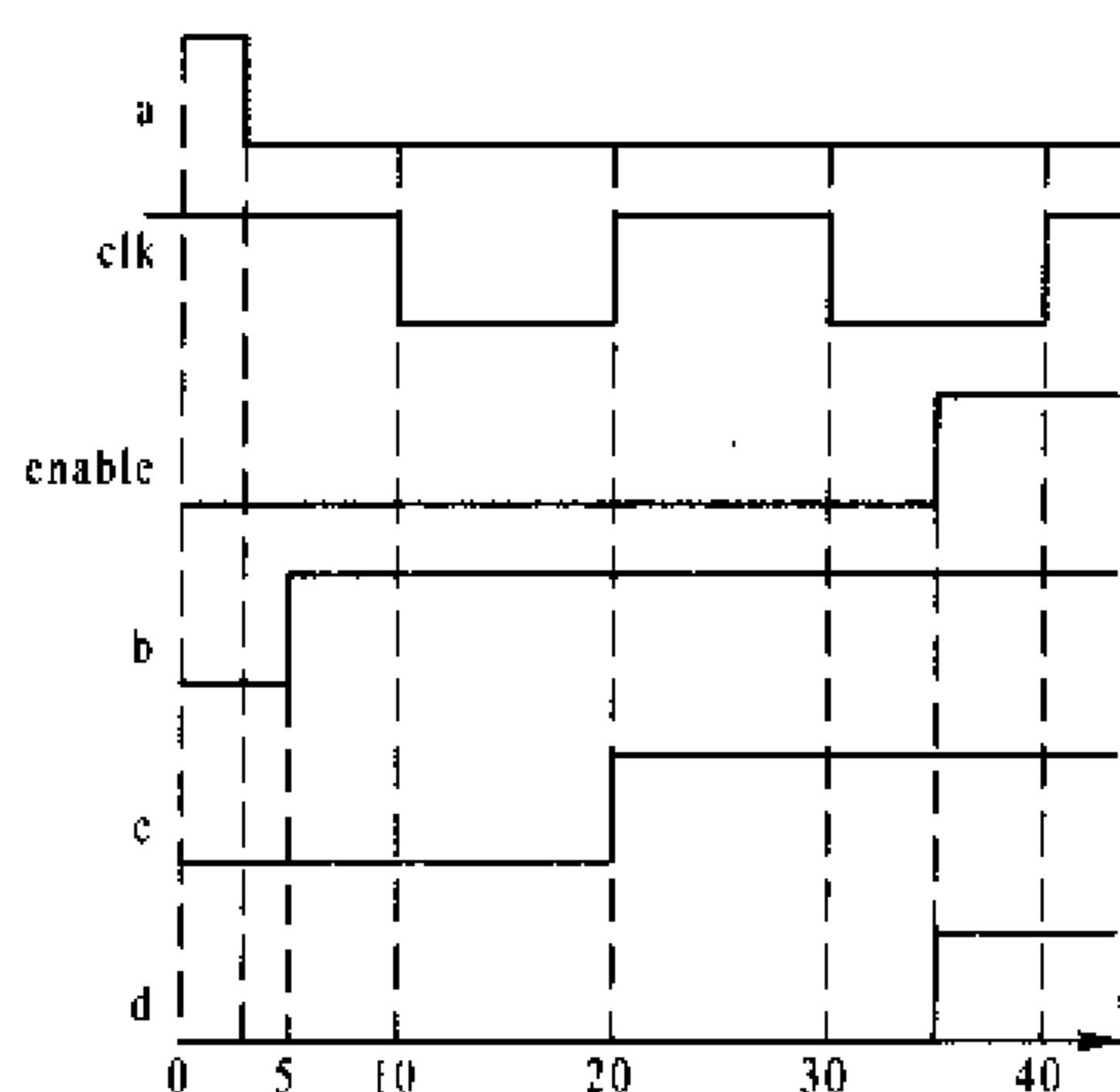


图 6.3 例 6-17 所示模块在进行仿真时的输入输出波形

由图 6.3 可以看到，在如图所示的输入信号 (a、clk 和 enable) 波形下，仿真进程进入模块中第二个 `initial` 过程块后，仿真进程同时遇到三条赋值语句 S1、S2 和 S3，这三条赋值语句同时开始执行 (在 $t=0$ 时刻)；由于 S1、S2 和 S3 都是内部时间控制方式的过程赋值语句，所以三条赋值语句中赋值表达式 “a” 的值都是在赋值语句开始执行后 ($t=0$ 时刻) 立即被计算的，其计算结果是输入信号 a 在 $t=0$ 时刻的取值 “1”；而 S1、S2 和 S3 所对应赋值操作得到执行的时刻则是由各条语句中的时间控制部分决定的：

对于语句 S1，仿真进程要在指定的 5 个单位的时间延迟量过去后 ($t=5$ 时刻) 才将已经计算得到的赋值表达式 “a” 的值 “1” 赋给 b。

对于语句 S2，仿真进程要在指定的触发事件发生时，也就是在 clk 发生正跳变时 ($t=20$ 时刻) 才将已计算得到的赋值表达式 “a” 的值 “1” 赋给 c。

对于语句 S3，仿真进程要在指定的触发条件得到满足时，也就是在信号 enable 为高电平时 ($t=35$ 时刻) 才将已计算得到的赋值表达式 “a” 的值 “1” 赋给 d。



由此可以看出: 尽管输入信号 a 的取值在赋值操作执行时刻已经变为“0”了, 但是由于赋值表达式“ a ”的计算是在 $t=0$ 时刻进行的, 所以经过赋值操作后变量 b 、 c 、 d 的最终取值都等于信号 a 在 $t=0$ 时刻的取值“1”。

6.2.3 阻塞型过程赋值

以赋值操作符“ $=$ ”来标识的赋值操作称为“阻塞型过程赋值(Blocking Assignment)”。在前面几个小节给出的例子内用到的赋值语句都属于阻塞型过程赋值语句。阻塞型过程赋值语句的特点如下:

(1) 串行块(begin-end 语句块)中的各条阻塞型过程赋值语句将以它们在顺序块中的先后排列次序依次得到执行; 而 fork-join 并行块中的各条阻塞型过程赋值语句则是同时得到执行的。

(2) 阻塞型过程赋值语句的执行过程是: 首先计算右端赋值表达式的取值, 然后立即将计算结果赋值给“ $=$ ”左端的被赋值变量。

阻塞型过程赋值的这两个特点表明: 仿真进程在遇到阻塞型过程赋值语句时将计算表达式的值并立即将其结果赋给等式左边的被赋值变量; 在串行语句块中, 下一条语句的执行会被本条阻塞型过程赋值语句所阻塞, 只有在当前这条阻塞型过程赋值语句所对应的赋值操作执行完毕后下一条语句才能开始执行。

比如如下程序:

```
initial
begin
    a=0;    //语句 S1
    a=1;    //语句 S2
end
```

中包含了两条阻塞型过程赋值语句 $S1$ 和 $S2$, 它们都是在仿真零时刻得到执行的, 其对应的赋值操作也都是在 0 时刻进行的。但由于它们是阻塞型赋值语句, 所以在执行 $S1$ 语句时 $S2$ 被“阻塞”而不能得到执行; 只有在 $S1$ 执行完毕、 a 被赋值“0”之后, $S2$ 才能开始执行。而 $S2$ 的执行将使 a 被重新赋值“1”, 所以上面这个过程块执行之后变量 a 的最终取值为“1”。

【例 6-18】阻塞型过程赋值语句带有延时控制时的情况。

```
initial
begin
    a=0;           //语句 S1
    a=#5 1;        //语句 S2
    a=#10 0;       //语句 S3
    a=#15 1;       //语句 S4
end
```

例 6-18 中的各条语句将依次得到执行, 并且在前一条语句所指定的赋值操作没有完成之前下一条语句不会开始执行。因此第一条语句 $S1$ 在 0 时刻开始执行, 对应的赋值操作在 0 时刻完成, a 被赋值“0”; 第二条语句 $S2$ 在 0 时刻开始执行, 对应的赋值操作在 t



$t=5$ 时刻完成, a 被赋值“1”; 第三条语句 $S3$ 在 $t=5$ 时刻开始执行, 对应的赋值操作在 $t=15$ 时刻完成, a 被赋值“0”; 第四条语句 $S4$ 在 $t=15$ 时刻开始执行, 对应的赋值操作在 $t=30$ 时刻完成, a 被赋值“1”; 整个 `initial` 过程块执行结束。

图 6.4 给出了例 6-18 所示模块仿真执行时输出信号 a 的波形。

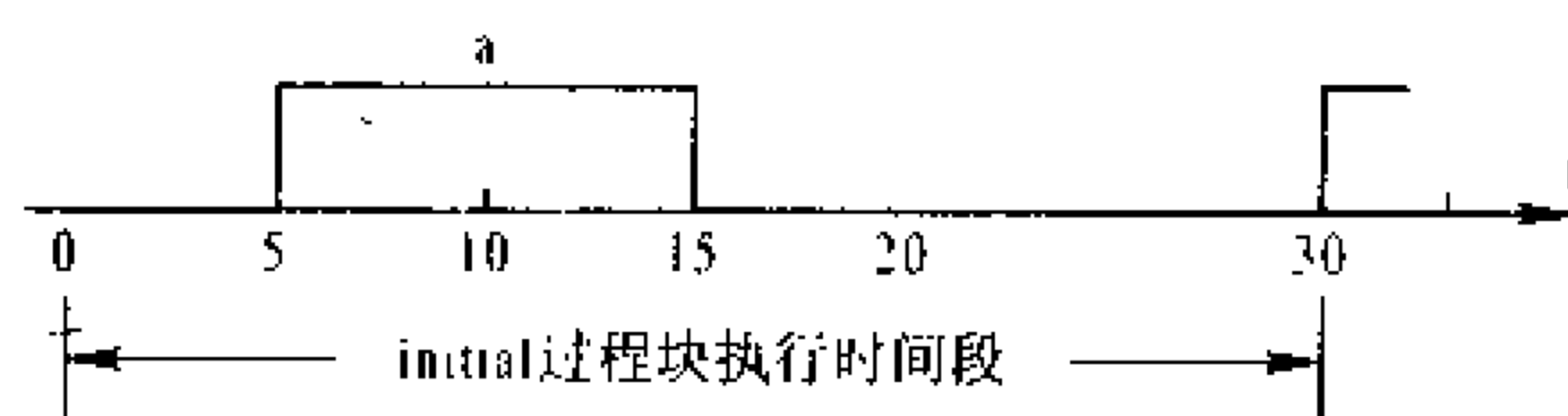


图 6.4 带有延时控制的阻塞型过程赋值产生的波形

6.2.4 非阻塞型过程赋值

以赋值操作符“ $\leq$ ”来标识的赋值操作称为“非阻塞型过程赋值(Nonblocking Assignment)”。非阻塞型过程赋值语句的特点是:

(1) 在 `begin-end` 串行语句块中, 一条非阻塞型过程赋值语句的执行不会阻塞下一条语句的执行, 也就是说在本条非阻塞型过程赋值语句对应的赋值操作执行完毕之前, 下一条语句也可以开始执行。

(2) 仿真进程在遇到非阻塞型过程赋值语句后首先计算其右端赋值表达式的值, 然后要等到当前仿真时间步结束时再将该计算结果赋值给赋值变量。也就是说, 这种情况下的赋值操作是在同一仿真时刻上的其它普通操作结束之后才得到执行的。“非阻塞型过程赋值语句”的这个特点是不同于“阻塞型过程赋值语句”的执行时序特点的。

为了了解非阻塞型过程赋值语句的上述两个特点, 我们可以观察如下语句:

```
initial
begin
    A <= B;      //语句 S1
    B <= A;      //语句 S2
end
```

上述语句中包含了两条非阻塞型过程赋值语句 $S1$ 和 $S2$, 当仿真进程遇到 `initial` 过程块后 (0 时刻), 语句 $S1$ 首先开始执行, 赋值表达式“ B ”的值得到计算 (但是对被赋值变量 A 的赋值操作要等到当前时间步结束时才执行), 同时由于 $S1$ 是一条非阻塞型赋值语句, 所以 $S1$ 的执行不会阻塞其下一条语句 $S2$ 的执行; 于是 $S2$ 也随即开始执行, 其对应的赋值表达式“ A ”的值得到计算, 由于这时 $S1$ 对 A 的赋值操作还没有执行, 所以此时计算得到的赋值表达式取值是 A 的初值。由于 $S2$ 也是一条非阻塞型赋值语句, 它对应的为变量 B 进行赋值的赋值操作也要等到当前时间步结束时才得到执行; 所以在当前时间步结束时, $S1$ 、 $S2$ 两条语句对应的赋值操作同时执行, 分别将已经计算得到的 A 和 B 的初值赋给变量 B 和 A , 这样就交换了 A 与 B 的取值。

如果非阻塞型过程赋值语句带有外部延时控制, 则它的执行相当于在指定的延时之后执行一条不带时间控制的非阻塞型过程赋值语句。并且这条非阻塞型过程赋值语句不会阻塞其后面语句的执行。

【例 6-19】带有外部延时控制的非阻塞型过程赋值语句。

```
initial          //过程块 1
begin
    #5 a <= b;    //语句 S1
    #10 c <= d;   //语句 S2
end
```

上面的过程块 1 相当于如下的过程块 2:

```
initial          //过程块 2
fork
begin            //语句块 1
    #5;
    a<=b;
end
begin            //语句块 2
    #10;
    c<=d;
end
join
```

上例中的“过程块 2”等价于“过程块 1”。其中，“语句块 1”和“语句块 2”分别等价于赋值语句 S1 和赋值语句 S2。“语句块 1”和“语句块 2”处于同一个 fork-join 并行块内，这表明了语句 S1 的执行不会阻塞语句 S2 的执行。而语句 S1 和 S2 的执行过程都是先延时等待然后再执行对应的赋值表达式计算和赋值操作。

所以上述“过程块 1”的执行流程将是：当仿真进程遇到 initial 过程块后 (0 时刻)，语句 S1 和语句 S2 同时开始执行。S1 的执行过程是在 5 个单位时间的延时后 (t=5 时刻) 计算赋值表达式“b”的值并将其结果赋给赋值变量 a，而 S2 的执行过程是在 10 个单位时间的延时后 (t=10 时刻) 计算赋值表达式“d”的值并将其结果赋给赋值变量 c。上例过程块的执行时序如图 6.5 所示。

如果非阻塞型过程赋值语句带有内部延时控制，则仿真进程在遇到这种赋值语句后将立即计算语句右端赋值表达式的值，然后进入延时控制部分所指定的等待状态，直到退出等待状态时刻所在时间步的末尾才执行赋值操作，将已计算得到的赋值表达式结果值赋给被赋值变量。例如如下语句：

```
a<=#10 b;
```

就是一条带有内部延时控制的非阻塞型过程赋值语句。在开始执行这条语句后，首先立即计算赋值表达式“b”的值 (在 t=0 时刻)，然后等待 10 个单位时间的延时量，一直等到延时结束时刻 (t=10 时刻) 所在时间步的末尾才将计算得到的赋值表达式取值结果赋给变量 a。

我们再举一个例子来说明非阻塞型过程赋值语句带有延时控制的情况。

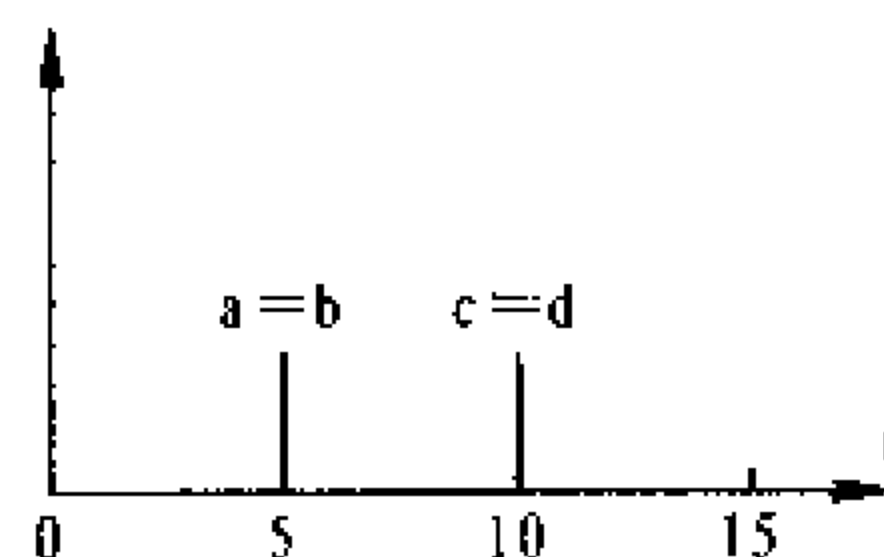


图 6.5 带有外部延时控制的非阻塞型过程赋值语句的执行时序



【例 6-20】非阻塞型过程赋值语句带有延时控制的情况。

```
initial
begin
    a <= 0;           //语句 S1
    a <= #5 1;        //语句 S2
    #10 a <= 0;       //语句 S3
    a <= #25 1;       //语句 S4
end
```

在例 6-20 内的各条非阻塞型过程赋值语句中，有的带有外部延时控制，有的带有内部延时控制，还有的没有延时控制。这些非阻塞型过程赋值语句都处于同一个 begin-end 串行块内，所以它们将依次开始得到执行：

(1) 语句 S1 在 0 时刻最先开始执行，由于 S1 是非阻塞型赋值语句，它的执行不会阻塞下一条语句 S2 的执行，所以语句 S2 也随即开始执行 (在 $t=0$ 时刻)；同时由于 S1 中没有延时控制部分，所以语句 S1 对应的赋值操作在 $t=0$ 时间步结束时得到执行，变量 a 被赋值“0”。

(2) 语句 S2 在开始执行后 (0 时刻)，由于它也是一条非阻塞型赋值语句，它的执行不会阻塞下一条语句 S3 的执行，所以语句 S3 也随即开始执行 ($t=0$ 时刻)。又由于 S2 中带有内部延时控制，所以语句 S2 所对应的赋值操作将在延时结束时刻 ($t=5$ 时刻) 所在时间步的末尾得到执行，变量 a 被赋值“1”。

(3) 语句 S3 在开始执行后 (0 时刻)，由于这条语句也是一条非阻塞型赋值语句，它的执行不会阻塞下一条语句 S4 的执行，所以语句 S4 也随即开始执行 (在 $t=0$ 时刻)。又由于 S3 带有外部延时控制，所以它的执行过程是先进入延时状态，等到延时结束后 ($t=10$ 时刻) 才开始语句“a <= 0”的执行，在 $t=10$ 时间步的末尾执行赋值操作，将值“0”赋给赋值变量 a。

(4) 语句 S4 在 $t=0$ 时刻开始执行后，由于带有内部延时控制，所以语句 S4 对应的赋值操作将在延时结束时刻 ($t=25$ 时刻) 所在时间步的末尾得到执行，a 被赋值“1”。

图 6.6 给出了上例过程块仿真执行时变量 a 的波形。

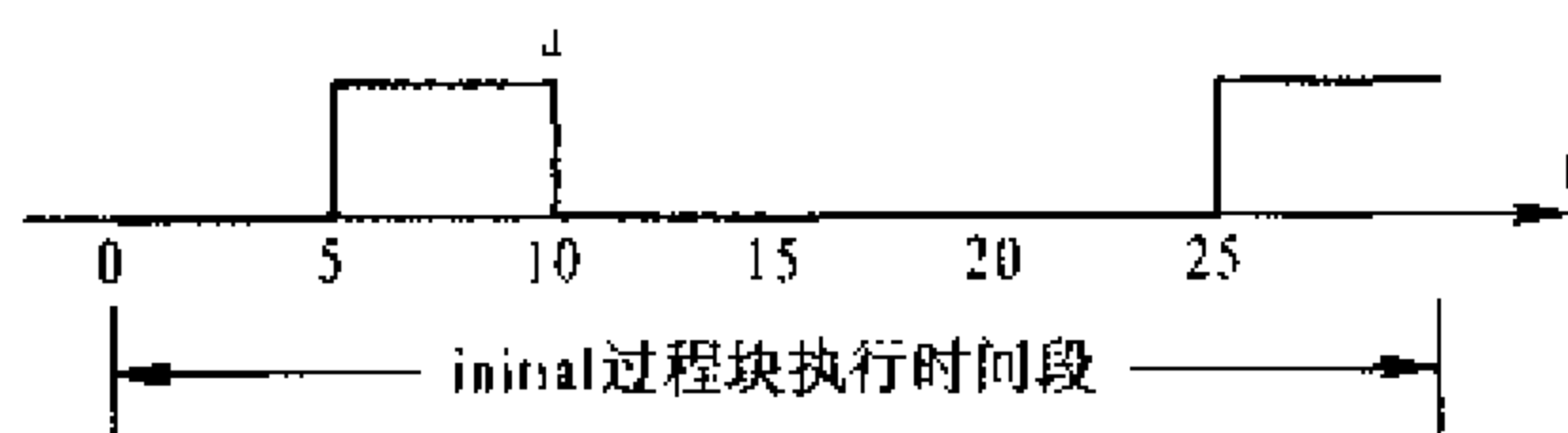


图 6.6 带有延时控制的非阻塞型过程赋值产生的波形

下面的例 6-21 是同时使用了阻塞型过程赋值语句和非阻塞型过程赋值语句的一个例子，注意两种赋值方式的区别。

【例 6-21】同时使用阻塞型过程赋值语句和非阻塞型过程赋值语句。

```
reg a, b, c;
initial
begin
```



```

a=0;    //语句 S1
a<=1;   //语句 S2
b=a;    //语句 S3
#5;     //等待一定的时延
c=a;    //语句 S4
end

```

为了确定例 6-21 内 initial 过程块执行后变量 a, b, c 的取值情况, 我们必须对其中各条赋值语句所指定的赋值操作的执行时序进行分析, 从而来判断各个变量的取值变化情况。

initial 过程块开始执行后, 语句 S1 首先得到执行, 由于它是一条阻塞型赋值语句, 所以它对应的赋值操作立即执行, 变量 a 被赋值“0”;

语句 S2 在语句 S1 执行完毕后开始执行, 由于 S2 是一条非阻塞型赋值语句, 因此它的下一条语句 S3 也随即开始执行, 并且 S2 所对应的赋值操作将在本时间步的结束时刻得到执行;

语句 S3 与语句 S2 同时开始执行, 在语句开始执行后立即计算表达式“a”的值, 因为此时还未到当前时间步的末尾, 由语句 S2 指定的给 a 赋值“1”的操作还未得到执行, 因此此时变量 a 的取值仍为“0”, 所以语句 S3 对应的赋值操作是将此刻 a 的取值“0”赋给变量 b;

然后在当前时间步 (0 时间步) 结束时, S2 对应的赋值操作得到执行, 变量 a 被赋值“1”;

随后仿真进程进入由延时控制语句“#5;”指定的延时等待状态; 在 5 个单位时间的等待过程结束后 (t=5 时刻), 语句 S4 开始执行, 其对应的操作是将此刻 a 的取值“1”赋给变量 c;

在语句 S4 执行结束后, initial 过程块的执行随之结束。

这样在整个 initial 过程块执行完毕后, 变量 a 的取值将是“1”, 变量 b 的取值是“0”, 变量 c 的取值将是“1”。

6.2.5 连续赋值语句

Verilog HDL 语言中的连续赋值语句与过程块一样也是一种行为描述语句 (在有的文献中将连续赋值语句归类于一种新的“数据流描述方式”, 而我们在这里则将其视为一种行为描述语句)。它主要用来对组合逻辑电路的行为进行描述。

连续赋值语句只能用来对连线型变量进行驱动 (赋值), 而不能对寄存器型变量进行赋值, 它可以采取如下两种格式:

(1) 显式连续赋值语句:

连线型变量类型 [连线型变量位宽] 连线型变量名;

assign #([延时量]) 连线型变量名 = 赋值表达式;

(2) 隐式连续赋值语句:

连线型变量类型 (赋值驱动强度) [连线型变量位宽] #([延时量]) 连线型变量名 = 赋值表达式;



第一种格式的连续赋值语句称为“显式连续赋值语句”。它包含了两条语句：第一条语句是对连线型变量进行类型说明的说明语句；第二条语句是对这个已得到声明的连线型变量进行连续赋值的赋值语句。

第二种格式的连续赋值语句称为“隐式连续赋值语句”。隐式连续赋值语句把连线型变量的说明语句以及对该连线型变量进行连续赋值的赋值语句结合到同一条语句内。利用它可以在对连线型变量进行类型说明的同时实现连续赋值。

在上述两种格式中：

关键词“assign”是连续赋值语句的标识。

“连线型变量类型”可以是除了 trireg 类型外的任何一种连线型数据类型。

“(赋值驱动强度)”是可选的，它只能在“隐式连续赋值语句”格式中得到指定。它用来对连线型变量受到的驱动强度进行指定。它是由“对 1 驱动强度”和“对 0 驱动强度”两项组成的，比如语句“wire (weak1, strong0) a = b&c ;”内的“(weak1, strong0)”就表示该语句指定的连续赋值对连线型变量“a”的驱动强度是：赋“1”值时的驱动强度为“弱(weak)”，而赋“0”值时的驱动强度为“强(strong)”。如果在格式中缺省了“赋值驱动强度”这一项，则驱动强度默认为(strong1, strong0)。

“延时量”这一项也是可选的。它指定了由赋值表达式内信号发生变化时刻到连线型变量取值被更新时刻之间的延迟时间量。它类似于基本元件原语实例定义时指定的延时量。

“延时量”的基本格式如下：

#(delay1, delay2, delay3)

其中 delay1, delay2 和 delay3 都是一个数值，其中的“delay1”指明了连线型变量转移到“1”状态时的延时值(称为上升延时)；“delay2”指明了连线型变量转移到“0”状态时的延时值(称为下降延时)；“delay3”指明了连线型变量转移到“高阻(z)”状态时的延时值(称为关断延时)。

在实际使用中“延时量”也可以省略为由一个或二个延时值构成，这种情况下三种延时值(“上升延时值”、“下降延时值”和“关断延时值”)的确定准则是：

- 若只给出一个延时值，则这个延时值将同时代表“上升延时值”、“下降延时值”和“关断延时值”。
- 如果只给出了两个延时值，那么这两个延时值将分别代表“上升延时值”和“下降延时值”，而“关断延时值”将由给出的两个延时值中较小的那一个指定。
- 如果“延时量”这一项缺省，则默认所有的延时值都为 0。

“赋值表达式”内可以包含连线型、寄存器或函数调用等任何数据类型的操作数，同时也可以包含任何操作符。

例 6-22 给出了一个显式连续赋值语句的例子。

【例 6-22】显式连续赋值语句。

```
module and_cont_assignment (z, x, y);  
input [3:0] x, y;  
output [3:0] z;  
wire [3:0] z, x, y;                                // 连线型变量说明
```




```
assign #(1.5, 1.0, 2.0) z = x & y;      //连续赋值语句  
endmodule
```

例 6-22 中连续赋值语句的赋值目标是连线型变量 z , 它的位宽为 4 位。连续赋值语句指定用表达式 “ $x \& y$ ” 的取值对连线型变量 z 进行连续驱动。其中指定的延时量为 “(1.5, 1.0, 2.0)”, 它指明了从信号 x 或 y 发生变化时刻到变量 z 被更新时刻之间的延时时间量: 上升延时为 1.5 个时间单位, 下降延时为 1 个时间单位, 关断延时为 2 个时间单位。连续赋值语句中并未出现 “驱动强度” 这一项, 所以连线型变量 z 受到的驱动强度是缺省的 “(strong1, strong0)”。

例 6-23 则给出了一个隐式连续赋值语句的例子, 它描述了与例 6-22 相同的硬件行为。

【例 6-23】隐式连续赋值语句。

```
module and_cont_assignment(z, x, y);  
input [3:0] x, y;  
output [3:0] z;  
wire [3:0] x, y;      // 输入连线型变量说明  
wire [3:0] #(1.5, 1.0, 2.0) z = x & y;  //隐式连续赋值语句  
endmodule
```

在实际使用中, 连续赋值语句的赋值目标可以是如下几种:

(1) 标量连线型变量。如

```
wire a, b;  
assign a = b;
```

(2) 向量连线型变量。如

```
wire [7:0] a, b;  
assign a = b;
```

(3) 向量连线型变量中的某一位。如

```
wire [7:0] a, b;  
assign a[3] = b[3];
```

(4) 向量连线型变量中的某几位。如:

```
wire [7:0] a, b;  
assign a[3:2] = b[3:2];
```

(5) 上面几种类型的任意拼接运算结果。如

```
wire a, c;  
wire [2:1] b;  
assign {a, c} = b;
```

在需要对多个连线型变量进行连续赋值时, 还可以采用如下形式的连续赋值语句, 它可以实现对多个变量进行连续赋值:

```
assign out_and = a&b;  
out_or = a|b;  
out_xor = a^b;
```




```
out_not_a = ~a;
```

```
out_not_b = ~b;
```

上述语句是下面 5 条独立的连续赋值语句的简化形式:

```
assign out_and = a&b;
```

```
assign out_or = a|b;
```

```
assign out_xor = a^b;
```

```
assign out_not_a = ~a;
```

```
assign out_not_b = ~b;
```

正如“连续赋值”这一名称所表述的含义,连续赋值语句对连线型变量(wire)进行的是连续的驱动。

与寄存器型变量类型不同,连线型变量(除了 trireg 类型外)没有数据保持能力,只有在被连续驱动后才能取得确定值(而寄存器型变量只要在某时刻得到一次过程赋值后就能一直保持其取值),若一个连线型变量没有得到任何连续驱动则它的取值将为不定态“x”。连续赋值就是实现对连线型变量进行连续驱动的一种方法(还有一种进行连续驱动的方法是由某个模块输出端来对连线型变量进行驱动,这归类于结构描述方式)。

一个连线型变量一旦被连续赋值语句赋值后,赋值语句右端赋值表达式的值将始终对被赋值连线型变量产生驱动(连续驱动)。在仿真执行时,只要右端赋值表达式内的任一操作数发生变化,就会立即触发对被赋值连线型变量的更新操作:重新计算赋值表达式的取值,然后将计算得到的结果赋值给被赋值连线型变量。也就是说赋值表达式内各个信号的变化将随时被反映到赋值表达式和被赋值变量的取值上。比如例 6-24 所示的模块实现了用输入信号 a、b 的“按位或”运算结果对连线型变量 c 进行连续驱动。a 或 b 在任意时刻发生的变化将立即被反映到变量 c 的取值上。例 6-24 所示模块和例 6-25 所示的用过程赋值语句描述的模块是等价的。

【例 6-24】连续赋值语句的例子。

```
module or_by_cont_assignment(c, a, b);  
input a, b;  
output c;  
assign c=a|b;  
endmodule
```

【例 6-25】与例 6-24 等价的描述。

```
module or_by_proc_assignment(c, a, b);  
input a, b;  
output c;  
reg c;  
initial           //initial 过程块用于对 c 赋初值  
begin  
    c=x;  
end  
always @(a or b)  //always 过程块用于更新 c 的取值
```



```
c=alb;
endmodule
```

在仿真执行时, 例 6-25 所示模块中的 always 过程块将对输入 a 和 b 进行监测, 每当 a 或 b 的取值发生变化时, 过程赋值语句都将通过 always 的循环控制, 启动一次对 c 的赋值操作过程; 而当 a 和 b 的取值没有发生变化时, always 过程块内的语句得不到执行, 输出 c 将保持原有取值。所以例 6-25 描述的硬件行为特征是与例 6-24 所描述的行为特征相一致的。但这种描述方式显然没有例 6-24 中的描述那么简单明了。

如果连续赋值语句带有延时(#delay), 则在仿真执行时, 只要右端赋值表达式中的任一信号发生变化, 都将立即对赋值表达式进行重新计算, 然后进入延时等待状态, 等到指定的延时时间过去后再将计算得到的赋值表达式取值结果赋值给被赋值连线型变量。

例如, 下面这条连续赋值语句:

```
assign #5 a = b;
```

指定的驱动方式就是: 当 b 的值发生变化时, 立即对赋值表达式“b”的取值进行计算 (也就是对 b 的取值进行采样, 但不是立即将其赋值给 a), 然后进入“#5”指定的延时等待状态, 直到过了 5 个时间单位的延时时间后 (相对于 b 值变化时刻的 5 个单位时间的延时) 才对 a 执行赋值操作, 赋值表达式取值的计算结果 (b 发生变化后的采样值) 赋值给 a。

图 6.7 给出了这条连续赋值语句在仿真执行时的时序图。

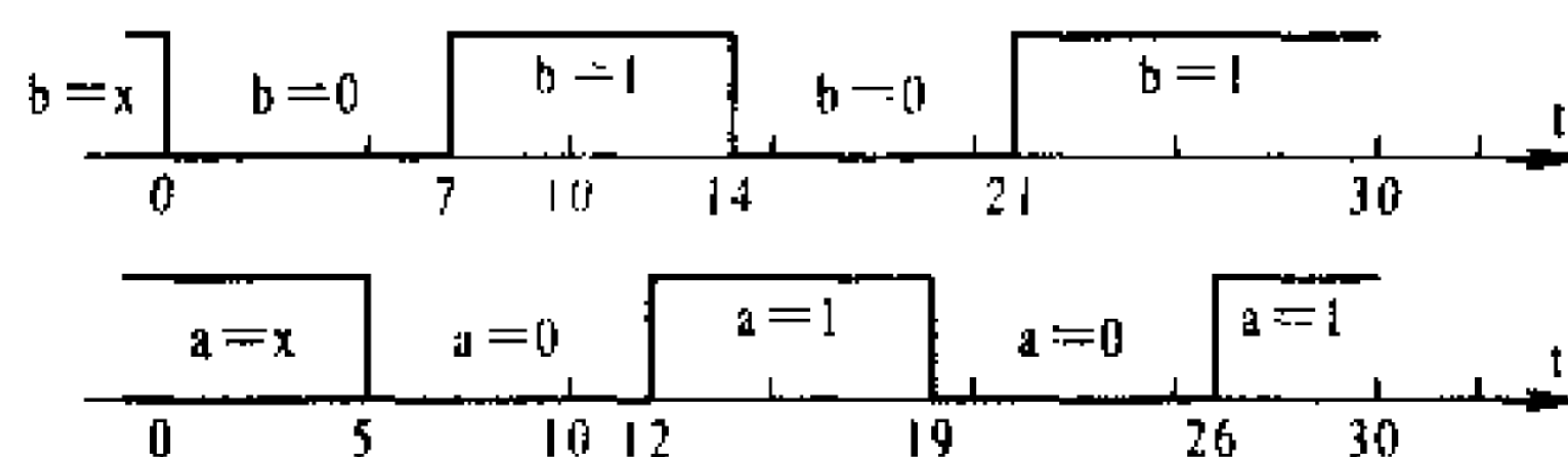


图 6.7 连续赋值语句“assign #5 a=b;”执行时的时序图

如果在一个模块内包含了多条连续赋值语句, 或者在一个模块中同时包含了连续赋值语句、过程块、模块实例或原语实例时, 各条连续赋值语句和其它的过程块、模块实例和原语实例之间是以并行方式执行的: 各条连续赋值语句、模块实例和原语实例都是同时在零仿真时刻产生作用, 并且过程块也是从零时刻开始执行的。

下面给出用连续赋值语句实现 4 位全加器的一个例子, 如例 6-26 所示。

【例 6-26】4 位全加器。

```
module adder_4(a, b, ci, sum, co);
input [3:0] a, b;
input ci;
output [3:0] sum;
output co;
assign {co, sum} = a + b + ci;    //连续赋值语句
endmodule
```

上例连续赋值语句内的被赋值目标是一个 4 位宽的组合变量“{co, sum}”, 它是由两个变量 co 和 sum 拼接而成的。这个组合变量的最高位就是变量 co 的取值, 低三位就是



变量 `sum` 的取值；连续赋值语句中的赋值表达式是“`a + b + ci`”，它是对输入信号 `a`、`b` 以及进位输入 `ci` 进行求和的一个求和表达式。因此这个求和表达式将对组合变量进行连续驱动；组合变量的低三位（变量 `sum`）将等于加法运算的本位和输出，而组合变量的最高位（变量 `co`）将等于加法运算的进位输出。

最后我们总结一下连续赋值语句和过程赋值语句之间的区别：

(1) 从语法上来看，连续赋值语句由“`assign`”关键词来标识，而过程赋值语句中则不包含这个关键词。

(2) 连续赋值语句左边的被赋值变量必须是连线类型的变量，而过程赋值语句中的被赋值变量则必须是寄存器类型的变量。

(3) 连续赋值语句不能出现在过程块（`initial` 过程块或 `always` 过程块）中，而过程赋值语句则只能出现在过程块内。

(4) 连续赋值语句主要用来对组合逻辑电路进行建模以及对连线型变量间的连接进行描述，而过程赋值语句则主要用来对时序逻辑电路进行行为描述。

(5) 连续赋值语句对被赋值变量（连线类型变量）的赋值是“连续”的（也就是说当连续赋值语句产生作用后，赋值表达式中信号的任何变化都将立即反映到被赋值变量的取值上，这也是我们有时会使用“连续驱动”这个术语的原因），而在过程赋值语句情况下，只有在过程赋值语句被执行时才执行赋值操作，语句执行完后被赋值变量的取值不再受到赋值表达式的影响。

6.2.6 过程连续赋值语句

与过程赋值语句一样，过程连续赋值语句也是一种过程性赋值语句，它用来实现过程连续赋值。正如它的名称所表示的那样，过程连续赋值是在过程块内对变量进行连续赋值的，而过程连续赋值语句就是用来实现过程连续赋值的过程性赋值语句。

过程连续赋值语句与连续赋值语句的不同之处在于：

- 过程连续赋值语句只能用在过程块（`initial` 过程块或 `always` 过程块）中，而连续赋值语句则不能出现在过程块中。

- 过程连续赋值语句可以对寄存器类变量进行连续赋值（`force-release` 语句组还可以对连线型变量进行连续赋值），它的赋值目标不可以是变量的某一位或者某几位，而连续赋值语句只能对连线型变量产生作用，它的赋值目标可以是变量的某一位或者某几位。

过程连续赋值语句执行的是—种“连续赋值”：一旦对某个变量进行了过程连续赋值，则该变量将一直受到过程连续赋值语句内“赋值表达式”的连续驱动，“赋值表达式”内操作数的任何变化都会引起被赋值变量取值的更新，直到对该变量执行了“撤销过程连续赋值”操作为止。

在 Verilog HDL 中有两组过程连续赋值语句可以实现过程连续赋值，它们是“`assign / deassign`”语句组和“`force / release`”语句组。

1. `assign` 与 `deassign`

“`assign` 语句”和“`deassign` 语句”构成了一组过程连续赋值语句。它们只能用于对寄存器类变量的连续赋值操作，而不能用来对连线类变量进行连续赋值操作。其中“`assign` 语句”用来实现对寄存器类变量的连续赋值，而“`deassign` 语句”则是一条撤销



连续赋值的语句, 它用来解除寄存器类变量上的、由 assign 语句实现的连续赋值状态。

(1) “assign 语句”的使用语法是:

assign <寄存器类变量> = <赋值表达式>;

其中“寄存器类变量”指明了连续赋值操作的目标变量, 而“赋值表达式”则指明了连续赋值的“驱动源”。

一旦 assign 语句得到执行, 寄存器类变量将由赋值表达式进行连续驱动, 它将进入被连续赋值的状态。如果这时有普通的过程赋值语句对该寄存器变量进行过程赋值操作, 由于过程连续赋值语句“assign”的优先级高于普通过程赋值语句, 所以处于连续赋值状态下的寄存器变量将忽视普通过程赋值语句对它的过程赋值操作, 它的逻辑状态仍然由过程连续赋值语句内的赋值表达式所决定。

如果先后有两条 assign 语句对同一寄存器变量进行了过程连续赋值, 那么第二条 assign 语句的执行将覆盖第一条 assign 语句的执行效果, 也就是说寄存器变量最终将进入由第二条 assign 语句实现的连续赋值状态。比如下面的语句:

```
reg out[3:0]
initial
begin
    out=0;           //S1, 过程赋值语句
    #10;
    assign out = a & b; //S2, 第一条过程连续赋值语句
    #10;
    assign out = a | b; //S3, 第二条过程连续赋值语句
end
```

上述语句的仿真执行过程是:

首先过程赋值语句 S1 得到执行, 在 t=0 时刻 out 被赋值 0 并保持这个取值。

在 t=10 时刻, 第一条过程连续赋值语句 S2 (第一条 assign 语句) 得到执行, 它实现了对变量 out 的连续赋值。所以从 t=10 时刻开始起 out 将处于被连续赋值的状态, 它将受到赋值表达式“a & b”的连续驱动, 信号 a 或 b 的任何变化都将触发 out 取值的更新操作。

在 t=20 时刻, 第二条过程连续赋值语句 S3 (第二条 assign 语句) 得到执行, S3 的执行将覆盖 S2 语句产生的作用, 所以从该时刻 (t=20 时刻) 起 out 将由赋值表达式“a | b”连续驱动。

(2) “deassign 语句”的使用语法是:

deassign <寄存器类变量>;

“deassign 语句”是一条撤销连续赋值语句。当它得到执行后, 原先由 assign 语句对变量进行的连续赋值操作将失效, 寄存器变量被连续赋值的状态将得到解除, 该变量又可以由普通的过程赋值语句进行赋值操作了。

需要注意的是: 当用 deassign 语句来撤销了对某个寄存器变量的连续赋值后, 该寄存器变量仍将保持 deassign 语句执行前的原有取值, 比如在如下语句:

```
reg[3: 0] out;
initial
```




```
begin
    out = 0;           //S1, 过程赋值语句
    assign out = 1;     //S2, 第一条过程连续赋值语句
    assign out = 2;     //S3, 第二条过程连续赋值语句
    deassign out;       //S4, 撤销连续赋值语句
end
```

的仿真执行过程中, 在语句 S4 得到执行之前, 变量 out 处于由第二条 assign 语句 (语句 S3) 实现的连续赋值状态, 其值被连续驱动为“2”; 当 deassign 语句 (语句 S4) 得到执行后, 变量 out 的连续赋值状态被解除, 其取值仍保持原值“2”。

例 6-27 给出了一个用 assign 语句和 deassign 语句来实现一个具有异步清零功能的下降沿 D 触发器的例子。

【例 6-27】具有异步清零输入的下降沿 D 触发器。

```
module dff_asyn_clear (q, d, clr, clk);
    output q;
    input d, clr, clk;
    reg q;
    always @(clr) //用于实现异步清零功能的 always 过程块
    begin
        if (!clr)
            assign q=0;
        else
            deassign q;
    end
    always @(negedge clk) //用于实现正常 D 触发器功能的 always 过程块
    begin
        q=d;
    end
endmodule
```

上例模块内的第一个 always 过程块描述了异步清零功能的实现, 这个 always 过程块是由清零控制信号 clr 的跳变触发执行的:

当输入 clr 由非零逻辑值变为零时, 过程连续赋值语句“assign 语句”得到执行, 变量 q 受到“0”的连续驱动, 因此 q 取值被强行置成“0”。从这时起一直到撤销连续赋值操作被执行以前, 由时钟 clk 下降沿启动的对 q 的过程赋值操作将被忽略 (这是由于: assign 过程连续赋值语句比普通的过程赋值语句“q=d;”具有更高的优先级), 输出信号 q 将一直保持取值“0”, 这样就实现了当 clr 为“1”时的清零功能。

当输入 clr 取值由零变为非零逻辑值时, 第一个 always 过程块中的“deassign 语句”得到执行, 它被用来解除对寄存器 q 的连续赋值, 因此寄存器 q 将结束由“0”连续驱动的状态 (结束对 q 端的强行清零控制), 此后当再有时钟下降沿出现时, 模块将实现 D 触发器的正常工作功能。



从例 6-27 中可以看出, 由于过程连续赋值语句比普通过程赋值语句具有更高的优先级, 当用一条 assign 语句对寄存器变量进行连续赋值以后, 过程赋值语句对该变量进行的过程赋值操作将不再起作用 (直到通过执行一条 “deassign 语句” 来撤销该寄存器的连续赋值状态以后)。所以过程连续赋值语句特别适合用来实现优先级高于正常时序逻辑功能的异步置位或异步清零逻辑功能。

2. force 与 release

“force 语句” 和 “release 语句” 也构成了一组过程连续赋值语句。这组过程连续赋值语句不仅能对寄存器类变量产生作用, 还能对连线型变量进行连续赋值操作。其中 “force 语句” 用来实现对寄存器或连线型变量的连续赋值, 我们称它为 “强制语句”。force 语句的优先级高于 “assign 语句”。“release 语句” 则类似于 “deassign 语句” 的作用, 它用来解除指定寄存器变量或连线型变量上的、由 “force 语句” 实现的连续赋值状态, 我们称它为 “释放语句”。

(1) “force 语句” 的使用语法如下:

force <寄存器或连线型变量> = <赋值表达式> ;

如果 force 语句内指定的被赋值目标变量是寄存器类变量, 则在 “force 语句” 得到执行后, 该寄存器类变量将强制地由 “<赋值表达式>” 进行连续驱动, 进入被连续赋值的状态, 这时将忽略其它较低优先级的赋值语句 (普通过程赋值语句或 assign 语句) 对这个寄存器变量的赋值操作, 直到执行了一条 “release 语句” 来释放对这个寄存器变量的连续赋值为止。比如例 6-28 中的过程块所示。

【例 6-28】force 语句的例子。

```
reg [2:0] var_reg1, var_reg2 ;
initial
begin
    var_reg1 = 0;           //S1, 过程赋值语句
    assign var_reg2 = 1;    //S2, 过程连续赋值语句, assign 语句
    #10;
    force var_reg1 = 4;     //S3, 过程连续赋值语句, force 语句
    force var_reg2 = 4;     //S4, 过程连续赋值语句, force 语句
end
```

上述语句实现了对两个寄存器变量 var_reg1 和 var_reg2 的过程连续赋值:

仿真执行过程开始后, 语句 S1 首先得到执行, 它用来实现对变量 “var_reg1” 的过程赋值操作, 变量 var_reg1 被赋值为 “0”。

随后语句 S2 得到执行, 它是一条过程连续赋值语句 (assign 语句), 用来实现对变量 “var_reg2” 的连续赋值, 这样语句 S2 得到执行后变量 var_reg2 将被连续赋值为 “1”。

在 t=10 时刻, 过程语句 S3 得到执行, 它也是一条过程连续赋值语句 (force 语句), 它实现对变量 var_reg1 的连续赋值操作, 这样在语句 S3 得到执行后变量 var_reg1 将被连续赋值为 “4”。

随后语句 S4 得到执行, 它也是一条 force 语句, 用来实现对变量 var_reg2 的连续赋值。由于语句 S4 执行之前变量 var_reg2 已经是处于由 “assign 语句” (语句 S2) 所实现的



连续赋值状态, 同时因为 force 语句的优先级高于 assign 语句的优先级, 所以在执行语句 S4 时要忽略 assign 语句 S2 对变量 var_reg2 的作用, var_reg2 将被连续赋值为“4”。

因此, 当例 6-28 所示的整个 initial 过程块结束执行后, 变量 var_reg1 和 var_reg2 都将处于被连续赋值为“4”的状态。

如果 force 语句指定的被赋值目标变量是连线型变量, 则在 force 语句得到执行后, 对应的连线型变量将得到“赋值表达式”的连续驱动, 此时将忽略该连线型变量上的较低优先级的驱动 (如 assign 连续赋值语句对该连线型变量的驱动, 实例输出端对连线型变量的驱动), 直到有一条 release 语句来释放对该变量的连续驱动为止。

【例 6-29】force 语句的例子。

```
module demo(var_net1, var_net2, in1, in2);
input in1, in2;
output var_net1, var_net2;
wire var_net1, var_net2, in1, in2;    // 本例中用到的都是连线型变量
assign var_net1=0;                    // S1, 连续赋值语句, 将 var_net1 连续赋值为 0
or #1 ( var_net2, in1, in2 );         // S2, 原语实例语句, 指定“或”门输出对 var_net2
//进行驱动
initial                                //initial 过程块
begin
    #5;
    force var_net1=in1 & in2;          // S3, 过程连续赋值语句, force 语句, 作用于
    // var_net1, 注意 force 语句只能出现在过程块中
    force var_net2 = (~in1) & in2;     // S4, 过程连续赋值语句, force 语句,
    // 作用于 var_net2
end
endmodule
```

上例模块中用 force 语句对两个连线型变量 var_net1 和 var_net2 进行了连续赋值操作。在 initial 过程块没有得到执行之前, 由于连续赋值语句 S1 的作用, var_net1 处于被连续赋值为“0”的状态; 同时由于“或门”实例语句 S2 的作用, var_net2 由或门输出端驱动。

在 initial 过程块开始执行后, 语句 S3 和语句 S4 在 t=5 时刻依次得到执行, 它们都是“force 过程连续赋值语句”, 分别实现对变量 var_net1 和变量 var_net2 的连续赋值, 由于 force 语句具有比 S1、S2 语句更高的优先级, 所以此时将忽略 S1、S2 语句对两个变量的作用。因此在上例模块执行之后, 变量 var_net1 将处于由“in1&in2”连续赋值的状态, 而变量 var_net2 将处于由“(~in1) & in2”连续赋值的状态。

(2) “release 语句”的使用语法如下:

release <寄存器或连线型变量> ;

与 deassign 语句的作用相类似, 在执行了 release 语句后, 原先由 force 语句对变量实施的连续赋值操作将失效, 变量将解除被连续赋值的状态, 从此刻起该变量又可以由其它低优先级的赋值语句进行赋值操作了。

在 `release` 语句内的目标变量是寄存器类变量的情况下, 如果在 `force` 语句执行之前该寄存器变量已经由于受“`assign` 语句”的作用而处于被连续赋值的状态 (就是例 6-28 中变量 `var_reg2` 的情况), 那么在执行了“`release` 语句”后, 这个寄存器变量将恢复到由 `assign` 语句实现的连续赋值状态; 如果在 `force` 语句执行之前该寄存器变量没有处于被连续赋值的状态 (就是例 6-28 中变量 `var_reg1` 的情况), 那么在执行了“`release` 语句”后, 这个寄存器变量将退出连续赋值状态, 其取值保持不变。

举例来说, 如果我们在例 6-28 所示的过程块中加上 `release` 语句。

【例 6-30】 `release` 语句的例子。

```
reg [2:0] var_reg1, var_reg2 ;
initial
begin
    var_reg1 = 0;           //S1, 过程赋值语句
    assign var_reg2 = 1;    //S2, 过程连续赋值语句, assign 语句
    #10;
    force var_reg1 = 4;     //S3, 过程连续赋值语句, force 语句
    force var_reg2 = 4;     //S4, 过程连续赋值语句, force 语句
    #10;
    release var_reg1;       //S5, release 语句, 作用于变量 var_reg1
    release var_reg2;       //S6, release 语句, 作用于变量 var_reg2
end
```

上例中, 由于在执行 `force` 语句 S3 之前变量 `var_reg1` 还未被 `assign` 语句执行过连续赋值操作, 所以在 `release` 语句 S5 执行后, 变量 `var_reg1` 将退出被连续赋值的状态, 其取值保持不变 (保持原有值“4”)。

而对于 `var_reg2` 来说, 由于在执行 `force` 语句 S4 之前变量 `var_reg2` 已经由 `assign` 语句 (语句 S2) 实现了连续赋值, 所以在 `release` 语句 S6 执行后, 变量 `var_reg2` 将恢复到由 `assign` 语句 S2 确定的连续赋值状态, 也就是被“1”连续赋值的状态。

在 `release` 语句内的目标变量是连线型变量的情况下, 如果在 `force` 语句执行之前该连线型变量已经通过“`assign` 语句”或原语实例语句指定了驱动源 (就是例 6-29 中变量 `var_net1` 和 `var_net2` 的情况), 那么在执行了 `release` 语句后, 这个连线型变量将恢复到原有的驱动方式。

举例来说, 如果我们在例 6-29 所示的过程块内加上 `release` 语句。

【例 6-31】 `release` 语句的例子。

```
module demo(var_net1, var_net2, in1, in2);
input in1, in2;
output var_net1, var_net2;
wire var_net1, var_net2, in1, in2;
assign var_net1 = 0;           // S1, 连续赋值语句, 将 var_net1 连续赋值为 0
or #1 (var_net2, in1, in2);    // S2, 原语实例语句, 指定“或”门输出对 var_net2
//进行驱动
```




```
initial                                // initial 过程块
begin
    #5;
    force var_net1= in1 & in2;         // S3, 过程连续赋值语句, force 语句, 作用于
    // var_net1
    force var_net2 = (~in1) & in2;     // S4, 过程连续赋值语句, force 语句,
    // 作用于 var_net2

    #5;
    release var_net1;                  // S5, release 语句, 用来撤销 force 语句 S3
    //对变量 var_net1 的作用
    release var_net2;                  // S6, release 语句, 用来撤销 force 语句 S4
    //对变量 var_net2 的作用
end
endmodule
```

在上例中, 由于在 force 语句 S3 得到执行前 assign 语句 S1 已经将连线型变量 var_net1 指定为由“0”连续驱动, 所以在执行了 release 语句 S5 之后, 变量 var_net1 将退出 S3 指定的连续赋值状态而恢复到语句 S1 指定的、由“0”进行连续驱动的状态。

同样, 由于在 force 语句 S4 得到执行前“或门实例语句”S2 已经将连线型变量 var_net2 指定为由或门实例的输出端进行驱动, 所以在执行了 release 语句 S6 之后, 连线型变量 var_net2 将退出 S4 指定的连续驱动状态而恢复到由语句 S2 指定的、受或门实例输出端连续驱动的状态。



第7章 行为描述(三): 高级程序语句、函数和任务

我们在前面讲过,过程块主要是由过程性赋值语句(包括过程赋值语句和过程连续赋值语句)以及高级程序语句(包括条件分支语句和循环控制语句)这两种行为语句所构成的。第6章已经对过程赋值语句进行了讨论,在本章我们将对高级程序语句以及函数和任务的使用进行介绍。

所谓高级程序语句,就是为了对硬件电路进行行为描述而从高级语言中直接借用的程序设计语句。高级程序语句被使用于对硬件电路的行为描述,它们只能出现在过程块中。Verilog HDL 中的高级程序语句是从 C 语言中引入的,它们可以分成分支语句和循环控制语句两类。我们将在 7.1 节和 7.2 节分别对分支语句和循环控制语句进行讨论。

除了过程块和连续赋值语句以外,行为描述模块内还可以包含任务定义和函数定义这两种结构部分。这两种结构在行为描述模块中都是可选的,它们是一种存在于模块中的“子程序”结构。引入任务和函数的目的是为了简化对需要多次执行的语句的描述,使之便于理解和调试。引入任务和函数还可以简化程序的结构,增强代码的易读性。我们将在 7.3 节介绍任务和函数的使用。

7.1 分支语句

Verilog HDL 语言中存在着如下两种分支语句:

- if-else 条件分支语句。
- case 分支控制语句。

本节我们将对这两种分支语句进行讨论。

7.1.1 if-else 条件分支语句

if-else 条件分支语句的作用是根据指定的判断条件是否满足来确定下一步要执行的操作。它在使用时可以采用如下三种形式:

(1) 使用形式 1:

if (<条件表达式>) 语句或语句块;

在 if-else 条件分支语句的这种使用形式中没有出现 else 项,这种情况下条件分支语句的执行过程将是:

- 如果指定的“<条件表达式>”成立(也就是这个条件表达式的逻辑值为“1”),则



执行条件分支语句内给出的“语句或语句块”，然后退出条件分支语句的执行。

● 如果“<条件表达式>”不成立(也就是条件表达式的逻辑值为“0”、“x”或“z”时)，则不执行条件分支语句内给出的“语句或语句块”，而是直接退出条件分支语句的执行。

例如下面这条条件分支语句：

```
if(enable == 1) out = data_in;
```

在执行时就会根据条件表达式“enable==1”是否成立来决定是否执行赋值语句“out=data_in;”：如果 enable 取值为“1”，则赋值语句得到执行，输出信号 out 得到 data 的值；如果 enable 的值为“0”、“x”或“z”(取值不为“1”)，则不执行指定的赋值语句，输出信号 out 保持原有值不变。

(2) 使用形式 2：

```
if(<条件表达式>) 语句或语句块 1
else              语句或语句块 2
```

这种形式的条件分支语句将以如下方式得到执行：

● 如果指定的“<条件表达式>”成立(也就是这个条件表达式的逻辑值为“1”)，则执行条件分支语句第一行所指定的“语句或语句块 1”，然后结束条件分支语句的执行。

● 如果“<条件表达式>”不成立，则执行由条件分支语句内第二行的 else 项所指定的“语句或语句块 2”，然后结束条件分支语句的执行。

这种形式的 if-else 条件分支语句实现了一种二路分支选择控制。

例如，下面这条条件分支语句：

```
if(select==1) out=input1;
else out=input0;
```

在执行时会根据条件表达式“select==1”是否成立来决定执行两条过程赋值语句中的哪一条。如果 select 取值为“1”，则第一行 if 这一项所指定的赋值语句“out=input1;”得到执行，输出信号 out 得到 input1 的取值；如果 select 取值不为“1”(取值为“0”、“x”或“z”)，则执行第二行 else 项所指定的赋值语句“out=input0;”，输出信号 out 将得到 input0 的取值。

(3) 使用形式 3：

```
if (<条件表达式 1>) 语句或语句块 1
else if (<条件表达式 2>) 语句或语句块 2
.....
else if (<条件表达式 n>) 语句或语句块 n
else 语句或语句块 n+1
```

在这种使用形式中一共出现了 n+1 个条件分支项，其中每个分支项都指定了当该分支项的条件满足时所要执行的操作。

在执行这种形式的 if-else 条件分支语句时，将按照各分支项的排列顺序对各个条件表达式是否成立作出判断，当遇到某一项的条件表达式成立时，就执行这一项所指定的“语句或语句块”。比如假设“<条件表达式 m>”成立，那么就执行“语句或语句块 m”。

如果所有的条件表达式都不成立，则执行最后的 else 项(这一项没有给出条件表达式)



所指定的操作(语句或语句块 $n+1$)。

这种形式的 if-else 条件分支语句实现了一种多路分支选择控制。

比如在例 7-1 给出的模块中就使用了这种形式的 if-else 条件分支语句。

【例 7-1】第三种形式的 if-else 条件分支语句。

```
module sel_from_three(q, sela, selb, a, b, c);  
input sela, selb, a, b, c;  
output q;  
reg q;  
always @(sela or selb or a or b or c)  
begin  
    if (sela)    q = a;  
    else if (selb) q = b;  
    else        q = c;  
end  
endmodule
```

在例 7-1 模块内的 if-else 条件分支语句中出现了三个分支项, 这个条件分支语句在执行时将依次对控制信号 sela 和 selb 的取值是否为“1”进行判断:

(1) 如果 sela 的取值为 1, 则第一个分支项的条件表达式“(sela)”成立, 因而第一个分支项所指定的赋值操作“q=a;”将得到执行。

(2) 如果 sela 的取值不为 1, 而 selb 的取值为 1, 则第二个分支项的条件表达式“(selb)”成立, 因而第二个分支项所指定的赋值操作“q=b;”将得到执行。

(3) 如果 sela 和 selb 的取值都不为 1, 则“(sela)”和“(selb)”这两个条件表达式都不成立, 这时 else 分支项所指定的赋值操作“q=c;”将得到执行。

(4) 如果 sela 和 selb 的取值都是 1, 那么第一个条件表达式“(sela)”以及第二个条件表达式“(selb)”都是成立的。在这种情形下, 由于首先被检测为成立的条件表达式是第一个分支项的条件表达式“(sela)”, 所以此时将执行第一分支项指定的赋值操作“q=a;”。

因此, 例 7-1 所示模块描述的是图 7.1 所示的硬件电路功能。

由上例条件表达式取值的第四种情况我们可以看出: 这种形式的 if-else 条件分支语句(第三种使用形式)内各个条件表达式的排列顺序决定了对各个条件进行判断的先后次序, 这种排列顺序隐含着一种优先级关系, 也就是说排在前面的分支项所指定的操作具有较高的优先级。

比如, 在前面讨论过的例 7-1 中条件分支语句的第四种执行情况下, 虽然两个条件表达式都是成立的, 但是由于条件表达式“(sela)”排在“(selb)”的前面, 所以“(sela)”这一项所对应的赋值语句“q=a;”具有较高优先级, 因此这时得到执行的将是语句“q=a;”而不是“q=b;”。

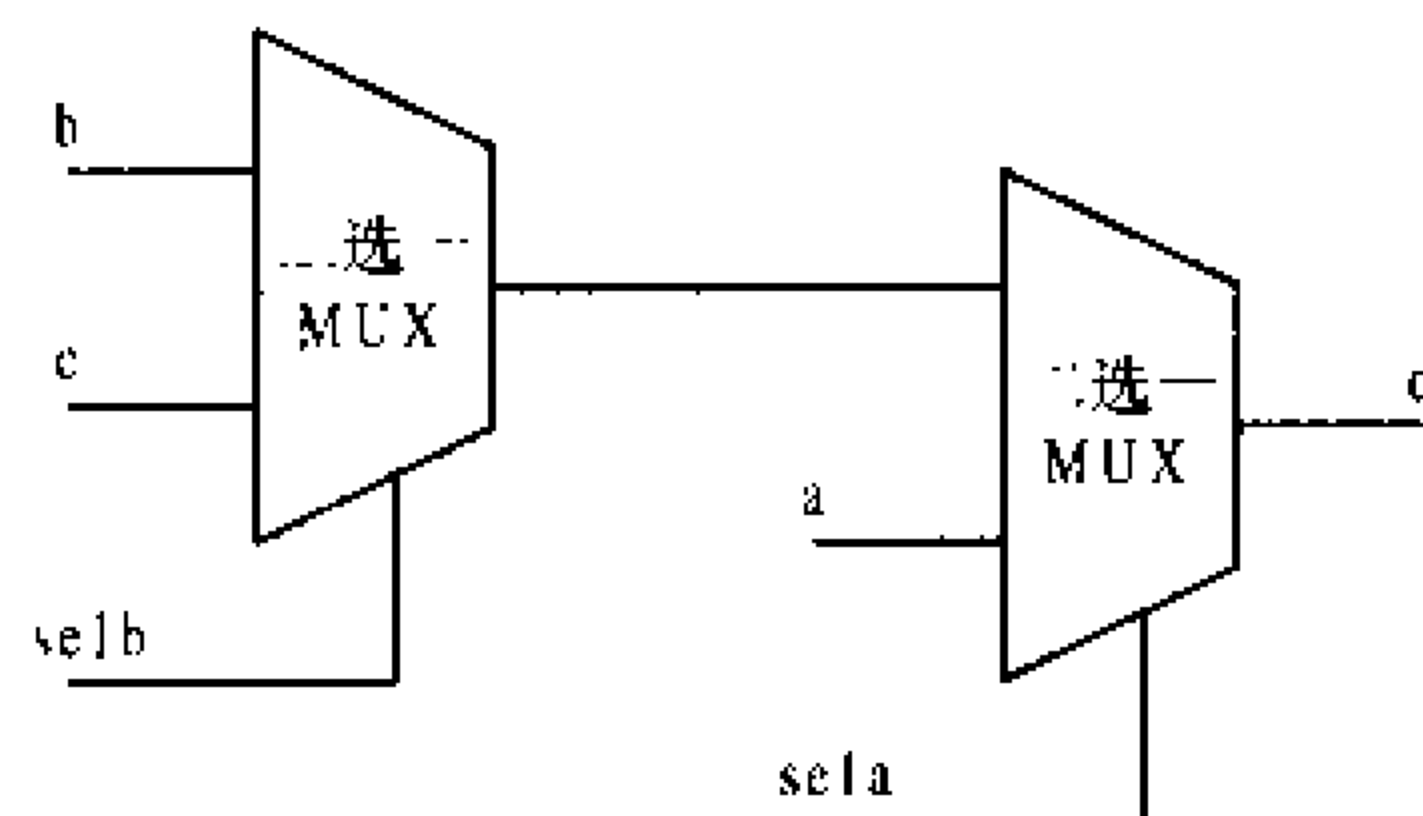


图 7.1 例 7-1 所示模块描述的电路功能



对于前面介绍的这三种形式的 if-else 条件分支语句，我们还必须说明如下几点：

(1) “<条件表达式>” 一般是一个逻辑表达式或关系表达式。如果这个逻辑表达式或关系表达式的取值为“1”，则认为该条件表达式成立，而如果这个逻辑表达式或关系表达式取值不为“1”（取值为“0”、“x”或“z”），则认为该条件表达式不成立。

(2) 尽管第二、三种形式的 if-else 条件分支语句在形式上是由多行和多条语句构成，但 Verilog 将把它们视为一条 if-else 条件分支语句来进行处理。

(3) 处于分支项内的、用于指定该分支项所对应操作的语句可以是一条行为语句，也可以是多条行为语句。在多条行为语句的情况下要用 begin 和 end 这两个关键词将这多条语句组合成一个语句块。

Verilog HDL 语言还允许 if-else 条件分支语句的嵌套使用，比如：

```
if (<条件表达式 1>)                //外层 if 语句
if (<条件表达式 2>)  语句 1          //内层 if 语句 1
else                语句 2          //内层 else 语句 1
else                //外层 else 语句
if (<条件表达式 3>)  语句 3          //内层 if 语句 2
else                语句 4          //内层 else 语句 2
```

在上面给出的语句结构中，外层 if-else 条件分支语句的两个分支语句分别是两个内层 if-else 条件分支语句。

在 if-else 条件分支语句嵌套使用的情况下，要注意 if 与 else 之间的配对关系。Verilog HDL 编译器会认为：每一个 else 总是与离它最近的上一个 if 进行配对。例如下面几条语句：

```
if (out_enable==1)                //外层 if 语句
if (with_not==1) output==~input;  //内层 if 语句
else  out= input;                  //?层 else 语句
```

在上面的语句中，第三行的 else 分支项是和哪一个 if 语句配对呢？由于第二行的 if 语句与这个 else 语句离得最近同时又没有和其它 else 配成对，所以这个 else 将被解释成和第二行的 if 配对，第二行和第三行语句构成了一个内嵌的 if-else 条件分支语句。

为了随意实现程序设计者的意图，我们可以用 begin-end 块语句来明确 if 与 else 的配对关系，举例来说，如果我们想让上面第三行的 else 语句与第一行的 if 语句进行配对，则可以采用如下描述方法来实现：

```
if (out_enable==1)                //外层 if 语句
begin
    if (with_not==1) output==~input;  //内层 if 语句
end
else  out= input;                  //外层 else 语句
```

其中的 begin, end 块语句限定了内嵌 if 条件分支语句的范围，而 else 语句处于该范围之外，因此 else 只能和第一个 if 配对。

最后我们再给出使用 if-else 条件分支语句的两个例子，它们利用 if-else 条件分支语句分别对具有同步清零功能的 D 触发器和具有异步清零功能的 D 触发器进行了描述。



【例 7-2】用 if-else 条件分支语句描述一个具有同步清零功能(低电平有效)的下降沿 D 触发器。

```
module dff_sync(q, d, clear, clk);
    output q;
    input d, clear, clk;
    reg q;
    always @(negedge clk)
        if(!clear) q=0;    //进行清零
        else      q=d;    //正常 D 触发器功能
endmodule
```

在例 7-2 中, 由于要实现的是同步清零方式, 所以触发器状态的改变都是发生在时钟下降沿的。always 过程块中的 if-else 条件分支语句首先对清零控制信号 clear 进行检测, 从而判断是否需要执行清零操作(因为清零操作具有较高优先级): 如果清零控制信号 clear 有效(取值为“0”), 则对 q 赋值“0”, 实现清零操作; 如果清零控制信号 clear 无效(取值为“1”), 则将输入端 d 的数据传送到输出端 q, 实现 D 触发器正常工作时的功能。

【例 7-3】用 if-else 条件分支语句描述一个具有异步清零功能(低电平有效)的下降沿 D 触发器。

```
module dff_asyn2(q, d, clear, clk);
    output q;
    input d, clear, clk;
    reg q;
    always @(clear)    //该过程块用来处理清零操作
        if(!clear) q=0;    //异步清零
    always @(negedge clk) //该过程块用来实现正常的 D 触发器功能
        if(!clear)
            q=0;    //考虑 clear 信号和 clk 信号在同一时刻发生变化的情况
        else
            q=d;    //正常 D 触发器功能
endmodule
```

上例中, 两个 always 过程块分别用来监测清零控制信号 clear 和时钟 clk 的变化, 其中第一个 always 过程块用于实现清零功能; 第二个 always 过程块则用于实现正常的 D 触发器功能。而且在第二个 always 过程块中还考虑了清零信号与时钟信号同时发生跳变的情况, 由于其中的 if-else 条件分支语句中对清零信号的处理优先于对时钟信号的处理, 所以在这种情况下也能正确实现异步清零功能。

7.1.2 case 分支控制语句

case 分支语句是另一种用来实现多路分支选择控制的分支语句。与使用 if-else 条件分支语句相比, 采用 case 分支语句来实现多路选择控制将显得更为方便与直观。

case 分支语句通常用于对微处理器指令译码功能的描述以及对有限状态机的描述。



case 分支语句有“case”、“casez”和“casex”三种形式。

1. case 语句

case 语句的格式如下：

case (<控制表达式>)

<分支项表达式 1> : 语句块 1;

<分支项表达式 2> : 语句块 2;

.....

<分支项表达式 n> : 语句块 n;

default: 语句块 n+1;

endcase

在上述格式中：

“<控制表达式>”代表着对程序流向进行控制的控制信号；各个“<分支项表达式>”则是控制表达式的某些具体状态取值，在实际使用中这些分支项表达式通常是一些常量表达式；各个“语句块”则指定了在各个分支下所要执行的操作，它们也可以是由单条语句构成的；处于最后的、以关键词“default”开头的那个分支项被称为“default 分支项”，它是可以缺省的。

case 语句的执行过程是：

(1) 当“控制表达式”的取值等于“分支项表达式 1”时，执行第一个分支项所包含的“语句块 1”。

(2) 当“控制表达式”的取值等于“分支项表达式 2”时，执行第二个分支项所包含的“语句块 2”。

.....

(3) 如果“控制表达式”的取值与所有 n 个分支项表达式的值都不相同，则执行“default 分支项”所包含的“语句块 n+1”。

(4) 在执行了某一支项内的语句块后，跳出 case 语句结构，终止 case 语句的执行。

case 语句中各个“分支项表达式”的取值必须是互不相同的，否则就会出现矛盾现象。

如果“控制表达式”有几种不同取值情况对应着同一操作，则上述格式可以这样进行简写：用逗号将代表着这几种不同取值情况的多个“分支项表达式”隔开，再将在这些情况下需要执行的语句块放在这几个分支项表达式的后面。比如下面的语句：

```
case(op_code)
```

```
2'b00: out=a|b;           // op_code 取值为“2'b00”时的分支项
```

```
2'b01,
```

```
2'b10,
```

```
2'b11: out=a&b;           // op_code 取值为“2'b01”、“2'b10”或“2'b11”时
```

```
                        // 要执行的操作都是“out=a&b;”
```

```
default: out=0;           // default 分支项
```

```
endcase
```

在上面的语句中，三个分支项表达式“2'b01”、“2'b10”和“2'b11”之间用“，”

隔开,同时在这几个分支项表达式后面指定了要执行的语句“out=a&b;”。这表明当控制信号 op_code 取值为 2'b01、2'b10 或 2'b11 时,对应的操作都是由语句“out=a&b;”指定的赋值操作。上例是如下多条语句的简写形式:

```
case(op_code)
  2'b00: out=a|b;           // op_code 取值为“2'b00”时的分支项
  2'b01: out=a&b;           // op_code 取值为“2'b01”时的分支项
  2'b10: out=a&b;           // op_code 取值为“2'b10”时的分支项
  2'b11: out=a&b;           // op_code 取值为“2'b11”时的分支项
default: out=0;             // default 分支项
endcase
```

case 语句在执行时,控制表达式和分支项表达式之间进行的比较是一种按位进行的“全等比较”,也就是说,只有在分支项表达式和控制表达式对应的每一位都彼此相等的情况下才认为分支项表达式和控制表达式两者是“相等”的。在进行对应位的比较时,“x”和“z”这两种逻辑状态也与“0”、“1”逻辑状态一样作为合法的状态参与比较。这些逻辑状态在进行位取值比较时的真值表如表 7.1 所示,其中第一行和第一列是参加比较的两个位的可能逻辑状态,在表中用符号“yes”表示比较结果为“两者相等”,用符号“no”表示比较结果为“两者不相等”。

表 7.1 case 语句的比较规则

case	0	1	x	z
0	Yes	no	no	no
1	No	yes	no	no
x	No	no	yes	no
z	No	no	no	yes

我们举一个例子来说明这种“全等比较”的特点,如例 7-4 所示。

【例 7-4】case 语句执行时对“x”状态和“z”状态的处理。

```
module demo_xz(a);
input a;
always @ (a)
  case(a)
    1'b1: $display("input signal value is 1");
    1'b0: $display("input signal value is 0");
    1'bx: $display("input signal is in unknown state");
    1'bz: $display("input signal is in high impedance state");
  endcase
endmodule
```

在例 7-4 所示模块中,控制表达式为“a”,同时还列出了四个分支项。在进行控制表达式和各个分支项表达式的比较时,“0”、“1”、“x”和“z”这四个逻辑状态将作为四个不同的合法状态参与比较。所以当输入信号 a 的取值为“x”时,只有第三个分支项表达式与控制表达式的比较结果是“两者相等”,此时第三个分支项后面指定的语句将得到执



行；同样，当输入 a 取值为“z”时，只有第四个分支项表达式与控制表达式的比较结果是“两者相等”，此时将执行第四个分支项后面所指定的语句。

由于 case 语句有上面这种按位进行全等比较的特点，case 语句中的控制表达式和所有分支项表达式必须具有相同的位宽，因为只有这样控制表达式和分支表达式才能进行对应位的比较；当各个分支项表达式以常数形式给出时，必须显式地标明每个常数的位宽，否则 Verilog 编译器会认为它们具有与机器字长相同的位宽。

例 7-5 是一个用 case 语句来实现操作码译码的例子。

【例 7-5】用 case 语句来实现操作码译码。

```
module decode_of_opcode_use_case(a, b, opcode, out);
input[7:0] a, b;
input[2:1] opcode;
output[7:0] out;
reg [7:0] out;
always @ (a or b or opcode)
begin
    case(opcode)
        2'b10:    out=a+b;
        2'b11:    out=a-b;
        2'b01:    out=(~a)+1;
        2'b00:    out=(~b)+1;
    endcase
end
endmodule
```

在例 7-5 所示模块中，输入信号 opcode 是宽度为两位的操作码，它的取值指定了对输入 a , b 执行的运算类型：

如果操作码为“2'b10”，则求 a , b 的和(第一个分支项)。

如果操作码为“2'b11”，则求 a , b 的差(第二个分支项)。

如果操作码为“2'b01”，则求 a 的补码(第三个分支项)。

如果操作码为“2'b00”，则求 b 的补码(第四个分支项)。

2. casez 和 casex 语句

Verilog HDL 还提供了另外两种形式的 case 分支语句，它们是 casez 语句和 casex 语句。可以利用它们来实现这样的分支控制：由控制表达式和分支项表达式的一部分数位的比较结果来决定程序的流向。

casez 语句执行时的比较过程将不考虑控制表达式及分支项表达式中处于高阻态“z”的那些位的比较，而 casex 语句则将高阻状态“z”和不定状态“x”都视为不必关心的情况。通过这两种形式的分支控制语句，就可以通过对控制表达式和分支项表达式的灵活设定来实现由一部分数位取值决定的分支控制。

1) casez 语句

casez 语句的格式如下：



```
casez (<控制表达式>)  
<分支项表达式 1> : 语句块 1;  
<分支项表达式 2> : 语句块 2;  
.....  
<分支项表达式 n> : 语句块 n;  
default :      语句块 n+1;  
endcase
```

从上面的格式我们可以看出: casez 语句与 case 语句的使用格式完全相同, 两者唯一的差别是关键词 “casez” 和 “case” 的不同。

在 casez 语句中, 如果控制表达式或分支项表达式的某一位取值为 “z”, 则在分支语句执行时将忽略该位的比较, 也就是说这种情况下控制表达式和分支项表达式在这一位的取值将不会对程序流程分支的选择产生任何影响。casez 语句进行位取值比较时的真值表如表 7.2 所示, 其中第一行和第一列是参加比较的两个位的可能取值状态, 我们在表中用符号 “yes” 来表示比较结果为 “两者相等”, 用符号 “no” 来表示比较结果为 “两者不相等”。

表 7.2 casez 语句的比较规则

casez	0	1	x	z
0	yes	no	no	yes
1	no	yes	no	yes
X	no	no	yes	yes
Z	yes	yes	yes	yes

例 7-6 是一个用 casez 语句实现分支控制的例子。

【例 7-6】用 casez 语句实现操作码译码。

```
module decode_of_opcode_use_casez(a, b, opcode, out);  
input[7:0] a, b;  
input[4:1] opcode;  
output[7:0] out;  
reg [7:0] out;  
always @ (a or b or opcode)  
begin  
    casez (opcode)  
        4'b1zzz: out=a+b;    //分支项 1  
        4'b01??: out=a-b;    //分支项 2  
        4'b001?: out=(~a)+1; //分支项 3  
        4'b0001: out=(~b)+1; //分支项 4  
    endcase  
end  
endmodule
```

在例 7-6 所示的模块中, 输入信号 opcode 为四位宽的操作码, 它用来指定对输入 a,



b 执行的运算类型, casez 语句内的控制表达式为操作码 “opcode”, 各个分支项表达式以常量形式给出:

第一个分支项表达式是 “4'b1zzz”, 它的第 1、2、3 位为 “z”, 所以控制表达式在与该分支项表达式进行比较时将忽略这三位而只对最高位进行比较。所以只要操作码 opcode 的最高位取值为 “1”, 两个表达式的比较结果就是 “相等”, 该分支项对应的语句 “out=a+b;” 就得到执行。

第二个分支项表达式是 “4'b01??”, 它的低二位数值为 “?”, 这里符号 “?” 的含义就代表 “z”。所以在进行两个表达式的比较时将忽略低二位而只对高二位数值进行比较。所以只要操作码 opcode 最高二位取值为 “01”, 就执行第二个分支项对应的语句 “out=a-b;”。

第三个分支项表达式是 “4'b001?”, 它的最低位数值为 “?”, 所以在进行两个表达式的比较时将忽略这个最低位而只对高三位数值进行比较。所以只要操作码 opcode 最高三位的取值为 “001”, 就执行第三个分支项对应的语句 “out=(~a)+1;”。

第四个分支项表达式是 “4'b0001”, 其中没有哪一位是高阻状态 “z”, 所以这时控制表达式的全部四位都要进行比较, 只有当操作码 opcode 等于 “4'b0001” 时, 该分支项所对应的语句 “out=(~b)+1;” 才能得到执行。

2) casex 语句

casex 语句的格式如下:

casex (<控制表达式>)

<分支项表达式 1> : 语句块 1;

<分支项表达式 2> : 语句块 2;

.....

<分支项表达式 n>: 语句块 n;

default: 语句块 n+1;

endcase

casex 语句与 case 语句以及 casez 语句所使用的格式完全相同, 唯一的差别是这里的关键词为 “casex”。

与 casez 语句一样, casex 语句中的控制表达式和分支项表达式在进行比较时将忽略某些位的比较, casex 语句和 casez 语句的不同之处在于: 在 casex 语句中, 处于 “x” 或 “z” 这两种逻辑状态的位都被忽略; 而在 casez 语句中只忽略处于 “z” 状态的位。

casex 语句在进行数位比较时的真值表如表 7.3 所示。

表 7.3 casex 语句的比较规则

casex	0	1	x	z
0	yes	no	yes	yes
1	no	yes	yes	yes
X	yes	yes	yes	yes
Z	yes	yes	yes	yes

【例 7-7】用 casex 语句实现操作码译码。

```
module decode_of_opcode_use_casex(a, b, opcode, out);
```




```
input[7:0] a, b;
input[4:1] opcode;
output[7:0] out;
reg [7:0] out;
always @ (a or b or opcode)
begin
    casex (opcode)
        4'b1zzx:    out=a+b;        //分支项 1
        4'b01xx:    out=a-b;        //分支项 2
        4'b001?:    out=(~a)+1;     //分支项 3
        4'b0001:    out=(~b)+1;     //分支项 4
    endcase
end
endmodule
```

例 7-7 所示模块中的控制表达式“opcode”在与各个分支项表达式进行比较时,将分别忽略其中取值为“z”、“x”、“?”的位。所以与例 7-6 的执行一样:

只要操作码 opcode 最高位取值为“1”,就执行第一个分支项对应的语句“out=a+b;”。

只要操作码 opcode 最高二位取值为“01”,就执行第二个分支项对应的语句“out=a-b;”。

只要操作码 opcode 最高三位取值为“001”,就执行第三个分支项对应的语句“out=(~a)+1;”。

当操作码 opcode 等于“4'b0001”时,执行第四个分支项对应的语句“out=(~b)+1;”。

7.2 循环控制语句

与条件分支语句一样,循环控制语句也是一种高级程序语句,它在 Verilog HDL 中被用来进行行为描述。Verilog HDL 语言中提供了如下几种循环控制语句:

- forever 循环语句。
- repeat 循环语句。
- while 循环语句。
- for 循环语句。

本节我们将对以上四种循环控制语句进行讨论。

7.2.1 forever 循环语句

forever 循环语句实现的是一种无限的循环,该循环语句内指定的循环体部分将不断重复地得到执行。

forever 语句的格式如下:

forever 语句或语句块;



在上述格式中，处于关键词 `forever` 后面的“语句或语句块”部分就是被重复执行的循环体。

`forever` 循环语句常用于产生周期性的波形来作为仿真测试信号，实例如下：

【例 7-8】一个由 $t=1000\text{ ns}$ 时刻开始的周期为 50 ns 的时钟产生器。

```
module clk_gen(clk);
output  clk;
initial
begin
    clk = 0;
    #1000;           //时间控制
    forever
        #25 clk = ~clk; //被指定循环执行的语句
end
endmodule
```

上例所产生的时钟信号 `clk` 的波形是：在 $t=0$ 时刻时钟信号首先被初始化为 0，并一直保持到 $t=1000\text{ ns}$ 时刻。此后每隔 25 ns ，时钟信号 `clk` 的取值翻转一次。这样就产生了周期为 50 ns 的时钟波形。

如果需要在某个时刻跳出 `forever` 循环语句所指定的无限循环，则可以通过在循环体语句块中使用中止语句 (`disable` 语句) 来实现这一目的，比如如下语句：

```
module clk_gen(clk);
output  clk;
integer counter;
initial
begin
    counter=0;
    clk = 0;
    #1000;
    begin:  FOREVER_PART           //外层块，有名块
        forever
            begin                  //内层块，被循环执行的语句块
                count=count+1;
                if(count>200) disable FOREVER_PART;
                #25 clk = ~clk;
            end                    //内层块结束标志
        end                        //FOREVER_PART 语句块的结束标志
    end
end
endmodule
```

上例所述模块用来产生长度为 100 个周期的时钟信号。其中，`initial` 过程块内的 `forever` 语句被置于一个有名语句块“`FOREVER_PART`”之内，同时 `forever` 语句的循环体

部分由另一个语句块构成。这个循环体语句块包含三条语句，当中止语句“disable FOREVER_PART;”得到执行时，有名语句块“FOREVER_PART”的执行将被中止，程序流程将进入“FOREVER_PART”语句块后的下一条语句。因此在执行第 201 次循环时，if 条件语句中的条件表达式“count>200”成立，“disable”中止语句得到执行，程序流程跳出“FOREVER_PART”语句块的执行，也就同时跳出了 forever 循环，这样就实现了跳出 forever 循环的目的。

7.2.2 repeat 循环语句

repeat 循环语句实现的是一种循环次数预先指定的循环，这种循环语句内的循环体部分将被重复执行指定的次数。

repeat 循环语句的格式如下：

repeat (<循环次数表达式>) 语句或语句块；

在上述格式中：

- “<循环次数表达式>”用于指定循环次数，它可以是一个整数、变量或一个数值表达式。如果是变量或数值表达式，其取值只在第一次进入循环时得到计算(从而得以事先确定循环次数)。

- “语句或语句块”是要被重复执行的循环体部分。

比如例 7-9 所示模块内就使用 repeat 循环语句实现了对输入信号 datain 的循环移位操作。

【例 7-9】用 repeat 循环语句来实现循环移位。

```
module drift(data, num, ctrl);
inout[16:1] data;
input[4:1] num;
input ctrl;
reg[16:1] data;
reg tmp;
always @ (ctrl)
if(ctrl==1)
repeat (num)
begin                                //循环体部分语句块
    tmp = data[16];
    data = {data << 1, tmp};
end
endmodule
```

在上例中，repeat 循环语句内的“循环次数表达式”是“num”，它表示后面循环体的重复执行次数由输入信号 num 的取值决定，输入信号 ctrl 用来启动移位操作的进行。循环体部分由一个 begin-end 语句块构成，该语句块每执行一次就对 data 实现一次 1 位循环移位操作。

又比如例 7-10 所示的模块，它利用 repeat 循环语句、采用移位相加法实现了一个 8



位乘法器，其中的输入数据为 8 位，输出为 16 位。

【例 7-10】用 repeat 循环语句来实现一个 8 位乘法器。

```
module 8_bits_multiplier(result, opa, opb);
parameter SIZE=8, LONGSIZE=16;
input [SIZE: 1] opa, opb;
output [LONGSIZE:1] result;
reg[SIZE: 1] opa, opb;
reg[LONGSIZE:1] result;
always @(opa or opb)
begin: mult
    reg [LONGSIZE:1] shift_opa, shift_opb; //局部变量定义
    shift_opa=opa;
    shift_opb=opb;
    result = 0;
    repeat(SIZE) // repeat 循环语句
    begin //循环体语句块
        if(shift_opb[1]) result = result+shift_opa;
        shift_opa = shift_opa<<1;
        shift_opb = shift_opb>>1;
    end //循环体语句块结束标志
end
endmodule
```

在上例中，repeat 循环语句内指定的“循环次数表达式”是一个参数“SIZE”，它代表了数值“8”，于是后面的循环体将被重复执行 8 次；循环体部分由一个 begin-end 语句块构成，这个语句块每执行一次就进行一次移位相加操作，在重复执行 8 次后就完成了两个 8 位输入操作数的相乘运算。

repeat 循环语句还有一种特别的用法在此必须加以说明，我们看如下语句块：

```
initial
begin
    out=0;
    repeat(NUM) @(posedge clk);
    out=1;
end
```

在上述 initial 过程块中，repeat 循环语句后面的循环体部分是一条时间控制语句“@(posedge clk);”。在这种情况下，这条 repeat 循环语句的作用是：等待时钟信号 clk 的 NUM 次正跳变。因此下一条语句“out=1;”将在时钟第 NUM 次正跳变后得到执行，out 的低电平状态共保持了 NUM 个时钟周期的时间。

7.2.3 while 循环语句

while 循环语句实现的循环是一种“条件循环”，就是说只有在指定的条件表达式取值为“真”时才会重复执行循环体，否则就不执行循环体。

while 循环语句的格式如下：

while (<条件表达式>) 语句或语句块；

在上述格式中：

“<条件表达式>”代表了循环体得到继续重复执行时必须满足的条件，它常常是一个逻辑表达式。在每一次执行循环体之前都要对这个条件表达式是否成立进行判断。

“语句或语句块”代表了被重复执行的循环体部分。

while 循环语句的执行过程是这样的：首先判断条件表达式是否成立，如果成立，则执行后面指定的“语句或语句块(循环体部分)”，然后再次对条件表达式是否成立作出判断，只要其取值为“真”就再次重复执行循环体，直到在某一次循环后判断出条件表达式不成立，循环过程才结束，程序流程退出 while 循环语句。假如条件表达式在一开始就不成立，则循环体一次也不被执行。

例 7-11 所示过程块是用 while 循环语句来实现从 0 计数到 100，并在第 101 次计数时退出循环。

【例 7-11】while 循环语句。

```
initial
begin
    count = 0;
    while (count < 100)
    begin                                //循环体语句块
        $display ("count = %d", count);
        #5 count = count + 1;
    end
end
```

在上例的描述中，while 循环控制语句执行时首先判断条件表达式“count < 100”是否成立，如果条件成立，则执行后面指定的 begin-end 语句块(也就是循环体部分)，这个语句块实现了对计数变量“count”进行修改和显示的操作。

在 while 循环语句开始执行时，count 变量初值为 0，条件表达式成立，循环体语句块得到执行，count 取值增 1 后再次对条件表达式是否成立进行判断，其判断结果决定着是否再次执行循环体语句块。这样，随着循环体语句块的不断重复执行，count 的取值不断增加，直到进行了第 100 次循环后，count 取值变为 100，这时条件表达式“count < 100”是不成立的，此时循环体语句块就不再被重复执行，因此循环过程结束，程序流程退出 while 循环语句的执行。这样，while 循环语句所指定的循环体实际上得到了 100 次重复执行，程序流程退出 while 循环语句的时刻是在 t=500 ns 时刻。



7.2.4 for 循环语句

与 while 循环语句一样, for 循环语句实现的循环也是一种“条件循环”, 只有在指定的条件表达式成立时才进行循环。

for 循环语句的格式如下:

for (<语句 1> ; <条件表达式> ; <语句 2>)

循环体语句或语句块;

在上述格式中:

“<语句 1>”和“<语句 2>”是两条过程赋值语句, 它们分别用来对循环计数变量执行赋初值操作和增值操作。

“<条件表达式>”代表着循环继续进行时必须满足的条件, 它常常是一个逻辑表达式。在每一次执行循环体之前都要对这个条件表达式是否成立进行判断。

“循环体语句或语句块”是要被重复执行的循环体部分。

for 循环语句的执行过程可以分成如下几步:

(1) 执行“<语句 1>”。

(2) 对“<条件表达式>”是否成立做出判断: 若“条件表达式”取值为真, 则执行 for 循环语句中指定的“循环体语句或语句块”, 然后继续执行下面的第(3)步; 若“条件表达式”取值不为真, 则不再执行循环体部分, 循环过程结束, 退出 for 循环语句的执行。

(3) 执行“<语句 2>”, 然后转到(2)继续执行。

由上面说明的执行步骤可以看出: “<语句 1>”只在第一次循环开始之前被执行了一次, 它通常是一条给循环计数变量赋初值的过程赋值语句; “<语句 2>”是在每次循环结束后、下一次循环开始前被执行的, 它通常用于对循环计数变量的取值进行修改; 而“条件表达式”取值的判断是在每次循环开始前发生的, 它的结果决定着“是否继续执行循环”。条件表达式的取值往往是随着循环的进行而发生变化的。这是因为: 如果条件表达式的取值在整个循环过程中一直保持不变, 那么循环体要么是一次都得不到执行, 要么是进入被执行无限次循环的死循环状态。

因此, for 循环语句实际上等价于由 while 循环语句构建的如下循环结构:

```
begin
    <语句 1>;
    while (<条件表达式>)
        begin
            循环体语句或语句块;
            <语句 2>;
        end
    end
end
```

这样, 对于使用 while 循环语句时需要 3 条语句才能完成的一个循环控制, for 循环语句只需一条语句就可以实现。

下面使用 for 循环语句来描述与例 7-11 相同的功能。

【例 7-12】for 循环语句。

```
initial
for(count=0; count<100; count=count+1)
begin    //循环体语句块
    $display ("count = %d", count);
    #5;
end
```

上例中 for 循环语句的执行过程是: 首先执行“count=0;”, 给循环变量 count 赋初值 0; 然后判断条件表达式“count < 100”是否成立, 根据判断结果确定是否要继续执行循环体语句块:

- 如果条件表达式成立, 则执行循环体语句块, 然后语句“count=count+1”得到执行, 实现循环变量 count 的增值操作, 然后再重新转回到判断条件表达式那一步。
- 如果条件表达式“count < 100”不成立, 则循环体语句块不被执行, 结束循环过程, 退出 for 循环语句的执行。

所以循环变量 count 将由初值 0 开始, 在每次循环后其值增 1。这样条件表达式在前 100 次循环时始终是成立的, 而在开始第 101 次循环前, count 取值为 100, 此时条件表达式“count < 100”不再成立, 因此这时将结束循环并退出 for 循环语句的执行。

下面再举两个使用 for 循环语句的例子。例 7-13 所示模块利用 for 循环语句实现了在例 7-10 中用 repeat 语句来实现的乘法器功能, 而例 7-14 则用 for 循环语句实现了一个 4 位左移器。

【例 7-13】用 for 循环语句来实现一个 8 位乘法器。

```
module 8_bits_multiplier(result, opa, opb);
parameter SIZE=8, LONGSIZE=16;
input [SIZE: 1] opa, opb;
output [LONGSIZE:1] result;
reg[SIZE: 1] opa, opb;
reg[LONGSIZE:1] result;
always @(opa or opb)
begin: mult
integer index;                                //循环变量(index)定义
result = 0;
for (index = 1; index<=SIZE; index=index+1)    // for 循环语句
if (opb[index]==1)                             //循环体语句是一条 if 语句
result=result+(opa<<(index-1));
end
endmodule
```

【例 7-14】4 位左移器。

```
module four-bit_left_shifter(result, a, ctrl);
input[7:0] a;
```




```

input ctrl;
output [7:0] result;
reg [7:0] result;
always @(a or ctrl)
begin : DRIFT
    integer i;                //循环变量(i)定义
    result = a;
    if (ctrl == 1)             //判断控制信号 ctrl 是否为 1
    begin                      //如 ctrl 为 1, 则执行 4 位左移操作
        for (i = 4; i <= 7; i = i + 1)
            result[i] = result[i-4];    //循环体语句
        result[3:0] = 0;                //低 4 位赋值为 0
    end
end
endmodule

```

7.3 任务(task)与函数(function)

除了在前面已经讨论过的过程块和连续赋值语句这两部分以外, 行为描述模块内还可以包含任务定义和函数定义这两部分。这两部分在行为描述模块中都是可选的, 它们是存在于模块中的一种“子程序”结构。引入任务和函数的目的是为了对需要多次执行的语句进行描述, 使之便于理解和调试, 引入任务和函数还可以简化程序的结构, 增强代码的易读性。

7.3.1 任务(task)

“任务(task)”类似于其它编程语言中的“过程”。任务的使用包括任务定义和任务调用。

1. 任务的定义

任务定义的语法如下:

```

task <任务名> ;
    端口与类型说明
    局部变量说明
begin
    <行为语句 1; >
    <行为语句 2; >
    .....
    <行为语句 n; >
end
endtask

```


其中: 关键词 `task` 和 `endtask` 将它们之间的内容标志成一个任务定义。其中关键词 `task` 标志着一个任务定义结构的开端, 其后的“<任务名>”是为被定义任务所取的一个名称。在“<任务名>”后面不能出现输入输出端口列表。

“端口与类型说明”用于对任务各个端口的宽度和类型进行说明, 其中端口类型由关键词 `input`, `output` 和 `inout` (分别表示: 输入、输出和双向端口)指定, 该说明语句的语法与进行模块定义时的相应说明语句语法是一致的。

“局部变量说明”用来对任务内用到的局部变量进行宽度和类型说明, 这个说明语句的语法与进行模块定义时的相应说明语句语法是一致的。

由“`begin`”和“`end`”关键词界定的一组行为语句指明了任务被调用时需要进行的操作。在任务被调用时, 这些行为语句将按串行方式得到执行。

任务定义与“过程块”、“连续赋值语句”及“函数定义”这三种成份以并列方式存在于行为描述模块中, 它们在层次级别上是相同的。任务定义结构不能出现在任何一个过程块的内部。

我们用下面的例 7-15 来对任务定义进行具体的说明。

【例 7-15】任务的定义。

```
task read_mem;                                //任务定义结构的开头, 指定任务名为“read_mem”
    input [15:0] address;                      //输入端口说明
    output [31:0] data;                       //输出端口说明
    reg [3:0] counter;                        //局部变量说明
    reg[7:0] temp [1:4];                      //局部变量说明
    begin                                     //语句块, 指明任务被调用时需要进行的操作
        for (counter=1 ; counter<=4; counter=counter+1)
            temp[counter]=mem[address+counter-1];
        data={temp[1], temp[2], temp[3], temp[4]};
    end
endtask                                       //任务定义结构的结尾
```

上例定义了一个名为“`read_mem`”的任务, 该任务有一个 16 位的输入端口“`address`”、一个 32 位的输出端口“`data`”、一个 4 位的局部变量“`counter`”和一个 8 位的存储器“`temp`”。

当上例所定义的任务被调用时, `begin` 和 `end` 中间的语句得到执行, 它们用来执行对存储器“`mem`”进行的四次读操作, 将其结果合并后输出到端口“`data`”。

在定义任务时必须注意以下几点:

- (1) 在第一行“`task`”语句中不能列出端口名列表。
- (2) 在任务定义结构中的“行为语句”部分可以有延时语句、敏感事件控制语句等时间控制语句出现。
- (3) 一个任务可以没有输入、输出和双向端口, 也可以有一个或多个输入、输出和双向端口。
- (4) 一个任务可以没有返回值, 也可以通过输出端口或双向端口返回一个或多个返回值。



(5) 在一个任务中可以调用其它的任务或函数，也可以调用该任务本身。

(6) 在任务定义结构内不允许出现过程块(initial 或 always 过程块)。

(7) 在任务定义结构内可以出现“disable 中止语句”，这条语句的执行将中断正在执行的任务。当任务被中断后，程序流程将返回到调用任务的地方继续向下执行。

2. 任务的调用

任务的调用是通过“任务调用语句”来实现的。任务调用语句的语法如下：

<任务名> (端口 1, 端口 2, ……, 端口 n);

其中，“(端口 1, 端口 2, ……, 端口 n)” 组成了一个端口名列表。

在调用任务时必须注意：

(1) 任务调用语句只能出现在过程块内。

(2) 任务调用语句就像一条普通的行为语句那样得到处理。

(3) 当被调用的任务具有输入或输出端口时，任务调用语句必须包含端口名列表，这个列表内各个端口名出现的顺序和类型必须与任务定义结构中端口说明部分的端口顺序和类型相一致，注意只有寄存器类的变量才能与任务的输出端口相对应。

例如，例 7-16 所示模块对例 7-15 中定义的任务“read_mem”进行了调用。

【例 7-16】对任务进行调用。

```
module demo_task_invo;
reg[7:0] mem [128:0];
reg[15:0] a;
reg[31:0] b;
initial
begin
    a=0;
    read_mem(a, b);          //第一次调用
    #10;
    a=64;
    read_mem(a, b);          //第二次调用
end
<任务“read_mem”定义部分>
endmodule
```

在上面的模块中，任务“read_mem”得到了两次调用，由于这个任务在定义时说明了输入端口和输出端口，所以任务调用语句内必须包含端口名列表“(a, b)”，其中变量 a 与任务的输入端口“address”相对应，变量 b 与任务的输出端口“data”相对应，并且这两个变量在宽度上也是与对应的端口相一致的。这样，在任务被调用执行时，变量 a 的值通过输入端口传给了 address；在任务调用完成后，输出信号 data 又通过对应的端口传给了变量 b。

任务的调用具有以下特点：

(1) 在任务内定义的局部变量都具有局部和静态的特点。也就是说，如果同时对同一任务进行两次调用，则两个调用进程使用的将是相同物理地址处的同一个局部变量。在这

种情况下要注意防止内存使用冲突。

(2) 在一个任务中可以直接访问上一级调用模块中的任何寄存器,例如上例中我们可以在任务“read_mem”中对存储器变量 mem 进行访问,这个 mem 变量是在上一级模块(调用模块)“demo_task_invo”中定义的。

(3) 由于任务内部可以包含时间控制语句,所以调用执行一个任务所需的时间可以是非零时间单位,也就是说任务启动时刻可以是不同于任务结束时刻的。

(4) 可以通过一个 disable 语句来中断任务的执行。任务被中断后,程序流程将返回到调用任务的地方继续往下执行。

下面我们举几个例子说明任务的调用。

【例 7-17】用任务来实现一个 8 位加法器。

```
module task_example (a, b, c);
input [7:0] a, b;
output [7:0] c;
reg [7:0] c;
task adder;                                //任务定义部分
input [7:0] a, b;
output [7:0] adder;
reg c;                                     //此处的变量 c 是任务内的局部变量
integer i;                                //任务内局部变量说明
begin
    c = 0;                                //c 存储着上一位数字相加时的进位
    for (i = 0; i <= 7; i = i+1)
    begin
        adder[i] = a[i] ^ b[i] ^ c;        //求一位数字相加的结果
        c = (a[i] & b[i]) | (a[i] & c) | (b[i] & c);
    end
end
endtask                                    //任务定义部分结束
always@(a or b)
adder (a, b, c);                           //任务调用语句
endmodule
```

上例所示的模块由“任务定义部分”和“always 过程块”两部分组成,其中定义的任务“adder”实现了一个 8 位加法器的运算功能,它有两个输入端口 a 和 b,有一个输出端口 adder。在任务被调用时,模块中的变量 a, b, c 分别和任务中的端口 a, b, adder 相互对应,这样在任务执行完毕后运算结果(即 adder 的值)可以通过输出端口传递给变量 c。要注意在模块和任务内都定义了变量“c”,它们是两个不同的变量,在任务内定义的局部变量只能在任务内部使用。所以在这种情况下,出现在任务定义结构内的标识符“c”都代表局部变量,而在任务定义结构外出现的“c”则代表着模块内定义的变量 c。

下面是一个具体的、实现交通灯控制的例子,在该模块中使用任务可以使程序简洁



易懂。

【例 7-18】 交通灯控制模块。

```
module traffic_lights(red, amber, green):  
    output red, amber, green;  
    reg [2:1] order;  
    reg clock, red, amber, green;  
    parameter ON=1, OFF=0, RED_TICS=350,  
        AMBER_TICS=30, GREEN_TICS=200;  
    //交通灯初始化  
    initial  
    begin  
        order=2'b00;  
        light(red, amber, green, 0, order) ;  
    end  
    //交通灯控制时序  
    always  
    begin  
        order=2'b01;  
        light(red, amber, green, RED_TICS, order);    //调用开灯任务, 开红灯  
        order=2'b10;  
        light(red, amber, green, GREEN_TICS, order); //调用开灯任务, 开绿灯  
        order=2'b11;  
        light(red, amber, green, AMBER_TICS, order); //调用开灯任务, 开黄灯  
    end  
    //任务的定义, 该任务用于实现交通灯的开启  
    task light;  
        output red;  
        output amber;  
        output green;  
        input [31:0] tic_time;  
        input [2:1] order;  
    begin  
        red=OFF;  
        green=OFF;  
        amber=OFF;  
        case (order)  
            2'b01: red=ON;  
            2'b10: green=ON;  
            2'b11: amber=ON;
```




```
    endcase
    repeat (tic_time) @(posedge clock);    //等待 tic_time 个时钟上升沿
    red=OFF;                                //关所有灯
    green=OFF;
    amber=OFF;
end
endtask
//产生时钟脉冲的 always 块
always
begin
    #100 clock=0;
    #100 clock=1;
end
endmodule
```

这个例子描述了一个简单的交通灯的时序控制, 它还带有自己的时钟产生器。模块的输出 red, amber 和 green 分别控制着红灯、黄灯和绿灯的开和关。

任务“light”用来使指定颜色的交通灯开通指定长度的时间, 输入变量“order”的取值决定着被开通交通灯的颜色: 如果 order 取值为 2'b00, 则不开任何灯; 如果 order 取值为 2'b01, 则开红灯; 如果 order 取值为 2'b10, 则开绿灯; 如果 order 取值为 2'b11, 则开黄灯。

输入变量“tic_time”的取值决定着对应交通灯被开通的持续时间, 这个持续时间将是 tic_time 个时钟周期长度。

7.3.2 函数(function)

函数(function)类似于其它编程语言中的函数概念。与任务一样, Verilog HDL 语言中的函数使用包括了函数的定义和函数的调用。

1. 函数的定义

函数定义的语法如下:

```
function <返回值类型或返回值宽度> <函数名>;
    <输入端口说明>
    <局部变量说明>
begin
    <行为语句1; >
    <行为语句2; >
    .....
    <行为语句n; >
end
endfunction
```

其中: 关键词 function 和 endfunction 表示这部分语句是一个函数定义结构, function 语句



标识着这个函数定义结构的开头；endfunction 语句标识着函数定义结构的结尾。

“<函数名>”是为被定义函数所取的一个名称，对被定义函数的调用是通过这个函数名来进行的。这个函数名在函数定义结构内部还代表着一个内部变量，函数调用后的返回值是通过这个函数名变量传递给调用语句(调用语句就是对函数进行了调用的语句)的。

“<返回值类型或返回值宽度>”是可选的，它用来对函数调用返回数据的类型或宽度进行说明(这个数据是通过函数名返回的)，它可以有如下三种形式：

(1) “[msb:lsb]”：这种形式说明函数名所代表的返回数据变量是一个多位的寄存器变量，它的位数由[msb:lsb]指定，比如如下函数定义语句：

```
function [7:0] adder;
```

就定义了一个函数“adder”，它的函数名“adder”还代表着一个 8 位宽的寄存器变量，其最高位为第 7 位，最低位为第 0 位。

(2) “integer”：这种形式说明函数名代表的返回变量是一个整数型变量。

(3) “real”：这种形式说明函数名代表的返回变量是一个实数型变量。

如果在函数的声明语句中 <返回值类型或返回值宽度>这一项缺省，则认为函数名代表的变量是一个一位的寄存器。

“<输入端口说明>”用来对函数各个输入端口的宽度和类型进行说明，在函数定义中必须至少有一个输入端口；该输入端口说明语句的语法与模块定义时的输入端口说明语句的语法是一致的。要注意在函数定义结构中不允许出现任何输出端口(output)和输入/输出端口(inout)。

“<局部变量说明>”是对函数内部局部变量进行的宽度和类型说明。

“行为语句”部分由关键词“begin”和“end”界定，这部分语句指明了函数被调用时要执行的操作，它们决定着函数实现的运算功能。在函数被调用时，这些行为语句将按串行方式得到执行。

我们可以用如下的例子来说明函数定义的格式。

【例 7-19】函数的定义。

```
function [7:0] getbyte ;           //函数定义结构的开头、注意此行中不能出现端口名列表
    input [63:0] word;              //说明第一个输入端口(输入端口 1)
    input [3:0] bytenum;            //说明第二个输入端口(输入端口 2)
    integer bit;                    //局部变量说明
    reg [7:0] temp;                 //局部变量说明
    begin
        for (bit=0; bit<=7; bit=bit+1)
            temp[bit] = word [((bytenum-1)*8)+bit]; //第一条行为语句
        getbyte = temp;              //第二条行为语句：将结果赋值给函数名变量 getbyte
    end
endfunction                        // 函数定义结束
```

上例定义了一个名为“getbyte”的函数，该函数有两个输入端口：一个 64 位的输入端口“word”和一个 4 位的输入端口“bytenum”。同时该函数还定义了两个局部变量 bit(整数型变量)和 temp(8 位寄存器变量)。

上例所定义的函数被调用时, begin 和 end 之间的行为语句部分得到执行, 其中第一条 for 循环语句实现了将 64 位寄存器 word 的一部分位的值赋给变量 temp, 需要赋值的位由输入 bytenum 指定。第二条赋值语句将得到的 temp 值赋给函数名变量 “getbyte”。这样就可以通过这个函数名变量将结果值返回给调用语句。

在进行函数定义时必须注意:

- (1) 与任务一样, 函数定义结构只能出现在模块中, 而不能出现在过程块内。
- (2) 函数至少必须有一个输入端口。
- (3) 函数不能有任何类型的输出端口(output 端口)和双向端口(inout 端口)。
- (4) 在函数定义结构中的行为语句部分内不能出现任何类型的时间控制描述, 也不允许使用 disable 终止语句。
- (5) 与任务定义一样, 函数定义结构内部不能出现过程块。
- (6) 在一个函数内可以对其它函数进行调用, 但是函数不能调用其它任务。
- (7) 在第一行 “function” 语句中不能出现端口名列表。

2. 函数的调用

函数调用的格式如下:

<函数名> (<输入表达式 1>, <输入表达式 2>, ……, <输入表达式 m>);

其中, m 个 “<输入表达式>” 与函数定义结构中说明的各个输入端口一一对应, 它们代表着各个输入端口的输入数据。这些输入表达式的排列顺序及类型必须与各个输入端口在函数定义结构中的排列顺序及类型保持严格一致。

在调用函数时必须注意如下两点:

- (1) 函数的调用不能单独作为一条语句出现, 它只能作为一个操作数出现在调用语句内。

例如, 下面这条语句对前面所定义的函数 “getbyte” 进行了调用:

```
out = getbyte( input1, number);
```

在这条调用语句中, 函数调用部分 “getbyte(input1, number)” 被看作是一个操作数, 这个操作数的取值就是函数调用的返回值。在整个调用语句中, 函数调用部分是作为 “赋值表达式” 出现在整条过程赋值语句中的, 函数调用部分不能单独地作为一条语句出现, 这就是说语句 “getbyte(input1, number);” 是非法的。

- (2) 函数调用既能出现在过程块中, 也能出现在 assign 连续赋值语句中。比如语句:

```
wire[7:0] net1;
```

```
reg [63:0] input1;
```

```
assign net1= getbyte( input1, 3 );
```

中的函数调用就出现在一条连续赋值语句内, 这条语句指定由函数调用返回值对 8 位连线型变量 net1 进行连续驱动。

进行函数调用后, 上述函数调用格式中的函数名将代表返回值参与调用语句(对函数进行调用的语句)的执行。

举例来说, 对上面定义的函数 “getbyte” 进行调用的模块可以是例 7-20 所示的模块:

**【例 7-20】** 函数的调用。

```
module demo_function_call;
reg[7:0] call_output;
reg[63:0] call_input1;
reg[3:0] call_input2;
initial
begin
    call_input1=64'h123456789abcdef0;
    call_input2=3;
    call_output=getbyte(input1, input2);           //第一次调用
    $display("after the first call, the returned value is : %b", call_output);
    #100;
    $display("second call, return value: %b ", getbyte(input1, 6)); //第二次调用
end
<函数 getbyte 定义部分>
endmodule
```

上例模块中的 initial 过程块对函数“getbyte”进行了两次调用：

函数 getbyte 的第一次调用是作为过程赋值语句“call_output=getbyte(input1, input2);”右端的赋值表达式出现的，调用时的输入表达式分别为两个寄存器变量 input1 和 input2，它们将与函数定义结构中的第一个和第二个输入端口相对应，因此这两个寄存器变量的取值将分别被传递给函数输入端口“word”和“bytenum”。函数调用完成后，过程赋值语句中的“getbyte”将具有函数调用的返回值，这个返回值将作为“赋值表达式”参与对变量 call_output 进行的过程赋值操作。

函数 getbyte 的第二次调用是作为系统任务 \$display 语句内的“输出变量表项”出现的，调用时的输入表达式分别是一个寄存器变量“input1”和一个常数“6”，它们的值将被分别传递给函数定义中的两个输入端口“word”和“bytenum”。函数调用完成后，\$display 语句中的“getbyte”将具有函数调用的返回值，这个返回值将作为“输出变量表项”参与 \$display 语句的执行。

由以上所给出的函数定义和函数调用的例子可以看出，在函数定义中必须有一条赋值语句来对函数名变量进行赋值，这样才能通过这个函数名变量来将函数调用的结果(返回值)传递给调用语句。在例 7-19 给出的函数定义中，语句“getbyte = temp;”就是这么一条赋值语句，它将函数调用的运算结果(变量 temp 的取值)赋值给函数名变量 getbyte。

函数的每一次调用只能通过函数名变量返回一个值。在有些时候我们要求一次函数调用能返回多个值，可以通过在函数定义和函数调用中使用合并操作符“{”、“}”来解决这一问题。

【例 7-21】 由一个函数返回多个值的方法。

```
module demo_multiout_function;
reg [7:0] a, b, c, d;
initial
```




```
begin
    a=8'h54;
    b=8'h32;
    {c, d}=multiout_fun(a, b);    //语句 S1, 进行了函数调用的过程赋值语句
    $display("the value of c is:%h  d is  %h", c, d);
end
function[15:0] multiout_fun;
    input[7:0] in1, in2;
    reg [7:0] out1;
    reg [7:0] out2;
    begin
        out1=in1 & in2;
        out2=in1 | in2;
        multiout_fun={out1, out2};    //语句 S2, 对函数名变量进行赋值
    end
endfunction
endmodule
```

上例中定义了一个函数 `multi_fun`, 我们想从这个函数中同时得到两个返回值 `out1` 和 `out2`, 由于函数调用时只能由函数名返回一个值, 因此 `out1` 和 `out2` 的取值不可能同时独立地被返回, 但是可以借助于合并操作符“{, }”来实现多个输出值的同时输出:

在函数定义内, 语句 `S2` 通过合并操作符把两个输出值 `out1` 和 `out2` 合并成一个值并将它赋值给函数名变量“`multiout_fun`”。这样, 虽然函数在形式上还是只有 `multiout_fun` 变量这一个返回值, 但是在这个返回值内实际上包含了两个输出数据。

在对函数进行了调用的过程赋值语句 `S1` 中, 函数调用返回值“`multiout_fun`”被赋值给由两个变量 `c` 和 `d` 组合而成的一个合并变量。这样, 在执行了赋值语句 `S1` 后, 函数调用返回值“`multiout_fun`”中所包含的两部分输出值将分别被赋值给变量 `c` 和 `d`, 这样也就是实现了将函数输出 `out1` 和 `out2` 的值分别传递给变量 `c` 和 `d`。

在使用函数时必须注意: 由于在函数定义结构中不能出现任何类型的时间控制语句, 所以函数调用执行所需的时间只能是零仿真时间, 也就是说, 函数调用启动时刻和函数调用返回时刻是相同的。

下面给出一个较复杂的函数使用的例子。例 7-22 描述了一个乘加器, 其结构如图 7.2 所示, 其中的乘法器是利用函数 `mult` 来实现的。

【例 7-22】乘加器。

```
`timescale 1 ns/ 10 ps
module mult_acc(out, ina, inb, clk, clr);
    input [7:0] ina, inb;
    input clk, clr;
    output [15:0] out;
    wire [15:0] mult_out, adder_out;
```




```

reg [15:0] out;
parameter set = 10;
parameter hld = 20;
assign adder_out = mult_out + out;      //描述了加法器
assign mult_out = mult (ina, inb);      //进行了函数调用，实现了乘法器
always @ (posedge clk or posedge clr)   //描述了时序逻辑，实现清零功能及输出锁存
begin
    if (clr)
        out = 16'h0000;
    else
        out = adder_out;
end
function [15:0] mult;                  //函数定义部分
input [7:0] a, b ;
reg [15:0] r;
integer i ;
begin
    if (a[0] == 1)
        r = b;
    else
        r = 0 ;
    for (i=1; i<=7; i=i+1)
    begin
        if (a[i] == 1)
            r = r + b <<i ;
        end
    mult = r;
end
endfunction
endmodule

```

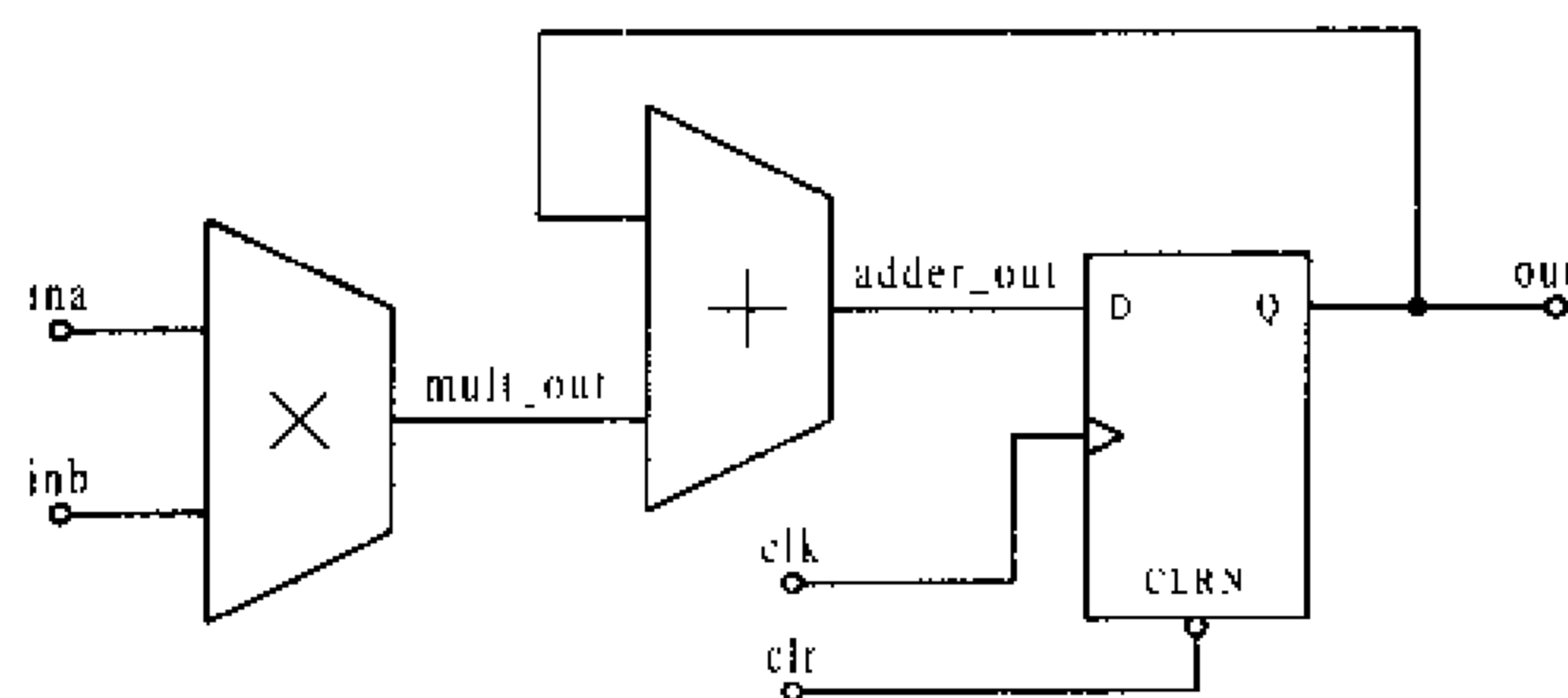


图 7.2 乘加器结构

上例所示模块由两条连续赋值语句、always 过程块、函数定义和延时说明块等几部分组成。其中:

第一条连续赋值语句描述了一个加法器, 其输入端为 multi_out 和 out, 输出端为 adder_out。

第二条连续赋值语句通过对函数 mult 的调用实现了一个乘法器, 其输入端为 ina 和 inb, 输出端为 mult_out, 其中的函数调用实现了乘法运算功能。

always 过程块对乘法器的时序逻辑进行了描述, 实现了异步清零和输出数据锁存功能, 每当清零信号 clr 变为高电平就进入清零操作状态, 对输出锁存器进行清零操作; 在正常工作状态下输出锁存器将在时钟 clk 的每个上升沿得到刷新。

函数定义部分对函数 mult 进行了定义, 该函数实现了一个 8 位乘法运算功能, 它有两个输入变量 a 和 b, 返回一个 16 位的运算结果。

综上所述, 函数和任务之间存在如下不同特点:

- (1) 函数至少必须有一个输入端口。而任务可以有一个或多个输入端口, 也可以没有输入端口。
- (2) 函数不能有输出端口。而任务可以有一个或多个输出端口, 也可以没有输出端口。
- (3) 函数调用通过函数名返回一个返回值, 而任务调用必须通过输出端口传递返回值。
- (4) 在函数中不能调用其它任务, 而在任务中则可以调用其它的任务或函数。
- (5) 函数调用不能单独作为一条语句出现, 它只能以语句的一部分的形式出现, 而任务的调用则是通过一条单独的任务调用语句实现的。
- (6) 函数调用可以出现在过程块或连续赋值语句中, 而任务调用只能出现在过程块中。
- (7) 函数定义内不允许存在时间控制语句, 函数的执行总是在零仿真时间内完成, 而任务定义内可以出现时间控制语句, 任务的执行可以占用非零时间单位。
- (8) 函数的执行不允许由 disable 语句进行中断, 而任务的执行可以由 disable 语句进行中断。



第8章 结构描述



前面讲过, Verilog HDL 语言对硬件功能的描述可以分成行为描述方式和结构描述方式两种类型。在一个模块内可以只采用其中的任何一种描述方式,也可以同时采用这两种描述方式(行为描述和结构描述混合的描述方式)。与此相对应,可以依据模块所采用的描述方式类型将其分为“行为描述模块”、“结构描述模块”以及“混合描述模块”三种类型。本章将对结构描述方式以及结构描述模块进行讨论。

8.1 结构描述方式

Verilog 语言的基本结构成分是模块。模块可以代表从简单门元件到复杂系统(比如一个微处理器)的任何一种硬件电路。一个复杂电路在进行 Verilog 建模后表现为一个顶级模块,该顶级模块的输入输出端口分别代表着电路的输入输出端;顶级模块可以由若干个子模块构成,每个子模块代表着整个电路内的具有特定功能的一个功能单元,各个子模块通过子模块端口相互连接起来;子模块又可以被分成若干个第二级子模块,代表着对上一级电路功能单元的更进一步的细分;这种细分一直可以进行到基本逻辑单元(门级元件和开关级元件)一级。这样,整个硬件电路在经过 Verilog 建模后将表现为分层次的、相互联系的一组模块。

结构描述方式就是将硬件电路描述成一个分级子模块系统(分级子模块互连网络)的一种描述方式。在这种描述方式下,组成硬件电路的各个子模块之间的相互层次关系以及相互连接关系都需要得到说明。由于任何硬件电路在结构上都是由一级级不同层次的若干功能单元组成的,所以结构描述方式很适合于用来对电路的结构特点进行说明,这也是“结构描述方式”这种称法的由来。结构描述方式的描述目标是电路的这种层次结构,组成硬件电路的各层功能单元将被描述成各个级别的子模块。

在 Verilog 语言中,一个模块可以对其它模块进行调用(也称模块的实例化),其中调用模块成为层次结构中的上级模块,而被调用模块成为下级模块。与函数调用不同,模块调用实现的不是程序流程的改变,而是代表着硬件电路的复制过程:调用模块每一次对被调用模块进行调用,被调用模块对应的硬件电路结构就会在调用模块内被复制一次,这个硬件拷贝在调用模块内被称为“被调用模块的一个实例”。

如果在某个模块内只包含对下级模块进行调用的语句(称为“模块实例语句”),而不包含任何行为描述语句时,则称该模块是采用了结构描述方式的一个“行为描述模块”。

根据所调用子模块的不同抽象级别,可以将模块的结构描述方式分成如下三类:

(1) 模块级结构描述(模块级建模):是指调用由用户设计生成的低级子模块来对硬件电路结构进行说明,这种情况下模块将由低级模块的实例组成。



(2) 门级结构描述(门级建模): 是指调用 Verilog 内部的基本门级元件来对硬件电路的结构进行说明, 这种情况下模块将由基本门级元件的实例组成。

(3) 开关级结构描述(开关级建模): 是指调用 Verilog 内部的基本开关级元件来说明硬件电路的结构, 这种情况下模块将由开关级元件的实例组成。

本章将分别对“模块级结构描述”以及“门级结构描述”这两种结构描述方式进行讨论。由于“开关级结构描述”主要用于对电路基本部件(门、缓冲器、驱动器等)的模型库进行设计, 而普通用户一般很少使用, 因此在本书中略去了对开关级描述方式的介绍。

8.2 模块级建模

8.2.1 模块的定义

在 Verilog HDL 中, 硬件电路中的各级功能单元被描述成模块, 模块具有如下的结构:

```
module <模块名> (模块端口列表);  
    声明语句  
    模块组项  
    函数和任务定义  
    specify 块  
endmodule
```

其中: “<模块名>”是给模块所取的一个名称, 它是模块的一个标识符。

“模块端口列表”是由输入端口、输出端口和双向端口的端口表达式按一定的次序组成的一个列表, 它用来指明模块所具有的端口。模块可以利用这些端口与其它外部模块进行通信, 利用端口列表可以将本模块和其它模块连接起来。

“声明语句”部分用来对模块端口类型(输入端口、输出端口或双向端口)、模块内部变量的类型(寄存器、存储器和连线类型)以及模块内参数(parameter)进行说明。其中要用到如下声明语句:

- parameter: 参数声明语句。
- input: 输入端口声明语句。
- output: 输出端口声明语句。
- inout: 双向端口声明语句。
- net-type: 连线类型变量声明语句。
- register-type: 寄存器类型变量声明语句。
- time: 时间类型变量声明语句。
- integer: 整数类型变量声明语句。
- real: 实数类型变量声明语句。
- event: 有名事件声明语句。

“模块组项”部分是模块功能的描述体部分, 用来对模块的功能特点进行描述, 在这一部分可以出现具有行为描述性质的语句结构(initial 过程块、always 过程块或连续赋值语



句),也可以出现结构描述语句(模块实例语句、门级元件实例语句、开关级元件实例语句)。如果在这个“模块组项”中只使用了结构描述语句,那么该模块就是一个“结构描述模块”。

“函数和任务定义”部分用来对“模块组项”内使用到的一些函数和任务进行定义。由于函数和任务都是行为描述性质的程序结构,所以在“结构描述模块”内不能使用函数或任务。

“specify 块”用于对模块输入、输出端口之间的路径延时时间量进行说明,同时还可以用来进行时序检验。

8.2.2 模块的端口

一个模块往往具有多个端口,它们代表了本模块和其它模块间的接口。在上一小节给出的模块定义格式中,“模块端口列表”列出了模块具有的外部可见端口,这个“模块端口列表”内的每一个端口项都代表着一个模块端口。

1. 模块端口列表

模块端口列表可以是如下两种形式之一:隐式端口名形式、显式端口名形式。

1) 隐式端口名形式

这种形式的模块端口列表的格式为:

(<端口表达式 1>, <端口表达式 2>, …… , <端口表达式 n>)

其中的各个“<端口表达式>”指明了各个模块端口与模块内变量的连接关系,一个端口表达式可以是:

- ①模块内某个变量的标识符;
- ②模块内某个矢量变量的某一位;
- ③模块内某个矢量变量的某几位;
- ④上面三种形式的组合。

在隐式端口名形式的端口列表中,没有显式地为各个模块端口指定端口名,这时各个模块端口的端口名将按照如下规则得到隐式定义:

- 如果给出的“端口表达式”是一个变量标识符(就是前面所列出的第 1 种端口表达式情况),则该端口表达式对应的端口名被默认地定义为该标识符本身。

- 如果给出的“端口表达式”是前面所列出的第 2、3、4 种端口表达式情况(此时该“端口表达式”不是一个标识符),那么对应的端口将处于“无名状态”(不存在端口名的状态)。

例 8-1 给出了模块端口列表的几个例子。

【例 8-1】隐式端口名形式的模块端口列表。

```
module ex1( a, b, z );
    input a, b;           //端口声明语句
    output z;             //端口声明语句
endmodule

module ex2( a[1], a[0], z );
    input [1: 0] a;       //端口声明语句
```



```
output z;           //端口声明语句
endmodule

module ex3( {a, b}, z );
input a, b;         //端口声明语句
output z;           //端口声明语句
endmodule
```

在例 8-1 中定义了三个模块“ex1”、“ex2”和“ex3”，这三个模块定义中都使用了隐式端口名形式的模块端口列表。

其中模块“ex1”的模块端口列表是由三个标识符“a”、“b”、“z”所组成的，它们指明模块“ex1”的三个端口将分别与三个模块内部变量“a”、“b”和“z”相连接，此时这三个端口将拥有隐式定义的端口名“a”、“b”和“z”。“ex1”模块定义中的两条端口声明语句(input 语句和 output 语句)指明模块端口“a”和“b”为输入端口，模块端口“z”为输出端口。

模块“ex2”的端口列表中的第 1、2 个端口表达式分别是矢量变量 a 的两个位“a[1]”和“a[2]”，这表明模块 ex2 的前两个端口将分别被连接到变量 a 的这两个位上，由于这时的端口表达式不是标识符的形式，所以这两个端口将没有端口名；而端口列表中的第三个端口表达式是标识符“z”，它表示模块 ex2 的第三个端口将与模块内部变量 z 相连接，同时这个端口具有隐式定义的端口名“z”。

模块“ex3”中端口列表内的第一个端口表达式是两个变量 a、b 的合并“{a, b}”，它表示模块 ex3 的第一个端口将被连接到连线变量 a、b 的合并“{a, b}”，也就是说这个端口的高位将被连接到变量 a，低位将被连接到变量 b。由于这个端口表达式不是标识符的形式，所以模块的第一个端口没有端口名；而端口列表中的第二个端口表达式是标识符“z”，它表示模块 ex3 的第二个端口将与变量 z 相连接，这个端口具有隐式定义的端口名“z”。

2) 显式端口名形式

模块端口列表在这种形式下的格式为：

```
(.<端口名 1>(<端口表达式 1>) .<端口名 2>(<端口表达式 2>) , ..... ,
.<端口名 n>(<端口表达式 n>))
```

在上述格式中，列表中的每一项都包含着一个“端口名”和一个“端口表达式”，在每个“端口名”前面都要加一个点号“.”来表明它是一个端口名；而“端口表达式”则位于“端口名”后面的圆括号内。

在这种端口名列表形式下，列表中的每一项都说明了一个端口：其中的“端口名”为该端口指定了一个端口名，“端口表达式”则指明了该端口与模块内部的连接关系。与隐式端口名的情况一样，这里的端口表达式也可以是前面给出的四种端口表达式形式之一。

在使用了这种“显式端口名形式的模块端口列表”后，各个模块端口都被显式地指定了一个端口名，在进行模块调用时可以通过这些端口名来指明外部信号端子与模块端口的连接关系。

例 8-2 给出了显式端口名形式模块端口列表的几个例子。



【例 8-2】显式端口名形式的模块端口列表。

```
module ex4( .in_a(a), .in_b(b), .out(z) );
input a, b;      //端口声明语句
output z;        //端口声明语句
endmodule

module ex5( .i1(a[1]), .i0(a[0]), .out(z) );
input [1: 0] a;   //端口声明语句
output z;         //端口声明语句
endmodule
```

例 8-2 中定义了两个模块 ex4 和 ex5，这两个模块定义都采用了显式端口名形式的端口列表。

其中模块 ex4 的端口列表是由三个端口项构成的，其中的每一个端口项定义说明了一个模块端口。第一个端口被取名为“in_a”，它与模块内变量 a 相连接；第二个端口被取名为“in_b”，它与模块内变量 b 相连接；第三个端口被取名为“out”，它与模块内变量 z 相连接。

模块 ex5 的端口列表也是由三个端口项构成的，它定义说明了端口名分别为“i1”，“i0”和“out”的三个模块端口。其中端口“i1”与变量 a 的第 1 位相连接，端口“i0”与变量 a 的第 0 位相连接，端口“out”与模块内变量 z 相连接。

模块定义中的端口列表也可以是由显式端口名形式和隐式端口名形式的端口项共同组成的。(参见下例)。

【例 8-3】由显式端口名形式和隐式端口名形式的端口项共同组成的模块端口列表。

```
module ex6( .i1(a), .i0(b), z );
input a, b ;      //端口声明语句
output z;         //端口声明语句
endmodule
```

例 8-3 中定义了模块“ex6”，在它的端口列表中同时包含了显式端口名形式和隐式端口名形式的端口项。其中第一个端口项“.i1(a)”和第二个端口项“.i0(b)”是显式端口名形式的，它们分别定义了端口名为“i1”和“i0”的两个端口，这两个端口分别与模块内变量 a 和 b 相连接；而端口列表中的第三个端口项“z”则是隐式端口名形式的，它定义的端口是与变量 z 相连接的，这个端口的端口名被隐式地定义为“z”。

模块端口列表中可以出现与模块内部没有任何连接的端口项，例如如下的模块定义：

```
module no_inner_connect ( .data( ), .control(ctrl), .out(out_data));
output[8: 0] out_data;
input ctrl;
.....
endmodule
```

模块“no_inner_connect”中的端口列表定义了三个模块端口，其端口名分别被指定为“data”、“control”和“out”。但是端口“data”所对应的端口表达式是空的(空括号)，这表明没有任何一个内部变量和该端口相连接。

也可以出现多个模块端口与同一个内部变量相连接的情况。例如如下模块:

```
module fanout (.a(ctrlin), .b(condout), .c(condout));
input ctrlin;
output condout;
assign condout= ctrlin;
endmodule
```

在模块“fanout”的端口列表中定义了三个模块端口,它们的端口名分别是“a”、“b”和“c”。其中与端口“b”和“c”对应的端口表达式都是“condout”,这表明这两个端口都是与内部变量“condout”相连接的,因此在模块端口“b”和“c”上输出的都是condout的取值。

2. 端口声明语句

在端口列表中出现的各个模块端口还必须由“端口声明语句”来进行端口方向的说明。Verilog 语言中有如下三种端口声明语句:

(1) input 输入端口声明语句:说明对应的端口是一个输入端口。

(2) output 输出端口声明语句:说明对应的端口是一个输出端口。

(3) inout 双向端口声明语句:说明对应的端口是一个双向端口,该端口既能用于输入也可用于输出。

端口声明语句的作用对象必须是与模块端口相连的模块内部变量,而不能是端口名(端口名和内部变量标识符同名的情况除外)。比如在例 8-2 中,模块“ex4”中的端口声明语句“input a, b;”以及“output z;”的作用对象就分别是与三个端口相连接的内部变量 a、b 和 z。这是由于端口名的作用(在进行模块调用时)是作为模块端口的标识来实现与外部信号的连接,在模块内部这些端口名是不可见的。模块内部对端口进行的输入输出操作是通过端口表达式中出现的那些内部变量进行的(这些变量与对应的端口相连),所以在模块内部必须对这些内部变量的输入输出特性进行说明。

8.2.3 模块的调用

一个 Verilog 模块可以被任意多个其它模块调用,“模块调用”与“函数调用”在本质上有着很大的差别:由于 Verilog HDL 所描述的是具体的硬件电路,一个模块代表着拥有特定功能的一个电路块,每当一个模块在其它模块内被调用一次,被调用模块所表示的电路结构就会在调用模块代表的电路内部复制一次(调用模块中的这部分复制电路结构被称为“被调用模块的一个实例”)。之所以模块调用不能像函数调用一样具有“退出调用”的操作,是因为硬件电路的结构是不会随着时间而发生变化的,被复制的电路块将一直存在下去。

模块调用是 Verilog 结构描述的基本描述方式。“基本门级元件调用”、“开关级元件调用”和“用户自定义元件(UDP)调用”都可以看作是特殊类型模块的调用。

通过模块调用,一个复杂电路可以被描述成是由各级模块像搭积木那样一层层组成的。在对一个硬件系统进行的描述中,必定存在着一个“顶级模块”,在整个硬件系统描述中只有这个“顶级模块”不被其它模块所调用,而其它各级模块都将被上一级模块调用。



在 Verilog 中, 模块被调用后就在上一级调用模块内生成了一个“模块实例”。模块实例代表着被调用模块电路结构在调用模块电路内的一个拷贝。

1. 模块实例语句

Verilog 语言中用“模块实例语句”来实现模块调用。模块实例语句的基本格式如下:

<模块名> <参数值列表> <实例名> (<端口连接表>);

在上面的格式中:

“<模块名>”就是在进行模块定义时(module 语句中)为被调用模块指定的模块名。

“<参数值列表>”在上面的格式中是可选项, 它是由一些参数值组成的一张有序列表, 这些参数值将被传递给被调用模块实例内的各个参数。我们将在 8.2.4 小节对这个参数值列表进行更为详细的讨论。

“<实例名>”是为本次模块调用后所生成的模块实例(被调用模块的模块实例)所取的一个名称, 它可以用来对所生成模块实例进行唯一标识。如果在某个模块内进行了多次模块调用, 那么各次调用所指定的实例名必须各不相同, 在同一模块中不能出现两个相同的实例名。这是因为在硬件描述语言中模块调用的含义是将被调用模块在调用模块中原原本本地复制一遍。同一个上级模块可以对多个下级模块进行调用, 也可以对一个下级模块进行多次调用。这样就会在同一电路中生成为多个一模一样的电路结构单元, 这些电路结构单元就是每次模块调用后生成的“模块实例”。所以为了对这些相同的电路结构单元进行区分, 为它们所取的实例名应该是各不相同的。实例名和模块名的区别为: 模块名标识着不同的模块, 用来区分电路单元的不同种类; 而实例名则标识着不同的模块实例, 用来区分电路系统中的不同硬件电路单元。

“<端口连接表>”是由“外部信号端子”组成的一张有序列表, 这些“外部信号端子”代表着与模块实例各端口相连的外部信号。所以“端口连接表”指明了模块实例端口与外部电路的连接情况。

如果需要在当前模块内对同一个被调用模块进行多次调用, 则可以采用如下所示的模块实例语句格式:

```
<模块名> <参数值列表 1> <实例名 1> ( <端口连接表 1> ),  
      <参数值列表 2> <实例名 2> ( <端口连接表 2> ),  
      .....  
      <参数值列表 n> <实例名 n> ( <端口连接表 n> );
```

在上面的模块实例语句格式中:

“<模块名>”指明了被调用的是哪一个模块。

格式中的每一行被称为一个“实例项”, 各个实例项之间用逗号隔开。

每个实例项实现了被调用模块的一次调用:

- (1) 所生成的模块实例被命名为该实例项内指定的“实例名”;
- (2) 传递给模块实例的参数值由该实例项内的“参数值列表”指明;
- (3) 模块实例与外部信号的连接由该实例项内的“端口连接表”指定。

整条模块实例语句实现了对同一模块的 n 次调用, 将在调用模块中生成 n 个模块实例。各个实例项对应的“实例名”必须是各不相同的, 否则将会出现不同模块实例具有相同实例名的情况。

下面给出一个例子来说明模块的调用和模块实例语句的使用。

【例 8-4】模块调用的例子。

```
// 对“与非门”进行建模的行为描述模块: NAND
module NAND(ina, inb, nand_out);
input ina, inb;
output nand_out;
// 连续赋值语句, 描述了“与非门”的输入输出关系
assign nand_out = ~(ina & inb);
endmodule

// 结构描述模块 AND: 由两个与非门来构成一个与门
module AND(in1, in2, and_out);
input in1, in2;
output and_out;
wire w1;           // 内部连线变量 w1
// 对模块 NAND 的两次调用
NAND NAND1(in1, in2, w1);
NAND NAND2(w1, w1, and_out);
endmodule
```

上例中定义了两个模块: NAND 和 AND。其中模块 NAND 采用的是行为描述方式, 它是一个行为描述模块, 其输出 nand_out 被描述成是输入 in1 和 in2 的“与非”运算结果, 所以模块 NAND 代表一个“与非门”; 而模块 AND 中只包含了两条模块实例语句, 其中没有出现任何行为描述语句, 所以模块 AND 是一个结构描述模块。

模块 AND 内的两条模块实例语句分别实现了对模块“NAND”的两次调用。

● 在第一条模块实例语句“NAND NAND1(in1, in2, w1);”内:

被调用模块名是“NAND”, 它表示被调用的是“NAND 模块”。

实例名是“NAND1”, 它表示本次调用所生成的模块实例被取名为 NAND1。

端口连接表是“(in1, in2, w1)”, 它表示模块实例 NAND1 的各个端口(端口 ina, inb 和 nand_out)将分别与模块 AND 内的变量 in1, in2 和 w1 相连接。

● 在第二条模块实例语句“NAND NAND2(w1, w1, and_out);”中:

被调用模块名是“NAND”, 它表示这次被调用的还是“NAND 模块”。

实例名是“NAND2”, 它表示第二次调用所生成的模块实例被取名为 NAND2。

端口连接表为“(w1, w1, and_out)”, 它表示模块实例 NAND2 的各个端口(端口 ina, inb 和 nand_out)将分别与模块 AND 内的变量 w1, w1 和 and_out 相连接。

根据上面的解释, 我们可以确定模块 AND 具有如图 8.1 所示的组成结构。

从图 8.1 可以看出, 模块 AND 是由两个与非门(实例 NAND1 和实例 NAND2)构成的。经过连线“w1”, 第一个与非门(实例 NAND1)的输出端口被同时连接到第二个与非门(实例 NAND2)的两个输入端口, 这样就构成了一个“与门”。

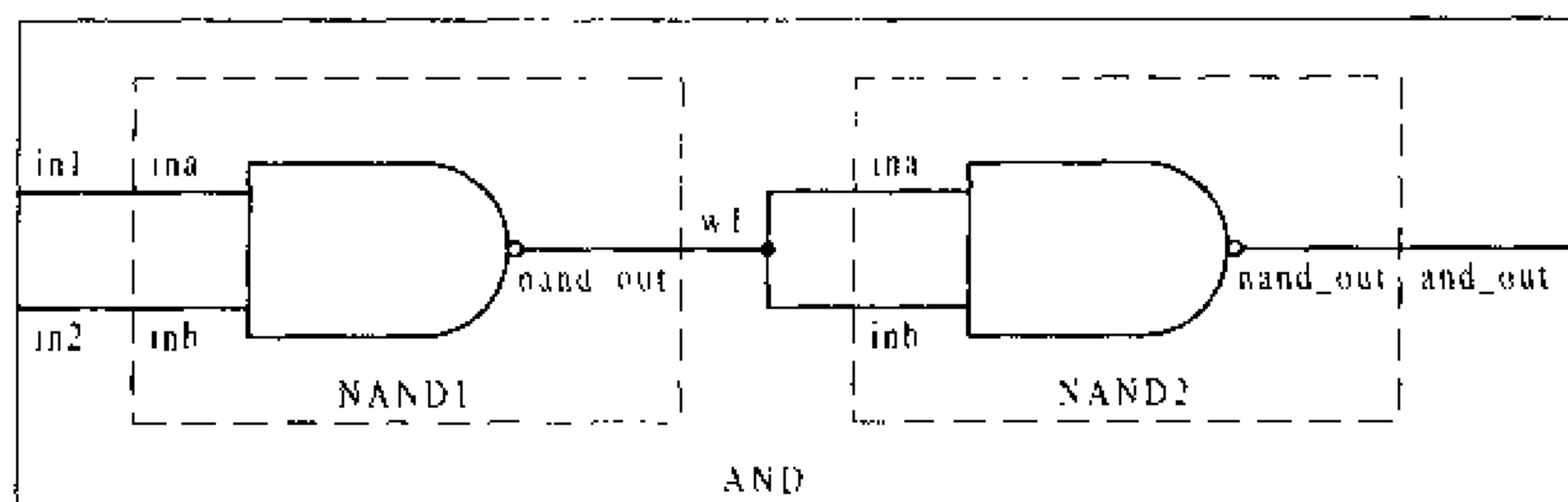


图 8.1 例 8-4 中模块 AND 的结构

2. 模块调用时的端口对应方式

在进行模块调用时，模块实例语句中的“<端口连接表>”用来指明模块实例端口与外部电路的连接情况。端口连接表是由被称为“外部信号端子”的一些表达式组成的，其中的各个信号端子将分别与对应的端口相连接。

在“端口连接表”中，可以采用如下两种方式指定模块实例端口与外部信号端子之间的连接关系。

1) 端口名称关联方式

在端口名称关联方式下，端口连接表的格式为：

(.<端口名 1> (<信号端子 1>), .<端口名 2> (<信号端子 2>),
……, .<端口名 n> (<信号端子 n>))

其中的“信号端子”可以是下面列出的任何一种类型：

- (1) 变量标识符(寄存器类变量或连线类变量标识符)。
- (2) 矢量变量的某一位。
- (3) 矢量变量的某几位。
- (4) 前面三种类型的合并。
- (5) 数值表达式(只适用于输入端口)。

在端口名称关联方式下，端口连接表内显式地指明了与每个外部信号端子相连的模块端口名。也就是说“端口名 1”所代表的端口将与“信号端子 1”相连接；“端口名 2”所代表的端口将与“信号端子 2”相连接，……，“端口名 n”所代表的端口将与“信号端子 n”相连接。例如在如下给出的例 8-5 中，模块“two_bits_full_adder”内的两条模块实例语句对子模块 full_adder 的调用就采用了“端口名称关联”的端口对应方式。

【例 8-5】 端口名称关联的端口对应方式。

```
module full_adder(a, b, cin, s, cout);    //一位全加器，采用行为描述方式进行说明
input a, b, cin;
output s, cout;
assign s=(a^b)^cin;                    //本位和输出 s
assign cout=(a & b) | (b & cin) | (a & cin);    //进位输出 cout
endmodule

module two_bits_full_adder(data1, data2, carry_in, result, carry_out);
input[2: 1] data1, data2;
```



```

input carry_in;           //进位输入 carry_in
output[2: 1] result;      //结果输出 result
output carry_out;         //进位输出 carry_out
wire w1;
//两条模块实例语句，所采用的端口对应方式是端口名称关联方式
full_adder U1(.a(data1[1]), .b(data2[1]), .cin(carry_in), .s(result[1]), .cout(w1));
full_adder U2(.a(data1[2]), .b(data2[2]), .cin(w1), .s(result[2]), .cout(carry_out));
endmodule

```

例 8-5 实现的是一个二位全加器。模块 `full_adder` 描述了一个一位全加器的功能，它采用的是行为描述方式。而模块 `two_bits_full_adder` 则是一个结构描述模块，在这个结构描述模块中对模块 `full_adder` 进行了两次调用，这两次调用分别生成了实例名为“U1”和“U2”的两个模块实例(模块“`full_adder`”的实例)，这两个实例在硬件电路中分别代表了两个一位全加器。两次模块调用所采用的端口对应方式都是端口名称关联方式。

第一条模块实例语句内的端口连接表指明了模块实例 U1 各个端口与外部信号端子的如下连接情况：

模块端口 a → 信号端子 data1[1]
 模块端口 b → 信号端子 data2[1]
 模块端口 cin → 信号端子 carry_in
 模块端口 s → 信号端子 result[1]
 模块端口 cout → 信号端子 w1

第二条模块实例语句内的端口连接表指明了模块实例 U2 各个端口与外部信号端子的如下连接情况：

模块端口 a → 信号端子 data1[2]
 模块端口 b → 信号端子 data2[2]
 模块端口 cin → 信号端子 w1
 模块端口 s → 信号端子 result[2]
 模块端口 cout → 信号端子 carry_out

这样，我们可以确定模块“`two_bits_full_adder`”具有如图 8.2 所示的模块结构：

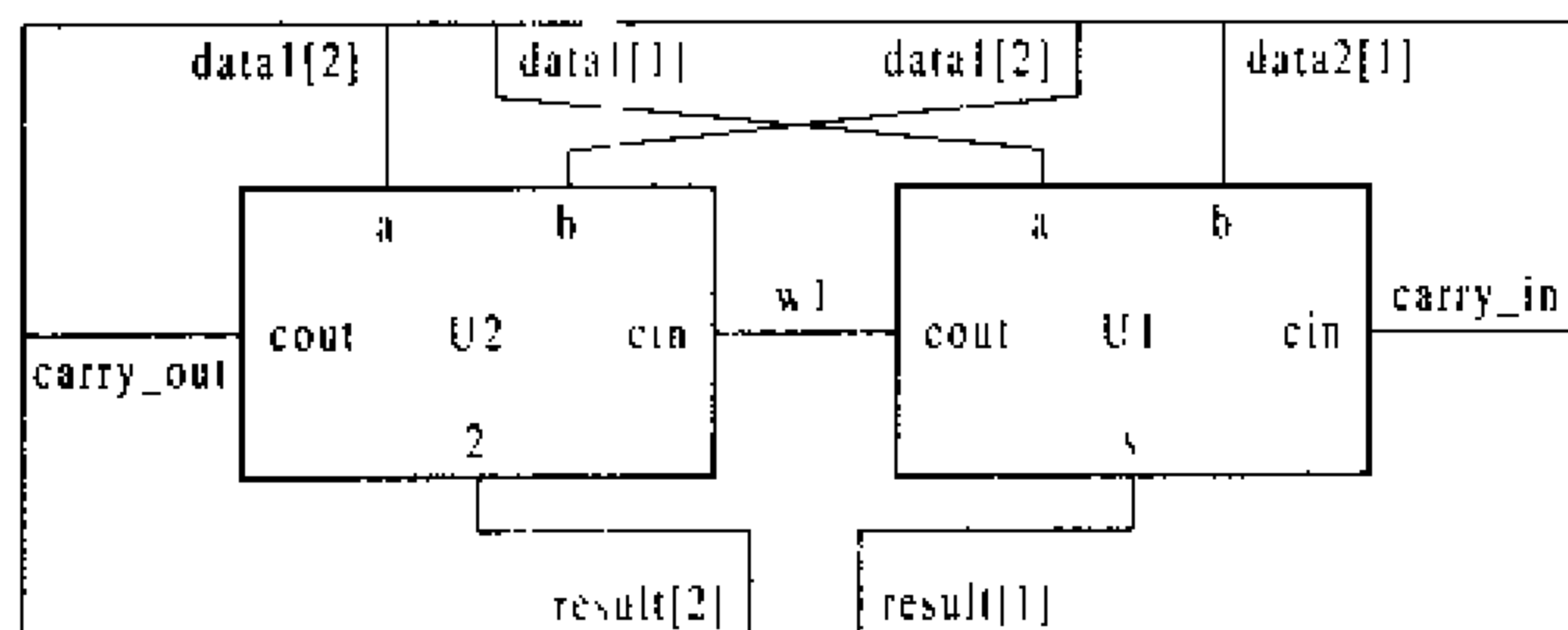


图 8.2 例 8-5 中模块“`two_bits_full_adder`”的结构

在端口名称关联方式下，模块端口和信号端子的连接关系被显式地说明，因此端口连



接表内各项的排列顺序对端口连接关系是没有影响的。例如我们可以用如下两条模块实例语句:

```
full_adder U1(.cin(carry_in), .a(data1[1]), .b(data2[1]), .s(result[1]), .cout(w1));
```

```
full_adder U2(.cout(carry_out), .a(data1[2]), .cin(w1), .s(result[2]), .b(data2[2]));
```

来替换例 8-5 中的两条模块实例语句, 这两组模块实例语句描述的都是相同的端口连接情况。

在端口名称关联方式下, 如果端口连接表内某一项的信号端子是缺省的(在“信号端子”位置处只出现一对空括号“()”), 那么这一项的端口名所代表的端口将处于悬空状态(不被连接)。比如在下面这条模块实例语句中:

```
full_adder U1(.a(data1[1]), .b( ), .cin(carry_in), .s(result[1]), .cout(w1));
```

端口连接表第二项内的信号端子是缺省的(空括号), 这种情况表明端口名“b”所代表的端口是悬空的。

如果被调用模块的某个端口不具有端口名(该端口处于无名状态), 则在对这个模块进行调用时不能采用端口名称关联方式, 而只能采用下面将介绍的“端口位置关联方式”。

2) 端口位置关联方式

在端口位置关联方式下, 端口连接表的格式为:

(<信号端子1>, <信号端子2>, ……, <信号端子n>)

其中各个“信号端子”可取的类型与前面讲述“端口名称关联方式”时的情况相同。

在端口位置关联方式下, 各个信号端子按一定的次序出现在端口连接表中, 这些信号端子将与在(模块定义时给出的)“端口列表”中出现的各个模块端口依次连接: 端口连接表中的第一个信号端子将与端口列表中的第一个模块端口相连接, 第二个信号端子将和第二个模块端口相连接, 依次类推。

举例来说, 我们可以将例 8-5 中的两条模块实例语句改写为如下两条(采用端口位置关联方式的)模块实例语句:

```
full_adder U1(data1[1], data2[1], carry_in, result[1], w1);
```

```
full_adder U2(data1[2], data2[2], w1, result[2], carry_out);
```

在定义模块“full_adder”时给出的端口列表是“(a, b, cin, s, cout)”, 这是一个隐式端口名形式的端口列表, 它为模块 full_adder 定义了四个模块端口, 它们被隐式地命名为端口名“a”, “b”, “cin”, “s”和“cout”。这样, 上面这两条模块实例语句内的各个信号端子将根据它们在端口连接表中出现的次序依次与模块端口“a”, “b”, “cin”, “s”和“cout”相连接:

第一条模块实例语句内的端口连接表指明模块实例“U1”的端口“a”、“b”、“cin”、“s”、“cout”将分别与外部信号端子“data1[1]”、“data2[1]”、“carry_in”、“result[1]”、“w1”依次相连。

第二条模块实例语句内的端口连接表指明模块实例“U2”的端口“a”、“b”、“cin”、“s”、“cout”将分别与外部信号端子“data1[2]”、“data2[2]”、“w1”、“result[2]”、“carry_out”依次相连。

这样, 在采用了上面两条模块实例语句后, 我们得到的还是如图 8.2 所示的模块结构。

若端口连接表中某一项的“信号端子”缺省，则表示这一项所对应的模块端口未被连接(悬空)。比如下面这条模块实例语句：

```
full_adder U1(data1[1], data2[1], , result[1], w1);
```

端口连接表内第三项的信号端子是缺省的，这种情况表明该项对应的模块端口(端口列表中的第三个端口“cin”)是悬空的。

需要注意的是，不能在同一个端口连接表内混合使用名称关联方式和位置关联方式。比如下面这条模块实例语句就是错误的：

```
full_adder U1(data1[1], data2[1], .b(carry_in), .s(result[1]), .cout(w1));
```

其中端口连接表内的第一项和第二项是位置关联方式的表项；而其中的第三、四、五项则是名称关联方式的表项。

3. 模块调用时的阵列调用方式

当需要对同一个模块进行重复性的多次调用时，我们可以采用如下所示的阵列调用方式：

<被调用模块名> <实例阵列名> [阵列左边界：阵列右边界] (<端口连接表>);

其中，“阵列左边界”和“阵列右边界”是两个常量表达式，它们用来指定调用后生成的模块实例阵列的大小。

【例 8-6】使用阵列调用方式的模块实例语句来进行结构描述。

```
module data_change (out, ina, inb);
input [3: 0] ina, inb;
output [3: 0] out;
wire [3: 0] out, ina, inb;
NAND NAND_ARRAY [3: 0] (out, ina, inb); //阵列调用方式的模块实例语句
endmodule

// 对“与非门”进行建模的行为描述模块：NAND
module NAND(ina, inb, nand_out);
input ina, inb;
output nand_out;
assign nand_out = ~(ina & inb); //连续赋值语句，描述了“与非门”的输入输出关系
endmodule
```

上例中的“data_change”模块是一个结构描述模块，该模块对“NAND”子模块进行了阵列调用方式的模块调用。其中的语句“NAND NAND_ARRAY [3: 0] (out, ina, inb);”就是一条阵列调用方式的模块实例语句，在这条语句中：

“调用模块名”是“NAND”，它指明被调用的是模块“NAND”；

“实例阵列名”是“NAND_ARRAY”，它用来对调用后生成的实例阵列进行标识；

“[3: 0]”定义了实例阵列的大小，它指明该实例阵列将包括四个实例，它们的实例名分别是 NAND_ARRAY3、NAND_ARRAY2、NAND_ARRAY1 和 NAND_ARRAY0；

“(out, ina, inb)”是模块实例语句中的“端口连接表”，它表明与实例阵列各端口相连的信号端子分别是 out、ina 和 inb。

所以，例 8-6 中出现的那条阵列调用方式的模块调用语句等价于如下几条语句：



```
NAND NAND_ARRAY3 (out[3], ina[3], inb[3]);
NAND NAND_ARRAY2 (out[2], ina[2], inb[2]);
NAND NAND_ARRAY1 (out[1], ina[1], inb[1]);
NAND NAND_ARRAY0 (out[0], ina[0], inb[0]);
```

由上面的例子可以看出,在对同一模块进行较多次数的调用时,采用阵列调用方式将会带来很大的方便。

8.2.4 在模块调用时对参数值的更改

当某个模块被另一个模块调用后,调用模块能够对被调用模块内的参数值进行更改。这样做的好处是:可以通过更改参数值来对被调用模块实现的功能进行控制。比如我们可以设计一个加法器模块,这个加法器的位数由模块内的参数指定。这样,在对这个加法器模块进行调用时,我们只需要在调用时进行一下参数更改,就可以将同一加法器模块作为8位、16位、32位等任意位的加法器加以使用。

在模块调用时对参数值进行更改可以通过下述两种方式实现:

- (1) 使用带有参数值的模块实例语句。
- (2) 使用参数重定义语句(defparam 语句)。

1. 使用带有参数值的模块实例语句

在这种方式下,模块实例语句内就包含有新的参数值,这些新参数值是在“参数值列表”中得到指定的。

前面给出的各种模块实例语句格式内都包含了“参数值列表”这一项,这一项在模块实例语句中是可选的。“参数值列表”是由若干参数值按一定次序组成的一张有序列表。

“参数值列表”具有如下的格式:

```
#(<参数值 1> , <参数值 2> , …… , <参数值 m>)
```

上述格式中指明的各个“<参数值>”将分别被传递给被调用模块实例内的各个参数。在只包含一个参数值的情况下,括号可以省略,即采用“#<参数值>”的格式。

下面我们用例 8-7 来说明这种参数值传递方式。

【例 8-7】利用带有参数值的模块实例语句来进行参数值传递。

//一个 4×4 位乘法器,采用行为描述方式,其中乘数的位数由参数 WIDE1、WIDE2 指定

```
module multibits_multiplier(data1, data2, out) ;
parameter WIDE1=4 ;           //说明参数 WIDE1
parameter WIDE2=4;           //说明参数 WIDE2
input [WIDE1: 1] data1 ;
input [WIDE2: 1] data2 ;
output [WIDE1+WIDE2 : 1] out ;
//连续赋值语句描述了乘法运算
assign out = data1 * data2 ;
endmodule

//一个  $8 \times 8$  位乘法器,采用结构描述方式,通过调用模块 multibits_multiplier
//和修改其参数值“WIDE1”、“WIDE2”来实现
```




```
module eight_bits_multiplier (a, b, result);
input [8: 1] a, b;           //8 位输入 a, b
output [16: 1] result;       //16 位结果输出 result
//带有参数值列表的模块实例语句，其中的参数值将被传递给模块实例 U1
multibits_multiplier #(8, 8) U1 (a, b, result);
endmodule

//一个 16×8 位乘法器，采用结构描述方式，通过调用模块 multibits_multiplier 并修改
//其参数值“WIDE1”，“WIDE2”来实现
module multiplier_16_8 (a, b, result);
input [16: 1] a;             //16 位输入 a
input [8: 1] b;              //8 位输入 b
output [24: 1] result;       //24 位结果输出 result
//带有参数值列表的模块实例语句，其中的参数值将被传递给模块实例 U2
multibits_multiplier #(16, 8) U2 (a, b, result);
endmodule
```

在例 8-7 中，模块 `multibits_multiplier` 内含有两条“parameter 参数说明语句”，这两条参数说明语句分别定义了两个参数“WIDE1”和“WIDE2”。这两个参数分别指明了两个乘数输入的位宽。这两个参数都被初始化为 4，表示该模块实现的是一个 4×4 位乘法器。

在模块 `eight_bits_multiplier` 内使用了一条带有参数值的模块实例语句：

```
multibits_multiplier #(8, 8) U1 (a, b, result);
```

来对模块 `multibits_multiplier` 进行调用，其参数值列表为“#(8, 8)”，其中的两个参数值将分别被传递给模块实例 U1 内的两个参数“WIDE1”和“WIDE2”。其中第一个参数值“8”被传递给第一个参数“WIDE1”；第二个参数值“8”被传递给第二个参数“WIDE2”。这样，模块调用后所生成的实例 U1 将是一个 8×8 位乘法器。

与模块 `eight_bits_multiplier` 相类似，在模块 `multiplier_16_8` 中也使用了一条带有参数值的模块实例语句：

```
multibits_multiplier #(16, 8) U2 (a, b, result);
```

来对模块 `multibits_multiplier` 进行调用，其参数值列表为“#(16, 8)”。其中的两个参数值“16”和“8”将分别被传递给模块实例 U2 内的参数“WIDE1”和参数“WIDE2”。这样，模块调用后所生成的实例 U2 内的两个参数将分别取值“16”和“8”，所以模块实例 U2 将是一个 16×8 位乘法器。

使用带有参数值的模块实例语句来实现参数传递时，参数值列表中各个参数值的排列次序必须与被调用模块中各个参数得到说明的次序保持一致，同时参数值和参数的个数也必须是相同的。在只希望对调用模块内的个别参数(而不是全部参数)进行更改的情况下，所有无需更改的参数值也必须按对应参数的顺序在参数值列表中全部列出(照抄原值)。

2. 使用参数重定义语句(defparam 语句)

在进行模块调用时更改被调用模块内参数值的第二种方法就是利用 Verilog HDL 的“参数重定义语句”(“defparam 语句”)。



参数重定义语句的语法形式如下:

```
defparam <参数名 1> = <参数值 1>,
        <参数名 2> = <参数值 2>,
        .....
        <参数名 n> = <参数值 n>;
```

上面格式中的每一行可以实现对一个参数值的修改。其中:

“defparam”是参数重定义语句的标识。

“<参数名>”用于指明被修改的是哪一个参数,这个参数名必须采用分级路径名的形式,因为在模块层次结构中,只有分级路径名形式的参数名才能唯一标识某个模块实例中的某个参数;

“<参数值>”就是“<参数名>”所对应的参数将被赋予的新参数值。

下面我们用例 8-8 来说明这种参数值传递方式。

【例 8-8】使用参数重定义语句来实现模块参数值的更改。

// 4 位乘法器,采用行为描述方式

```
module multibits_multiplier(data1, data2, out);
    parameter WIDE1=4;           //参数 WIDE1 说明
    parameter WIDE2=4;           //参数 WIDE2 说明
    input [WIDE1: 1] data1;
    input [WIDE2: 1] data2;
    output [WIDE1+WIDE2: 1] out;
    //连续赋值语句描述了乘法功能
    assign out = data1 * data2;
endmodule

//一个 8×6 位乘法器,采用结构描述方式,通过调用模块 multibits_multiplier
//以及修改其参数值“WIDE1”、“WIDE2”来实现
module multiplier_8_6(a, b, result);
    input [8: 1] a;               //8 位输入 a
    input [6: 1] b;               //6 位输入 b
    output [14: 1] result;        //14 位结果输出 result
    //参数重定义语句,对被调用模块 multibits_multiplier 的实例 U1 内的参数值进行修改
    defparam U1.WIDE1 = 8,        //实例 U1 内参数“WIDE1”的参数值被修改为 8
    U1.WIDE2 = 6;                 //实例 U1 内参数“WIDE2”的参数值被修改为 6
    //模块实例语句,对模块 multibits_multiplier 进行调用,生成实例 U1
    multibits_multiplier U1(a, b, result);
endmodule
```

在例 8-8 中,模块 multibits_multiplier 与例 8-7 中的同名模块完全相同,其中定义了两个参数“WIDE1”和“WIDE2”。它们分别用来指明两个输入乘数的位宽。这两个参数的值都被初始化为 4,所以模块 multibits_multiplier 所实现的是一个 4×4 乘法器。

在模块 multiplier_8_6 中使用了一条参数重定义语句:



```
defparam    U1.WIDE1 = 8,  
            U1.WIDE2 = 6;
```

这条 defparam 语句包括了两项(两行), 这两项分别对两个参数进行了参数值修改。

- 在第一项“U1.WIDE1 = 8”中:

指定的“参数名”是“U1.WIDE1”, 它指定了需要被修改的参数。这个参数名是一个分级路径名形式的参数名, 它表明需要进行修改的参数是当前模块(模块 multiplier_8_6)包含的模块实例 U1 内的参数“WIDE1”。

指定的“参数值”是“8”, 它表明“U1.WIDE1”所对应参数的参数值将被修改为 8。

- 在第二项“U1.WIDE2 = 6”中:

“参数名”是“U1.WIDE2”, 它表明需要进行修改的参数是当前模块(模块 multiplier_8_6)包含的模块实例 U1 内的参数“WIDE2”。

“参数值”是“6”, 它表明“U1.WIDE2”所对应参数的参数值将被修改为 6。

所以, 实例 U1 内的两个参数“WIDE1”和“WIDE2”的参数值将分别被修改为 8 和 6, 这样实例 U1 就实现了一个 8×6 位乘法器。

3. 两种参数值更改方式的区别

在采用第一种参数值修改方式(使用带有参数值的模块实例语句)的情况下, 由于需要在模块实例语句中列出新的参数值, 而模块实例语句只能用来对下一级模块进行调用。所以这种方式只能用于对本模块直接调用的下一级模块内的参数进行修改, 而不能对不是由本模块直接调用的模块或更低层次模块内的参数进行修改。

在采用第二种参数值修改方式(使用参数重定义语句)的情况下, 由于 defparam 参数重定义语句是独立于模块实例语句的, 并且由于 defparam 参数重定义语句内的“参数名”是一种分级路径名形式的参数名。所以这种参数值修改方式可以用来实现更为灵活的参数值修改操作。下面举例说明。

【例 8-9】 defparam 参数重定义语句的灵活使用。

```
module top (...);  
... .. //端口说明和数据类型说明  
parameter PARA_2=2; //参数 PARA_2 的初始参数值为 2  
medium U1(out, in1, in2); //模块实例语句, 调用下一级模块 medium  
defparam U1.U2.PARA_1=3; //参数重定义语句  
... .. //其它语句  
endmodule  
module medium(out, in1, in2);  
... .. //端口说明和数据类型说明  
bottom U2(a, b, c) //模块实例语句, 调用下一级模块 bottom  
... .. //其它语句  
endmodule  
module bottom(a, b, c);  
... .. //端口说明和数据类型说明
```




```

parameter PARA_1=1,      //参数 PARA_1 的初始参数值为 1
.....                  //其它语句

endmodule

module change_para;      //一个用来对参数值进行修改的独立模块
defparam top. PARA_2=10; //参数重定义语句
endmodule

```

例 8-9 中，我们定义了四个模块：top、medium、bottom 和 change_para。其中前三个模块是依次被调用的关系，它们依次成为模块分层结构中的顶层模块、中间层模块和底层模块；而第四个模块 change_para 与前三个模块之间没有调用或被调用的关系，它是独立于前三个模块的。这四个模块构成了如图 8.3 所示的层次关系。

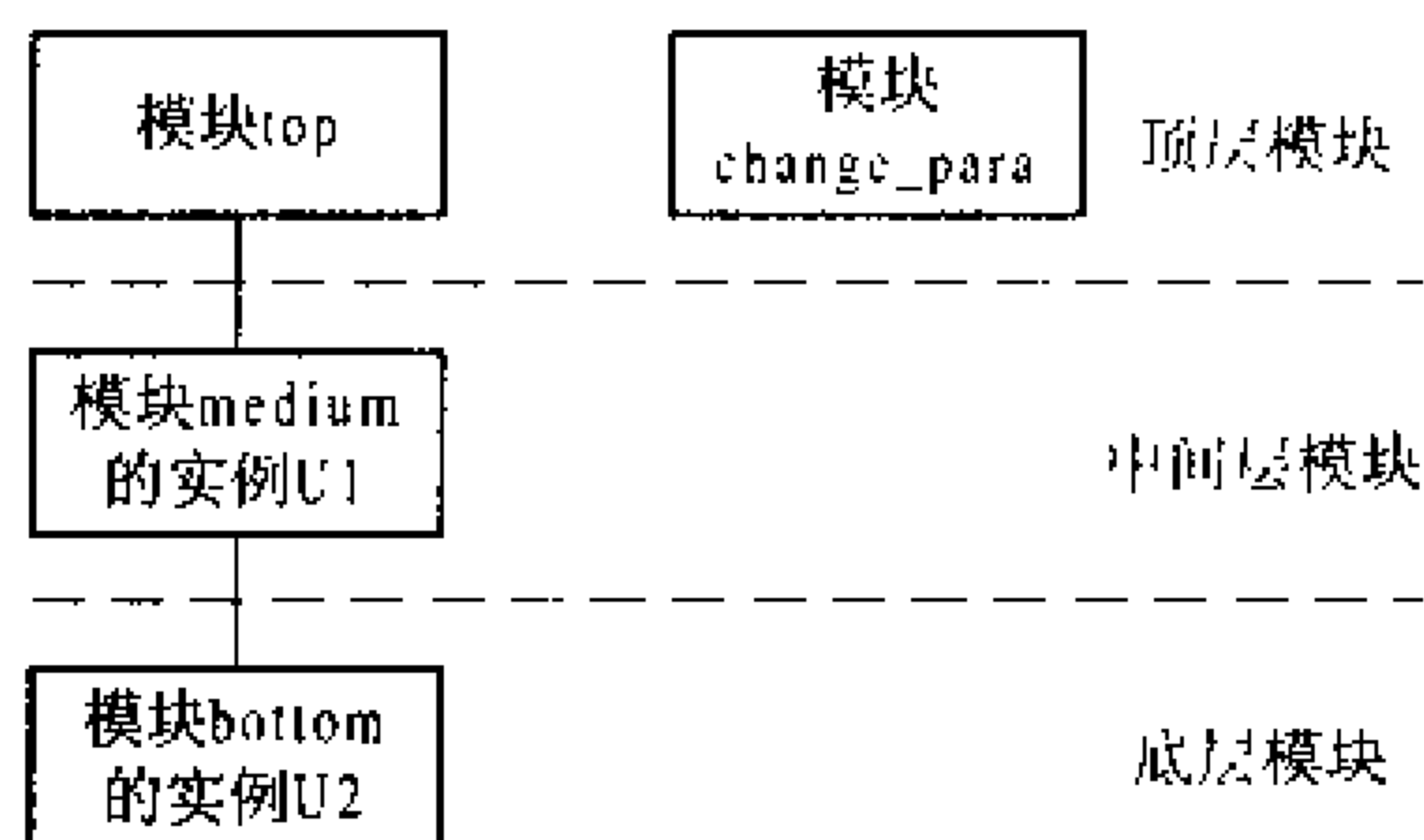


图 8.3 例 8-9 中各模块的层次关系

底层模块 bottom 内定义了一个参数 PARA_1，其初始参数值被指定为 1；顶层模块 top 内定义了一个参数 PARA_2，它的初始参数值被指定为 2。

在顶层模块 top 内有一条参数重定义语句：“defparam U1.U2.PARA_1=3;”。其中的参数名是“U1.U2.PARA_1”，它指定了进行参数值修改的目标参数。这个参数名所代表的参数是：底层模块实例 U2(模块 bottom 的实例)中的参数“PARA_1”，并且这个 U2 实例是出现在“顶层模块 top 所引入的模块实例 U1”内的那个底层模块实例。这样我们就在顶层模块 top 中实现了对底层模块实例 U2 内参数的修改，对应的“PARA_1”参数将拥有新参数值“3”。

在独立模块“change_para”中也有一条参数重定义语句：“defparam top. PARA_2=10;”，其中的参数名是“top.PARA_2”，它指明了进行参数值修改的目标参数。由这个参数名指定的、需要进行修改的参数是模块 top 中的参数“PARA_2”。由于模块“change_para”和模块“top”是相互独立的两个模块，这样就实现了：“在一个模块中定义某个参数，而在另外一个独立的模块中对该参数进行参数值更改”。在这种情况下，defparam 语句中包含的“参数名”必须是从顶层开始的一个“全局分级名”(举例来说，参数名“top.PARA_2”中的第一个域“top”必须是一个顶层模块名)。

8.2.5 举例

例 8-10 中的模块“shifter”对如图 8.4 所示的一个 4 位移位寄存器结构进行了描述。这个移位寄存器是由四个 D 触发器(U1、U2、U3 和 U4)构成的。其中 seri_in 是这个移位



寄存器的串行输入；clk 为移位时钟脉冲输入；clrb 为清零控制信号输入；Q[0] ~ Q[3] 则是移位寄存器的并行输出。

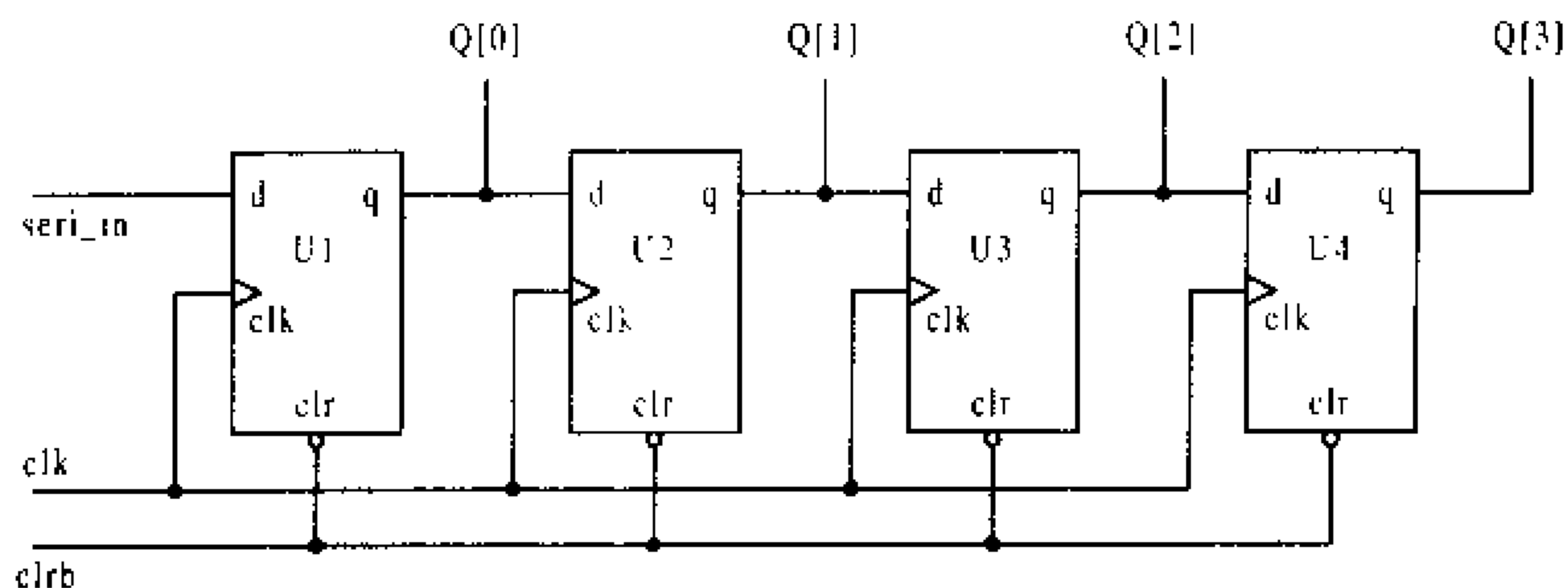


图 8.4 四位移位寄存器

【例 8-10】图 8.4 所示移位寄存器的模块级结构描述。

// 模块 “d_flop” 描述了一个带有清零输入的上升沿 D 触发器

```
module d_flop(q, d, clr, clk);
```

```
output q;
```

```
input d, clr, clk;
```

```
reg q;
```

```
always @(clr) //用于实现异步清零功能的 always 过程块
```

```
if (!clr) assign q=0;
```

```
else deassign q;
```

```
always @(negedge clk) //用于实现正常 D 触发器功能的 always 过程块
```

```
q=d;
```

```
endmodule
```

// 模块 shifter 是移位寄存器的结构描述模块

```
module shifter(seri_in, clk, clrb, Q);
```

```
input seri_in, clk, clrb;
```

```
output [3:0] Q;
```

// 模块实例语句，对 D 触发器模块进行调用

```
d_flop U1(Q[0], seri_in, clrb, clk);
```

```
U2(Q[1], Q[0], clrb, clk);
```

```
U3(Q[2], Q[1], clrb, clk);
```

```
U4(Q[3], Q[2], clrb, clk);
```

```
endmodule
```

例 8-10 中模块 d_flop 对 D 触发器进行了行为建模。在模块 “shifter” (描述了移位寄存器的模块)内，d_flop 模块得到了四次调用，分别引入了四个 D 触发器实例：U1、U2、U3 和 U4。模块实例语句内包含的“端口连接表”指明了 D 触发器输入、输出端与外部信号端子的连接。这样，结构图内各单元之间的相互连接关系就得到了描述，实现了移位寄存器的结构描述。



8.3 门级建模

门级建模(门级结构描述)是指调用 Verilog 内部定义的“基本门级元件(Basic Gate-Level Primitives)”来对硬件电路进行结构描述。

门级建模方式采用的是一种特殊的模块调用方式,这时所调用的模块是 Verilog 内部预先定义好的一些“基本门级元件”,在门级建模方式下硬件电路将被描述成由一组基本门级元件的实例所组成。

由于一个数字电路系统最终是由一个个逻辑门和开关所组成的,所以用逻辑门单元和开关单元来对硬件电路的组成结构进行描述是最直观的。考虑到这一点,Verilog HDL 把一些常用的基本逻辑门单元和开关单元的模型包含到语言内部,这些模型被称为“基本门级元件(Basic Gate-Level Primitives)”和“基本开关级元件(Basic Switch-Level Primitives)”。本节我们将只讨论如何利用基本门级元件进行结构描述。

在进行门级建模时,可以像调用子模块那样对各种基本门级元件进行调用,某个基本门级元件得到调用后就在当前模块内生成该元件的一个实例。实际上我们可以将基本门级元件模型看作是一种特殊模块,这种模块是由 Verilog HDL 语言本身所提供的,不需要用户自己进行定义。

8.3.1 内置基本门级元件

Verilog HDL 内含的基本元件模型(Basic Primitives)共有 26 种,其中 14 种为基本门级元件,12 种为开关级元件(Switch-Level Primitives),这一节我们只介绍其中的门级元件部分。

Verilog 中的基本门级元件包括:and(与门)、nand(与非门)、or(或门)、nor(或非门)、xor(异或门)、xnor(异或非门)、buf(缓冲器)、not(非门)、bufif1(高电平使能缓冲器)、bufif0(低电平使能缓冲器)、notif1(高电平使能非门)、notif0(低电平使能非门)、pullup(上拉电阻)、pulldown(下拉电阻)。它们可以被分成如下四类加以说明:

- (1) 多输入门: and, nand, or, nor, xor, xnor。
- (2) 多输出门: buf, not。
- (3) 三态门: bufif0, bufif1, notif0, notif1。
- (4) 上拉、下拉电阻: pullup, pulldown。

1. 基本门级元件的调用

基本门级元件的调用是通过“门级元件实例语句”来实现的,这种实例语句的基本格式为:

<门级元件名> <驱动强度说明> #(<门级延时量>) <实例名> (端口连接表);

其中用斜体表示的项是可以缺省的项,各项的意义分别说明如下:

“<门级元件名>”就是前面列出的 Verilog HDL 内含的 14 种基本门级元件类型中的任意一种,它用于指明被当前模块调用的是哪一种基本门级元件。“<门级元件名>”这一项在门级元件实例语句中是必须出现的,不能缺省。

“<驱动强度说明>”这一部分是可以缺省的,它的格式为:



(<对高电平的驱动强度> , <对低电平的驱动强度>)

“驱动强度说明”用来对本次(基本门级元件)调用所引入的门级元件实例的输出端驱动能力进行说明。这是因为:在结构描述方式下,一条连线可能会由多个前级输出端同时驱动,该连线最终的逻辑状态将取决于各个驱动端的不同驱动能力,因此有必要对元件实例的输出驱动能力进行说明。对于门级元件,驱动强度分为对高电平(逻辑“1”)的驱动强度和对低电平(逻辑“0”)的驱动强度,因此“驱动强度说明”部分是由“对高电平的驱动强度”和“对低电平的驱动强度”这两种成分组成的。其中的每一种驱动强度成分可以是“supply”、“strong”、“pull”、“weak”和“highz”中的某一个等级,所以可以有如下关键词出现在门级元件的“驱动强度说明”中:

```
supply1 strong1 pull1 weak1 highz1
supply0 strong0 pull0 weak0 highz0
```

上面第一行表示“对高电平驱动强度”的五个等级;第二行表示了“对低电平驱动强度”的五个等级。在基本门级元件实例语句的驱动强度说明中,“对高电平的驱动强度”和“对低电平的驱动强度”这两个成分在括号内的前后位置是可以互换的,比如如下两条门级元件实例语句:

```
and (strong0, weak1) AND_1 (out, in1, in2);
and (weak1, strong0) AND_2 (out, in1, in2);
```

其中引入了两个与门实例:“AND_1”与“AND_2”。这两个实例拥有相同的输出端驱动强度:高电平输出强度为“weak”,低电平输出强度为“strong”。

如果基本门级元件实例语句中的“驱动强度说明”部分被省略,则默认的输出驱动强度为“(strong0, strong1)”。

我们在以后的章节还会对驱动强度作进一步介绍。

门级元件实例语句中的“# (<门级延时量>)”部分是可以缺省的,它说明了信号从门级元件实例的输入端流动到输出端时所经历的延时时间长度。其中的“<门级延时量>”是由若干个延时值组成的,当它只包含一个延时值时,其外部的圆括号可以省略,比如语句:

```
and #10 AND_3 (out, in1, in2);
```

就引入了一个与门实例“AND_3”,其中的“#10”部分指定了对应与门实例的门级延时量:实例 AND_3 的输入端到输出端的延时量是 10 个单位时间。

当语句格式中的“# (<门级延时量>)”部分缺省时,延时量被默认为 0。

值得注意的是,以上所述的、对门级元件实例的门级延时量进行指定的方式与我们在 8.2.4 小节中讲述的对模块参数值进行修改的第二种方式(即使用“带有参数值的模块实例语句”)是相似的。在这两种情况下,对应的实例语句中都出现了“# (<数值量>)”部分。这部分内容在其中一种情况下代表着模块参数的新参数值;而在另一种情况下则代表了门级元件实例输入输出端口间的门级延时量。由于这两种情况所对应的调用分别是“模块调用”和“基本门级元件调用”,在进行模块调用时不能采用像门级元件调用那样的方式来指定延时值(而只能利用“specify 块”结构来指定延时值);而在进行基本门级元件调用时又不能像模块调用那样对参数值进行修改(只有用户自定义的模块才能有参数值),所以在上述的两种情况下不会发生混淆。



在以后的章节我们还会对门级延时量的表示方式作进一步的介绍。

门级元件实例语句格式中的“<实例名>”是为本次元件调用后生成的门级元件实例所取的一个名称。与进行模块调用的情况不同，在对基本门级元件进行调用时这个“实例名”部分是可以省略的。在缺省“实例名”的情况下，Verilog 编译系统将自动以“<元件名><实例序号>”的形式来为门级元件实例取名。

门级元件实例语句格式中的“端口连接表”类似于模块实例语句中的端口连接表，它是由若干“外部信号端子”组成的一张有序列表，用来明确门级元件实例和外部电路的连接情况。

与进行模块调用时的情况不同，在进行基本门级元件调用时只能采用“端口位置关联方式”的端口关联方式。在这种关联方式下，门级元件实例的各个端口将依次与端口连接表中的各个信号端子相连接：元件的第一个端口与端口连接表内的第一个信号端子与相连，第二个端口与第二个信号端子相连，依次类推。

还有一点必须注意，与基本门级元件实例各个端口相连的信号端子只能是宽度为一位的表达式或变量标识符。

如果需要对同一个基本门级元件进行多次调用，则可以采用如下所示的一种元件实例语句格式：

```
<门级元件名> <驱动强度说明> #( <门级延时量> )
    <实例名 1> ( 端口连接表 1 ) ,
    <实例名 2> ( 端口连接表 2 ) ,
    .....
    <实例名 n> ( 端口连接表 n ) ;
```

上面这条元件调用语句对“<门级元件名>”所代表的基本门级元件进行了 n 次调用，分别生成了由“实例名 1”、……、“实例名 n”标识的 n 个门级元件实例。该调用语句各行内的“端口连接表”分别说明了对应元件实例端口和外部信号端子的连接关系。注意，要用逗号将每个实例项分隔开。

例如如下语句：

```
and (strong0, weak1) #10
    AND_1(out1, in1, in2),
    AND_2(out2, in3, in4);
```

就对“与门 and”进行了两次调用，分别生成了取名为“AND_1”和“AND_2”的两个元件实例(与门实例)。其中元件实例“AND_1”的输出端被连接到 out1，输入端被连接到 in1 和 in2；元件实例“AND_2”的输出端被连接到 out2，输入端被连接到 in3 和 in4。

必须注意，在采用上述门级元件实例语句格式的情况下，元件调用进行后所生成的各个元件实例将具有相同的门级延时量(都由语句格式中的“门级延时量”进行指定)和输出驱动强度(都由语句格式中的“驱动强度说明”进行指定)。比如前面的例子所生成的两个元件实例“AND_1”和“AND_2”都将具有由“(strong0, weak1)”代表的驱动强度和 10 个单位时间的门级延时量。

我们可以由上面的描述看出，基本门级元件的调用很像模块级建模中的模块调用。二者的区别仅在于：前者所调用的是包含在 Verilog HDL 语言系统内部的基本门级元件模



型，而在后者情况下被调用的是由用户自己编写的模块；在对基本门级元件进行调用时可以指定驱动强度与门级延时量，而在对模块进行调用时却不能包含这些内容。

2. 多输入门

内置多输入门包括如下几种门级元件：

- (1) and：与门。
- (2) nand：与非门。
- (3) or：或门。
- (4) nor：或非门。
- (5) xor：异或门。
- (6) xnor：异或非门。

这六种逻辑门的特点都是可以有一个或多个输入但只能有一个输出。

这些多输入门的元件模型可以表示为：

<门级元件名> (<输出端口>, <输入端口 1>, <输入端口 2>, ……., <输入端口 n>)

其中的“<门级元件名>”就是上面给出的六种多输入门名称之一；在端口列表中出现的第一个端口是输出端口，其后是多个输入端口。

表 8.1 给出了上述六种多输入门的逻辑真值表，其中第一行和第一列代表着两个输入端的输入状态，而其它的表项代表着输出端的输出状态。

表 8.1 and、nand、or、nor、xor、xnor 元件的真值表

and	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

nand	0	1	x	z
0	1	1	1	1
1	1	0	x	x
x	1	x	x	x
z	1	x	x	x

or	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

nor	0	1	x	z
0	1	0	x	x
1	0	0	0	0
x	x	0	x	x
z	x	0	x	x



xor	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

xnor	0	1	x	z
0	1	0	x	X
1	0	1	x	X
x	x	x	x	X
z	x	x	x	X

由表 8.1 给出的各个逻辑真值表可以看出, 多输入门对输入逻辑状态“x”和“z”的处理方式是相同的: 多输入门的输出状态不会是“z”。

对多输入门进行调用的元件实例语句将采用如下格式:

<多输入门元件名> <驱动强度说明> #(<门级延时量>)

<实例名> (<输出信号端子>, <输入信号端子 1>, <输入信号端子 2>,
……, <输入信号端子 n>);

在上面所示的元件实例语句格式中, 与多输入门元件输出端相连的外部信号端子(“<输出信号端子>”)必须出现在端口连接表的第一项中, 与多输入门的各个输入端相连的外部信号端子则只能出现在端口连接表的后面几项内。这些输入信号端子将依次与多输入门的各个输入端相连接。信号端子和元件端口之间的连接关系是采用“端口位置关联方式”来确定的。比如下面几条语句:

```
and A1(out1, in1, in2);
```

```
or O2(a, b, c, d);
```

```
xor X1(x_out, p1, p2);
```

分别对多输入门“and”、“or”和“xor”进行了调用, 其中:

第一条元件调用语句引入了二输入与门的一个实例, 它的实例名被指定为“A1”。该实例的输出端被连接到信号端子“out1”, 两个输入端分别被连接到信号端子“in1”和“in2”。

第二条元件调用语句引入了三输入或门的一个实例。这个实例的实例名为“O2”, 它的输出端被连接到信号端子“a”, 三个输入端被分别连接到信号端子“b”、“c”和“d”。

第三条元件调用语句引入了二输入异或门的一个实例, 其实例名为“X1”, 其输出端被连接到信号端子“x_out”, 两个输入端分别被连接到信号端子“p1”和“p2”。

图 8.5 给出了上述多输入门元件实例的逻辑示意图。

也可以利用一条调用语句来实现对同一多输入门的多次调用, 比如下面这条语句:

```
nand NA1(out1, in11, in12),
```

```
NA2(out2, in21, in22),
```

```
NA3(out3, in31, in32, in33);
```

就对与非门“nand”进行了三次调用, 分别引入了实例名为“NA1”、“NA2”和“NA3”的三个实例, 这些实例与外部信号的连接关系如图 8.6 所示。

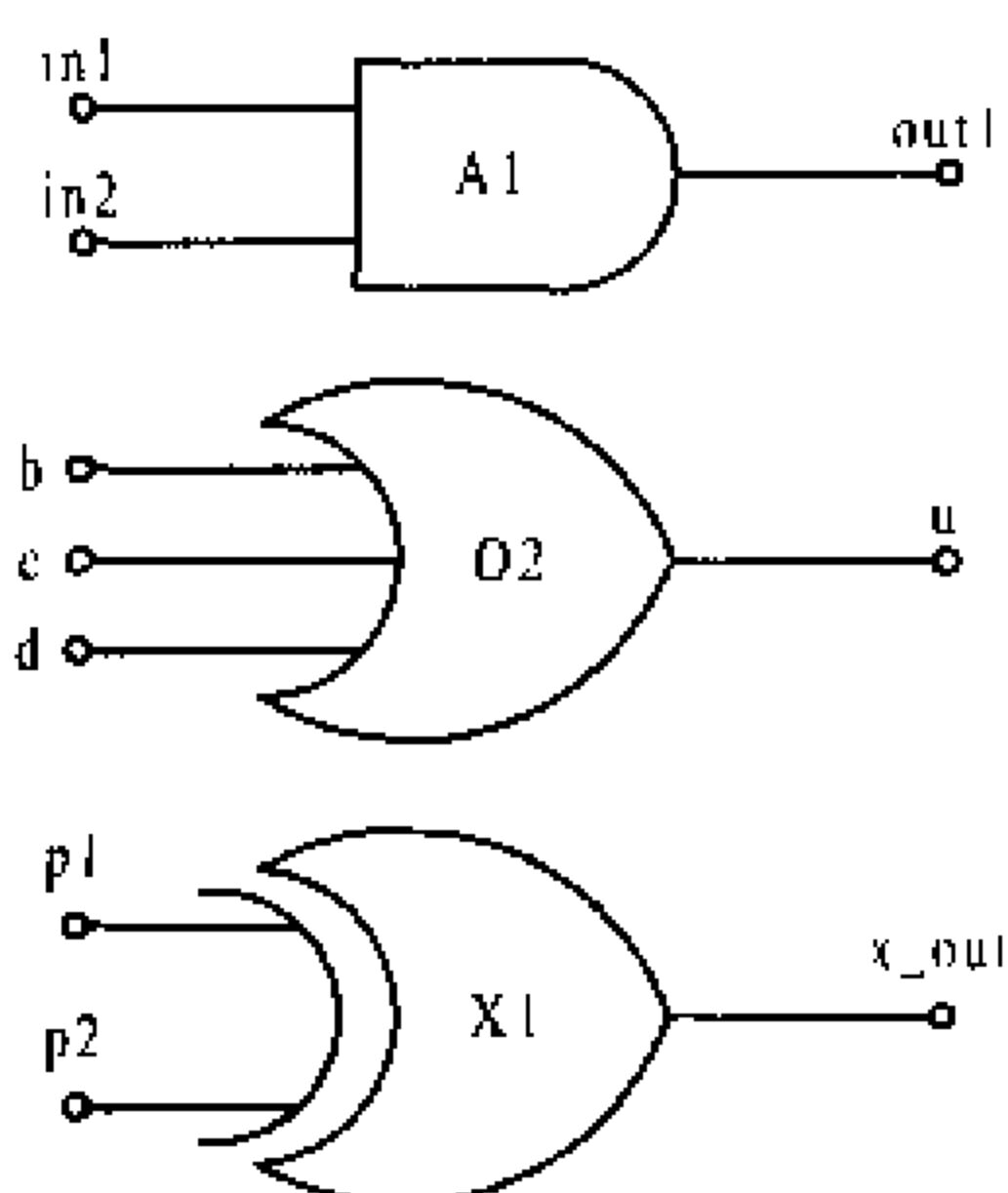


图 8.5 多输入门元件实例

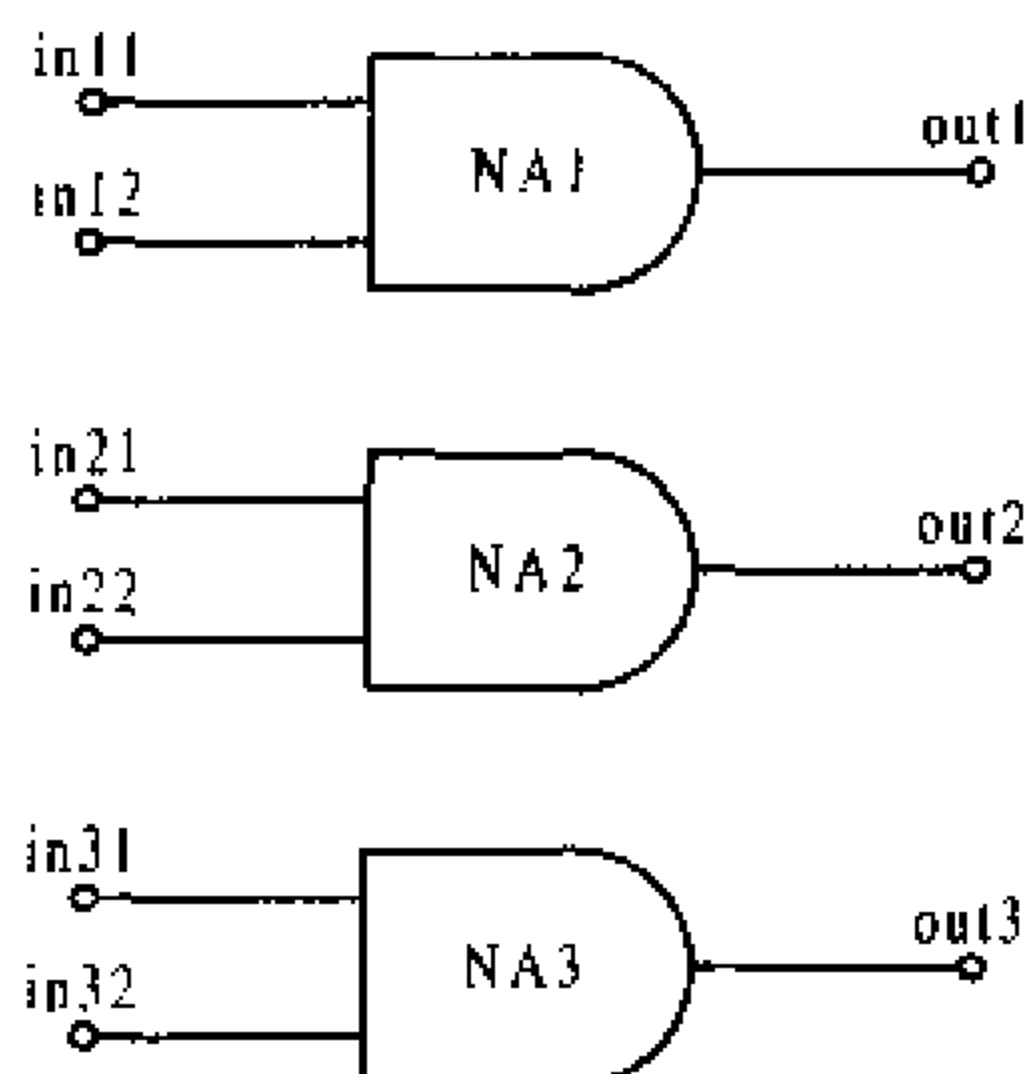


图 8.6 多输入门实例

3. 多输出门

内置多输出门包括如下两种门级元件：

- (1) buf: 缓冲器。
- (2) not: 非门。

这两种逻辑门的共同特点是，允许有多个输出，但只能有一个输入。

这两种多输出门的元件模型可以表示为：

<元件名> (<输出端口 1>, <输出端口 2>, …… , <输出端口 n>, <输入端口>)

其中，“<元件名>”是元件名称“buf”和“not”之一；端口列表中的最后一项是输入端口，前面的其余端口为输出端口。

表 8.2 给出了这两种多输出门的逻辑真值表。

表 8.2 buf 和 not 两种元件的真值表

buf		not	
输入	输出	输入	输出
0	0	0	1
1	1	1	0
X	x	x	x
Z	x	z	x

由表 8.2 可以看出，多输出门“buf”和“not”对输入状态“x”和“z”的处理方式是相同的；并且这两种元件的输出状态不会是“z”。

对多输出门进行调用的元件实例语句将采用如下格式：

<多输出门元件名> <驱动强度说明> #(<门级延时量>)

<实例名>(<输出信号端子 1>, <输出信号端子 2>, …… , <输出信号端子 n> ,
<输入信号端子>);

在上述格式中，端口连接表中出现的最后一个信号端子将被连接到多输出门的输入端，而排在前面的其余信号端子将与多输出门的各个输出端依次相连。信号端子和元件端



口之间的连接关系是采用“端口位置关联方式”来确定的。

举例来说，如下语句分别对非门和缓冲器进行了调用。

```
not NOT_1(out1, out2, in);
```

```
buf BUF_1(bufout1, bufout2, bufout3, bufin);
```

其中：第一条调用语句引入了二输出非门的一个实例，其实例名为“NOT_1”，该实例的两个输出端分别被连接到信号端子“out1”和“out2”，其输入端被连接到信号端子“in”。

第二条调用语句引入了三输出缓冲器的一个实例，其实例名为“BUF_1”，该实例的三个输出端分别被连接到信号端子“bufout1”、“bufout2”和“bufout3”，其输入端被连接到信号端子“bufin”。

图 8.7 给出了这些多输出门元件实例的逻辑示意图。

4. 三态门

内置三态门包括如下四种元件：

- (1) buifl：高电平使能缓冲器。
- (2) bufif0：低电平使能缓冲器。
- (3) notifl：高电平使能非门。
- (4) notif0：低电平使能非门。

这四种逻辑门用来对三态驱动器建模，它们都有一个数据输出端、一个数据输入端和一个控制输入端。

这几种三态门的特点是：它们的数据输出端可以实现三态输出。其元件模型可以表示为：

<元件名> (<数据输出端口>, <数据输入端口>, <控制输入端口>)

其中的“<元件名>”就是前面给出的四种元件名称“buifl”、“bufif0”、“notifl”和“notif0”中的一个；端口列表中的第一项是数据输出端口，第二项是数据输入端口，最后一项是控制输入端口。

表 8.3 给出了这几种三态门元件的逻辑真值表。其中的第一行是控制输入端的各种状态，第一列是数据输入端的各种输入状态，其它的表项代表着数据输出端的输出状态。真值表中的某些表项是不确定的。比如，“0/z”所代表的输出状态就是“既可能取 0 值也可能取 z 值”。

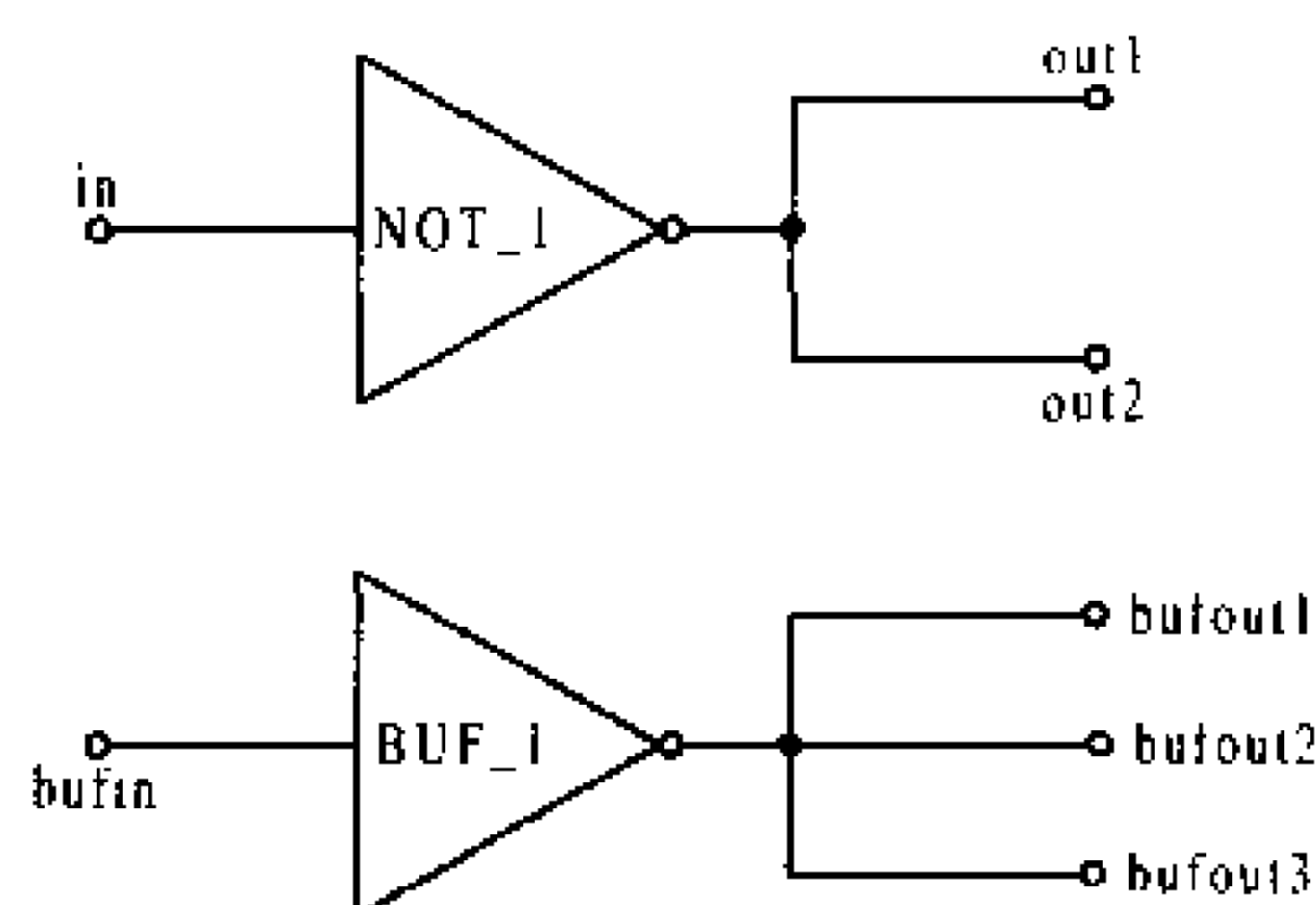


图 8.7 多输出门元件实例

表 8.3 buifl、bufif0、notifl、notif0 元件的真值表

buifl		控制输入			
		0	1	x	z
数据输入	0	z	0	0/z	0/z
	1	z	1	1/z	1/z
	x	z	x	x	x
	z	z	x	x	x

bufif0		控 制 输 入			
		0	1	x	z
数 据 输 入	0	0	z	0/z	0/z
	1	1	z	1/z	1/z
	x	x	z	x	x
	z	x	z	x	x

notif1		控 制 输 入			
		0	1	x	z
数 据 输 入	0	z	1	1/z	1/z
	1	z	0	0/z	0/z
	x	z	x	x	x
	z	z	x	x	x

notif0		控 制 输 入			
		0	1	x	z
数 据 输 入	0	1	z	1/z	1/z
	1	0	z	0/z	0/z
	x	x	z	x	x
	z	x	z	x	x

由这些逻辑真值表可以看出，这几种三态门元件的输出端可以被驱动到高阻状态“z”(在特定的控制输入状态下)。

对于高电平使能缓冲器“bufif1”，若控制输入为 1，则输入数据被传送到数据输出端；若控制输入为 0，则数据输出端处于高阻状态“z”。

对于低电平使能缓冲器“bufif0”，若控制输入为 1，则数据输出端处于高阻状态“z”；若控制输入为 0，则输入数据被传送到数据输出端。

对于高电平使能非门“notif1”，若控制输入为 1，则数据输出端的逻辑状态是输入的“逻辑非”；若控制输入为 0，则数据输出端处于高阻状态“z”。

对于低电平使能非门“notif0”，若控制输入为 1，则数据输出端处于高阻状态“z”；若控制输入为 0，则数据输出端的逻辑状态是输入的“逻辑非”。

对三态门进行调用的元件实例语句具有如下所示的格式：

<元件名> <驱动强度说明> #(<门级延时量>)

<实例名> (<输出信号端子>, <输入信号端子>, <控制信号端子>);

在上述格式中，端口连接表内出现的第一个信号端子(“<输出信号端子>”)将与三态门的数据输出端相连接；第二个信号端子(“<输入信号端子>”)将与三态门的数据输入端相连；第三个信号端子(“<控制信号端子>”)将与三态门的控制输入端相连。各个信号端子与三态门各端口之间的连接关系是采用“端口位置关联方式”来确定的。

举例来说，下面的四条元件实例语句分别对四种三态门元件进行了调用。

bufif1 BF1 (data_bus, mem_data, enable);

bufif0 BF0 (a, b, c);



```
notif1 NT1 (out, in, ctrl);
```

```
notif0 NT0 (addr, a_bus, select);
```

其中：第一条语句对“高电平使能缓冲器 bufif1”进行了调用，引入了该元件的一个实例，其实例名为“BF1”。该实例的数据输出端与信号端子“data_bus”相连；其数据输入端与信号端子“mem_data”相连；其控制输入端与信号端子“enable”相连。

第二条语句对“低电平使能缓冲器 bufif0”进行了调用，引入了该元件的一个实例，其实例名为“BF0”。该实例的数据输出端与信号端子“a”相连；其数据输入端与信号端子“b”相连；其控制输入端与信号端子“c”相连。

第三条语句对“高电平使能非门 notif1”进行了调用，引入了该元件的一个实例，其实例名为“NT1”。该实例的数据输出端与信号端子“out”相连；其数据输入端与信号端子“in”相连；其控制输入端与信号端子“ctrl”相连。

第四条语句对“低电平使能非门 notif0”进行了调用，引入了该元件的一个实例，其实例名为“NT0”。该实例的数据输出端与信号端子“addr”相连；其数据输入端与信号端子“a_bus”相连；其控制输入端与信号端子“select”相连。

图 8.8 给出了这些三态门实例的逻辑示意图。

5. 上拉、下拉电阻

上拉、下拉电阻包括两种元件：

(1) pullup: 上拉电阻。

(2) pulldown: 下拉电阻。

上拉电阻和下拉电阻这两种门级元件都是只有一个输出端口而没有输入端口。上拉电阻将输出置为 1，下拉电阻将输出置为 0。

上拉、下拉电阻的元件模型可以表示为：

<元件名> (<输出端口>)

其中，“<元件名>”就是元件名称“pullup”和“pulldown”之一；在端口列表中只能包含一个输出端口。

对上拉、下拉电阻进行调用的元件实例语句具有如下的语法格式：

<元件名> <驱动强度说明> <实例名> (<输出信号端子>);

在上述格式中，端口连接表内只能出现一个信号端子(输出信号端子)，该信号端子将被连接到元件实例的输出端口。由于上拉电阻只能输出逻辑状态 1，下拉电阻只能输出逻辑状态 0，所以在进行驱动强度说明时(在需要进行说明的情况下)，对 pullup 元件只需指出“对高电平的驱动强度”；而对 pulldown 元件则只需指出“对低电平的驱动强度”。

举例来说，如下两条语句就分别对上拉电阻和下拉电阻进行了调用。

```
pullup (supply1) U1 (power);
```

```
pulldown (pull0) U2 (gnd);
```

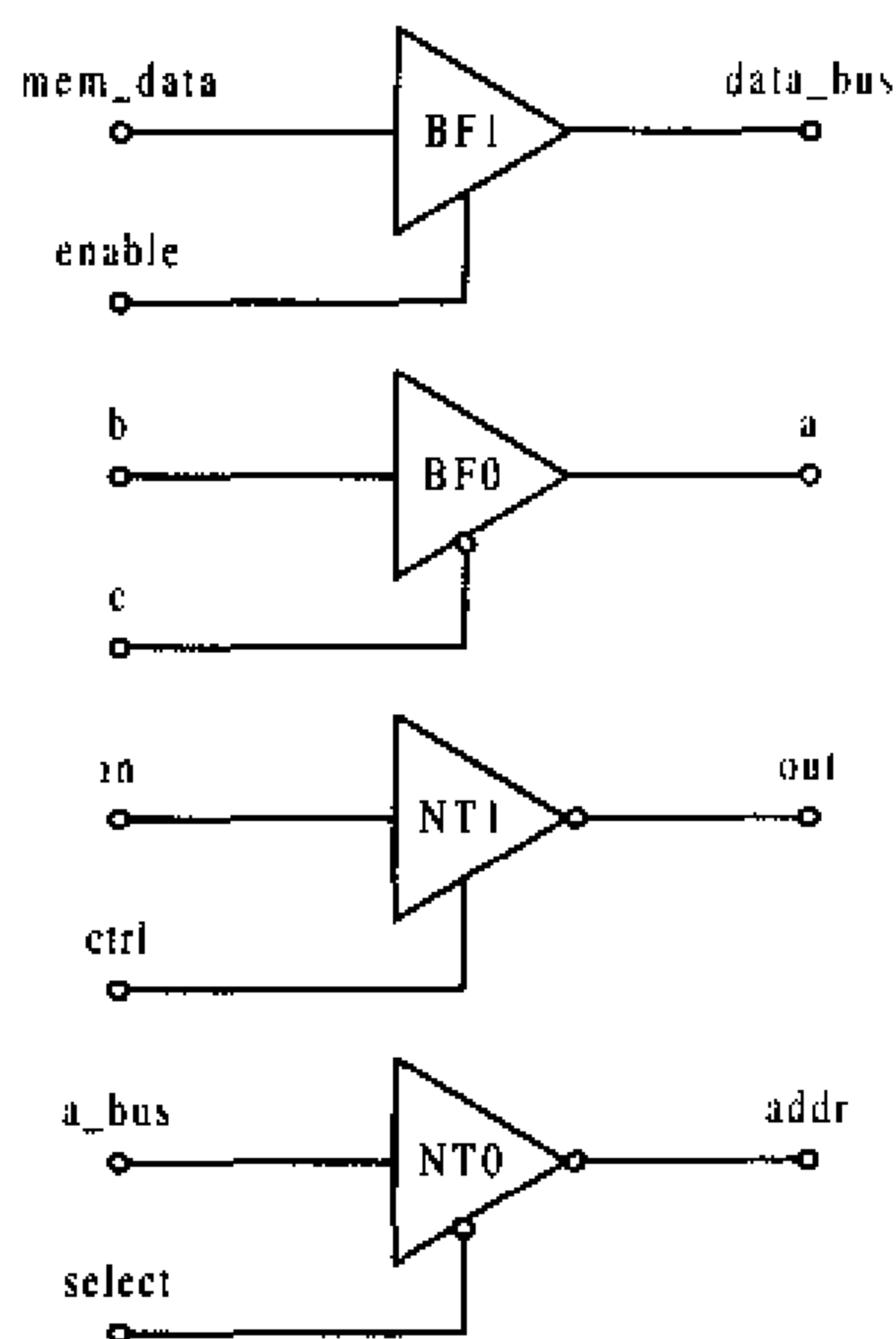


图 8.8 三态门实例



其中：第一条语句引入了上拉电阻的一个实例，其实例名为“U1”。该实例的输出端被连接到信号端子“power”；这个输出端的驱动强度被指定为“supply1”。

第二条语句引入了下拉电阻的一个实例，其实例名为“U2”。该实例的输出端被连接到信号端子“gnd”；该输出端的驱动强度被指定为“pull0”。

6. 门级延时的说明

门级延时反映的是信号的变化从门级元件的任一输入端口流动到任一输出端口所经历的传输延时。

门级延时可以分为如下几类：

(1) “上升延时”：表示信号由“0”、“x”或“z”状态变化到“1”状态时受到的门传输延时。

(2) “下降延时”：表示信号由“1”、“x”或“z”状态变化到“0”状态时受到的门传输延时。

(3) “截止延时”：表示信号由“0”、“1”或“x”状态变化到“z”状态时受到的门传输延时。

(4) “到不定态的延时”：表示信号由“0”、“1”或“z”状态变化到“x”状态时受到的门传输延时。

由于多输入门(and, nand, or, nor, xor 和 xnor)以及多输出门(buf 和 not)的输出不可能是高阻态“z”，所以多输入门和多输出门这两类元件没有“截止延时”这类门级延时，只能有“上升延时”、“下降延时”和“到不定态的延时”这三种门级延时种类。

对于三态门(bufif1, bufif0, notif1 和 notif0)，由于这类元件的输出可以取所有四种可能逻辑状态“0”、“1”、“x”和“z”，所以三态门这类元件可以具有全部四种门级延时种类。

对于上拉电阻(pullup)和下拉电阻(pulldown)，由于这两种元件没有输入端口，所以不存在从输入到输出的延时。因此这两种元件不能有任何形式的门级延时说明。

在对门级元件进行调用时，可以对元件实例的门级延时进行说明，这是通过元件实例语句内的“门级延时说明”部分来实现的。如果“门级延时说明”部分被缺省，则门级延时被默认为是零延时。

元件实例语句内的“门级延时说明”部分的格式为：

#(门级延时量)

“门级延时量”是由一个或多个延时值组成的，它可以有如下两种表示方式：

- 基本表示方式。

- “最小延时”、“典型延时”和“最大延时”的表示方式。

下面我们将分别对门级延时量的这两种表示方式进行讨论，需要注意的一点是：Verilog HDL 中的所有延时量都是以“单位时间”表示的，单位时间与实际时间的关联可以通过timescale 编译指令来实现。

1) 门级延时量的基本表示方式

在这种表示方式下，“门级延时量”内可以包含1个、2个或3个延时值，下面分别对这三种具有不同延时值个数的情况进行说明。

(1) 当“门级延时量”只包含一个延时值时，也就是说当门级延时说明部分是“# d”



形式时(其中 d 是一个给定的延时值), 延时值“ d ”将同时代表元件实例的“上升延时值”、“下降延时值”、“截止延时值”和“到不定态的延时值”(在多输入门或多输出门情况下没有“到不定态的延时值”)。

例如, 在元件实例语句“`notif1 #20 U1 (out, in, ctrl);`”中, 门级延时说明部分是“`#20`”, 其中门级延时量“`20`”内只包含了一个延时值(20)。这表明元件实例 U1 的所有类型的门级延时都是 20 个单位时间。

(2) 当“门级延时量”包含两个延时值时, 也就是当门级延时说明部分是“`# (d1, d2)`”形式时(其中 $d1$ 、 $d2$ 为两个给定的延时值), 元件实例的“上升延时值”将由“ $d1$ ”指定; “下降延时值”将由“ $d2$ ”指定; 而“截止延时值”和“到不定态的延时值”将由“ $d1$ ”和“ $d2$ ”中的较小者指定。

比如, 在元件实例语句“`notif1 #(10, 20) U2 (out, in, ctrl);`”中, 门级延时说明部分是“`#(10, 20)`”, 其中门级延时量“`(10, 20)`”内包含了两个延时值(10 和 20)。这表明: 元件实例 U2 将具有 10 个单位时间的“上升延时”和 20 个单位时间的“下降延时”, 而它的“截止延时”和“到不定态的延时”将是 10 和 20 中的较小值(也就是 10 个单位时间)。

(3) 当“门级延时量”包含三个延时值时, 也就是当门级延时说明部分是“`# (dA, dB, dC)`”形式时(其中 dA 、 dB 、 dC 为三个给定的延时值), 元件实例的“上升延时值”将由“ dA ”指定; “下降延时值”将由“ dB ”指定; “截止延时值”将由“ dC ”指定; 而“到不定态的延时值”将由 dA 、 dB 和 dC 中的较小者指定。

比如, 在元件实例语句“`notif1 #(10, 20, 30) U3 (out, in, ctrl);`”中, 门级延时说明部分是“`#(10, 20, 30)`”, 其中门级延时量“`(10, 20, 30)`”内包含了三个延时值(10、20 和 30)。这表明: 元件实例 U3 将具有 10 个单位时间的“上升延时”、20 个单位时间的“下降延时”以及 30 个单位时间的“截止延时”, 而它的“到不定态的延时”将是 10、20 和 30 中的较小值也就是 10 个单位时间。

我们可以将上面讨论的各种情况下的门级延时值确定方式归纳在表 8.4 中, 其中的各个列分别代表着“门级延时量”包含 1、2、3 个延时值的情况, 而其中的每一行则分别给出了四类门级延时值在对应情况下的确定方式。

表 8.4 各类门级延时值的确定

	给出 1 个延时值时 $\#d$	给出 2 个延时值时 $\#(d1, d2)$	给出 3 个延时值时 $\#(dA, dB, dC)$
上升延时值	d	$d1$	dA
下降延时值	d	$d2$	dB
到不定态的延时值	d	$\min(d1, d2)$	$\min(dA, dB, dC)$
截止延时值	d	$\min(d1, d2)$	dC

2) “最小延时”、“典型延时”和“最大延时”的表示方式

除了基本表示方式外, 门级延时量还可以采用一种更为复杂的表示方式, 在这种表示方式下, 门级延时量中的每一项将由“最小延时”、“典型延时”和“最大延时”三个值来



表示。这种表示方式下的“门级延时量”将具有如下格式：

(dA_min: dA_typ: dA_max, dB_min: dB_typ: dB_max, dC_min: dC_typ: dC_max)

在上述格式中，门级延时量包括了三部分(dA、dB、dC 三部分)，这三部分之间用逗号进行分隔；而每一部分又是由三个延时值组成的，这些延时值之间用冒号“:”分隔。在这三个延时值中：min 表示最小延时，typ 表示典型延时，max 表示最大延时。

在采用这种表示方式时，每一类门级延时(上升延时、下降延时、截止延时和到不定态的延时)将相应地由“最小延时”，“典型延时”和“最大延时”三个延时值共同确定。

表 8.5 给出了这种表示方式下各类门级延时值的确定方式。

表 8.5 各类门级延时值的确定

	最小延时	典型延时	最大延时
上升延时	DA_min	dA_typ	dA_max
下降延时	DB_min	dB_typ	dB_max
到不定态的延时	min(dA_min, dB_min, dC_min)	min(dA_typ, dB_typ, dC_typ)	min(dA_max, dB_max, dC_max)
截止延时	DC_min	dC_typ	dC_max

比如，在元件实例语句“notif1 #2: 3: 4, 5: 6: 7, 1: 3: 4) U4 (out, in, ctrl);”中，门级延时说明部分是“#2: 3: 4, 5: 6: 7, 1: 3: 4)”，其中的门级延时量“(2: 3: 4, 5: 6: 7, 1: 3: 4)”包含了三部分延时值“2: 3: 4”、“5: 6: 7”和“1: 3: 4”。这表明元件实例 U4 将具有如表 8.6 所示的门级延时情况。

表 8.6 元件实例 U4 所具有的门级延时

	最小延时	典型延时	最大延进
上升延时	2	3	4
下降延时	5	6	7
到不定态的延时	1	3	4
截止延时	1	3	4

选择使用那种延时(最小延时、典型延时或最大延时)是模拟运行时的一个选项，我们可以通过设置该选项来选择使用某一种延时。相应的模拟运行过程可以看作是最好情况分析、典型情况分析和最坏情况分析。

8.3.2 门级建模的例子

前面已经对各种基本门级元件以及它们的调用方法进行了介绍，这一小节将给出综合使用这些元件来进行门级建模的几个例子。



图 8.9 所示是一个 1 位全加器，其中“a_in”和“b_in”是数据输入端，“c_in”是进位输入端；“s_out”和“c_out”分别是和输出端和进位输出端。例 8-11 给出了这个全加器的门级描述。

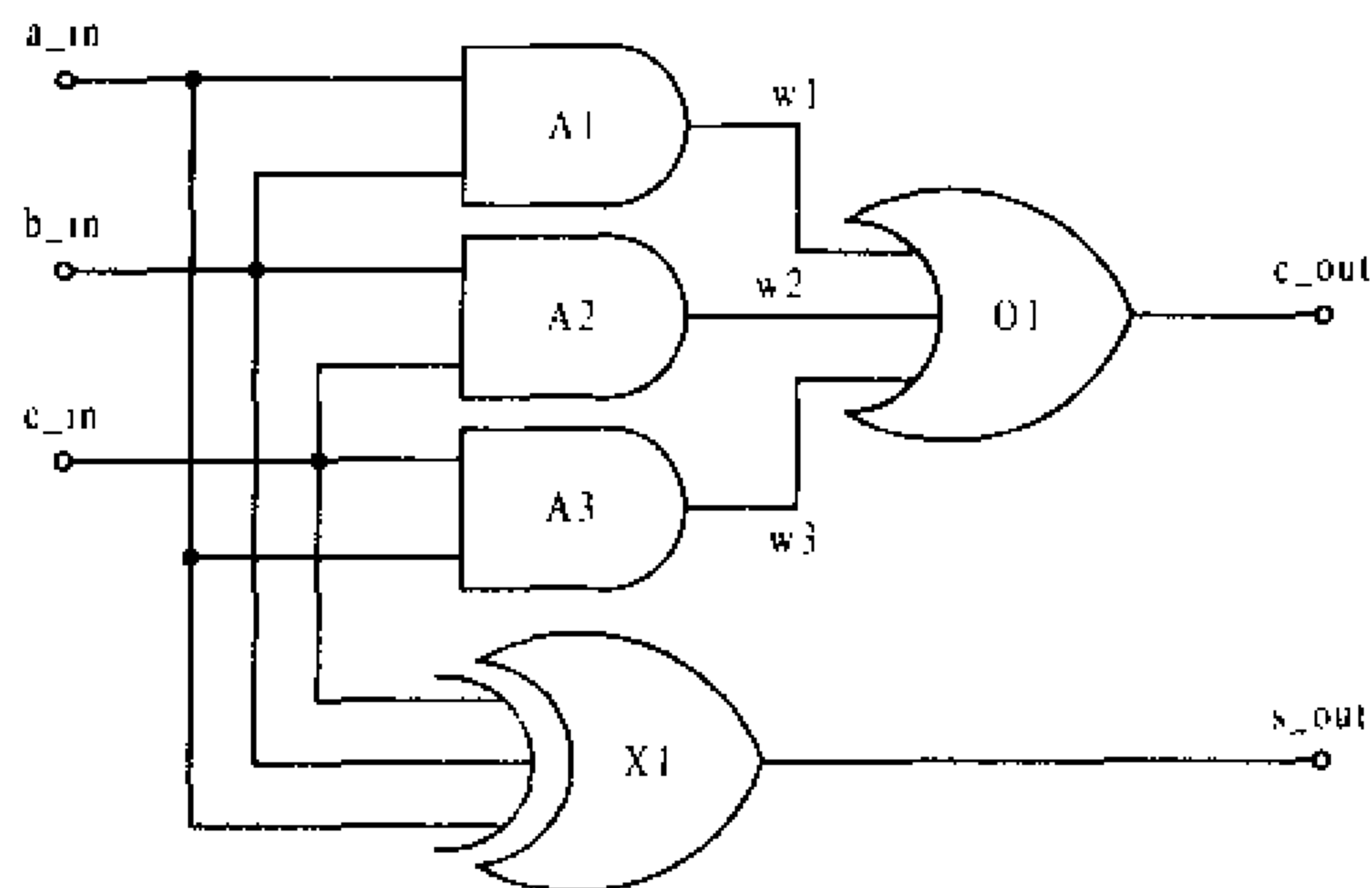


图 8.9 一位全加器

【例 8-11】一位全加器的门级建模。

```
module full_adder (c_out, s_out, a_in, b_in, c_in);
    output c_out, s_out;
    input a_in, b_in, c_in;
    wire w1, w2, w3;
    parameter AND_DELAY=2;
    parameter OR_DELAY=2;
    parameter XOR_DELAY=4;
    //元件实例语句
    and #AND_DELAY A1 (w1, a_in, b_in);
    and #AND_DELAY A2 (w2, b_in, c_in);
    and #AND_DELAY A3 (w3, c_in, a_in);
    or #OR_DELAY O1 (c_out, w1, w2, w3);
    xor #XOR_DELAY X1 (s_out, a_in, b_in, c_in);
endmodule
```

在例 8-11 所示的结构描述模块“full_adder”中：与门“and”得到了三次调用，分别生成了三个与门实例“A1”、“A2”和“A3”；或门“or”得到了一次调用，生成了一个或门实例“O1”；异或门“xor”得到了一次调用，生成了一个异或门实例“X1”。各个实例之间的连接关系由各条元件实例语句内的端口连接表共同确定。

我们再来看如图 8.10 所示的一个 2-4 译码器，其中“a0”和“a1”是译码代码输入端，“enable”是使能控制端，而“y0”，“y1”，“y2”和“y3”则是 4 个译码结果输出端。该译码器输入输出真值表为如表 8.7 所示。

表 8.7 2-4 译码器的真值表

输入		输出			
a1	a0	y3	y2	y1	Y0
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	1

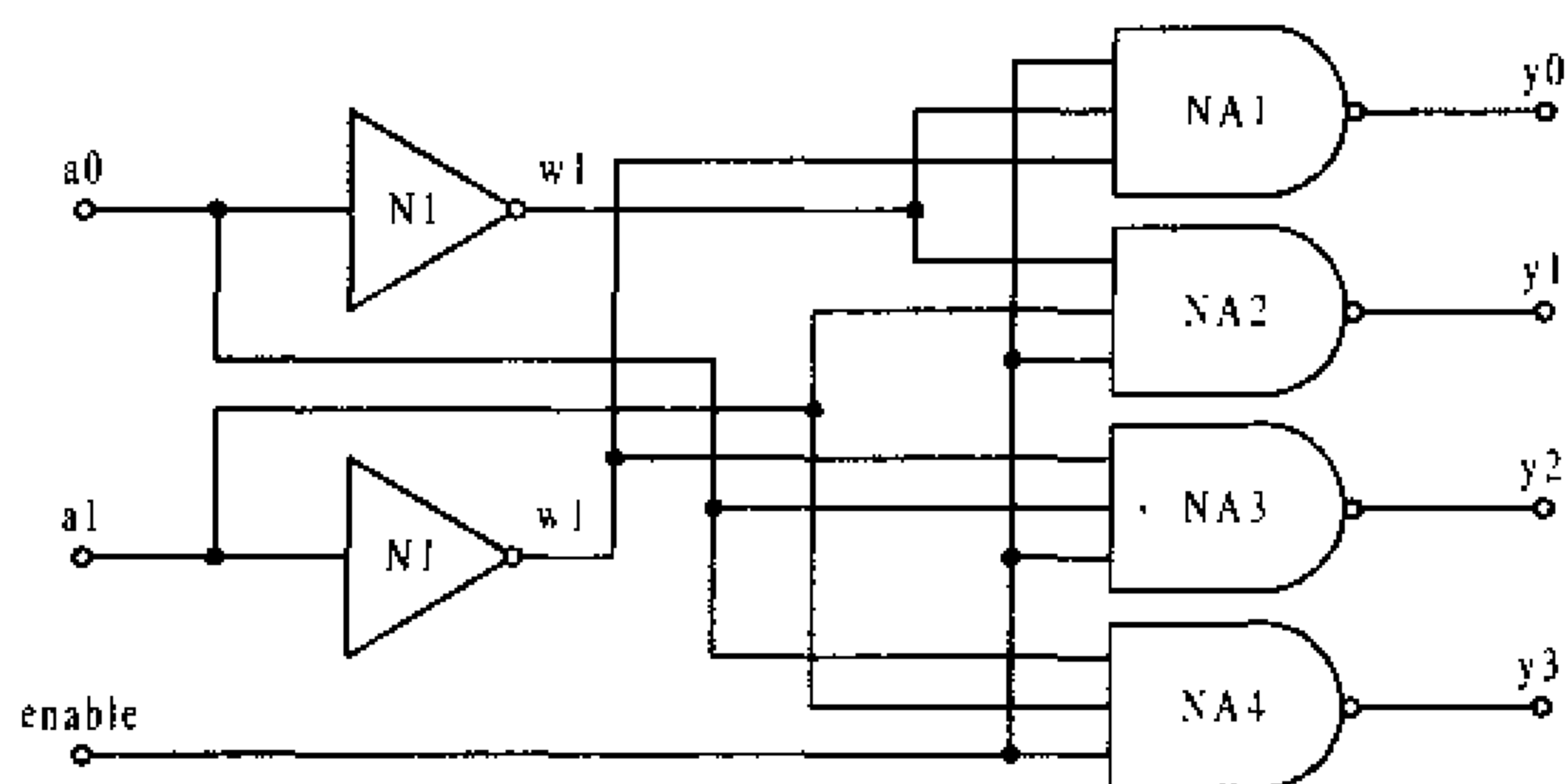


图 8.10 2-4 译码器

例 8-12 给出了这个 2-4 译码器的门级描述。

【例 8-12】一位 2-4 译码器的门级建模。

```

module decoder (y0, y1, y2, y3, a0, a1, enable);
output y0, y1, y2, y3;
input a0, a1, enable;
wire w1, w2;
//元件实例语句
not #(1, 2)
    N1 (w1, a0),
    N2 (w2, a1);
nand #(4, 3)
    NA1 (y0, enable, w1, w2),
    NA2 (y1, enable, w1, a1),
    NA3 (y2, enable, a0, w2),
    NA4 (y3, enable, a0, a1);
endmodule
    
```

在上例给出的模块“decoder”中，通过各条元件实例语句引入了：

- 两个非门实例“N1”和“N2”。
- 四个与非门实例“NA1”、“NA2”、“NA3”和“NA4”。

这些门级元件实例之间的连接关系由各条实例语句中的端口连接表共同确定。



最后举一个对“D 型主从触发器”进行门级建模的 Verilog 结构描述模块的例子。“D 型主从触发器”的结构如图 8.11 所示。例 8-13 给出了它的门级描述。

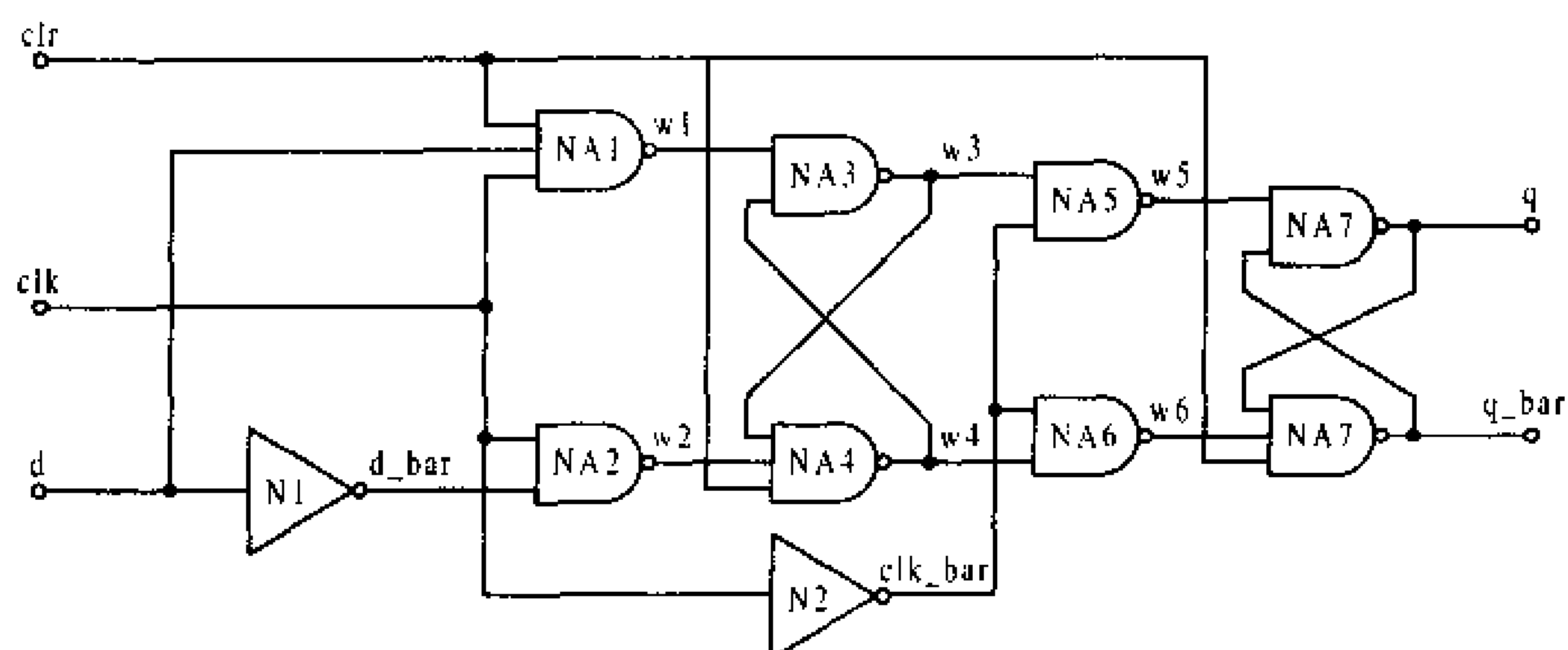


图 8.11 D 型主从触发器的结构

【例 8-13】D 型主从触发器的门级建模。

```
module d_flop(d, clk, clr, q, q_bar);
input  d, clk, clr;
output q, q_bar;
wire   w1, w2, w3, w4, w5, w6, d_bar, clk_bar;
//元件实例语句
nand #10
NA1(w1, clr, d, clk),
NA2(w2, clk, d_bar),
NA4(w4, w3, w2, clr),
NA5(w5, w3, clk_bar),
NA6(w6, w4, clk_bar),
NA8(q_bar, q, w6, clr);
nand #9
NA3(w3, w1, w4),
NA7(q, w5, q_bar);
not #10
N1(d_bar, d),
N2(clk_bar, clk);
endmodule
```

8.4 specify 说明块和时序检验

在调用基本门级元件时，可以在调用语句内加入“门级延时说明”这一项来对元件实例输入输出端口之间的传输延时量进行说明。而对于由用户自己定义设计的模块来说，如

果要对模块输入端和输出端之间的延时关系进行说明，则不能采用前面的“加入门级延时说明”的办法，只能通过一个被称为“延时说明块(Specify Block)”的结构来对模块的传输延时进行说明。注意，延时说明块既可以出现在行为描述模块内，也可以出现在结构描述模块内。

【例 8-14】包含延时说明块的模块。

```
'timescale 1ns / 1ns
module specify_block_example(out, a, b, c);
output out;
input a, b, c;
nand #3 NA1(out, w1, c);
xor #2 X1(w1, a, b);
//延时说明块
specify
specparam a_out=6, b_out=5, c_out=1;
(a⇒out)=a_out;
(b⇒out)=b_out;
(c⇒out)=c_out;
endspecify
endmodule
```

在上面给出的例子中，模块 `specify_block_example` 有三个输入端口“a”、“b”、“c”以及一个输出端口“out”，其结构如图 8.12 所示。

关键词“specify”与“endspecify”之间的内容就是一个“延时说明块”。“specify”与“endspecify”分别是延时说明块的起始标识符和终止标识符。

延时说明块是模块内部的一个独立结构成份，它与如下结构成份是并列的：

- (1) 过程块(initial 过程块或 always 过程块)。
- (2) 连续赋值语句。
- (3) 任务定义(task)。
- (4) 函数定义(function)。
- (5) 模块实例语句。
- (6) 基本元件实例语句。

在延时说明块内可以实现：

- (1) 对一些代表延时量的延时参数进行定义(specparam 语句)。
- (2) 对模块输入端和输出端之间的信号延时量进行说明。
- (3) 借助时序检验系统任务对模块输入输出时序进行时序检验。

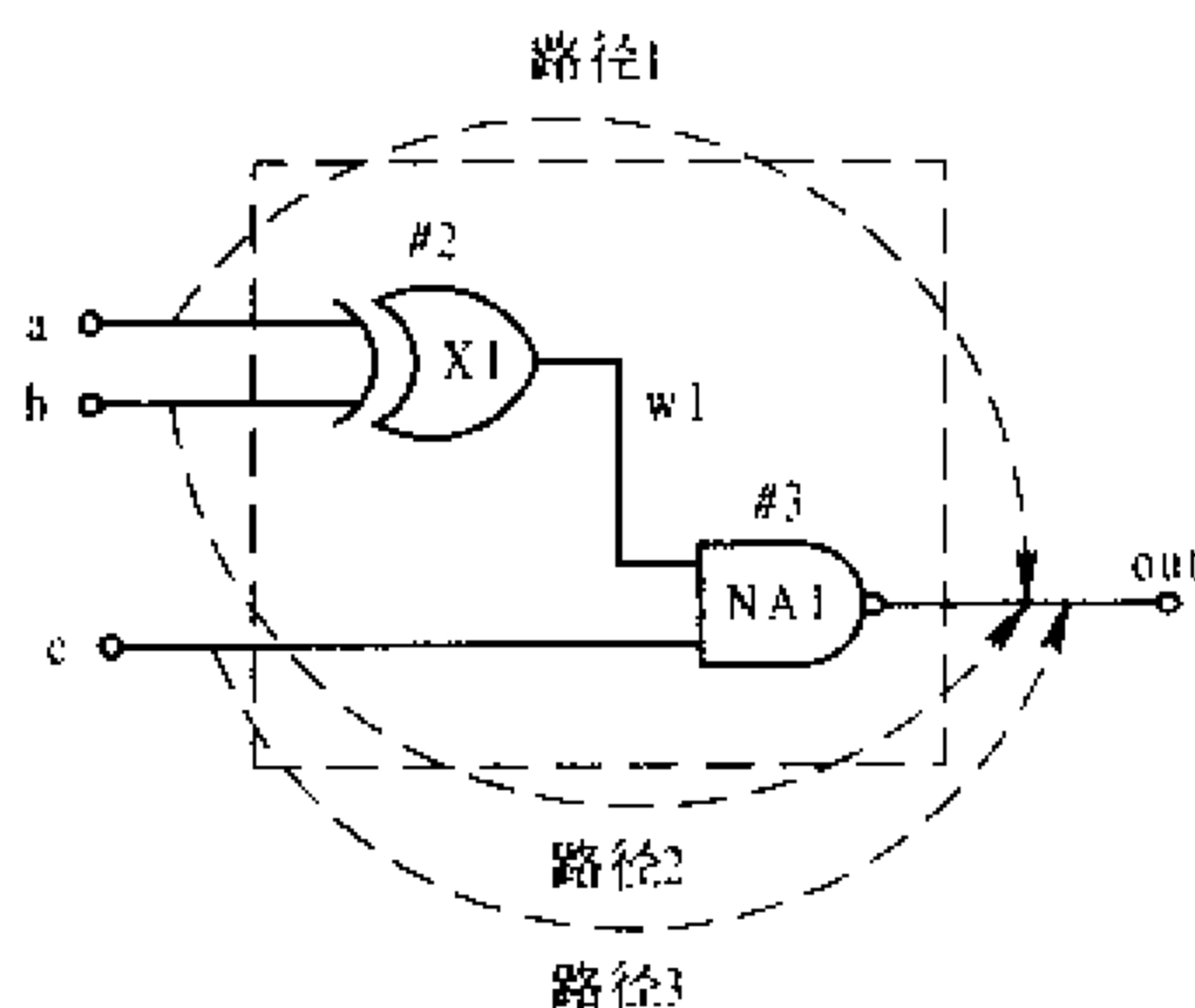


图 8.12 延时说明块的例子



8.4.1 延时参数的定义: specparam 语句

例 8-14 所示延时说明块内的如下这条语句:

```
specparam a_out=6, b_out=5, c_out=1;
```

就是一条延时参数定义语句,它定义了三个延时参数:

- (1) 延时参数“a_out”,它的参数值为 6,代表了 6 个单位时间的延时量。
- (2) 延时参数“b_out”,它的参数值为 5,代表了 5 个单位时间的延时量。
- (3) 延时参数“c_out”,它的参数值为 1,代表了 1 个单位时间的延时量。

specparam 语句的格式和作用都类似于前面介绍过的“parameter 参数说明语句”,两者之间的不同之处在于:

- (1) specparam 语句只能在延时说明块(specify 块)中出现;而 parameter 语句则不能在延时说明块内出现。
- (2) 由 specparam 语句进行定义参数只能是延时参数;而由 parameter 语句定义参数可以是任何数据类型的常数参数。
- (3) 由 specparam 语句定义的延时参数只能在延时说明块内使用;而由 parameter 语句定义的参数则可以在模块内(该 parameter 语句之后)的任意位置处使用。

8.4.2 对模块输入输出端口之间的路径延时进行说明

例 8-14 所示的延时说明块内的如下三条语句:

```
(a⇒out)=a_out;
```

```
(b⇒out)=b_out;
```

```
(c⇒out)=c_out;
```

对模块输入输出端口间的路径延时进行了说明,其中:

- (1) 从输入端口“a”到输出端口“out”(路径 1)的延时量由延时参数“a_out”的参数值(6)指定,所以该路径上的延时量为 6 ns。
- (2) 从输入端口“b”到输出端口“out”(路径 2)的延时量由延时参数“b_out”的参数值(5)指定,所以该路径上的延时量为 5 ns。
- (3) 从输入端口“c”到输出端口“out”(路径 3)的延时量由延时参数“c_out”的参数值(1)进行指定,所以该路径上的延时量为 1 ns。

这些在延时说明块内指定的延时量被称为“路径延时”。与此相对应,将模块内某条路径上各个基本单元(可以是基本门级元件、开关级元件或子模块)的延时加起来所构成的延时被称为“分布延时”。

在使用延时说明块来对路径延时进行定义时,其中的每条“路径”都是由一个模块输入端到一个模块输出端所构成的路径。路径的起点必须是模块输入端(而不能是输出端或模块内部连线),路径的终点必须是模块的输出端(而不能是输入端或模块内部连线)。

例如,在例 8-14 所示的模块中,如下通路不能在延时说明块内进行路径延时定义(参见图 8.12):

a	→	b	起止点都是输入端口
out	→	a	起点是输出端口,终点是输入端口



a → w1 终点是模块内部的连线而不是一个模块端口

w1 → out 起点是模块内部的连线而不是一个模块端口

延时说明块所定义的某个路径延时有可能会不同于该路径具有的分布延时，例如在图 8.12 所示的各条路径上：

a → out 分布延时为 $2+3=5$ ns 路径延时被指定为 6 ns

b → out 分布延时为 $2+3=5$ ns 路径延时被指定为 5 ns

c → out 分布延时为 3 ns 路径延时被指定为 1 ns

在这种情况下，程序仿真执行时将以不同延时值中的较大者为准。比如以上各条路径的延时值将分别是：6 ns，5 ns 和 3 ns。

路径延时的说明可以采用如下两种方式：

1) 并行连接说明方式

并行连接说明方式的格式为：

(<输入端口 1>, …… , <输入端口 n> ⇒ <输出端口 1>, …… , <输出端口 n>) = <延时量>;

其中“<延时量>”的表示可以采用 8.3.1 节所描述的由一个、二个或三个延时值组成延时量的基本表示方式，也可以采用“最小延时、典型延时和最大延时的表示方式”。在延时量由多个值组成的情况下，应在延时量的外面加上一对括号。如：

(a ⇒ out) = (10, 9, 8);

在上述格式的并行连接说明方式下，如下 n 条路径的路径延时得到了说明：

输入端口 1 → 输出端口 1

输入端口 2 → 输出端口 2

……

输入端口 n → 输出端口 n

这些路径所对应的延时时间都由<延时量>决定。

比如语句：

(in1, in2, in3 ⇒ out2, out3, out1) = 3.4;

就同时对三条路径进行了路径延时定义：

端口 in1 → 端口 out2 路径延时时间为 3.4 个单位时间

端口 in2 → 端口 out3 路径延时时间为 3.4 个单位时间

端口 in3 → 端口 out1 路径延时时间为 3.4 个单位时间

由于在这种说明方式下，连接符“⇒”两边的各个端口将依据其排列顺序一一对应并构成 n 条路径，因此我们称其为“并行连接说明方式”。在这种连接方式下连接符“⇒”两边的端口数目必须相等。

在“输入端口”和“输出端口”是矢量端口的情况下，对应端口的各个位将依次相连构成多条路径。比如如下模块：

module sample(a, b);

input [0: 2] a ;

output [5: 3] b;

……



```

specify
(a⇒b)=10;
endspecify
endmodule

```

中的延时说明块就说明了如下路径的延时量(被说明为 10 个单位时间的延时量):

```

a[0] → b[5]
a[1] → b[4]
a[2] → b[3]

```

2) 全连接说明方式

全连接说明方式的格式为:

(<输入端口 1>, ……,<输入端口 n> *> <输出端口 1>, ……,<输出端口 m>) = <延时量> ;

其中的连接符“*>”不同于并行连接说明方式下的“⇒”连接符。与并行连接说明方式一样,上面格式中的“延时量”可以采用由一个、二个或三个延时值组成的基本表示方式,也可以采用“最小延时、典型延时和最大延时的表示方式”。

在上述格式的全连接说明方式下,如下 $m \times n$ 条路径的路径延时得到了说明:

```

输入端口 1 → 输出端口 1
输入端口 1 → 输出端口 2
.....
输入端口 1 → 输出端口 m
输入端口 2 → 输出端口 1
.....
输入端口 2 → 输出端口 m
.....
输入端口 n → 输出端口 m

```

这些路径所对应的延时时间都由“<延时量>”决定。

在这种连接方式下,连接符“*>”左边的各个输入端口将与右边的每一个输出端口构成路径连接关系,所以一共构成了 $m \times n$ 条路径,这些路径具有的延时量都由“延时量”指定。在这种方式下,连接符“*>”两边的端口数目不必相等。

比如语句:

```
(in1, in2 *> out1, out2) = 4 ;
```

就同时对如下四条路径进行了路径延时说明:

端口 in1	→	端口 out1	路径延时时间为 4 个单位时间
端口 in1	→	端口 out2	路径延时时间为 4 个单位时间
端口 in2	→	端口 out1	路径延时时间为 4 个单位时间
端口 in2	→	端口 out2	路径延时时间为 4 个单位时间

它相当于如下四条语句的作用:

```
(in1 ⇒ out1) = 4 ;
```



```
(in1 ⇒ out2) = 4 ;
```

```
(in2 ⇒ out1) = 4 ;
```

```
(in2 ⇒ out2) = 4 ;
```

如果“输入端口”以及“输出端口”是具有多个位的矢量端口，那么输入端口的各个位将分别与输出端口的每一个位构成路径连接关系。比如模块：

```
module sample2(b, a);
```

```
    input [1: 2] a ;
```

```
    output [1: 2] b;
```

```
    .....
```

```
    specify
```

```
    (a *> b)=10;
```

```
    endspecify
```

```
endmodule
```

中的延时说明块就对如下的各条路径进行了延时量说明，它们都具有 10 个单位时间的延时量：

```
a[1] → b[1]
```

```
a[1] → b[2]
```

```
a[2] → b[1]
```

```
a[2] → b[2]
```

8.4.3 借助时序检验系统任务对模块输入输出时序进行时序检验

除了对模块输入输出端口之间的路径延时进行说明外，延时说明块还可以用来对模块输入输出时序进行时序检验。

Verilog 提供了多种“时序检验系统任务”，这些系统任务只能使用在延时说明块内，在此只介绍其中的两种：建立时间检验系统任务(\$setup)和保持时间检验系统任务(\$hold)。

1. 建立时间检验系统任务：\$setup

建立时间检验系统任务\$setup 用来对“信号的建立时间是否满足给定的最低要求”进行检测。其调用格式为：

```
$setup(<目标信号名>, <参考事件>, <建立时间下限>, <违规触发标志位>);
```

其中：“<目标信号名>”指定了受监测的目标信号。

“<参考事件>”指定了一个时间参考信号，其作用是为了确定时间量的参考点。

“<建立时间下限>”给出了对目标信号建立时间长度的最低要求，它是一个常数时间值。

“<违规触发标志位>”部分是可以缺省的，它必须是一个一位的寄存器变量，它是为用户提供的—个检验结果标志位。它可以作为模块的输出信号得到输出。

例如，语句：

```
$setup (data, posedge CLOCK, 10, indicator1);
```

就对信号 data 的建立时间进行了时序检验。其中指定的最小建立时间为 10 个单位时间，



参考事件是信号 CLOCK 的上跳变(posedge CLOCK), 检验结果将通过寄存器 indicator1 得到表示。

在执行“\$setup 系统任务”时, 如果目标信号由非稳定态进入稳定态的时刻(图 8.13 中标出的“时刻 1”)与“参考事件”发生的时刻(“时刻 2”)之间的时间差(“建立时间”)小于给定的“建立时间下限”, 则仿真程序会自动进行违规报警, 同时“违规触发标志位”的取值反转(如果是第一次违规报警则“违规触发标志位”被置为 1)。可以根据这个标志位的取值变化来启动相应的处理操作。

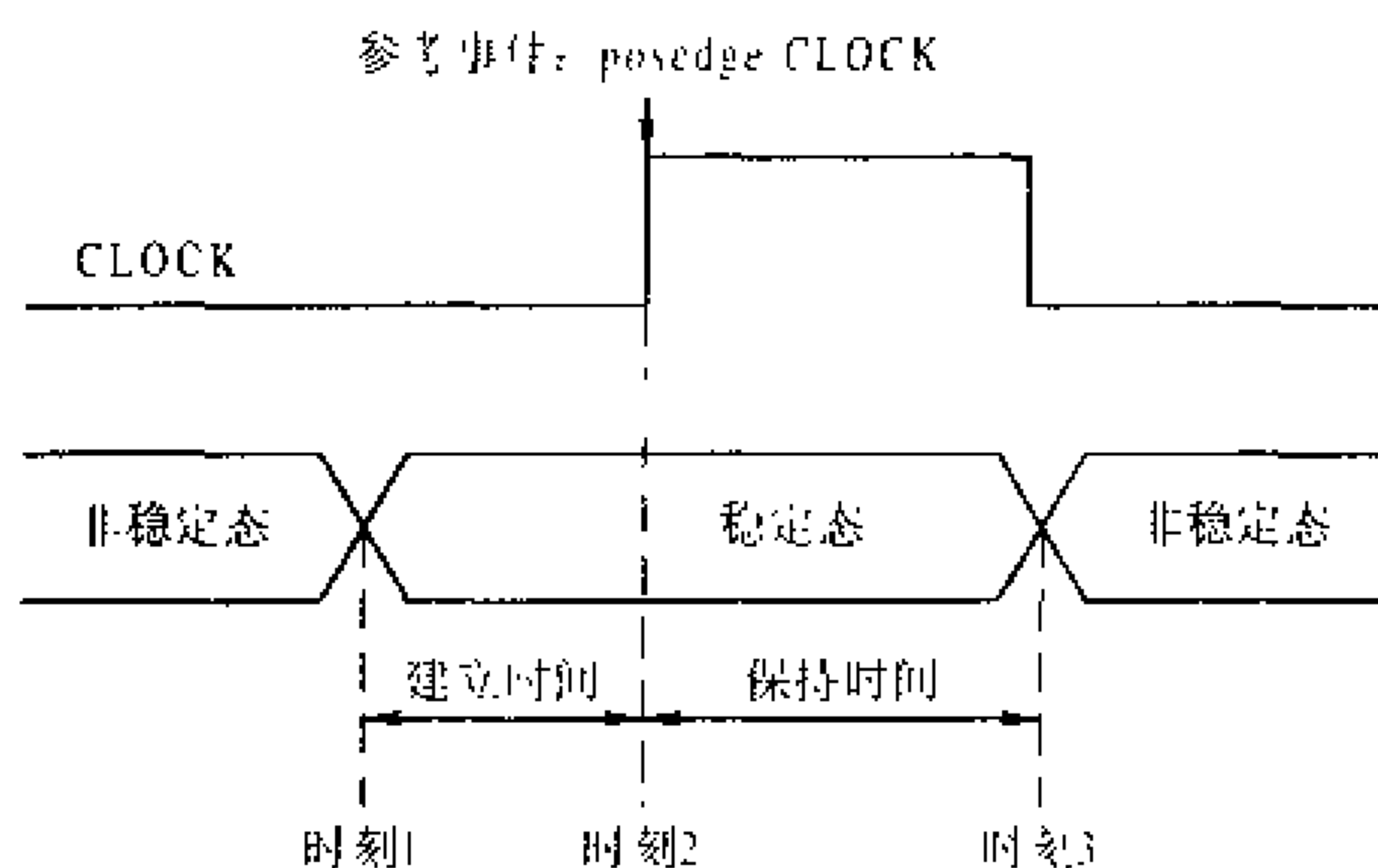


图 8.13 建立时间和保持时间

2. 保持时间检验系统任务: \$hold

保持时间检验系统任务\$hold 用来对“信号的保持时间是否满足给定的最低要求”进行检测。其调用格式为:

\$hold (<参考事件>, <目标信号名>, <保持时间下限>, <违规触发标志位>);

其中: “<参考事件>”指定了一个时间参考信号, 其作用是为了确定时间量的参考点。

“<目标信号名>”指定了受监测的目标信号。

“<保持时间下限>”给出了对目标信号保持时间长度的最低要求, 它是一个常数时间值。

“<违规触发标志位>”是可以缺省的, 它必须是一个一位的寄存器变量, 它是为用户提供的的一个检验结果标志位。它可以作为模块的输出信号得到输出。

例如, 语句:

\$hold (posedge CLOCK, data, 8, indicator2);

就对信号 data 的保持时间进行了时序检验, 其中指定的最小保持时间为 8 个单位时间, 参考事件是信号 CLOCK 的上跳变(posedge CLOCK), 检验结果将通过寄存器 indicator2 表示。

在执行“\$hold 系统任务”时, 如果目标信号由稳定态变为非稳定态的时刻(如图 8.13 中标出的“时刻 3”)与“参考事件”发生时刻(“时刻 2”)之间的时间差(“保持时间”)小于给定的“保持时间下限”, 则仿真程序会自动进行违规报警, 同时“违规触发标志位”的取值被反转(如果是第一次违规报警, 则“违规触发标志位”被置为 1)。

第9章 测试与仿真



为了检验设计模块功能的正确性，常常需要将一系列的信号输入到已设计的模块，并检查已设计模块的输出，看它们是否符合设计要求。这就要求我们编写测试模块。本章首先讨论测试与仿真的流程，然后对测试模块的描述方法进行说明，最后提供了几个常用电路的测试模块实例，并给出仿真结果。

9.1 测试与仿真的流程

对已设计模块的测试与仿真通常可分为以下三个步骤：

- (1) 产生输入向量，包括输入向量的初始化与产生测试波形。
- (2) 将输入向量添加到已设计模块并给出相应的输出结果。
- (3) 将输出结果与设计要求相比较

9.1.1 产生输入向量

对输入向量的初始化可通过 initial 过程块来实现。

【例 9-1】输入向量的初始化。

```
initial
begin
    clk = 1'b0;
    globalReset = 1'b1;
    in = 1'b1;
end
```

在上面的例子中，将二进制表示形式的 0 值赋给变量 clk。实际上，对于值 0 与 1，没有必要采用二进制表示形式 1'b0 与 1'b1，直接用 0 与 1 给出即可。

当给输入向量赋值时，可以利用一定的延迟时间来控制测试与仿真程序(参见下面的例子)。

【例 9-2】带延迟的向量赋值。

```
#100 globalReset = 0;
#100 in = 0;
#100 in = 1;
#300 in = 0;
```

经过 100 个时间单位的延迟后将变量 globalReset 的值置为 0，再经过 100 个时间单位的延迟把变量 in 变为 0，依次类推。这里需要注意的是，时延是一个相对的概念，即相对于上一条语句执行完后的时刻而言，并非相对于仿真的起始时间。



如果测试模块中没有控制仿真结束时刻的语句,那么当对测试模块进行仿真实验时,程序就会陷入死循环。通过在 initial 过程块中加入带有延迟的系统任务 \$finish 或 \$stop,就可轻易解决该问题。

【例 9-3】仿真结束语句的应用

```
initial
begin
    #100 globalReset = 0;
    #100 in = 0;
    #100 in = 1;
    #300 in = 0;
    #400 $finish
end
```

采用 always 过程块可以很容易实现时钟信号,下面是一个产生某测试模块的时钟信号的例子。

【例 9-4】时钟信号的产生。

```
always
begin
    #10 clk = ~clk;
end
```

该例用来产生一个周期为 20 个时间单位的时钟信号。

如果输入信号的值有规律地变化时,例如按相同的延迟重复出现 n 次,那么就可以通过 repeat 循环语句来实现。

【例 9-5】repeat 循环语句的例子。

```
repeat(10)
begin
    #40 in = 0;
    #20 in = 1;
end
```

9.1.2 测试模块

为了更好地理解测试与仿真的流程,可将测试模块与功能模块进行比较。这里以一个 4 选 1 的多路选择器的例子进行详细的说明。

【例 9-6】4 选 1 多路选择器的功能模块与测试模块。

//功能模块部分

```
module multiplexor4_1(out, in1, in2, in3, in4, cntrl1, cntrl2);
    output out;
    input in1, in2, in3, in4, cntrl1, cntrl2;
    reg out;
    always @(in1 or in2 or in3 or in4 or cntrl1 or cntrl2)
```



```

        case ({cntrl1, cntrl2})
            2'b00: out = in1;
            2'b01: out = in2;
            2'b10: out = in3;
            2'b11: out = in4;
        default: $display("Please check control bits");
    endcase
endmodule
//测试模块部分
module muxstimulus;
    reg IN1, IN2, IN3, IN4, CNTRL1, CNTRL2;
    wire OUT;
    multiplexor4_1 mux1_4(OUT, IN1, IN2, IN3, IN4, CNTRL1, CNTRL2);
    initial begin
        IN1 = 1; IN2 = 0; IN3 = 1; IN4 = 0;
        $display("Initial arbitrary values");
        #0 $display("input1 = %b, input2 = %b, input3 = %b, input4 = %b\n",
            IN1, IN2, IN3, IN4);
        {CNTRL1, CNTRL2} = 2'b00;
        #1 $display("cntrl1=%b, cntrl2=%b, output is %b", CNTRL1, CNTRL2, OUT);
        {CNTRL1, CNTRL2} = 2'b01;
        #1 $display("cntrl1=%b, cntrl2=%b, output is %b", CNTRL1, CNTRL2, OUT);
        {CNTRL1, CNTRL2} = 2'b10;
        #1 $display("cntrl1=%b, cntrl2=%b, output is %b", CNTRL1, CNTRL2, OUT);
        {CNTRL1, CNTRL2} = 2'b11;
        #1 $display("cntrl1=%b, cntrl2=%b, output is %b", CNTRL1, CNTRL2, OUT);
    end
endmodule

```

测试模块与功能模块相类似，都是以关键字 `module` 开头的。但由于测试模块只是为了进行仿真验证，因而不需要端口列表，也不能被其它模块调用。而功能模块则必须有端口列表。

由于测试模块没有端口列表，因而不需进行端口声明。为了对功能模块的参数进行赋值，测试模块中也需进行数据类型的说明，即

```

reg IN1, IN2, IN3, IN4, CNTRL1, CNTRL2;
wire OUT;

```

接下来就是调用功能模块的部分：

```

multiplexor4_1 mux1_4(OUT, IN1, IN2, IN3, IN4, CNTRL1, CNTRL2);

```

可以看出，其调用格式如下：

```

<module_name> <instance_name> (port list);

```




其中<module_name>代表已设计的功能模块的名字；<instance_name>是不能省略的，且调用部分的端口列表必须与功能模块的端口列表(包括端口的个数与排列顺序)相对应。

测试模块的主体部分包含在 initial 过程块中，即

```
initial begin
    IN1 = 1; IN2 = 0; IN3 = 1; IN4 = 0;
    $display("Initial arbitrary values");
    #0 $display("input1 = %b, input2 = %b, input3 = %b, input4 = %b\n",
    IN1, IN2, IN3, IN4);
    {CNTRL1, CNTRL2} = 2'b00;
    #1 $display("cntrl1=%b, cntrl2=%b, output is %b", CNTRL1, CNTRL2, OUT);
    {CNTRL1, CNTRL2} = 2'b01;
    #1 $display("cntrl1=%b, cntrl2=%b output is %b", CNTRL1, CNTRL2, OUT);
    {CNTRL1, CNTRL2} = 2'b10;
    #1 $display("cntrl1=%b, cntrl2=%b output is %b", CNTRL1, CNTRL2, OUT);
    {CNTRL1, CNTRL2} = 2'b11;
    #1 $display("cntrl1=%b, cntrl2=%b output is %b", CNTRL1, CNTRL2, OUT);
end
```

首先对输入向量进行赋值，并通过系统任务\$display 显示其结果，

```
IN1 = 1; IN2 = 0; IN3 = 1; IN4 = 0;
$display("Initial arbitrary values");
#0 $display("input1 = %b, input2 = %b, input3 = %b, input4 = %b\n",
    IN1, IN2, IN3, IN4);
```

延迟表示#0 是为了保证当给输入变量赋值后立即显示。然后通过给变量 CNTRL1 与 CNTRL2 赋值并用连接符号{ }将它们连接起来作为一个整体判决条件，来验证已设计模块的输出与设计要求是否吻合。

9.2 测试举例

通过上一节对测试与仿真流程的介绍，我们已对测试模块有了一个清晰的认识。在此基础上，本节给出一些常用电路的功能模块与测试模块的例子，以供读者参考。

【例 9-7】模为 16 的增 1 计数器。

// 4 位二进制增 1 计数器

```
module counter4_bit(q, d, increment, load_data, global_reset, clock);
    output [3: 0] q;
    input [3: 0] d;
    input load_data, global_reset, clock, increment;
    wire t1, t2, t3; //内部连线
    wire see15, reset; //内部连线
    //内部连线用来在模块间传递信息、
```




```
//它们可被当作临时变量、用来储存
//某个门的输出并输入另一个门
et_ff etff0(q[0], d[0], increment, load_data, reset, clock);
et_ff etff1(q[1], d[1], t1, load_data, reset, clock);
et_ff etff2(q[2], d[2], t2, load_data, reset, clock);
et_ff etff3(q[3], d[3], t3, load_data, reset, clock);
and a1(t1, increment, q[0]);
and a2(t2, t1, q[1]);
and a3(t3, t2, q[2]);
recog15 r1(see15, q);
or or1(reset, see15, global_reset);
endmodule

module et_ff(q, data, toggle, load_data, reset, clock);
    output q;
    input data, toggle, load_data, reset, clock;
    wire m1, m2;
    mux mux0(m1, ~q, q, toggle);
    mux mux1(m2, data, m1, load_data);
    dff dff0(q, m2, reset, clock);
endmodule

module mux(out, in1, in2, ctrl):
    output out;
    input in1, in2, ctrl;
    assign out = ctrl ? in1 : in2;
    //这是一个连续赋值,任一操作数的改变,输出将相应地改变
    //此语句相当于 if (ctrl==1) out = in1; else out = in2;
endmodule

module dff(q, data, reset, clock);
    output q;
    input data, reset, clock;
    reg q;
    always @(posedge clock) //在时钟的上升沿,如果 reset 为 1, q 复位
        if (reset == 1) //成 0, 否则 q 等于 data
            q = 0;
        else q = data;
endmodule

module recog15(flag, in);
    input [3: 0] in;
    output flag;
```




```

    assign flag = (in == 4'b1111) ? 1: 0;
    // 如果输入为 15 , 则 flag 被设置为 1,
    // 否则为 0
endmodule

module stimulus;
    wire [3: 0] q;
    reg [3: 0] d;
    reg          load_data, global_reset, clk, increment;
    counter4_bit mod1 (q, d, increment, load_data, global_reset, clk);
    initial begin
        global_reset = 0;
        clk = 0;
        increment = 0;
        load_data = 0;
        d = 4'b0010;
        #10 global_reset = 1;
        #20 global_reset = 0;
        #20 load_data = 1;
        #20 load_data = 0;
        #20 increment = 1;
        #200 global_reset = 1;
        #20 global_reset = 0;
        #50 load_data = 1;
        #20 load_data = 0;
        #10 increment = 0;
        #20 $finish;
    end // initial begin
    always #5 clk = ~clk;
    always #10 $display ($time, "   %b %b %b %d -> %b %d",
        increment, load_data, global_reset, d, q, q);
endmodule

```

仿真测试的输出结果如下:

```

10  0 0 1  2 -> xxxx  x
20  0 0 1  2 -> 0000  0
30  0 0 0  2 -> 0000  0
40  0 0 0  2 -> 0000  0
50  0 1 0  2 -> 0000  0
60  0 1 0  2 -> 0010  2
70  0 0 0  2 -> 0010  2

```




```
80 000 2->0010 2
90 100 2->0010 2
100 100 2->0011 3
110 100 2->0100 4
120 100 2->0101 5
130 100 2->0110 6
140 100 2->0111 7
150 100 2->1000 8
160 100 2->1001 9
170 100 2->1010 10
180 100 2->1011 11
190 100 2->1100 12
200 100 2->1101 13
210 100 2->1110 14
220 100 2->1111 15
230 100 2->0000 0
240 100 2->0001 1
250 100 2->0010 2
260 100 2->0011 3
270 100 2->0100 4
280 100 2->0101 5
290 101 2->0110 6
300 101 2->0000 0
310 100 2->0000 0
320 100 2->0001 1
330 100 2->0010 2
340 100 2->0011 3
350 100 2->0100 4
360 110 2->0101 5
370 110 2->0010 2
380 100 2->0010 2
390 000 2->0011 3
400 000 2->0011 3
```

【例9-8】2选1的多路选择器的例子(用4种方法实现)。

//多路选择器的行为描述1

```
module mux_beh(out, cnt, a, b);
    output out;
    input  cnt, a, b;
    reg    out;
```




```
        always @(cnt or a or b)
            if (cnt == 0) out = a;
            else out = b;
    endmodule//mux_gate
//多路选择器的行为描述 2
module mux_beh_2 (out, cnt, a, b);
    output out;
    input  cnt, a, b;
    assign out = (cnt == 0) ? a: b;
endmodule//mux_gate
//多路选择器的功能描述
module mux_func (out, cnt, a, b);
    output out;
    input  cnt, a, b;
    assign out = ((~cnt & a) |(cnt & b));
endmodule//mux_gate
//多路选择器的门级描述
module mux_gate (out, cnt, a, b);
    output out;
    input  cnt, a, b;
    wire    aval, bval;
    wire    cntbar;
    not (cntbar,cnt);
    and (aval, cntbar, a);
    and (bval, cnt, b);
    or (out, aval, bval);
endmodule//mux_gate
//测试模块
module stimulus;
    reg    a, b, c;
    wire    out1, out2, out3, out4;
    integer i;
    mux_beh_2 mod1 (out1, c, a, b);
    mux_beh    mod2 (out2, c, a, b);
    mux_func   mod3 (out3, c, a, b);
    mux_gate   mod4 (out4, c, a, b);
    initial begin
        for (i = 0; i < 8; i = i + 1) begin
            {a, b, c} = i[2:0];
```




```
#10 $display("simulation time is: ", $time);  
$display ("inputs is: %b %b %b ->output is: %b %b %b %b",  
          a, b, c, out1, out2, out3, out4);  
end  
#10 $finish;  
end  
endmodule
```

仿真测试的输出结果如下:

```
simulation time is:          10  
inputs is: 0 0 0 ->output is: 0 0 0 0  
simulation time is:          20  
inputs is: 0 0 1 ->output is: 0 0 0 0  
simulation time is:          30  
inputs is: 0 1 0 ->output is: 0 0 0 0  
simulation time is:          40  
inputs is: 0 1 1 ->output is: 1 1 1 1  
simulation time is:          50  
inputs is: 1 0 0 ->output is: 1 1 1 1  
simulation time is:          60  
inputs is: 1 0 1 ->output is: 0 0 0 0  
simulation time is:          70  
inputs is: 1 1 0 ->output is: 1 1 1 1  
simulation time is:          80  
inputs is: 1 1 1 ->output is: 1 1 1 1
```

【例 9-9】触发器的例子。

```
module ff(q, data, enable, reset, clock);  
    output q;  
    input  data, enable, reset, clock;  
    wire    norout, muxout;  
    mux2_1 mod1 (muxout, data, q, enable);  
    nor    (norout, reset, muxout);  
    dff    dff0 (q, norout, clock);  
endmodule  
  
module mux2_1(out, in1, in2, cntrl);  
    output out;  
    input  in1, in2, cntrl;  
    assign out = cntrl ? ~in1 : ~in2;  
endmodule  
  
module dff(q, data, clock);
```




```
output q;
input data, clock;
reg q;
always @(posedge clock)
    q = data;
endmodule

module sti_ff;
reg data, enable, reset, clock;
wire q;
ff mod1 (q, data, enable, reset, clock);
initial begin
    clock = 0;
    forever clock = #5 ~clock;
end
initial begin
    enable = 0;    // 初始化使能变量
    reset = 1;
    #10 reset = 0;
    data = 1;      // 设置 data 为 1
    #10 enable = 1; // 触发器使能
    #10 data = 0;   // 改变 data 值
    #20 data = 1;   // 改变 data 值
    #10 data = 0;   // 改变 data 值
    #10 data = 1;   // 改变 data 值
    #10 data = 0;   // 改变 data 值
    #20 enable = 0; // 触发器不使能
    #10 data = 1;   // 改变 data 值 - 输出不变
    #10 reset = 1;  // 复位
    #20 $finish;    // 结束仿真任务
end
always@(reset or enable or data)
begin
    #10 $display("simulation time is:", $time);
    $display(" reset, enable, data, q ");
    $display("      %d      %d      %d      %d",
              reset, enable, data, q);
end
endmodule
```

仿真测试的输出结果如下：



```
simulation time is: 20
    reset,    enable,    data,    q
      0      1      1      0
simulation time is: 40
    reset,    enable,    data,    q
      0      1      0      0
simulation time is: 60
    reset,    enable,    data,    q
      0      1      0      1
simulation time is: 80
    reset,    enable,    data,    q
      0      1      0      1
simulation time is: 110
    reset,    enable,    data,    q
      0      0      1      0
simulation time is: 130
    reset,    enable,    data,    q
      1      0      1      0
```




第 10 章 设计举例与设计技巧

通过对前面几章的学习，我们对 Verilog HDL 有了一个比较系统的认识。比如对同一个电路而言，可以有两种不同的描述方法：行为描述与结构描述，而行为描述可以采用多种形式来实现。本章结合作者的工作实践，对常用电路的 Verilog HDL 实现给出了例示。在这些实例的功能模块实现中，尽量使用多种方法，以便读者在进行实际电路设计时参考。

10.1 加 法 器

加法器是常用的组合电路之一，本节给出了几类加法器的设计实例。

10.1.1 带进位输入的 8 位加法器

【例 10-1】带进位输入的 8 位加法器。

//功能模块部分

`timescale 1ns/1ps

module EightBitFA(FA,FB,FCin,FSum,FCout);

parameter Size=8;

input [Size:1] FA,FB;

output [Size:1] FSum;

input FCin;

output FCout;

wire [1:Size-1] Ftemp;

OneBitFA

FA1(.A(FA[1]),.B(FB[1]),.Cin(FCin),.Sum(FSum[1]),.Cout(Ftemp[1])),

FA2(.A(FA[2]),.B(FB[2]),.Cin(Ftemp[1]),.Sum(FSum[2]),.Cout(Ftemp[2])),

FA3(.A(FA[3]),.B(FB[3]),.Cin(Ftemp[2]),.Sum(FSum[3]),.Cout(Ftemp[3])),

FA4(.A(FA[4]),.B(FB[4]),.Cin(Ftemp[3]),.Sum(FSum[4]),.Cout(Ftemp[4])),

FA5(.A(FA[5]),.B(FB[5]),.Cin(Ftemp[4]),.Sum(FSum[5]),.Cout(Ftemp[5])),

FA6(.A(FA[6]),.B(FB[6]),.Cin(Ftemp[5]),.Sum(FSum[6]),.Cout(Ftemp[6])),

FA7(.A(FA[7]),.B(FB[7]),.Cin(Ftemp[6]),.Sum(FSum[7]),.Cout(Ftemp[7])),

FA8(.A(FA[8]),.B(FB[8]),.Cin(Ftemp[7]),.Sum(FSum[8]),.Cout(FCout));

endmodule

module OneBitFA(A,B,Cin,Sum,Cout);


```

input  A,B;
output Sum;
input Cin;
output Cout;
wire S1,T1,T2,T3;
xor
    X1(S1,A,B),
    X2(Sum,S1,Cin);
and
    A1(T3,A,B),
    A2(T2,B,Cin),
    A3(T1,A,Cin);
or
    O1(Cout,T1,T2,T3);
endmodule
//测试模块部分
module stimulus_EightBitFA;
    parameter Size=8;
    reg [Size:1] DA,DB;
    wire [Size:1] DSum;
    wire DCout;
    reg  DCin;
    EightBitFA
    F1(DA,DB,DCin,DSum,DCout);
    initial
        begin
            DA=8'b01100100;
            DB=8'b11100000;
            DCin=1'b1;
            #5;
            $display("input A is %b",DA);
            $display("input B is %b",DB);
            $display("input Cin is %b",DCin);
            $display("output Sum is: %b",DSum);
            $display("output Cout is: %b",DCout);
        end
endmodule

```

仿真结果如下:

input A is 01100100



```
input B is 11100000
input Cin is 1
output Sum is: 01000101
output Cout is: 1
```

在上面的例子中, 模块 **EightBitFA** 对于模块 **OneBitFA** 的调用采用的是端口名对应调用法。另外, 还可采用位置对应调用法, 相应部分的描述如下:

```
OneBitFA
    FA1(FA[1], FB[1], FCin, FSum[1], Ftemp[1]),
    FA2(FA[2], FB[2], Ftemp[1], FSum[2], Ftemp[2]),
    FA3(FA[3], FB[3], Ftemp[2], FSum[3], Ftemp[3]),
    FA4(FA[4], FB[4], Ftemp[3], FSum[4], Ftemp[4]),
    FA5(FA[5], FB[5], Ftemp[4], FSum[5], Ftemp[5]),
    FA6(FA[6], FB[6], Ftemp[5], FSum[6], Ftemp[6]),
    FA7(FA[7], FB[7], Ftemp[6], FSum[7], Ftemp[7]),
    FA8(FA[8], FB[8], Ftemp[7], FSum[8], FCout);
```

在进行模块定义时, 有几点需要注意:

- (1) 模块例示名(如 **FA1**)不能省略
- (2) 多个模块例示之间用“,”隔开, 而不是“;”。
- (3) 多个模块例示可以采用端口名对应与位置对应两种调用的混合形式, 但对同一个例示只能采用一种调用形式。

该例中的模块 **OneBitFA** 采用的是结构描述方式, 另外可采用行为描述方式或两种描述方式的混合设计, 这里只给出行为描述方式的两种不同形式。

利用连续赋值语句实现:

```
module OneBitFA(A, B, Cin, Sum, Cout);
    input A, B, Cin;
    output Sum, Cout;
    wire Sum, Cout;
    assign Sum=A^B^Cin;
    assign Cout=(A&Cin)|(B&Cin)|(A&B);
endmodule
```

利用过程赋值语句实现:

```
module OneBitFA(A, B, Cin, Sum, Cout);
    input A, B, Cin;
    output Sum, Cout;
    reg Sum, Cout;
    reg T1, T2, T3;
    always
        @(A or B or Cin)
begin
```




```
Sum=(A^B)^Cin;  
T1=A&Cin;  
T2=B&Cin;  
T3=A&B;  
Cout=(T1|T2)/T3;  
end  
endmodule
```

行为描述的仿真结果与结构描述的仿真结果相同。在接下来所讨论的例子中, 如果功能模块的某一子模块可以用多种描述方式实现, 则一并给出, 不再分开说明(且端口调用采用位置对应调用格式)。

10.1.2 带进位的通用加法器

【例 10-2】带进位的通用加法器!

```
`timescale 1ns/1ps  
//功能模块部分  
module GlobalFA(FA,FB,FCin,FSum,FCout);  
    parameter Size=8;  
    integer i;  
    input [Size:1] FA,FB;  
    output [Size:1] FSum;  
    input FCin;  
    output FCout;  
    assign {FCout,FSum}=FA+FB+FCin;  
endmodule  
//测试模块部分  
module stimulus_GlobalFA;  
    parameter Size=10;  
    reg [Size:1] DA,DB;  
    wire [Size:1] DSum;  
    wire DCout;  
    reg DCin;  
    GlobalFA  
    F1(DA,DB,DCin,DSum,DCout);  
    defparam F1.Size=Size;  
    initial  
        begin  
            DA=10'b00_1111_0110;  
            DB=10'b11_1000_1110;  
            DCin=1'b1;  
        end  
    end  
endmodule
```




```

        #5;
        $display("input A is %b",DA);
        $display("input B is %b",DB);
        $display("input Cin is %b",DCin);
        $display("output Sum is: %b",DSum);
        $display("output Cout is: %b",DCout);
    end
endmodule

```

仿真结果如下:

```

input A is 0011110110
input B is 1110001110
input Cin is 1
output Sum is: 0010000101
output Cout is: 1

```

如果实现其它位数的加法, 只需对测试模块中的两处稍作修改即可:

- (1) 将参数 Size 改为相应的位数值。
- (2) 在 initial 过程块中输入相应位数的加数与进位数。

例如, 求 $A=12'b0100_1111_0110$ 与 $B=12'b0011_1000_1110$ 的和, 且 $Cin=1'b0$ 。则在上面的程序中将测试模块中的参数 Size 改为 12, 将 DA、DB 与 DCin 用相应的 A、B 与 Cin 代替, 运行程序得到如下仿真结果:

```

input A is 010011110110
input B is 001110001110
input Cin is 0
output Sum is: 100010000100
output Cout is: 0

```

10.1.3 长度为 N 的向量加法器

【例 10-3】 长度为 8 的向量加法器

```

`timescale 1ns/1ps
//功能模块部分
module VectorA(FA,FB,FSum);
    parameter Size=4;
    input [Size:1] FA,FB;
    output [Size+1:1] FSum;
    assign FSum=FA+FB;
endmodule
//测试模块部分
module stimulus_VectorA;
    parameter Size=8;

```




```
reg [Size:1] DA,DB;
wire [Size+1:1] DSum;
wire DCount;
VectorA
F1(DA,DB,DSum);
defparam F1.Size=Size;
initial
begin
    DA=8'b1111_0110;
    DB=8'b1000_1110;
    #5;
    $display("input A is %b",DA);
    $display("input B is %b",DB);
    $display("output Sum is: %b",DSum);
end
endmodule
```

仿真结果如下:

```
input A is 11110110
input B is 10001110
output Sum is: 110000100
```

10.2 向量乘法器

该例是使用移位加的方法实现两个位向量相乘的乘法器。

【例 10-4】向量乘法器。

//功能模块部分

```
module multiply(product,mul_a,mul_b);
    parameter Size=2;
    input [Size:1] mul_a,mul_b;
    output {2*Size:1} product;
    reg [2*Size:1] shift_mula,product;
    reg [Size:1] shift_mulb;
    always @(mul_a or mul_b)
    begin
        product=0;
        shift_mula=mul_a;
        shift_mulb=mul_b;
        repeat(Size)
            begin
```




```

        #100 if(shift_mulb[1])
            product=product+shift_mula;
        shift_mula=shift_mula<<1;
        shift_mulb=shift_mulb>>1;
    end
end
endmodule
//测试模块部分
module stimulus_mul;
    parameter Size=4;
    reg [Size:1] mul_a,mul_b;
    wire [2*Size:1] product;
    multiply
        m1(product,mul_a,mul_b);
    defparam m1.Size=Size;
    initial
        begin
            mul_a='b1010;
            mul_b='b01;
            #110;
            $display("input mul_a is: %b",mul_a);
            $display("input mul_b is: %b",mul_b);
            $display("output product is: %b",product);
        end
endmodule

```

仿真结果如下：

input mul_a is: 1010

input mul_b is: 0001

output product is: 00001010

上面的例子中，输入变量 mul_a、mul_b 分别代表被乘数与乘数，输出变量 product 为它们相乘后的结果。由于两个长度为 N 的数相乘，结果的长度为 2N，所以变量 product 的位数定为 2*Size。内部变量 shift_mula、shift_mulb 用来作为被乘数与乘数的移位寄存器。乘法开始时，依据乘数的最低位确定被乘数是否进入部分积。之后被乘数相对于部分积左移一位，而乘数向右移一位，使得乘数移位寄存器中原来的第二位被移到最低位，接着开始第二次循环，直到所有的乘数位被全部右移至乘数移位寄存器而结束。

该例可以实现两个长度不同的位向量的乘法，只需在测试模块中以较长的位向量的长度定义输入变量即可。



10.3 比 较 器

该例是一个比较器，用于比较两个位串所代表的整数的大小，并给出了两种不同的实现形式。

【例 10-5】比较器。

//功能模块部分

//实现形式 1:

```
module compare_1(result_1,a,b);
    parameter size=1;
    input [size:1] a,b;
    output result_1;
    reg result_1;
    always @(a or b)
        if(a>=b)
            result_1=1;
        else
            result_1=0;
endmodule
```

//实现形式 2:

```
module compare_2(result_2,a,b);
    parameter size=1;
    input [size:1] a,b;
    output result_2;
    assign result_2=(a>=b)?1:0;
endmodule
```

//测试模块部分

```
module sti_com;
    parameter size=4;
    reg [size:1] a,b;
    wire result_1,result_2;
    compare_1 c1(result_1,a,b);
    compare_2 c2(result_2,a,b);
    defparam c1.size=size;
    defparam c2.size=size;
    initial
        begin
            a='b1101;
            b='b1111;
```




```

        #5;
        $display("a is:%b",a);
        $display("b is:%b",b);
        $display("if 1(a>=b),else 0(a<b)");
        $display("result_1 is:%b",result_1);
        $display("result_2 is:%b",result_2);
    end
endmodule

```

仿真结果如下:

```

a is: 1101
b is: 1111
if 1(a>=b), else 0(a<b)
result_1 is: 0
result_2 is: 0

```

10.4 多路选择器与译码器

多路选择器和译码器是组合电路中较简单的两种通用电路,它们可以由简单的门电路组合连接而构成。利用 Verilog 语言,从行为、功能描述多路选择器和译码器,不仅逻辑设计变得非常简单,而且阅读起来也会很方便。

10.4.1 8 选 1 多路选择器

选择器常用于信号之间的切换,8 选 1 多路选择器可以用于 8 路信号的切换。该选择器有 8 个输入端,3 个信号选择端及 1 个信号输出端。

【例 10-6】8 选 1 多路选择器。

//功能模块部分

//行为描述:形式 1

```

module multiplexor8_1bc (out, in1, in2, in3, in4, in5, in6, in7, in8, cntrl1, cntrl2, cntrl3);
    output out;
    input  in1, in2, in3, in4, in5, in6, in7, in8, cntrl1, cntrl2, cntrl3;
    reg    out;
    always
        @(in1 or in2 or in3 or in4 or in5 or in6 or in7 or in8 or cntrl1 or cntrl2 or cntrl3)
            case ({cntrl1, cntrl2, cntrl3})
                3'b000 : out = in1;
                3'b001 : out = in2;
                3'b010 : out = in3;
                3'b011 : out = in4;
                3'b100 : out = in5;
            endcase
    end
endmodule

```




```
3'b101 : out = in6;  
3'b110 : out = in7;  
3'b111 : out = in8;  
default : $display("Please check control bits");  
endcase
```

```
endmodule
```

//行为描述: 形式 2

```
module multiplexor8_1b1(out, in1, in2, in3, in4, in5, in6, in7, in8, cntrl1, cntrl2, cntrl3);  
    output out;  
    input in1, in2, in3, in4, in5, in6, in7, in8, cntrl1, cntrl2, cntrl3;  
    assign out = (in1 & ~cntrl1 & ~cntrl2 & ~cntrl3) |  
                 (in2 & ~cntrl1 & ~cntrl2 & cntrl3) |  
                 (in3 & ~cntrl1 & cntrl2 & ~cntrl3) |  
                 (in4 & ~cntrl1 & cntrl2 & cntrl3) |  
                 (in5 & cntrl1 & ~cntrl2 & ~cntrl3) |  
                 (in6 & cntrl1 & ~cntrl2 & cntrl3) |  
                 (in7 & cntrl1 & cntrl2 & ~cntrl3) |  
                 (in8 & cntrl1 & cntrl2 & cntrl3);  
endmodule
```

//结构描述

```
module multiplexor8_1g(out, in1, in2, in3, in4, in5, in6, in7, in8, cntrl1, cntrl2, cntrl3);  
    output out;  
    input in1, in2, in3, in4, in5, in6, in7, in8, cntrl1, cntrl2, cntrl3;  
    wire notcntrl1, notcntrl2, notcntrl3, s, t, u, v, w, x, y, z;  
    not (notcntrl1, cntrl1);  
    not (notcntrl2, cntrl2);  
    not (notcntrl3, cntrl3);  
    and (s, in1, notcntrl1, notcntrl2, notcntrl3);  
    and (t, in2, notcntrl1, notcntrl2, cntrl3);  
    and (u, in3, notcntrl1, cntrl2, notcntrl3);  
    and (v, in4, notcntrl1, cntrl2, cntrl3);  
    and (w, in5, cntrl1, notcntrl2, notcntrl3);  
    and (x, in6, cntrl1, notcntrl2, cntrl3);  
    and (y, in7, cntrl1, cntrl2, notcntrl3);  
    and (z, in8, cntrl1, cntrl2, cntrl3);  
    or (out, s, t, u, v, w, x, y, z);  
endmodule
```

//测试模块部分

```
module muxstimulus;
```




```

    reg IN1, IN2, IN3, IN4, IN5, IN6, IN7, IN8, CNTRL1, CNTRL2, CNTRL3;
    wire OUT_1, OUT_2, OUT_3;

    multiplexor8_1bc
mux1_8bc(OUT_1, IN1, IN2, IN3, IN4, IN1, IN2, IN3, IN4, CNTRL1, CNTRL2, CNTRL3);
    multiplexor8_1bl
mux1_8bl(OUT_2, IN1, IN2, IN3, IN4, IN1, IN2, IN3, IN4, CNTRL1, CNTRL2, CNTRL3);
    multiplexor8_1g
mux1_8g(OUT_3, IN1, IN2, IN3, IN4, IN1, IN2, IN3, IN4, CNTRL1, CNTRL2, CNTRL3);

    initial begin
        IN1 = 1; IN2 = 0; IN3 = 1; IN4 = 0; IN5 = 1; IN6 = 0; IN7 = 1; IN8 = 0;
        $display("Initial arbitrary values");
        #0 $display("input1 = %b, input2 = %b, input3 = %b, input4 = %b, input5 = %b, input6 = %b, input7
                    = %b, input8 = %b\n",
                    IN1, IN2, IN3, IN4, IN5, IN6, IN7, IN8);
        {CNTRL1, CNTRL2, CNTRL3} = 3'b000;
        #1 $display("cntrl1=%b, cntrl2=%b, cntrl3=%b -> out_1=%b out_2=%b out_3=%b",
                    CNTRL1, CNTRL2, CNTRL3, OUT_1, OUT_2, OUT_3);
        {CNTRL1, CNTRL2, CNTRL3} = 3'b001;
        #1 $display("cntrl1=%b, cntrl2=%b, cntrl3=%b -> out_1=%b out_2=%b out_3=%b",
                    CNTRL1, CNTRL2, CNTRL3, OUT_1, OUT_2, OUT_3);
        {CNTRL1, CNTRL2, CNTRL3} = 3'b010;
        #1 $display("cntrl1=%b, cntrl2=%b, cntrl3=%b -> out_1=%b out_2=%b out_3=%b",
                    CNTRL1, CNTRL2, CNTRL3, OUT_1, OUT_2, OUT_3);
        {CNTRL1, CNTRL2, CNTRL3} = 3'b011;
        #1 $display("cntrl1=%b, cntrl2=%b, cntrl3=%b -> out_1=%b out_2=%b out_3=%b",
                    CNTRL1, CNTRL2, CNTRL3, OUT_1, OUT_2, OUT_3);
        {CNTRL1, CNTRL2, CNTRL3} = 3'b100;
        #1 $display("cntrl1=%b, cntrl2=%b, cntrl3=%b -> out_1=%b out_2=%b out_3=%b",
                    CNTRL1, CNTRL2, CNTRL3, OUT_1, OUT_2, OUT_3);
        {CNTRL1, CNTRL2, CNTRL3} = 3'b101;
        #1 $display("cntrl1=%b, cntrl2=%b, cntrl3=%b -> out_1=%b out_2=%b out_3=%b",
                    CNTRL1, CNTRL2, CNTRL3, OUT_1, OUT_2, OUT_3);
        {CNTRL1, CNTRL2, CNTRL3} = 3'b110;
        #1 $display("cntrl1=%b, cntrl2=%b, cntrl3=%b -> out_1=%b out_2=%b out_3=%b",
                    CNTRL1, CNTRL2, CNTRL3, OUT_1, OUT_2, OUT_3);
        {CNTRL1, CNTRL2, CNTRL3} = 3'b111;
        #1 $display("cntrl1=%b, cntrl2=%b, cntrl3=%b -> out_1=%b out_2=%b out_3=%b",
                    CNTRL1, CNTRL2, CNTRL3, OUT_1, OUT_2, OUT_3);
    end

```




```
end  
endmodule
```

仿真结果如下:

```
input1 = 1, input2 = 0, input3 = 1, input4 = 0, input5 = 1, input6 = 0, input7 = 1, input8 = 0  
cntrl1=0, cntrl2=0, cntrl3=0 -> out_1=1 out_2=1 out_3=1  
cntrl1=0, cntrl2=0, cntrl3=1 -> out_1=0 out_2=0 out_3=0  
cntrl1=0, cntrl2=1, cntrl3=0 -> out_1=1 out_2=1 out_3=1  
cntrl1=0, cntrl2=1, cntrl3=1 -> out_1=0 out_2=0 out_3=0  
cntrl1=1, cntrl2=0, cntrl3=0 -> out_1=1 out_2=1 out_3=1  
cntrl1=1, cntrl2=0, cntrl3=1 -> out_1=0 out_2=0 out_3=0  
cntrl1=1, cntrl2=1, cntrl3=0 -> out_1=1 out_2=1 out_3=1  
cntrl1=1, cntrl2=1, cntrl3=1 -> out_1=0 out_2=0 out_3=0
```

10.4.2 3-8 译码器

3-8 译码器是最常用的一种小规模集成电路, 它有 3 个二进制输入端, 8 个译码输出端与 1 个译码使能端。

【例 10-7】3-8 译码器。

//功能模块部分

`timescale 1ns/1ns

//结构描述

```
module dec3x8_g(a, b, c, enable, z);  
    input a, b, c, enable;  
    output[0: 7] z;  
    wire aw, bw, cw;  
    not  
        v0(aw, a),  
        v1(bw, b),  
        v2(cw, c);  
    nand  
        n0(z[0], enable, aw, bw, cw),  
        n1(z[1], enable, aw, bw, c),  
        n2(z[2], enable, aw, b, cw),  
        n3(z[3], enable, aw, b, c),  
        n4(z[4], enable, a, bw, cw),  
        n5(z[5], enable, a, bw, c),  
        n6(z[6], enable, aw, b, c),  
        n7(z[7], enable, a, b, c);  
endmodule
```

//行为描述



```

module dec3x8_b(a, b, c, enable, z);
    input a, b, c, enable;
    output[0: 7] z;
    assign z[0]= ~(enable&~a&~b&~c);
    assign z[1]= ~(enable&~a&~b&c);
    assign z[2]= ~(enable&~a&b&~c);
    assign z[3]= ~(enable&~a&b&c);
    assign z[4]= ~(enable&a&~b&~c);
    assign z[5]= ~(enable&a&~b&c);
    assign z[6]= ~(enable&a&b&~c);
    assign z[7]= ~(enable&a&b&c);
endmodule

//测试模块部分
module sti_dec;
    reg sa, sb, sc, sena;
    wire [0: 7] sz_1, sz_2;
    dec3x8_g d1(sa, sb, sc, sena, sz_1);
    dec3x8_b d2(sa, sb, sc, sena, sz_2);
    initial
    begin
        sena=0;
        sa=0;
        sb=0;
        sc=0;
        #10 sena=1;
        #10 sc=1;
        #10 sb=1; sc=0;
        #10 sc=1;
        #10 sa=1; sb=0; sc=0;
        #10 sc=1;
        #10 sb=1; sc=0;
        #10 sc=1;
        #10 $stop;
    end
    always
    @(sena or sa or sb or sc or sz_1 or sz_2)
    begin
        #5 $display("at time %t, enable is %b, input is %b%b%b", $time, sena, sa, sb, sc);
        $display("structural output is: %b <--> behavioural output is: %b", sz_1, sz_2);
    end
endmodule

```




```
end
```

```
endmodule
```

仿真结果如下:

```
at time          5, enable is 0, input is 000
structural output is: 11111111 <---> behavioural output is: 11111111
at time          15, enable is 1, input is 000
structural output is: 01111111 <---> behavioural output is: 01111111
at time          25, enable is 1, input is 001
structural output is: 10111111 <---> behavioural output is: 10111111
at time          35, enable is 1, input is 010
structural output is: 11011111 <---> behavioural output is: 11011111
at time          45, enable is 1, input is 011
structural output is: 11101111 <---> behavioural output is: 11101111
at time          55, enable is 1, input is 100
structural output is: 11110111 <---> behavioural output is: 11110111
at time          65, enable is 1, input is 101
structural output is: 11111011 <---> behavioural output is: 11111011
at time          75, enable is 1, input is 110
structural output is: 11111111 <---> behavioural output is: 11111101
at time          85, enable is 1, input is 111
structural output is: 11111110 <---> behavioural output is: 11111110
```

10.5 寄 存 器

触发器种类很多,常用的两种触发器为锁存器与 JK 触发器。而锁存器根据触发边沿、复位和预置方式以及输出端的多少的不同,也可以有多种不同形式的锁存器,例如简单的 D 锁存器、同步复位的 D 锁存器等。寄存器一般由多位触发器连接而成,通常有锁存寄存器与移位寄存器等。本节主要讨论锁存寄存器,移位寄存器将在 10.8 节进行讨论。

10.5.1 带同步复位的边沿触发器

【例 10-8】带同步复位的边沿触发器。

//功能模块部分

```
module SDlatch(q,qb,d,clk,reset);
```

```
    input d,clk,reset;
```

```
    output q,qb;
```

```
    reg q,qb;
```

```
    always @(posedge clk)
```

```
        if(!reset)
```

```
            begin
```




```
        q=0;
        qb=~q;
    end
    else
    begin
        q=d;
        qb=~q;
    end
endmodule
//测试模块部分
module sti_SD;
    reg sd,sreset,sclk;
    wire sq,sqb;
    SDlatch sdl(sq,sqb,sd,sclk,sreset);
    initial
    begin
        sreset=0;
        sclk=0;
        sd=1;
        #50 sd=0;
        #50 sd=1;
        #50 sd=0;
        #50 sd=1;
        $finish;
    end
    always
    begin
        #10 sclk=~sclk;
        #40 sreset=~sreset;
    end
    always @(sclk or sreset or sd or sq)
    begin
        $display("time=%t:",$time,"clk=%b,reset=%b",sclk,sreset);
        $display("input=%b,output=%b %b",sd,sq,sqb);
    end
endmodule
```

仿真结果如下:

```
time=          0: clk=0, reset=0
input=1, output=x x
```




```
time=          10: clk=1, reset=0
input=1, output=x x
time=          10: clk=1, reset=0
input=1, output=0 1
time=          50: clk=1, reset=1
input=0, output=0 1
time=          60: clk=0, reset=1
input=0, output=0 1
time=         100: clk=0, reset=0
input=1, output=0 1
time=         110: clk=1, reset=0
input=1, output=0 1
time=         150: clk=1, reset=1
input=0, output=0 1
time=         160: clk=0, reset=1
input=0, output=0 1
```

10.5.2 带异步复位和置位的边沿触发器

【例 10-9】带异步复位和置位的边沿触发器。

//功能模块部分

```
module ADlatch(q,qb,d,clk,reset,preset);
    input d,clk,reset,preset;
    output q,qb;
    reg q,qb;
    always @(posedge clk or reset or preset)
        if(!reset&&preset)
            begin
                q=0;
                qb=~q;
            end
        else if(!preset&&reset)
            begin
                q=1;
                qb=~q;
            end
        else if(!reset&&!preset)
            begin
                q=1'bx;
                qb=~q;
            end
endmodule
```




```

        end
    else
        begin
            q=d;
            qb=~q;
        end
    endmodule
//测试模块部分
module sti_SD;
    reg sd,sreset,spreset,sclk;
    wire sq,sqb;
    ADlatch sdl(sq,sqb,sd,sclk,sreset,spreset);
    initial
    begin
        sreset=0;
        spreset=1;
        sclk=0;
        sd=1;
        #50 sd=0;
        #50 sd=1;
        #50 sd=0;
        #50 sd=1;
        $finish;
    end
    always
    begin
        #10 sclk=~sclk;
        #20 sreset=~sreset;
        #60 spreset=~spreset;
    end
    always @(sclk or sreset or spreset or sd or sq)
    begin
        #5 $display("time=%t:",$time,"clk=%b,reset=%b,preset=%b",sclk,sreset,spreset);
        $display("input=%b,<--->output=%b %b",sd,sq,sqb);
    end
endmodule

```

仿真结果如下:

```

time=                    5: clk=0, reset=0, preset=1
input=1, <--->output=0 1

```




```
time=          15: clk=1, reset=0, preset=1
input=1, <--->output=0 1
time=          35: clk=1, reset=1, preset=1
input=1, <--->output=1 0
time=          55: clk=1, reset=1, preset=1
input=0, <--->output=1 0
time=          95: clk=1, reset=1, preset=0
input=0, <--->output=1 0
time=         105: clk=0, reset=1, preset=0
input=1, <--->output=1 0
time=         125: clk=0, reset=0, preset=0
input=1, <--->output=x x
time=         155: clk=0, reset=0, preset=0
input=0, <--->output=x x
time=         185: clk=0, reset=0, preset=1
input=0, <--->output=0 1
time=         195: clk=1, reset=0, preset=1
input=0, <--->output=0 1
```

10.5.3 带使能和异步复位的 8 位寄存器

【例 10-10】带使能和异步复位的 8 位寄存器。

```
`define size 8    //宏替换定义
//功能模块部分
module ERDlatch(q,d,clk,reset,en);
    input clk,reset,en;
    input [`size:1] d;
    output [`size:1] q;
    reg [`size:1] q;
    always @(posedge clk or reset)
        if(!reset)
            q=0;
        else if(en)
            q=d;
endmodule
//测试模块部分
module sti_ERD;
    reg sreset,sen,sclk;
    reg [`size:1] sd;
    wire [`size:1] sq;
```




```

ERDlatch erdl(sq,sd,sclk,sreset,sen);
initial
begin
    sreset=0;
    sen=1;
    sclk=0;
    sd=`size'b0000_0011;
repeat(8)
begin
    #15 sd=`size'b0000_1010;
    #15 sd=`size'b0010_1000;
    #15 sd=`size'b0100_0011;
    #15 sd=`size'b1111_1111;
end
    #100 $finish;
end
always
begin
    #5 sclk=~sclk;
    #40 sreset=~sreset;
    #50 sen=~sen;
end
always @(sclk or sreset or sen or sd or sq)
begin
    #5 $display("time=%t:",$time,"clk=%b,reset=%b,en=%b",sclk,sreset,sen);
    $display("d=%b,q=%b",sd,sq);
end
endmodule

```

仿真结果如下:

```

time=          5: clk=1, reset=0, en=1
d=00000011, q=00000000
time=         20: clk=1, reset=0, en=1
d=00001010, q=00000000
time=         35: clk=1, reset=0, en=1
d=00101000, q=00000000
time=         50: clk=1, reset=1, en=1
d=01000011, q=01000011
time=         65: clk=1, reset=1, en=1
d=11111111, q=01000011

```




```
time=          80: clk=1, reset=1, en=1
d=00001010, q=01000011
time=          95: clk=1, reset=1, en=0
d=00101000, q=01000011
time=         105: clk=0, reset=1, en=0
d=01000011, q=01000011
time=         125: clk=0, reset=1, en=0
d=11111111, q=01000011
time=         140: clk=0, reset=0, en=0
d=00001010, q=01000011
time=         145: clk=0, reset=0, en=0
d=00001010, q=00000000
time=         155: clk=0, reset=0, en=0
d=00101000, q=00000000
time=         170: clk=0, reset=0, en=0
d=01000011, q=00000000
time=         185: clk=0, reset=0, en=0
d=11111111, q=00000000
time=         195: clk=1, reset=0, en=1
d=00001010, q=00000000
time=         215: clk=1, reset=0, en=1
d=00101000, q=00000000
time=         230: clk=1, reset=0, en=1
d=01000011, q=00000000
time=         240: clk=1, reset=1, en=1
d=11111111, q=01000011
time=         260: clk=1, reset=1, en=1
d=00001010, q=01000011
time=         275: clk=1, reset=1, en=1
d=00101000, q=01000011
time=         290: clk=0, reset=1, en=0
d=01000011, q=01000011
time=         305: clk=0, reset=1, en=0
d=11111111, q=01000011
time=         320: clk=0, reset=1, en=0
d=00001010, q=01000011
time=         335: clk=0, reset=0, en=0
d=00101000, q=00000000
time=         350: clk=0, reset=0, en=0
```




```

d=01000011, q=00000000
time=          365: clk=0, reset=0, en=0
d=11111111, q=00000000
time=          380: clk=0, reset=0, en=1
d=00001010, q=00000000
time=          390: clk=1, reset=0, en=1
d=00101000, q=00000000
time=          410: clk=1, reset=0, en=1
d=01000011, q=00000000
time=          425: clk=1, reset=1, en=1
d=11111111, q=00000000
time=          430: clk=1, reset=1, en=1
d=11111111, q=11111111
time=          440: clk=1, reset=1, en=1
d=00001010, q=11111111
time=          455: clk=1, reset=1, en=1
d=00101000, q=11111111
time=          470: clk=1, reset=1, en=1
d=01000011, q=11111111
time=          480: clk=0, reset=1, en=0
d=11111111, q=11111111
time=          525: clk=0, reset=0, en=0
d=11111111, q=00000000
time=          575: clk=1, reset=0, en=1
d=11111111, q=00000000

```

10.6 边沿控制的脉冲发生器

【例 10-11】边沿控制的脉冲发生器。

//功能模块部分

```

module pul_g(d_in,clk,reset,ctr,puls);
    input d_in,clk,reset,ctr;
    output puls;
    reg puls;
    always@(posedge clk or reset)
        if(!reset)
            begin
                puls=0;
            end
end

```



```

        else
            begin
                if(d_in==ctr)
                    puls=1;
                else
                    puls=0;
            end
        endmodule
//测试模块部分
module sti_pul;
    reg clk,reset,ctr,d_in;
    wire puls;
    pul_g p1(d_in,clk,reset,ctr,puls);
    initial
        begin
            reset=0;
            ctr=1;
            clk=0;
            d_in=1;
            #10 reset=1;
            repeat(10)
                begin
                    #10 d_in=0;
                    #20 d_in=1;
                end
            #5 $finish;
        end
    always
        #5 clk=~clk;
    initial
        $display("          time    reset    d_in    puls");
    always @(clk or reset or d_in)
        #5 $display("%t    %b    %b    %b", $time,reset,d_in,puls);
endmodule

```

仿真结果如下:

time	reset	d_in	puls
10	1	1	0
20	1	0	1
30	1	0	0



40	1	1	0
50	1	0	1
60	1	0	0
70	1	1	0
80	1	0	1
90	1	0	0
100	1	1	0
110	1	0	1
120	1	0	0
130	1	1	0
140	1	0	1
150	1	0	0
160	1	1	0
170	1	0	1
180	1	0	0
190	1	1	0
200	1	0	1
210	1	0	0
220	1	1	0
230	1	0	1
240	1	0	0
250	1	1	0
260	1	0	1
270	1	0	0
280	1	1	0
290	1	0	1
300	1	0	0
310	1	1	0

10.7 计 数 器

上一章测试举例部分给出了一种上升沿触发的增 1 计数器实例，它是采用结构描述的方式来实现的。本节给出另外两种计数器的实例，一种是下降沿触发的增 1 计数器，另一种是采用上升沿触发的增 1/减 1 计数器，以便读者根据自己的需要参照不同的例子。

这两种计数器均采用行为描述的方式来实现，其结构描述实现方式可参照上一章的有关内容，只需稍作修改即可。

10.7.1 带使能和进位输出的 4 位计数器

【例 10-12】带使能和进位输出的 4 位计数器(下降沿触发)。



//功能模块部分

//行为建模

// 4 位二进制增 1 计数器, 模为 16

```
module counter4_bit(q, cout, d, increment, load_data, global_reset, clock);
    output [3:0] q;
    output cout;
    input [3:0] d;
    input load_data, global_reset, clock, increment;
    reg [3:0] q;
    reg cout;
    always @(posedge clock)
        if (global_reset)
            begin
                q = 4'b0000;
                cout=0;
            end
        else if (load_data)
            begin
                q = d;
                cout=0;
            end
        else if (increment) begin
            if (q == 15)
                begin
                    q = 0;
                    cout = 1;
                end
            else
                begin
                    q = q + 1;
                    cout = 0;
                end
            end
        end
endmodule
```

//测试模块部分

module stimulus;

wire [3:0] q;

wire cout;

reg [3:0] d;



```

reg          load_data, global_reset, clk, increment;
counter4_bit mod1 (q, cout, d, increment, load_data, global_reset, clk);
initial begin
    global_reset = 0;
    clk = 0;
    increment = 0;
    load_data = 0;
    d = 4'b0011;
    #10 global_reset = 1;
    #20 global_reset = 0;
    #20 load_data = 1;
    #20 load_data = 0;
    #20 increment = 1;
    #50 global_reset = 1;
    #20 global_reset = 0;
    #50 load_data = 1;
    #20 load_data = 0;
    #10 increment = 0;
    #20 $finish;
end // initial begin
always #5 clk = ~clk;
always
begin
    #10 $display ("simulation time:", $time);
        $display("count_en=%b,load_data=%b,global_reset=%b",
                increment, load_data, global_reset);
        $display("counter output(binary)=%b<--->decimal=%d",q,q);
        $display("Cout is:%b",cout);
    end
endmodule

```

仿真结果如下:

```

simulation time:                10
count_en=0, load_data=0, global_reset=1
counter output(binary)=xxxx<--->decimal= x
Cout is: x
simulation time:                20
count_en=0, load_data=0, global_reset=1
counter output(binary)=0000<--->decimal= 0
Cout is: 0

```




```
simulation time:          30
count_en=0, load_data=0, global_reset=0
counter output(binary)=0000<--->decimal= 0
Cout is: 0

simulation time:          40
count_en=0, load_data=0, global_reset=0
counter output(binary)=0000<--->decimal= 0
Cout is: 0

simulation time:          50
count_en=0, load_data=1, global_reset=0
counter output(binary)=0000<--->decimal= 0
Cout is: 0

simulation time:          60
count_en=0, load_data=1, global_reset=0
counter output(binary)=0011<--->decimal= 3
Cout is: 0

simulation time:          70
count_en=0, load_data=0, global_reset=0
counter output(binary)=0011<--->decimal= 3
Cout is: 0

simulation time:          80
count_en=0, load_data=0, global_reset=0
counter output(binary)=0011<--->decimal= 3
Cout is: 0

simulation time:          90
count_en=1, load_data=0, global_reset=0
counter output(binary)=0011<--->decimal= 3
Cout is: 0

simulation time:         100
count_en=1, load_data=0, global_reset=0
counter output(binary)=0100<--->decimal= 4
Cout is: 0

simulation time:         110
count_en=1, load_data=0, global_reset=0
counter output(binary)=0101<--->decimal= 5
Cout is: 0

simulation time:         120
count_en=1, load_data=0, global_reset=0
counter output(binary)=0110<--->decimal= 6
```




```
Cout is: 0
simulation time:          130
count_en=1, load_data=0, global_reset=0
counter output(binary)=0111<--->decimal= 7
Cout is: 0
simulation time:          140
count_en=1, load_data=0, global_reset=0
counter output(binary)=1000<--->decimal= 8
Cout is: 0
simulation time:          150
count_en=1, load_data=0, global_reset=0
counter output(binary)=1001<--->decimal= 9
Cout is: 0
simulation time:          160
count_en=1, load_data=0, global_reset=0
counter output(binary)=1010<--->decimal=10
Cout is: 0
simulation time:          170
count_en=1, load_data=0, global_reset=0
counter output(binary)=1011<--->decimal=11
Cout is: 0
simulation time:          180
count_en=1, load_data=0, global_reset=0
counter output(binary)=1100<--->decimal=12
Cout is: 0
simulation time:          190
count_en=1, load_data=0, global_reset=0
counter output(binary)=1101<--->decimal=13
Cout is: 0
simulation time:          200
count_en=1, load_data=0, global_reset=0
counter output(binary)=1110<--->decimal=14
Cout is: 0
simulation time:          210
count_en=1, load_data=0, global_reset=0
counter output(binary)=1111<--->decimal=15
Cout is: 0
simulation time:          220
count_en=1, load_data=0, global_reset=0
```




counter output(binary)=0000<--->decimal= 0

Cout is: 1

simulation time: 230

count_en=1, load_data=0, global_reset=0

counter output(binary)=0001<--->decimal= 1

Cout is: 0

simulation time: 240

count_en=1, load_data=0, global_reset=0

counter output(binary)=0010<--->decimal= 2

Cout is: 0

simulation time: 250

count_en=1, load_data=0, global_reset=0

counter output(binary)=0011<--->decimal= 3

Cout is: 0

simulation time: 260

count_en=1, load_data=0, global_reset=0

counter output(binary)=0100<--->decimal= 4

Cout is: 0

simulation time: 270

count_en=1, load_data=0, global_reset=0

counter output(binary)=0101<--->decimal= 5

Cout is: 0

simulation time: 280

count_en=1, load_data=0, global_reset=0

counter output(binary)=0110<--->decimal= 6

Cout is: 0

simulation time: 290

count_en=1, load_data=0, global_reset=1

counter output(binary)=0111<--->decimal= 7

Cout is: 0

simulation time: 300

count_en=1, load_data=0, global_reset=1

counter output(binary)=0000<--->decimal= 0

Cout is: 0

simulation time: 310

count_en=1, load_data=0, global_reset=0

counter output(binary)=0000<--->decimal= 0

Cout is: 0

simulation time: 320



```
count_en=1, load_data=0, global_reset=0
counter output(binary)=0001<--->decimal= 1
Cout is: 0
simulation time:                330
count_en=1, load_data=0, global_reset=0
counter output(binary)=0010<--->decimal= 2
Cout is: 0
simulation time:                340
count_en=1, load_data=0, global_reset=0
counter output(binary)=0011<--->decimal= 3
Cout is: 0
simulation time:                350
count_en=1, load_data=0, global_reset=0
counter output(binary)=0100<--->decimal= 4
Cout is: 0
simulation time:                360
count_en=1, load_data=1, global_reset=0
counter output(binary)=0101<--->decimal= 5
Cout is: 0
simulation time:                370
count_en=1, load_data=1, global_reset=0
counter output(binary)=0011<--->decimal= 3
Cout is: 0
simulation time:                380
count_en=1, load_data=0, global_reset=0
counter output(binary)=0011<--->decimal= 3
Cout is: 0
simulation time:                390
count_en=0, load_data=0, global_reset=0
counter output(binary)=0100<--->decimal= 4
Cout is: 0
simulation time:                400
count_en=0, load_data=0, global_reset=0
counter output(binary)=0100<--->decimal= 4
Cout is: 0
```

10.7.2 并行加载的通用增 1/减 1 计数器

【例 10-13】并行加载的通用增 1/减 1 计数器。

//通用增 1/减 1 计数器, 周期为 2^N



//功能模块部分

```
module counter_bit(q, d, updown, count_en, load_data, global_reset, clock);
    parameter bitsize=4;
    output [bitsize:1] q;
    input [bitsize:1] d;
    input load_data, global_reset, clock, updown, count_en;
    reg [bitsize:1] q;
    always @(posedge clock)
        if (global_reset)
            q = 0;
        else if (load_data)
            q = d;
        else if (count_en)
            begin
                if (updown)
                    q = q + 1;
                else
                    q = q - 1;
            end
    endmodule
```

//测试模块部分

```
module stimulus;
    parameter size=3;
    wire [size:1] q;
    reg [size:1] d;
    reg count_en, load_data, global_reset, clk, updown;
    counter_bit mod1 (q, d, updown, count_en, load_data, global_reset, clk);
    defparam mod1.bitsize=size;
    initial begin
        count_en=1;
        global_reset = 0;
        clk = 0;
        updown = 0;
        load_data = 0;
        d = 3'b010;
        #10 global_reset = 1;
        #20 global_reset = 0;
        #20 load_data = 1;
        #20 load_data = 0;
    end
endmodule
```




```

        #20 updown = 1;
        #50 count_en=0;
        #50 count_en=1;
        #50 global_reset = 1;
        #20 global_reset = 0;
        #50 load_data = 1;
        #20 load_data = 0;
        #10 updown = 0;
        #20 $finish;
    end // initial begin
    always #5 clk = ~clk;
    initial
    #1 $display("          time  en  pdown  load  reset  binary  dicimal");
    always
    #10 $display ($time,"    %b          %b          %b          %b          %b
        %d", count_en, updown, load_data, global_reset, q, q);
endmodule

```

仿真结果如下:

time	en	pdown	load	reset	binary	dicimal
10	1	0	0	1	xxx	x
20	1	0	0	1	000	0
30	1	0	0	0	000	0
40	1	0	0	0	111	7
50	1	0	1	0	110	6
60	1	0	1	0	010	2
70	1	0	0	0	010	2
80	1	0	0	0	001	1
90	1	1	0	0	000	0
100	1	1	0	0	001	1
110	1	1	0	0	010	2
120	1	1	0	0	011	3
130	1	1	0	0	100	4
140	0	1	0	0	101	5
150	0	1	0	0	101	5
160	0	1	0	0	101	5
170	0	1	0	0	101	5
180	0	1	0	0	101	5
190	1	1	0	0	101	5
200	1	1	0	0	110	6



210	1	1	0	0	111	7
220	1	1	0	0	000	0
230	1	1	0	0	001	1
240	1	1	0	1	010	2
250	1	1	0	1	000	0
260	1	1	0	0	000	0
270	1	1	0	0	001	1
280	1	1	0	0	010	2
290	1	1	0	0	011	3
300	1	1	0	0	100	4
310	1	1	1	0	101	5
320	1	1	1	0	010	2
330	1	1	0	0	010	2
340	1	0	0	0	011	3
350	1	0	0	0	010	2

10.8 移位寄存器

在 10.5 节中我们讨论了几种锁存寄存器的 Verilog HDL 实现方法, 本节主要讨论另外一种寄存器形式移位寄存器的 Verilog HDL 实现方法, 包括以下两种:

- (1) 串行输入/并行输出的移位寄存器。
- (2) 并行输入/串行输出的移位寄存器。

10.8.1 串行输入/并行输出的移位寄存器

【例 10-14】 串行输入/并行输出的移位寄存器。

//功能模块部分

```
module shifter_sipo(data_in,clk,clr,shift_en,data_out);
    parameter size=4;
    input data_in,clk,clr,shift_en;
    output [size:1] data_out;
    reg [size:1] data_out;
    always@(posedge clk)
        if(!clr)
            data_out='b0;
        else if(shift_en)
            begin
                data_out=data_out<<1;
                data_out[1]=data_in;
            end
end
```




```

endmodule
//测试模块部分
module sti_shifter;
    parameter size=8;
    wire {size:1} data_out;
    reg clk,clr,data_in,shift_en;
    shifter_sipo sh1(data_in,clk,clr,shift_en,data_out);
    defparam sh1.size=size;
    initial begin
        clk = 0;
        clr = 1;
        shift_en=1;
        data_in = 0;
        #10 data_in=1;
        #20 data_in=0;
        #30 data_in=1;
        #40 data_in=0;
        #50 clr=0;
        #10 data_in=1;
        #20 clr=1;
        #50 shift_en=0;
        #40 data_in=0;
        #10 shift_en=1;
        #40 data_in=1;
        #20 $finish;
    end // initial begin
    always #5 clk = ~clk;
    always #10 $display ($time," clr=%b shift_en=%b data_in=%b -> data_out=%b ",clr,shift_en, data_in,
        data_out);
endmodule // stimulus

```

仿真结果如下:

```

10 clr=1 shift_en=1 data_in=1 -> data_out=xxxxxxxx0
20 clr=1 shift_en=1 data_in=1 -> data_out=xxxxxxxx01
30 clr=1 shift_en=1 data_in=0 -> data_out=xxxxxxxx011
40 clr=1 shift_en=1 data_in=0 -> data_out=xxxxx0110
50 clr=1 shift_en=1 data_in=0 -> data_out=xxx01100
60 clr=1 shift_en=1 data_in=1 -> data_out=xx011000
70 clr=1 shift_en=1 data_in=1 -> data_out=x0110001
80 clr=1 shift_en=1 data_in=1 -> data_out=01100011

```



```

90 clr=1 shift_en=1 data_in=1 -> data_out=11000111
100 clr=1 shift_en=1 data_in=0 -> data_out=10001111
110 clr=1 shift_en=1 data_in=0 -> data_out=00011110
120 clr=1 shift_en=1 data_in=0 -> data_out=00111100
130 clr=1 shift_en=1 data_in=0 -> data_out=01111000
140 clr=1 shift_en=1 data_in=0 -> data_out=11110000
150 clr=0 shift_en=1 data_in=0 -> data_out=11100000
160 clr=0 shift_en=1 data_in=1 -> data_out=00000000
170 clr=0 shift_en=1 data_in=1 -> data_out=00000000
180 clr=1 shift_en=1 data_in=1 -> data_out=00000000
190 clr=1 shift_en=1 data_in=1 -> data_out=00000001
200 clr=1 shift_en=1 data_in=1 -> data_out=00000011
210 clr=1 shift_en=1 data_in=1 -> data_out=00000111
220 clr=1 shift_en=1 data_in=1 -> data_out=00001111
230 clr=1 shift_en=0 data_in=1 -> data_out=00011111
240 clr=1 shift_en=0 data_in=1 -> data_out=00011111
250 clr=1 shift_en=0 data_in=1 -> data_out=00011111
260 clr=1 shift_en=0 data_in=1 -> data_out=00011111
270 clr=1 shift_en=0 data_in=0 -> data_out=00011111
280 clr=1 shift_en=1 data_in=0 -> data_out=00011111
290 clr=1 shift_en=1 data_in=0 -> data_out=00111110
300 clr=1 shift_en=1 data_in=0 -> data_out=01111100
310 clr=1 shift_en=1 data_in=0 -> data_out=11111000
320 clr=1 shift_en=1 data_in=1 -> data_out=11110000
330 clr=1 shift_en=1 data_in=1 -> data_out=11100001

```

10.8.2 并行输入/串行输出的移位寄存器

【例 10-15】并行输入/串行输出的移位寄存器。

//功能模块部分

```

module shifter_sipo(data_in,load,clk,clr,shift_en,data_out);
    parameter size=4;
    input [size:1] data_in;
    input load,clk,clr,shift_en;
    output data_out;
    reg data_out;
    reg [size:1] shift_reg;
    always@(posedge clk)
    begin
        if(!clr)

```




```
        shift_reg='b0;
    else
        if(load)
            shift_reg=data_in;
        else if(shift_en)
            begin
                shift_reg=shift_reg<<1;
                shift_reg[1]=0;
            end
        data_out=shift_reg[size];
    end
endmodule
//测试模块部分
module sti_shifter;
    parameter size=8;
    wire data_out;
    reg load,clk,clr,shift_en;
    reg [size:1] data_in;
    shifter_sipo sh1(data_in,load,clk,clr,shift_en,data_out);
    defparam sh1.size=size;
    initial begin
        clk = 0;
        clr = 1;
        load=1;
        shift_en=1;
        data_in = 8'b1101_0101;
        #15 load=0;
        #50 clr=0;
        #20 clr=1;
        #10 load=1;
        #20 load=0;
        #50 shift_en=0;
        #10 shift_en=1;
        #20 $finish;
    end // initial begin
    always #5 clk = ~clk;
    initial
        $display("data_in=%b",data_in);
    always #10 $display ($time," clr=%b shift_en=%b load=%b data_out=%b ",
```




```
clr,shift_en,load,data_out);
```

```
endmodule // stimulus
```

仿真结果如下:

```
data_in=11010101
10 clr=1 shift_en=1 load=1 data_out=1
20 clr=1 shift_en=1 load=0 data_out=1
30 clr=1 shift_en=1 load=0 data_out=0
40 clr=1 shift_en=1 load=0 data_out=1
50 clr=1 shift_en=1 load=0 data_out=0
60 clr=1 shift_en=1 load=0 data_out=1
70 clr=0 shift_en=1 load=0 data_out=0
80 clr=0 shift_en=1 load=0 data_out=0
90 clr=1 shift_en=1 load=0 data_out=0
100 clr=1 shift_en=1 load=1 data_out=1
110 clr=1 shift_en=1 load=1 data_out=1
120 clr=1 shift_en=1 load=0 data_out=1
130 clr=1 shift_en=1 load=0 data_out=0
140 clr=1 shift_en=1 load=0 data_out=1
150 clr=1 shift_en=1 load=0 data_out=0
160 clr=1 shift_en=1 load=0 data_out=1
170 clr=1 shift_en=0 load=0 data_out=1
180 clr=1 shift_en=1 load=0 data_out=0
190 clr=1 shift_en=1 load=0 data_out=1
```

10.9 分 频 器

【例 10-16】4 分频。

//功能模块部分

```
module div(clk_in,reset,clk_out);
    parameter bitsize=2;
    parameter L=3;
    input clk_in,reset;
    output clk_out;
    reg [bitsize:1] count;
    reg clk_out;
    always
        @(posedge clk_in)
        begin
            if(!reset)
```




```
        count=0;
    else
        begin
            if(count<(L))
                count=count+1;
            else
                count=0;
            end
        clk_out=count[bitsize];
    end
endmodule
//测试模块部分
module sti_div;
    integer file_out;
    reg clk_in,reset;
    wire clk_out;
    div d1(clk_in,reset,clk_out);
    initial
        file_out=$fopen("result.vec");
    initial
        begin
            reset=0;
            clk_in=0;
            #10 reset=1;
            #200 reset=0;
            #10 $finish;
            $fclose(file_out);
        end
    always
        #5 clk_in=~clk_in;
    initial
        $fmonitor(file_out,"clk_out=%b at time %t",clk_out,$time);
endmodule
```

仿真结果如下:

clk_out=x at time	0
clk_out=0 at time	5
clk_out=1 at time	25
clk_out=0 at time	45
clk_out=1 at time	65


```

clk_out=0 at time      85
clk_out=1 at time     105
clk_out=0 at time     125
clk_out=1 at time     145
clk_out=0 at time     165
clk_out=1 at time     185
clk_out=0 at time     205

```

该程序虽然用来计算 4 分频，但只需对功能模块中的参数 L、bitsize 进行修改，即可得到任意整数倍的分频。例如要进行 5 倍分频，令参数 bitsize=3，参数 L=4，则可得到如下的仿真结果：

```

clk_out=x at time      0
clk_out=0 at time      5
clk_out=1 at time     45
clk_out=0 at time     55
clk_out=1 at time     95
clk_out=0 at time    105
clk_out=1 at time    145
clk_out=0 at time    155
clk_out=1 at time    195
clk_out=0 at time    205

```

10.10 FIR 滤波器

本节介绍 N 点 FIR 滤波器的设计方法。对于 N 点 FIR 滤波器来说，每 N 个时钟周期要读入一个采样 x，通过滤波处理，每 N 个时钟周期输出一个结果 y。因而在设计 FIR 滤波器时，必须定义两个数组来存储系数与前一次的输入。下面的例子就是一个 8 点 FIR 滤波器的设计实例，它采用了两种不同的实现方法。

【例 10-17】8 点 FIR 滤波器。

//功能模块部分

//实现方案 1

```

module FIRFilter_1(clock,reset,x,y);
    input      clock,reset;
    input  [7:0] x;
    output [7:0] y;
    reg  [7:0] coef_array[7:0];
    reg  [7:0] acc,y;
    reg  [7:0] x_array[7:0];
    reg  [2:0] index,start_pos;
    initial

```




```

        forever @ (posedge reset) begin
            disable FIRmain;
            start_pos=0;
        end
always begin:FIRmain
    wait(!reset);
    @ (negedge clock);
    x_array[start_pos]=x;
    acc=x*coef_array[start_pos];
    index=start_pos+1;
    begin:loop
        forever begin
            @(negedge clock);
            acc=acc+x_array[index]*coef_array[index];
            index=index+1;
            if(index==start_pos) disable loop;
        end
    end//loop
    y<=acc;
    start_pos=start_pos+1;
end
endmodule//FIRFilter_1
//实现方案 2
module FIRFilter_2(clock,reset,x,y);
    input      clock,reset;
    input  [7:0] x;
    output [7:0] y;
    reg  [7:0] coef_array[7:0];
    reg  [7:0] acc,y;
    reg  [7:0] x_array[7:0];
    reg  [2:0] index,start_pos;
    initial
        forever @ (posedge reset) begin
            disable FIRmain;
            start_pos=0;
            index=0;
        end
always begin:FIRmain
    wait(!reset);

```




```
begin:loop
  forever begin
    @(negedge clock);
    if(index==start_pos)
      begin
        x_array[index]=x;
        acc=x*coef_array[index];
        index=start_pos+1;
      end
    else
      begin
        acc=acc+x_array[index]*coef_array[index];
        index=index+1;
        if(index==start_pos) disable loop;
      end
    end
  end//loop
  y<=acc;
  start_pos=start_pos+1;
  index=start_pos;
end
endmodule//FIRFilter_2
```




第 11 章 综合设计实例



上一章给出了一些简单组合电路与时序电路的 Verilog HDL 实现。为了进一步加深对 Verilog HDL 设计语言的了解与掌握,本章给出一个较为复杂的综合设计实例。首先介绍有限状态机的概念,并给出三个设计实例,然后给出用于教学的、经简化只有 8 条指令的 RISC(Reduced Instruction Set Computer)中央处理单元(CPU)的顶层设计。

11.1 有限状态机的概念及其设计实例

11.1.1 有限状态机的概念

有限状态机是时序电路的通用模型,任何时序电路都可以表示为有限状态机。有限状态机通常可分为两类:Moore 型与 Mealy 型。下一状态只由当前状态决定的有限状态机称为 Moore 型有限状态机;下一状态不但与当前状态有关,而且与当前输入值有关的有限状态机为 Mealy 型有限状态机。实际上,Moore 型有限状态机可看作 Mealy 型的特例。

有限状态机从本质上讲是由寄存器与组合逻辑构成的时序电路,各个状态之间的转移总是在时钟的触发下进行的。通常可使用带有 always 语句的 case 语句建模,状态信息存储在寄存器中,case 语句的多个分支包含每个状态的行为。下面通过一个例子进行说明。

【例 11-1】一个说明有限状态机描述的例子。

//Moore 型

```
module sta_machine1(x, clock, current_state, z);
```

```
    parameter [1:0] set0 = 0, hold0 = 1, set1 = 2;
```

```
    input x, clock;
```

```
    output [1:0] current_state;
```

```
    output z;
```

```
    reg [1:0] current_state;
```

```
    reg z;
```

```
    /* 重新声明为寄存器型变量,以便于在 always 语句中赋值。缺省时,端口是连线型,不能在  
    always 语句中赋值*/
```

```
    reg [1:0] next_state;
```

```
    always @ (current_state)
```

```
    begin
```

```
        case (current_state)
```

```
            set0:
```




```
hold0:
    begin
        z = 0;
        if (x == 0)
            next_state = hold0;
        else
            next_state = set1;
        end
```

```
set1:
    begin
        z = 1;
        next_state = set0;
    end
```

```

        endcase
    end
always @ (posedge clock)
    current_state = next_state;

```

endmodule

```
//Mealy 型
```

```
module sta_machine2(x, clock, current_state, z);
```

```
parameter [1:0] set0 = 0, hold0 = 1, set1 = 2;
```

```
input x, clock;
```

```
output [1:0]current_state;
```

output z;

```
reg [1:0] current_state;
```

```
reg Z;
```

/* 重新声明为寄存器型变量, 以便于在 always 语句中赋值。缺省时, 端口是连线型, 不能在 always 语句中赋值*/

```
reg [1:0] next_state;
```

always @ (x or current_state)

begin

```
case (current_state)
```

set0:

```
if(x==1)
```

begin



```
        z = 0 ;
        ncxt_state = hold0;
    end
    else
        z=1;
    hold0:
        if (x == 0)
            begin
                ncxt_state = hold0;
                z=0;
            end
        else
            begin
                next_state = set1;
                z=1;
            end
        set1:
            if(x==1)
                begin
                    z = 1;
                    next_state = set0;
                end
            else
                z=0;
        endcase
    end
    always @ (posedge clock)
        begin
            current_state = next_state;
        end
endmodule
```

11.1.2 有限状态机的设计实例

在熟悉有限状态机基本描述方法的基础上，下面给出三个有限状态机的设计实例：饮料销售机、交通灯控制器与闹钟系统控制器。

1. 饮料销售机

这是一个饮料自动售货机的例子，每听饮料的售价为 35 美分，可以使用的硬币有三种：5 美分(nickel)、一角(dime)、25 美分(quarter)，同时售货机具有找零功能。

【例 11-2】 饮料销售机的例子。



```
//功能模块部分
`define vend_a_drink{d,dispense,collect}={idle,2'b11}
module drink_machine(nickel_in,dime_in,quarter_in,collect,
                    nickel_out,dime_out,dispense,reset,clk);
    parameter idle=0,five=1,ten=2,twenty_five=3,
               fifteen=4,thirty=5,twenty=6,owe_dime=7;
    input nickel_in,dime_in,quarter_in,reset,clk;
    output collect,nickel_out,dime_out,dispense;
    reg collect,nickel_out,dime_out,dispense;
    reg [2:0] d,q;
    always @(nickel_in or dime_in or quarter_in or reset)
    begin
        nickel_out=0;
        dime_out=0;
        dispense=0;
        collect=0;
        if(!reset)
            d=idle;
        else
            begin
                d=q;
                case(q)
                    idle:
                        if(nickel_in)
                            d=five;
                        else if (dime_in)
                            d=ten;
                        else if(quarter_in)
                            d=twenty_five;
                    five:
                        if(nickel_in)
                            d=ten;
                        else if (dime_in)
                            d=fifteen;
                        else if(quarter_in)
                            d=thirty;
                    ten:
                        if(nickel_in)
                            d=fifteen;
```




```
    else if (dime_in)
        d=twenty;
    else if(quarter_in)
        `vend_a_drink;
twenty_five:
    if(nickel_in)
        d=thirty;
    else if (dime_in)
        `vend_a_drink;
    else if(quarter_in)
        begin
            `vend_a_drink;
            nickel_out=1;
            dime_out=1;
        end
fifteen:
    if(nickel_in)
        d=twenty;
    else if (dime_in)
        d=twenty_five;
    else if(quarter_in)
        begin
            `vend_a_drink;
            nickel_out=1;
        end
thirty:
    if(nickel_in)
        `vend_a_drink;
    else if (dime_in)
        begin
            `vend_a_drink;
            nickel_out=1;
        end
    else if(quarter_in)
        begin
            `vend_a_drink;
            nickel_out=1;
            d=owe_dime;
        end
end
```




```
        twenty:
            if(nickel_in)
                d=twenty_five;
            else if (dime_in)
                d=thirty;
            else if(quarter_in)
                begin
                    `vend_a_drink;
                    dime_out=1;
                end
        owe_dime:
            begin
                dime_out=1;
                d=idle;
            end
        endcase
    end
end
always @(posedge clk)
    begin
        q=d;
    end
endmodule
//测试模块部分
module stimulus;
    reg nickel_in,dime_in,quarter_in,reset,clk;
    wire collect,nickel_out,dime_out,dispense;
    drink_machine
    dr1(nickel_in,dime_in,quarter_in,collect,nickel_out,dime_out,dispense,reset,clk);
    initial
        begin
            reset=1;
            clk=1;
            nickel_in=0;
            dime_in=0;
            quarter_in=0;
            #5 reset=0;
            #5 nickel_in=1;
            #2 reset=1;
        end
end
```




```

        #3 nickel_in=0;
        dime_in=1;
    #10 dime_in=0;
        quarter_in=1;
    #10 nickel_in=1;
        quarter_in=0;
    #10 nickel_in=0;
    repeat(5)
        begin
            #5 nickel_in=1;
            #5 nickel_in=0;
        end
    #5 $finish;
end
always
    #5 clk=~clk;
endmodule

```

仿真结果如图 11.1 所示。

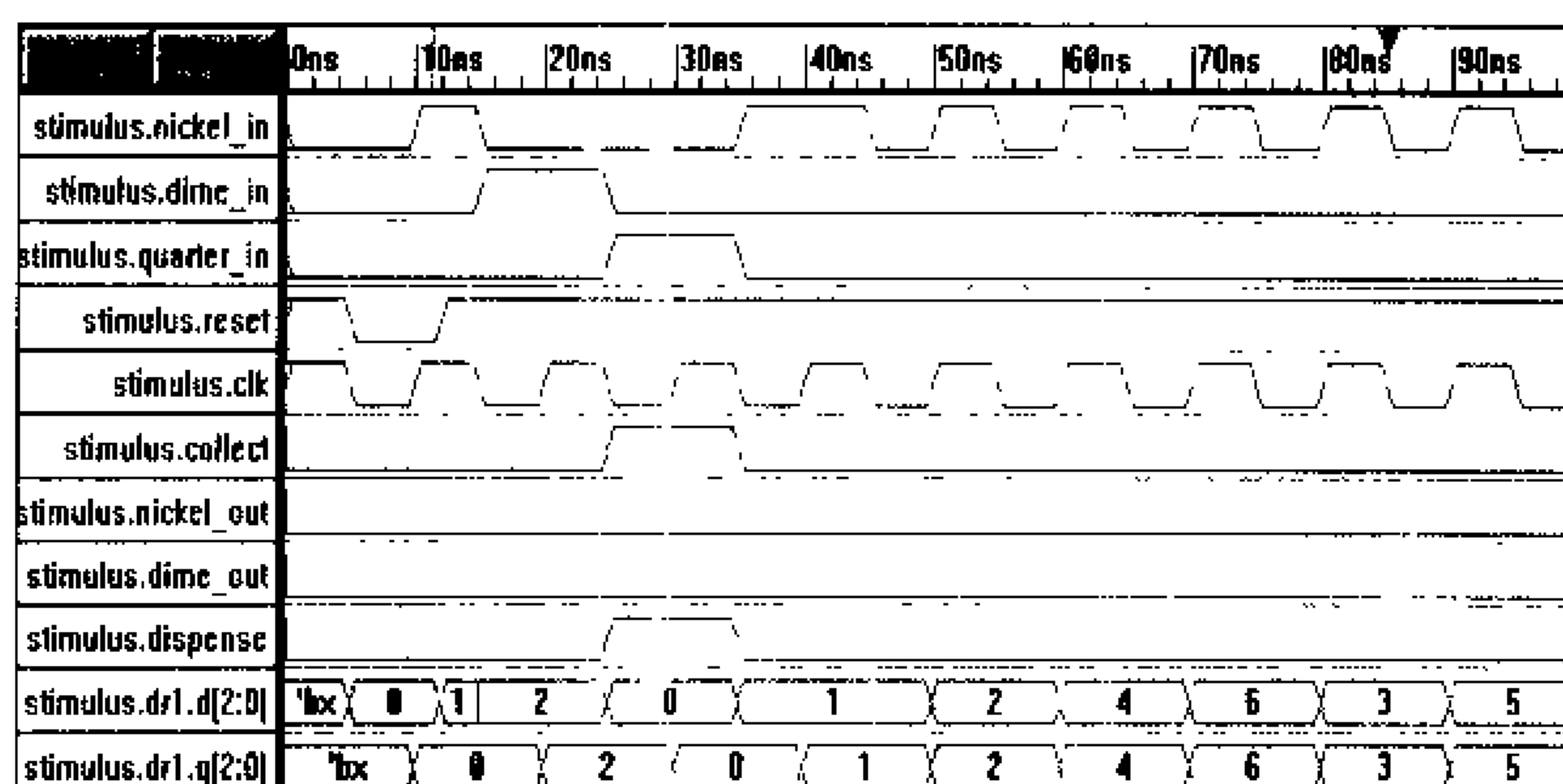


图 11.1 饮料销售机的仿真结果

在上面的例子中，将三种币值的投币信号(nickel_in、dime_in、quarter_in)作为模块的输入信号，系统的复位能力由输入信号 reset 控制，并且状态的转换是在输入时钟信号 clk 的上升沿完成的。为了便于比较，这里给出另外一种基于计数的实现方式，可供读者参考。

```

module drink_machine2(nickel_in,dime_in,quarter_in,collect,
    nickel_out,dime_out,dispense,reset,clk);
    input nickel_in, dime_in, quarter_in, reset, clk;
    output nickel_out, dime_out, collect, dispense;
    reg nickel_out, dime_out, dispense, collect;

```




```
reg [3:0] nickel_count, temp_nickel_count;
reg temp_return_change, return_change;
always @ ( nickel_in or dime_in or quarter_in or
           collect or temp_nickel_count or
           reset or nickel_count or return_change)
begin
    nickel_out = 0;
    dime_out = 0;
    dispense = 0;
    collect = 0;
    temp_nickel_count = 0;
    temp_return_change = 0;
    // 检查是否有钱放入
    if (! reset)
        begin
            temp_nickel_count = nickel_count;
            if (nickel_in)
                temp_nickel_count = temp_nickel_count + 1;
            else if (dime_in)
                temp_nickel_count = temp_nickel_count + 2;
            else if (quarter_in)
                temp_nickel_count = temp_nickel_count + 5;
            if (temp_nickel_count >= 7)
                begin
                    temp_nickel_count = temp_nickel_count - 7;
                    dispense = 1;
                    collect = 1;
                end
            if (return_change || collect)
                begin
                    if (temp_nickel_count >= 2)
                        begin
                            dime_out = 1;
                            temp_nickel_count = temp_nickel_count - 2;
                            temp_return_change = 1;
                        end
                    if (temp_nickel_count == 1)
                        begin
                            nickel_out = 1;
                        end
                    end
                end
            end
        end
```




```

        temp_nickel_count = temp_nickel_count - 1;
    end
end
end
end
always @ (posedge clk )
begin
    nickel_count = temp_nickel_count;
    return_change = temp_return_change;
end
endmodule

```

2. 交通灯控制器

本例实现高速公路与乡间小路红绿灯的控制。实现如下要求：

- (1) 只有在小路上发现汽车时，高速公路上的交通灯才可能变为红灯。
- (2) 当有汽车在小路上时，小路的交通灯保持为绿灯，但不能超过给定的延迟时间。
- (3) 公路灯转为绿灯后，即使小路上有汽车出现，而公路上并无汽车，也将在给定的时间内保持绿灯。

此交通灯控制器可用一个有限自动机来实现。由上所述，它定义了 5 种状态，如表 11.1 所示。

其状态转换图如图 11.2 所示。

表 11.1 交通控制器的 5 种状态

状态	公路灯	小路灯
0	绿	红
1	黄	红
2	红	绿
3	红	黄
4	红	红

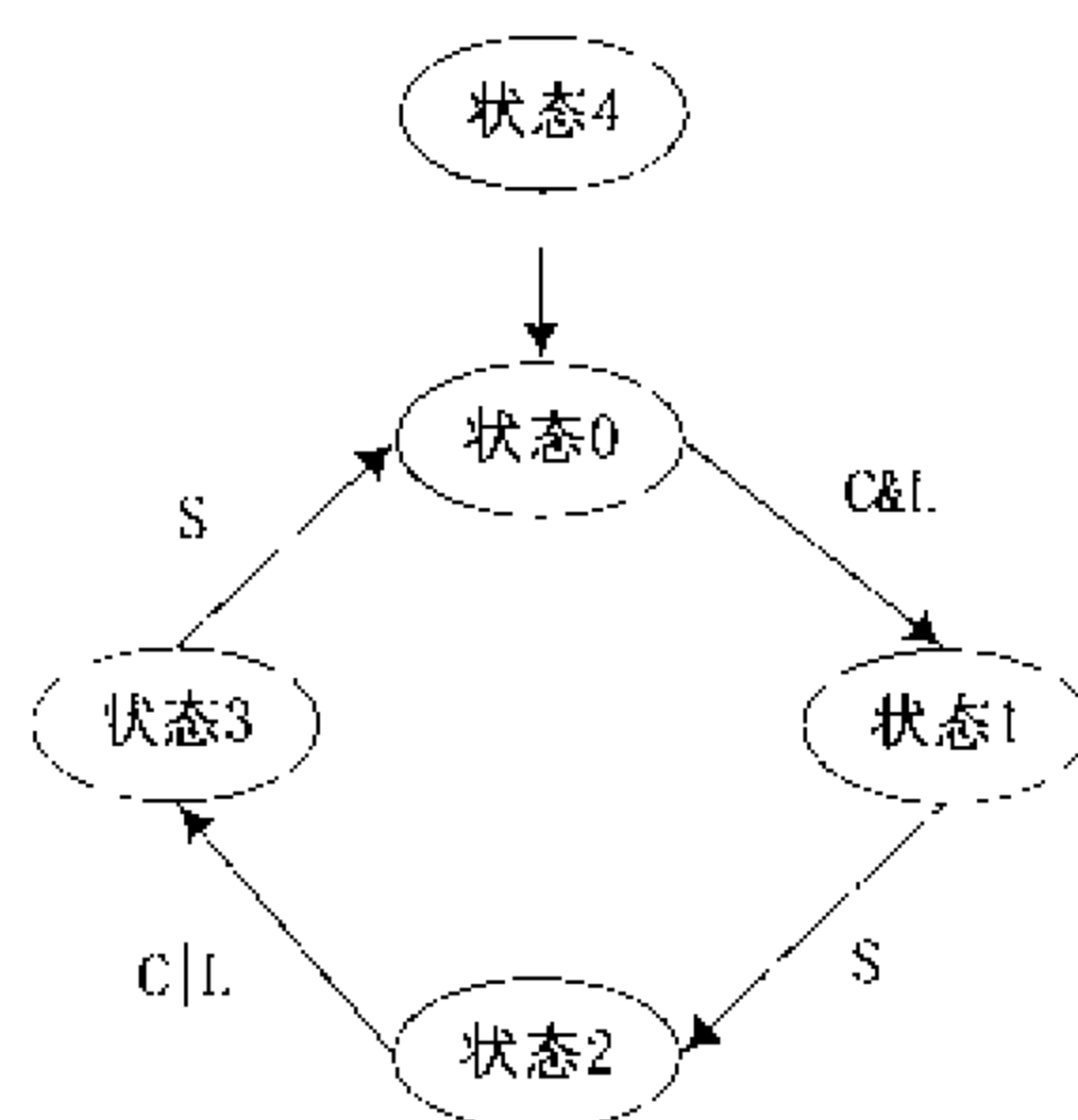


图 11.2 交通灯控制器的状态转换图

图 11.2 中，C 表示小路上有车；L 表示过了一段长的时间；S 表示过了一段短的时间；操作符“&”表示逻辑与的关系；操作符|表示逻辑或的关系。基于上述有限状态机的状态转换关系，可以得到如下的 Verilog HDL 实现程序。

【例 11-3】交通灯控制器。

//功能模块部分

```
module traffic(car,timeoutL,timeoutS,newHL,newFL,newST);
```

```
    input car,timeoutL,timeoutS;
```




```
output [3:1] newHL,newFL;
output newST;
reg [3:1] newHL,newFL;
reg newST;
reg [3:1] cur_state,next_state;
initial
begin
    cur_state=3'b111;
    newHL=3'b100;
    newFL=3'b001;
    newST=0;
end
always @(car or timeoutL or timeoutS)
begin
    case(cur_state)
        3'b000:
            begin
                newHL=3'b100;
                newFL=3'b001;
                if((car==1) && (timeoutL==1))
                begin
                    next_state=3'b100;
                    newST=1;
                    newHL=3'b010;
                    newFL=3'b001;
                end
            end
        else
            begin
                next_state=3'b000;
                newST=0;
            end
        end
    end
    3'b100:
        begin
            newHL=3'b010;
            newFL=3'b001;
            if(timeoutS==1)
            begin
                next_state=3'b010;
```




```
        newST=1;
        newHL=3'b001;
        newFL=3'b100;
    end
else
    begin
        next_state=3'b100;
        newST=0;
    end
end
3'b010:
begin
    newHL=3'b001;
    newFL=3'b100;
    if((car==0) || (timeoutL==1))
    begin
        next_state=3'b110;
        newST=1;
        newHL=3'b001;
        newFL=3'b010;
    end
    else
    begin
        next_state=3'b010;
        newST=0;
    end
end
3'b110:
begin
    newHL=3'b001;
    newFL=3'b010;
    if(timeoutS==1)
    begin
        next_state=3'b000;
        newST=1;
        newHL=3'b100;
        newFL=3'b001;
    end
    else
```



```
        begin
            next_state=3'b110;
            newST=0;
        end
    end
3'b111:
    begin
        next_state=3'b000;
        newHL=3'b100;
        newFL=3'b001;
        newST=0;
    end
endcase
cur_state=next_state;
end
endmodule
//测试模块部分
module sti_traffic;
    reg car,timeoutL,timeoutS;
    wire [3:1] newHL,newFL;
    wire newST;
    traffic t1(car,timeoutL,timeoutS,newHL,newFL,newST);
    initial
    begin
        car=0;
        timeoutL=0;
        timeoutS=0;
        #20 car=1;
        timeoutL=1;
        #20 timeoutS=1;
        timeoutL=0;
        car=0;
        #20 car=1;
        #20 car=0;
        timeoutS=0;
        #20 timeoutS=1;
        #20 timeoutS=0;
        timeoutL=1;
        car=1;
```




```

#10 timeoutL=0;
    car=0;
#10 timeoutL=1;
#10 timeoutS=1;
#10 timeoutS=0;
    timeoutL=0;
    car=1;
#10 timeoutS=1;
#10 timeoutS=0;
    timeoutL=1;
#10 timeoutS=1;
#10 timeoutS=0;
    timeoutL=0;
#10 timeoutL=1;
#10 timeoutL=0;
    timeoutS=1;
#30 $finish;
end

```

endmodule

仿真结果如图 11.3 所示。

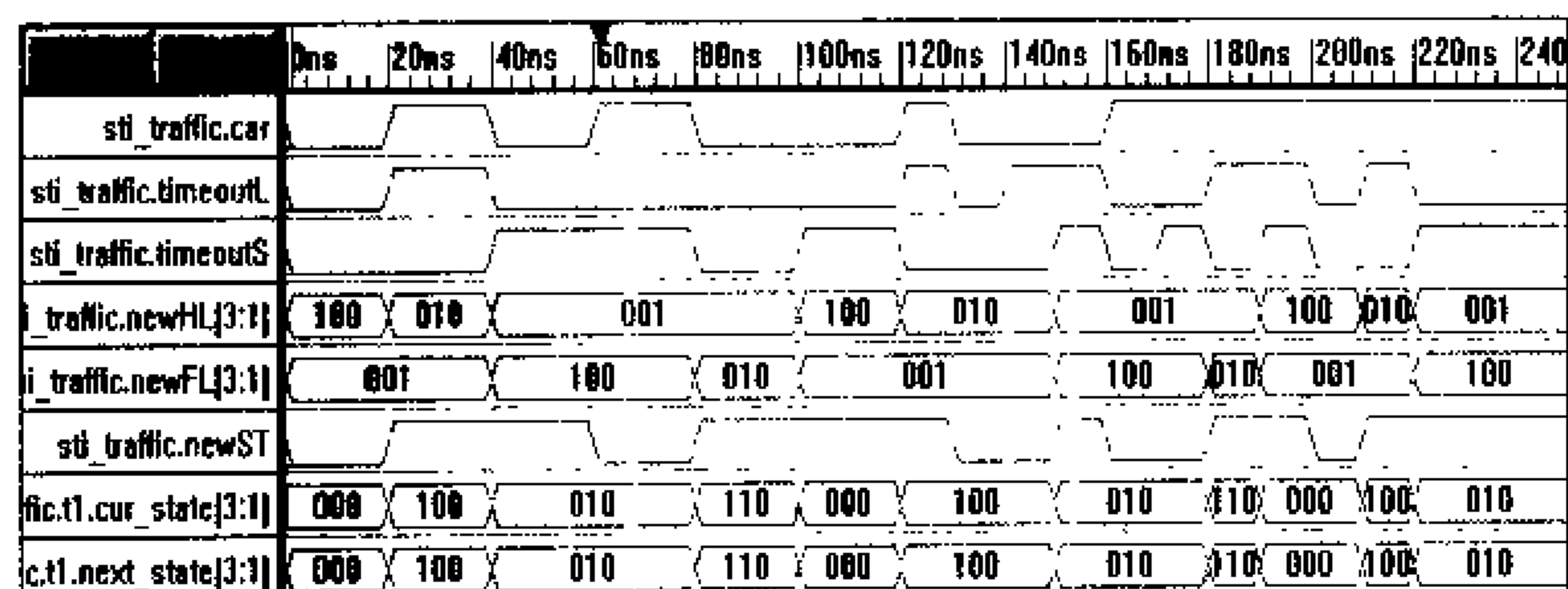


图 11.3 交通灯控制器的仿真结果

在上面的例子中，变量 `cur_state` 表示当前的状态值，5 种状态分别由它的 5 个值来代表：

值	状态
000	0
100	1
010	2
110	3
111	4

其中状态 4 被设为初始状态；变量 `new_state` 表示下一个状态值，由输入变量 `car`、

timeoutL、timeoutS 的值来决定，其中变量 car 表示有无汽车，变量 timeoutL 表示状态转换图中的 L，变量 timeoutS 表示状态转换图中的 S；输出变量 newHL 代表高速公路灯的状态，输出变量 newFL 代表乡间小路灯的状态，它们都是三位位长的二进制位串，每一位分别表示绿、黄、红灯的亮、灭状态，如“100”表示路灯为红灯；输出变量 NewST 为用于启动外部计时器的输出位。

3. 闹钟系统控制器

该例设计一带闹钟功能的 24 小时计时系统的控制器。计时系统要完成如下功能：

- (1) 计时功能：每隔一分钟计时一次，并在显示屏上显示当前时间。
- (2) 闹钟功能：如果当前时间与设置的闹钟时间相同，则扬声器发出蜂鸣声。
- (3) 设置新的计时时间：用户通过数字键“0”~“9”输入新的时间，然后按“time”键确认。
- (4) 设置新的闹钟时间：用户用数字键“0”~“9”输入新的闹钟时间，然后按“alarm”键确认。
- (5) 显示所设置的闹钟时间：在正常计时显示状态下，用户直接按下“alarm”键，则已设置的闹钟时间显示在显示屏上。

根据上面所描述的功能，可以得到控制器的外部端口，如图 11.4 所示。

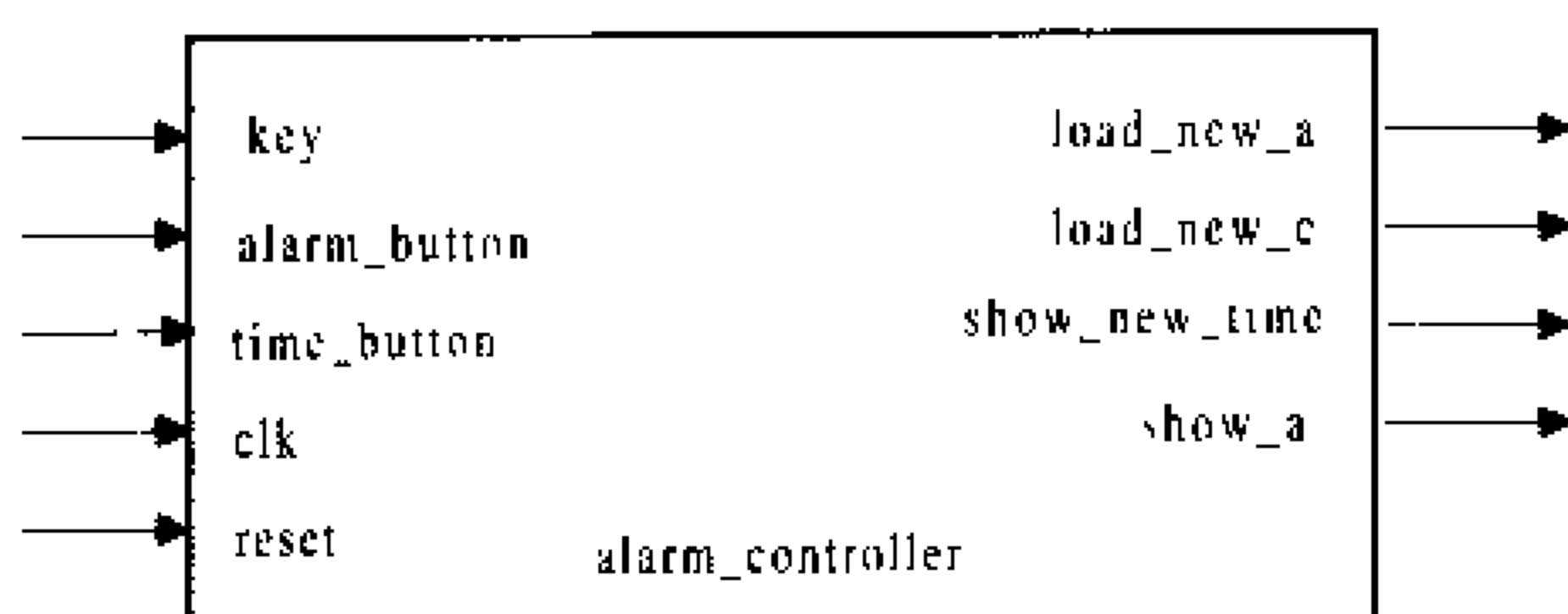


图 11.4 控制器的外部端口

可以通过有限状态机的方式来实现控制器的功能。根据设计要求及端口设置，需要 5 个状态来实现：

hold：表示电路初态即正常计数状态。

receive：接收键盘输入状态。在状态 hold 时用户按下数字键后进入此状态。在此状态下，显示屏上显示的是用户键入的数字。

set_at：设置新的闹钟时间。在状态 receive 时用户按下 alarm 键后进入此状态。

set_ct：设置新的计时时间。在状态 receive 时用户按下 time 键后进入此状态。

show_at：显示闹钟时间。在状态 hold 时用户直接按下 alarm 键后进入此状态。

相应的状态转换及控制输出如表 11.2 所示。表中没有显式说明的控制信号赋值，表示信号的值为 0。

在表 11.2 中，状态 receive 与状态 show_at 涉及到超时判断的问题：如果没有超时，则一直停留在当前状态；否则，根据条件转到其他状态。具体描述时，可以利用外部时钟信号进行计数。例如，若外部时钟周期为 10 ms，预定超时时间为 200 ms，则对时钟计数 20 次后经过的时间就是超时时间。但何时开始计数，如何得知计数结束即超时，必须引入新的内部信号。这样，整个控制器实际上分为两部分，一部分是有限状态机，另一部分完



成计数功能用以超时判断。

表 11.2 控制器状态转换及控制输出表

当前状态	控制输入(条件)		下一状态	控制输出(动作)
hold	key==1		receive	show_new_time=1
	alarm_button==1		show_at	show_a=1
	否则		hold	
receive	key==1		receive	show_new_time=1
	alarm_button==1		set_at	load_new_a=1
	time_button==1		set_ct	load_new_c=1
	否则(超时?)	否	receive	show_new_time=1
		是	hold	
set_at	alarm_button==1		set_at	load_new_a=1
	否则		hold	
set_ct	time_button==1		set_ct	load_new_c=1
	否则		hold	
show_at	alarm_button==1		show_at	show_a=1
	否则(超时?)	否	show_at	show_a=1
		是	hold	

基于上面的分析,利用控制器状态转换及控制输出表,可得到如下的控制器的 Verilog HDL 实现程序。

【例 11-4】闹钟系统控制器。

```

`timescale 1ms/1ms
//功能模块部分
module alarm_ctr(key,alarm_button,time_button,clk,reset,
                load_new_a,load_new_c,show_new_time,show_a);
    parameter hold=3'b000,receive=3'b001,set_at=3'b010,
                set_ct=3'b011,show_at=3'b100;
    parameter key_timeout=20;
    input key,alarm_button,time_button,clk,reset;
    output load_new_a,load_new_c,show_new_time,show_a;
    reg load_new_a,load_new_c,show_new_time,show_a;
    reg [3:1] cur_state,next_state;
    reg count_k_end,count_a_end,enable_count_k,enable_count_a;
    integer counter_k,counter_a;
    always @(posedge clk or reset)
    begin
        if(!reset)
            cur_state=hold;
        else

```




```
        cur_state=next_state;
    end
always @(cur_state or key or alarm_button or time_button or count_k_end or count_a_end)
begin
    load_new_a=0;
    load_new_c=0;
    show_new_time=0;
    show_a=0;
    enable_count_k=0;
    enable_count_a=0;
    case (cur_state)
        hold:
            if(key==1)
                begin
                    next_state=receive;
                    show_new_time=1;
                end
            else if(alarm_button==1)
                begin
                    next_state=show_at;
                    show_a=1;
                end
            else
                next_state=hold;
        receive:
            begin
                if(key==1)
                    begin
                        next_state=receive;
                        show_new_time=1;
                    end
                if(alarm_button==1)
                    begin
                        next_state=set_at;
                        load_new_a=1;
                    end
                if(time_button==1)
                    begin
                        next_state=set_ct;
```




```
        load_new_c=1;
    end
else
    if(count_k_end==1)
        next_state=hold;
    else
        begin
            show_new_time=1;
            next_state=receive;
        end
        enable_count_k=1;
    end
set_at:
    if(alarm_button==1)
        begin
            next_state=set_at;
            load_new_a=1;
        end
    else
        next_state=hold;
set_ct:
    if(time_button==1)
        begin
            next_state=set_ct;
            load_new_c=1;
        end
    else
        next_state=hold;
show_at:
    begin
        if(alarm_button==1)
            begin
                next_state=show_at;
                show_a=1;
            end
        else
            if(count_a_end==1)
                next_state=hold;
            else
```




```
        begin
            show_a=1;
            next_state=show_at;
        end
        enable_count_a=1;
    end
endcase
end
always @(posedge clk or enable_count_k)
    if(enable_count_k==0)
        begin
            counter_k=0;
            count_k_end=0;
        end
    else
        if(counter_k>=key_timeout)
            count_k_end=1;
        else
            counter_k=counter_k+1;
    end
always @(posedge clk or enable_count_a)
    if(enable_count_a==0)
        begin
            counter_a=0;
            count_a_end=0;
        end
    else
        if(counter_a>=key_timeout)
            count_a_end=1;
        else
            counter_a=counter_a+1;
    end
endmodule
//测试模块部分
module stimulus;
    reg key,alarm_button,time_button,clk,reset;
    wire load_new_a,load_new_c,show_new_time,show_a;
    alarm_ctr    alarm1(key,alarm_button,time_button,clk,reset,
                        load_new_a,load_new_c,show_new_time,show_a);

    initial
        begin
```




```

key=0;
alarm_button=0;
time_button=0;
clk=0;
reset=1;
#5 reset=0;
#5 reset=1;
#10 key=1;
#10 key=0;
    alarm_button=1;
#20 alarm_button=0;
    key=1;
#10 time_button=1;
    key=0;
#20 time_button=0;
    alarm_button=1;
#150 key=0;
#20 reset=0;
#10 $finish;
end
always
    #5 clk=~clk;
endmodule

```

测试结果如图 11.5 所示。

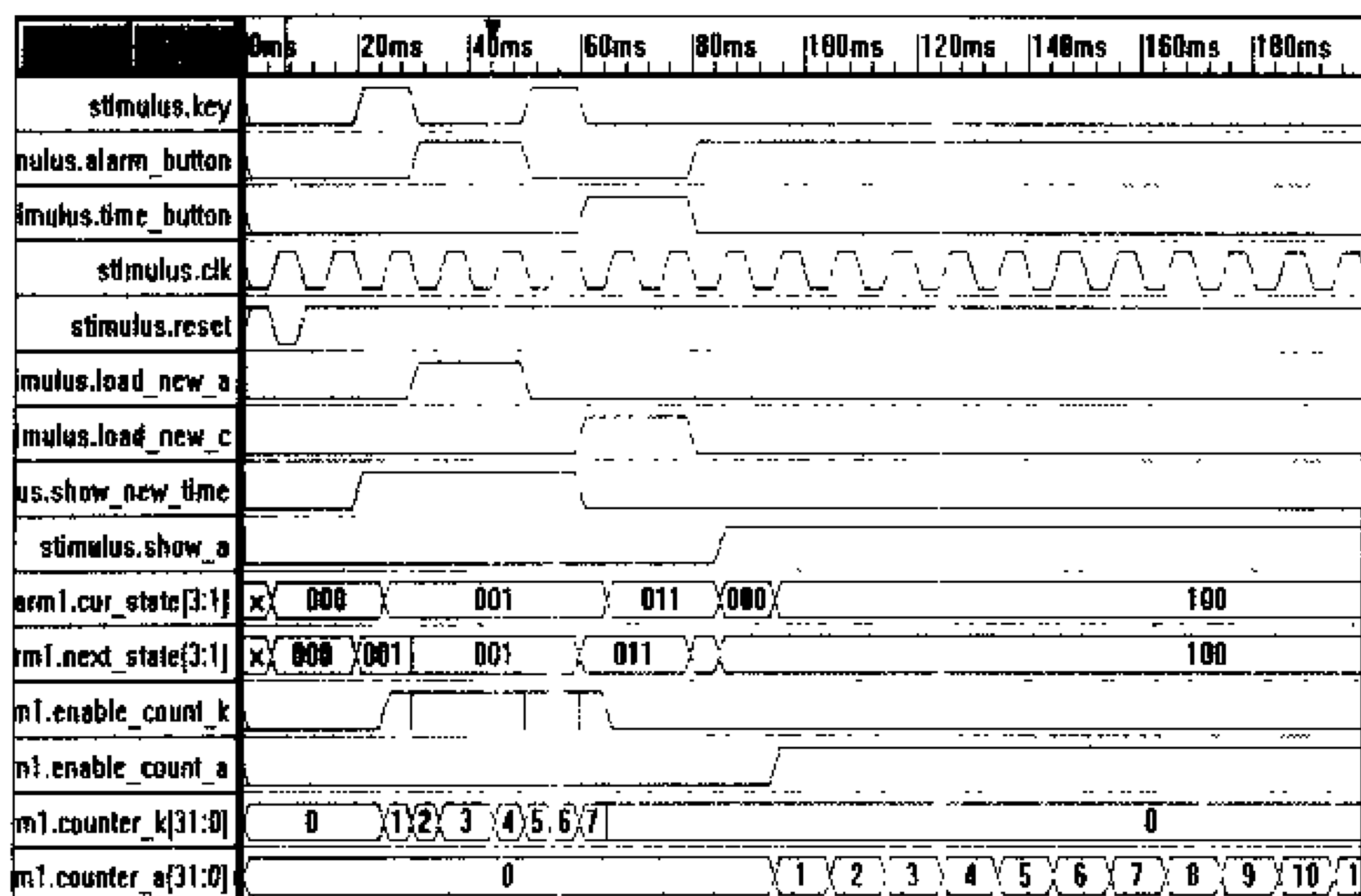


图 11.5 闹钟控制器的仿真结果

11.2 RISC 中央处理单元(CPU)的顶层设计

RISC(Reduced Instruction Set Computer)的含义是精简指令集计算机,这意味着计算机的指令集非常简单,所谓指令集简单是与复杂指令集计算机(CISC)相对比而言。RISC 很适合于用超大规模集成电路实现,由于指令简单,所以译码复杂度低,同时可以针对各条指令对硬件进行优化,因此指令的执行速度很高,从而弥补了其指令个数少的缺点。本节通过一个经简化只有 8 条指令的、字长为 16 位的 RISC 中央处理单元(CPU)的顶层设计实例来说明用 Verilog HDL 对复杂电路进行建模的方法。

RISC_CPU 本身是一个极其复杂的数字电路,可分成 9 个基本部件:累加器(accumulator)、RISC 算术运算单元(alu)、数据控制器(datactrl)、动态存储器(memory)、指令寄存器(instruction register)、状态控制器(state controller)、程序计数器(programm counter)、地址多路器(addrmux)和时钟发生器(clkgen)等,其基本部件的逻辑比较简单。利用 Verilog HDL 将基本部件的功能描述清楚,并对各部件的输入、输出逻辑关系进行仿真验证,再利用结构建模的方法将基本部件组合成一个顶层模块,由此就完成了 RISC 中央处理单元(CPU)的顶层设计。下面各节分别讨论各基本部件的 Verilog HDL 描述,最后给出顶层设计模块。

11.2.1 累加器用寄存器

累加器用寄存器用于存放当前的结果,它也是双目运算中的一个数据来源,其外部端口如图 11.6 所示。

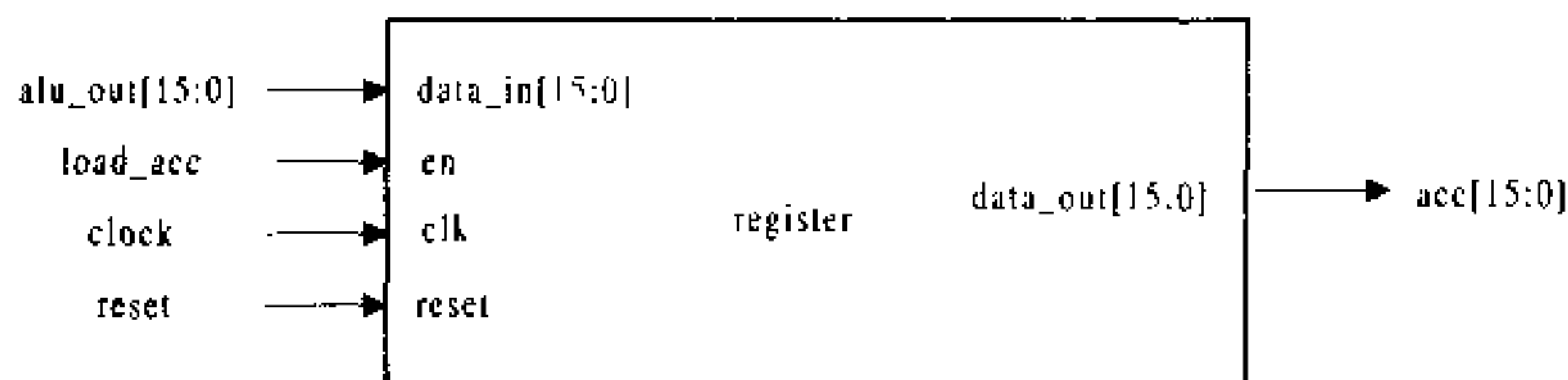


图 11.6 累加器用寄存器的外部端口

当 reset 信号有效(低电平有效)时,该寄存器清零。当来自 CPU 状态控制器的信号 load_acc 有效时,累加器用寄存器被启动,在 clock 时钟的上升沿接收数据总线的数据。累加器用寄存器的 Verilog HDL 实现程序如下:

```
module cpu_register(data_out,clk,reset,en,data_in);
    output [15:0] data_out;
    input [15:0] data_in;
    input clk,reset,en;
    reg [15:0] data_out;
    always @(posedge clk or reset or en)
        if(reset)
            data_out=16'b0;
```




```

    else if(en)
        data_out=data_in;
endmodule

```

11.2.2 RISC 算术运算单元

RISC 算术运算单元通过判断输入的不同操作码，来分别完成相应的加、与、异或等 8 种基本操作。然后通过这几种基本运算来实现多种其他运算以及逻辑判断等操作。RISC 算术运算单元的外部端口如图 11.7 所示。

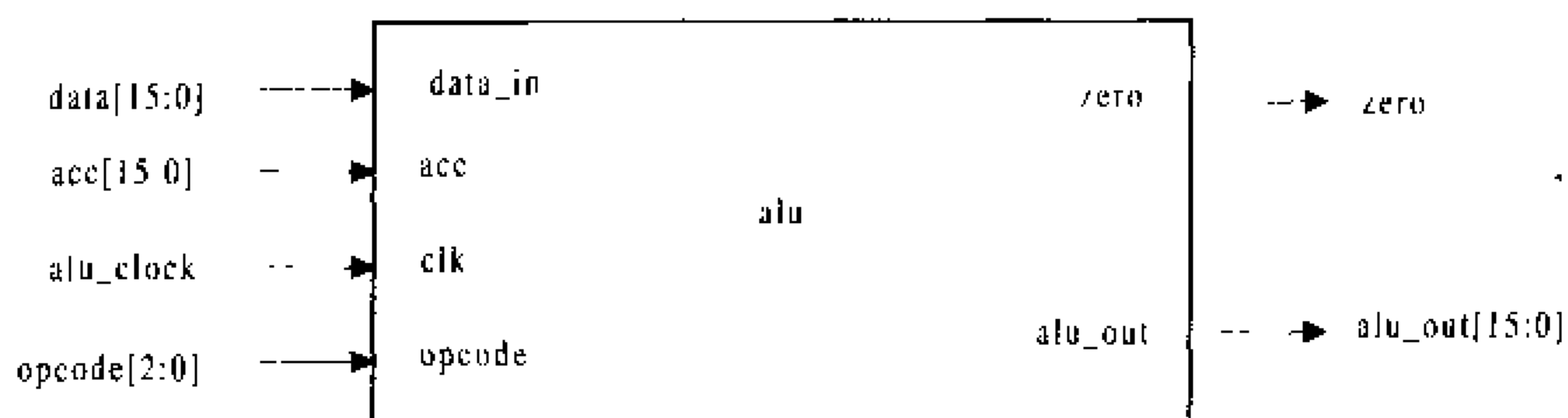


图 11.7 算术逻辑单元的外部端口

RISC 算术运算单元的 Verilog HDL 实现程序如下：

```

`timescale 1ns/1ns
module cpu_alu(alu_out,zero,opcode,data_in,acc,clk),
    input [2:0] opcode;
    input [15:0] data_in,acc;
    input clk;
    output [15:0] alu_out;
    output zero;
    reg [15:0] alu_out;
    parameter z_delay=1.5, alu_delay=4.5;
    always @(posedge clk)
    begin
        case(opcode)
            3'b000: #alu_delay alu_out=accum;
            3'b001: #alu_delay alu_out=accum;
            3'b010: #alu_delay alu_out=data_in+accum;
            3'b011: #alu_delay alu_out=data_in&accum;
            3'b100: #alu_delay alu_out=data_in^accum;
            3'b101: #alu_delay alu_out=data_in;
            3'b110: #alu_delay alu_out=accum;
            3'b111: #alu_delay alu_out=accum;
            default: begin
                $display("unknown opcode");
            end
        endcase
    end
endmodule

```



```

        #alu_delay alu_out=16'bx;
    end
endcase
end
assign #z_delay zero=(accum==0)?1:0;
endmodule

```

11.2.3 数据控制器

数据控制器用于控制累加器的数据输出，其外部端口如图 11.8 所示。数据总线是不同操作时传送数据的共用通道，不同情况下传送的内容不同。例如，当来自状态控制器的 `fetch` 信号有效时，数据控制器要传输指令；而当 `mem_r` 信号有效时，将累加器的数据写入到 RAM 区，否则应呈现高阻态，以允许其他部件使用数据总线。

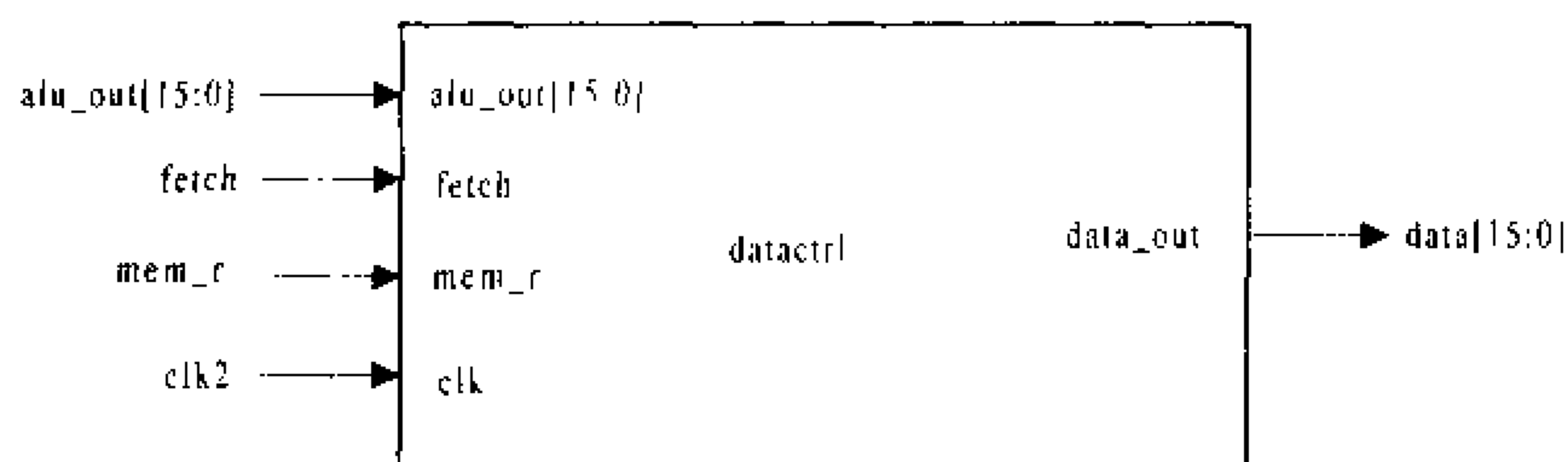


图 11.8 数据控制器的外部端口

数据控制器的 Verilog HDL 实现程序如下：

```

module cpu_datactrl(data_out,alu_out,fetch,mem_r,clk);
    output [15:0] data_out;
    reg [15:0] data_out;
    input [15:0] alu_out;
    input fetch,mem_r,clk;
    always @(fetch or mem_r or posedge clk)
        if(!fetch & !mem_rd)
            data_out=alu_out;
        else
            data_out=16'bz;
endmodule

```

11.2.4 指令寄存器

指令寄存器用于存储指令，其外部端口如图 11.9 所示。当 `reset` 信号有效时，指令寄存器被清零；然后在 `clock` 的上升沿触发下，将数据总线送来的指令存入到寄存器中去。但并非每个上升沿寄存器都储存来自数据总线的的数据，这由来自状态控制器的 `load_ir` 信号决定。这是因为数据总线上有时传输指令，有时传输数据。只有当指令寄存器的使能端有效时，才可以进行存储数据操作。

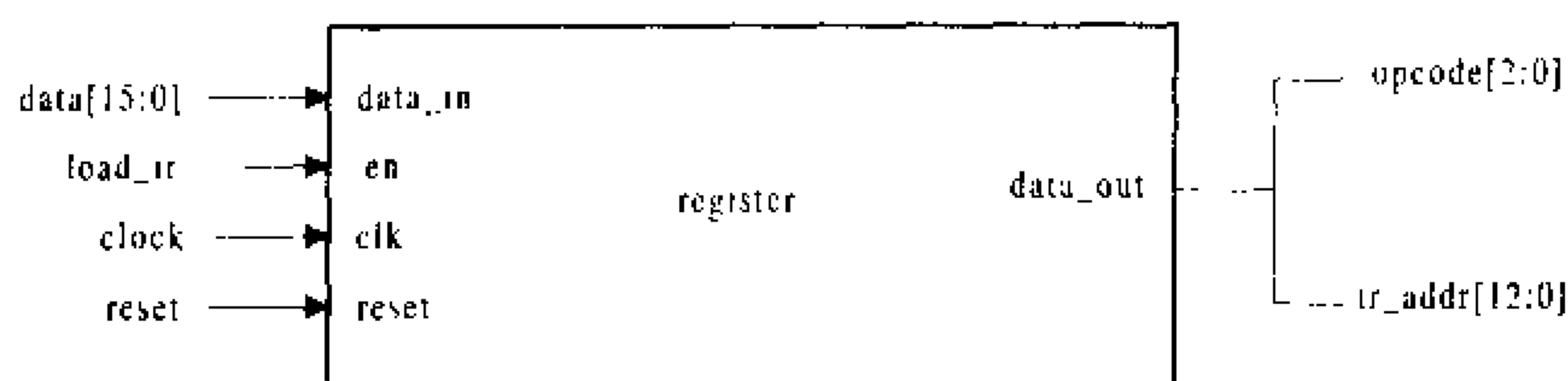


图 11.9 指令寄存器的外部端口

指令寄存器的 Verilog HDL 实现程序如下:

```
module cpu_register(data_out,clk,reset,en,data_in);
    output [15:0] data_out;
    input [15:0] data_in;
    input clk,reset,en;
    reg [15:0] data_out;
    always @(posedge clk or reset or en)
        if(!reset)
            data_out=16'b0;
        else if(en)
            data_out=data_in;
endmodule
```

11.2.5 状态控制器

状态控制器是 CPU 的核心部分,它产生一系列的控制信号,用于启动或停止某些部件。例如 CPU 何时进行指令的读写操作,数据总线传输指令或数据、程序计数器的计数时钟等都是由该控制器来完成的。状态控制器的外部端口如图 11.10 所示。

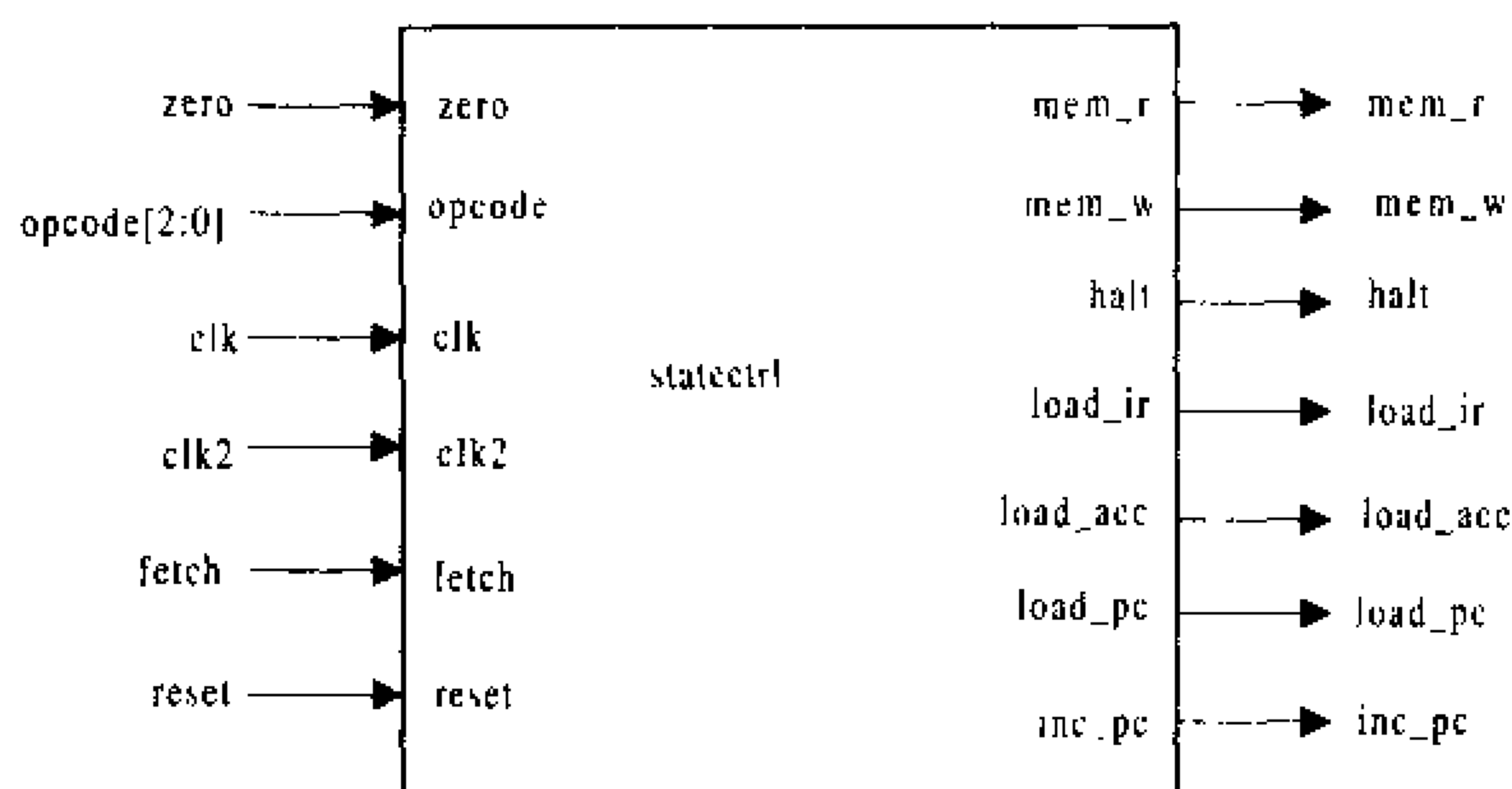


图 11.10 状态控制器的外部端口

状态控制器的 Verilog HDL 实现程序如下:

```
module cpu_sctrl(load_acc,mem_r,mem_w,inc_pc,load_pc,load_ir,
    halt,opcode,fetch,zero,clk,clk2,reset);
```




```
output load_acc,mem_r,mem_w,inc_pc,load_pc,load_ir,halt;
reg load_acc,mem_r,mem_w,inc_pc,load_pc,load_ir,halt;
input [2:0] opcode;
input fetch,zero,clk,clk2,reset;
parameter Hlt=3'b000,Skz=3'b001,Add=3'b010,And=3'b011,
          Xor=3'b100,Lda=3'b101,Sto=3'b110,Jmp=3'b111;
always @(posedge fetch)
    if(reset)
        ctl_state;
always @(negedge reset)
    begin
        disable ctl_state;
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0;
    end
always @(posedge reset)
    @(posedge fetch) ctl_state;
task ctl_state;
    begin
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0000000;
        @(posedge clk)
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0000100;
        @(negedge clk)
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0000110;
        @(posedge clk)
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0000110;
        @(negedge clk)
        if(opcode==Hlt)
            {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b1000001;
        else
            {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b1000000;
        @(posedge clk)
        if((opcode==Add)||(opcode==And)||(opcode==Xor)||(opcode==Lda))
            {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0000100;
        else
            {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0000000;
        @(negedge clk)
        if(opcode==Jmp)
            {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0010000;
        else if((opcode==Skz)&&zero)
```




```

        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b1000000;
    else if((opcode==Add)||(opcode==And)||(opcode==Xor)||(opcode==Lda))
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0100100;
    else
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0000000;
    @(posedge clk)
    if(opcode==Jmp)
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b1010000;
    else if(opcode==Sto)
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0001000;
    else if((opcode==Skz)&&zero)
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b1000000;
    else if((opcode==Add)||(opcode==And)||(opcode==Xor)||(opcode==Lda))
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0100100;
    else
        {inc_pc,load_acc,load_pc,mem_w,mem_r,load_ir,halt}=7'b0000000;
    end
endtask
endmodule

```

11.2.6 动态存储器

动态存储器用于存储地址，其外部端口如图 11.11 所示。

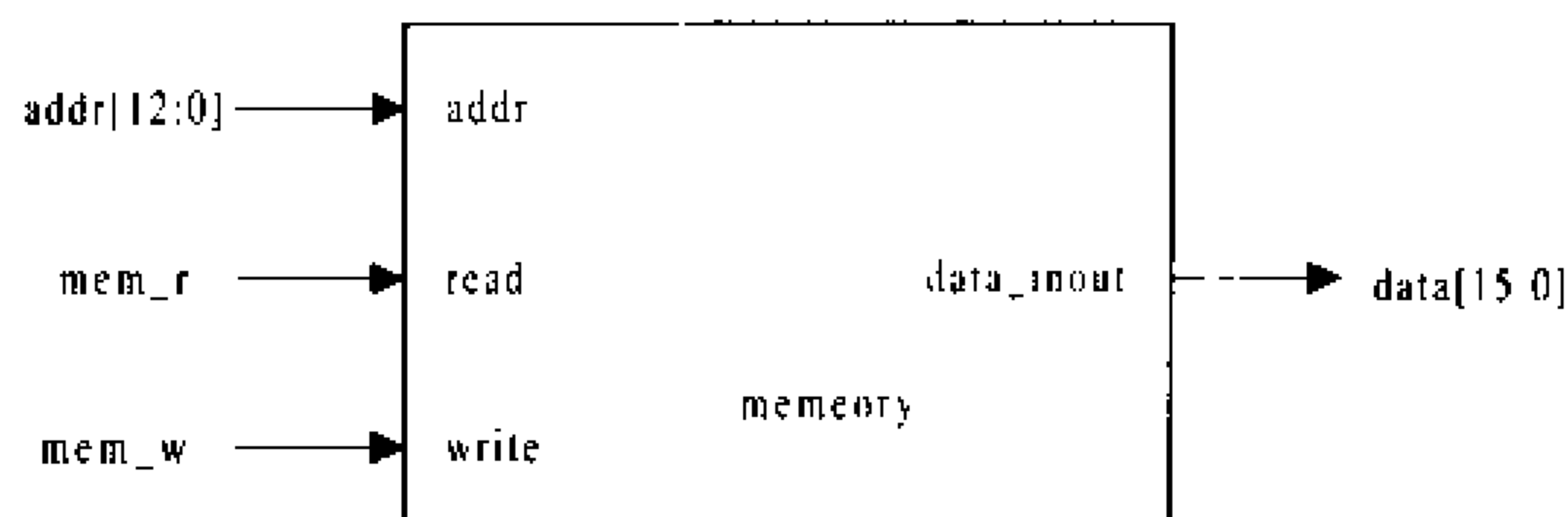


图 11.11 动态存储器的外部端口

动态存储器的 Verilog HDL 实现程序如下：

```

`timescale 1ns/100ps
module cpu_mem(data_inout,addr,read,write);
    inout [15:0] data_inout;
    input [12:0] addr;
    input read,write;
    reg [15:0] memory[0:'h1FFF];
    wire [15:0] data_inout=read?memory[addr]:16'bz;
    always @(posedge write)

```




```
begin
    memory{addr}=data_inout;
end
endmodule
```

11.2.7 程序计数器

程序计数器用于提供指令地址，以便读取指令，其外部端口如图 11.12 所示。

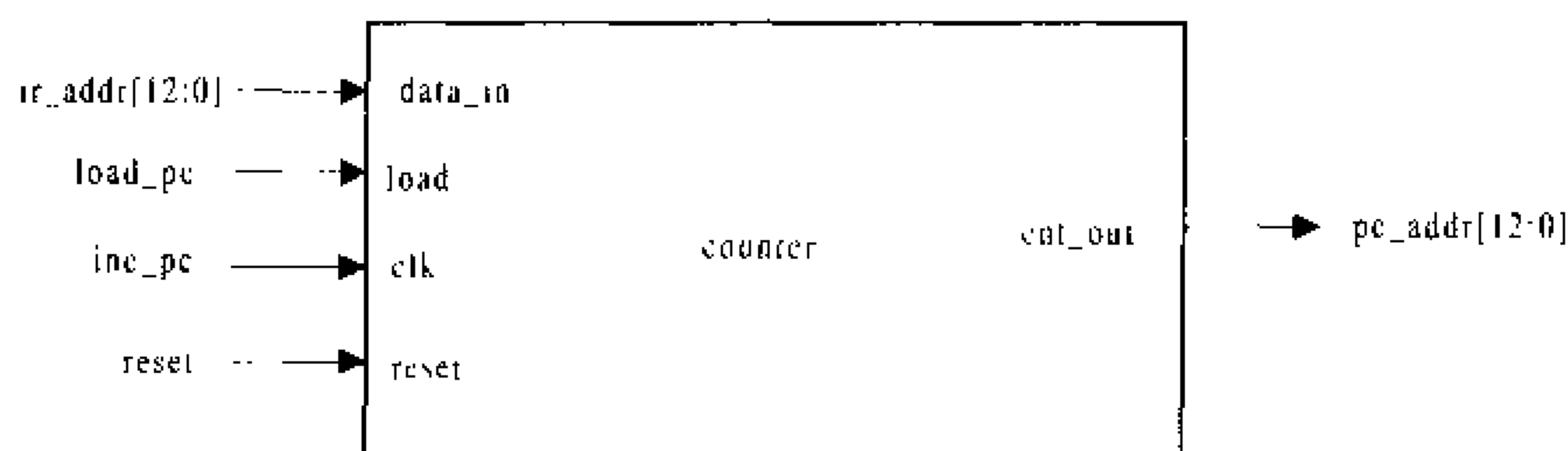


图 11.12 程序计数器的外部端口

程序计数器的 Verilog HDL 实现程序如下：

```
module cpu_pcounter(cnt_out,clk,data_in,reset,load);
    input [12:0] data_in;
    input clk,reset,load;
    output [12:0] cnt_out;
    reg [12:0] cnt_out;
    always @(posedge clk or reset)
        if(!reset)
            cnt_out=13'b0;
        else
            if(load==1)
                cnt_out=data_in;
            else
                if(cnt_out==5'b1FFF)
                    cnt_out=13'b0;
                else
                    cnt_out=cnt_out+1;
endmodule
```

11.2.8 地址多路器

地址多路器用于选择输出的地址是程序计数地址还是数据/端口地址，其外部端口如图 11.13 所示。

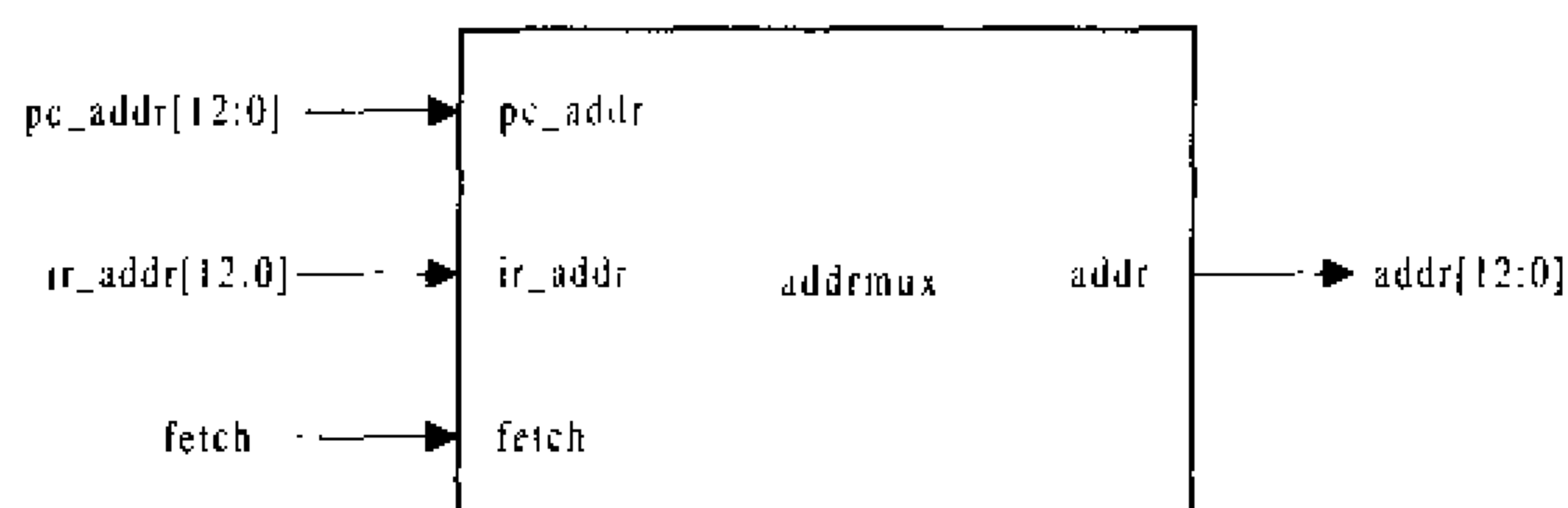


图 11.13 地址多路器的外部端口

地址多路器的 Verilog HDL 的实现程序如下:

```
module cpu_admux(addr,pc_addr,ir_addr,fetch);
    output [12:0] addr;
    input [12:0] pc_addr,ir_addr;
    input fetch;
    if(fetch)
        addr=pc_addr;
    else
        addr=ir_addr;
endmodule
```

11.2.9 时钟发生器

时钟发生器用于生成不同的时钟信号 clock、clk2、fetch 与 alu_clk, 其外部端口如图 11.14 所示。产生的时钟信号 clock 送往寄存器与状态控制器, 时钟信号 clk2 送往数据控制器与状态控制器, 信号 fetch 送往数据控制器、状态控制器与地址多路器, 信号 alu_clk 送往算术逻辑单元。

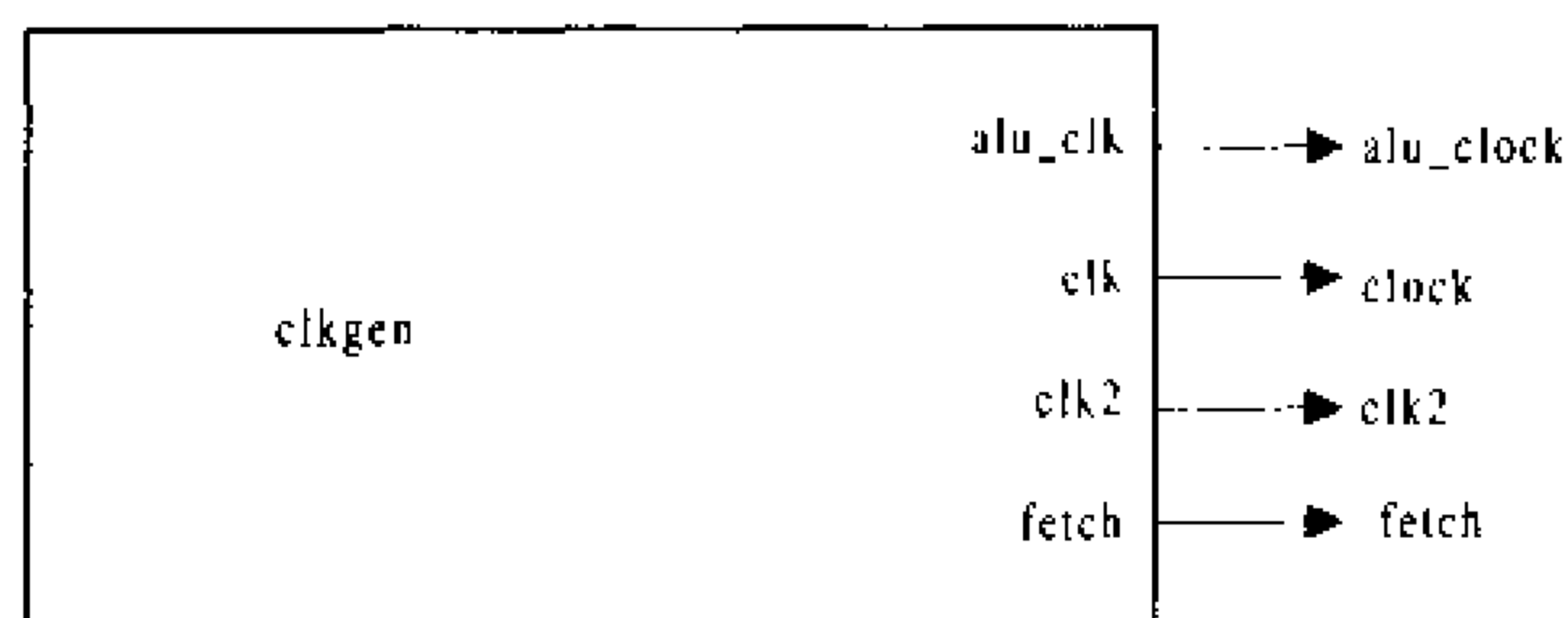


图 11.14 时钟发生器的外部端口

时钟发生器的 Verilog HDL 实现程序如下:

```
`timescale 1ns/10ps
module cpu_clkg(fetch,clk2,clk,alu_clk);
    output fetch,clk2,clk,alu_clk;
    reg fetch,clk2,clk;
    parameter period=60;
    assign alu_clk=(fetch!clk2!clk);
    initial
```



```

fork
    clk=0;
    clk2=1;
    fetch=1;
    forever #(period/2) clk=~clk;
    forever #(period)   clk2=~clk2;
    forever #(2*period) fetch=~fetch;
join
endmodule

```

11.2.10 顶层设计模块

上述几节完成了各基本模块的设计，本节用结构建模的方法把它们组成一个顶层模块，由此完成 RISC 中央处理单元(CPU)的顶层设计。

顶层设计模块程序如下：

```

`timescale 1ns/1ps
module cpu_top;
    wire reset,load_acc,load_ir,load_pc,halt,zero;
    wire clock,clk2,alu_clock,fetch,inc_pc;
    wire [15:0] alu_out,acc,data,opcode_iraddr;
    wire [12:0] addr,ir_addr,pc_addr;
    wire [2:0] opcode;
    assign {opcode,ir_addr}=opcode_iraddr;
    cpu_register accumulator(accum,clock,alu_out,load_acc,reset);
    cpu_alu    alu(alu_out,zero,opcode,data,accum,alu_clock);
    cpu_datactrl data_ctrl(data,alu_out,fetch,mem_r,clk2);
    cpu_mem    mem(data,addr,mem_r,mem_w);
    cpu_register instr_register(opcode_iraddr,clock,data,load_ir,reset);
    cpu_sctrl  state_ctrl(load_acc,mem_r,mem_w,inc_pc,load_pc,load_ir,
                        halt,opcode,fetch,zero,clock,clk2,reset);
    cpu_pcounter program_counter(pc_addr,inc_pc,ir_addr,reset,load_pc);
    cpu_admux  address_mux(addr,pc_addr,ir_addr,fetch);
    cpu_clkg  clock_g(fetch,clk2,clk,alu_clock);
endmodule

```




第 12 章 开发工具介绍



本章首先简单介绍 EDA 基本工具与 Verilog HDL 开发工具，然后对目前最流行的一种 HDL 语言仿真器 VeriLogger Pro 的概况与使用进行阐述。希望通过本章的介绍，为读者利用 Verilog HDL 语言进行产品开发提供一定的帮助。

12.1 EDA 基本工具

集成电路技术的发展不断对 EDA 技术提出新的要求，并且促进了 EDA 技术的发展。但是总的来说，EDA 系统的设计能力一直难以赶上集成电路技术发展的要求。EDA 工具的发展经历了两大阶段：物理工具阶段和逻辑工具阶段。现在的 EDA 和系统设计工具正被理解成一个整体的概念——电子设计自动化。

物理工具用来完成设计中的实际物理问题，如芯片布局、印刷电路板布线等。另外它还能提供一些设计的电气性能分析，如设计规则检查等。这些工作现在主要由芯片生产厂家来完成。

逻辑工具是基于网表、布尔逻辑、传输时序等概念的。首先进行设计输入，包括原理图输入和硬件描述语言的输入，然后逻辑工具对设计输入进行逻辑综合、仿真、优化等操作，最后生成物理工具可以接受的网表和 VHDL、Verilog HDL 的结构化描述。

在过去的 30 多年里，人们开发了大量的 EDA 工具来进行集成电路的设计，这些设计工具的分类如图 12.1 所示。

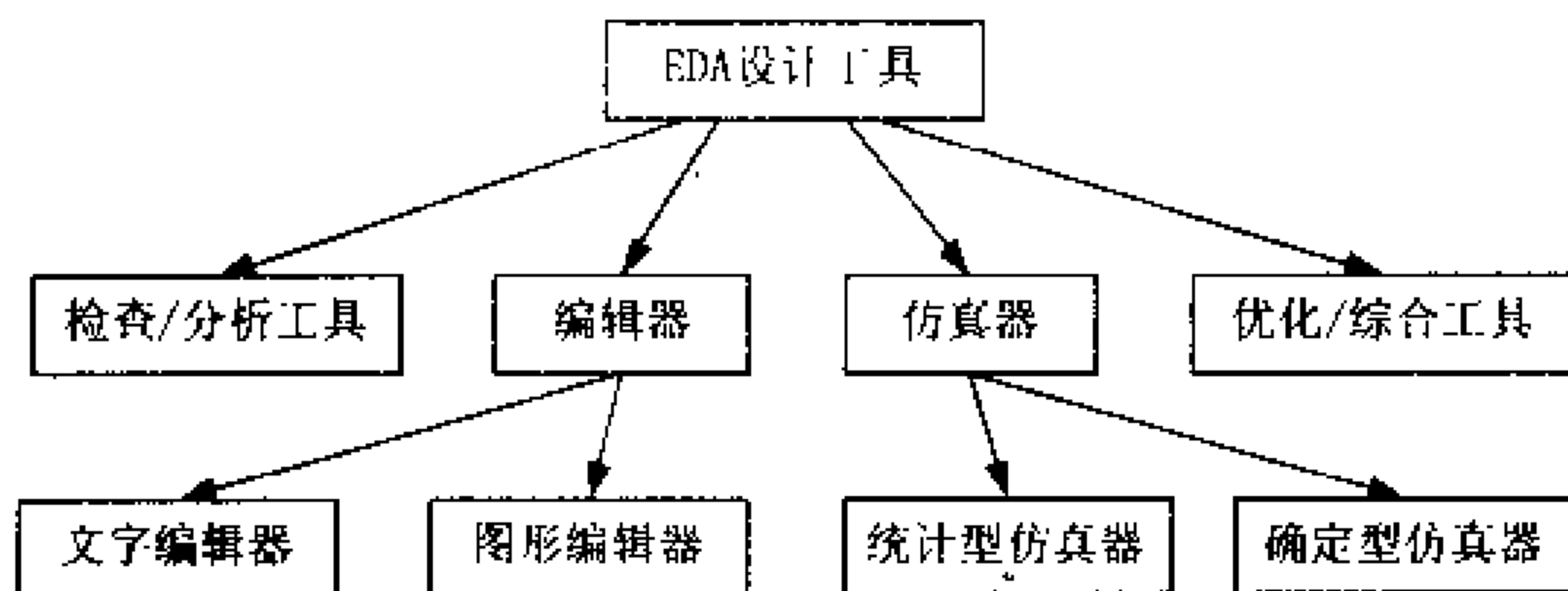


图 12.1 设计工具分类

12.1.1 编辑器

编辑器包括文字编辑器和图形编辑器。文件编辑器用来编辑硬件描述语言，比如 VHDL 和 Verilog HDL。图形编辑器可用于硬件设计的各个层次。在版图级，图形编辑器可以用来编辑表示硅工艺加工过程的几何图形。在其它层次，图形编辑器还可以用来编辑硬件系统的方框图、原理图等。典型的原理图输入工具一般都具备下面三个功能：



- (1) 基本单元符号库，主要包括基本单元的图形符号和仿真模型。
- (2) 原理图编辑器的编辑功能。
- (3) 产生网表的功能。

12.1.2 仿真器

仿真器又称模拟器，主要用来帮助设计者验证设计的正确性。硬件系统设计的各个层次都要用到仿真器。在数字系统中，硬件系统是由数字逻辑器件以及它们之间的互连来表示的。仿真器的用途是确定系统的输入/输出关系，所采用的方法是把每一个数字逻辑器件映射为一个或几个进程，把整个系统映射为由进程互连构成的进程网络，这种由进程互连组成的网络就是设计的仿真模型。

12.1.3 检查/分析工具

在集成电路设计的各个层次都会用到检查/分析工具。在版图级，必须用设计规则检查工具来保证版图所表示的电路能被可靠地制造出来。在逻辑门级，检查/分析工具可以用来检查是否有违反“扇出规则”的连接关系。时序分析器一般用来检查最坏情况下电路中的最大和最小延时。

12.1.4 优化/综合工具

优化/综合工具用来把一种硬件描述转换为另一种描述，这种转换通常伴随着某种设计上的改进。在逻辑门级，可以用逻辑最小化对布尔表达式进行简化。在 RTL 级，优化工具可以用来确定控制序列和数据路径的最优组合。各个层次的综合工具可以将硬件的高层次描述转换为低层次描述，也可以将硬件的行为描述转换为结构描述。

12.2 Verilog HDL 开发工具

Verilog HDL 的开发工具大致可分为两大类：综合工具与仿真器。由于 Verilog HDL 高级行为描述用于综合的历史还不长，可综合的 Verilog HDL 的语法只是它们自己语言的一个子集，又由于 HDL 的可综合性研究近来发展很快，可综合子集的国际标准日前尚未形成，因此各厂商的综合器所支持的可综合 HDL 子集也略有不同。鉴于此，本节只简单给出目前比较通用的几家公司的综合工具与仿真器，以便用户查阅。

12.2.1 综合工具

各公司的综合工具如表 12.1 所示。

表 12.1 各公司的综合工具

公 司 名 称	综 合 工 具
Avanti(ACEO Technology, Inc.)	Asyn
Cadence Design Systems Inc.	• InVisia
	• Exemplar Logic
	• Leonardo Spectrum



续表

公 司 名 称	综 合 工 具
Mentor Graphics Corp.	AutoLogic and PLDSynthesis
Synopsys, Inc.	Design Compiler and Behavioral Compiler
Synplicity, Inc.	Synplify

12.2.2 仿真器

各公司的仿真器及其网址如表 12.2 所示。

表 12.2 各公司的仿真器

公 司 名 称	仿 真 器	网 址
Avanti Corporation	Polaris	http://www.avanticorp.com
Cadence Design Systems, Inc.	NC Verilog & Verilog-XL	http://www.cadence.com
Synopsys Inc.	VCS	http://www.synopsys.com
SynaptiCAD Inc.	VeriLogger Pro	http://www.syncad.com
Fintronic USA, Inc.	FinSim	http://www.fintronic.com
InnoLogic Systems Inc.	ESP Symbolic Simulator	http://www.innologic-systems.com
Mentor Graphics	QuickHDL	http://www.mentorg.com
Model Technology	ModelSim	http://www.model.com
Dolphin Integration	SMASH	http://www.dolphin.fr
Simucad	Silos III	http://www.simucad.com

12.3 VeriLogger Pro 概况

VeriLogger Pro 是 SynaptiCAD 公司开发的 HDL 语言仿真器, 它支持 VHDL 与 VerilogHDL 两种硬件描述语言, 并支持多种标准, 适用于多种操作平台。它是 EDA 设计中不可缺少的一种有效工具, 也是目前最流行的 HDL 语言仿真器之一。

12.3.1 VeriLogger Pro 适用平台

VeriLogger Pro 适用于以下平台:

- (1) Microsoft Windows 95/98/NT。
- (2) HP/UX 11。
- (3) Solaris 2.6/7.0/8.0。

12.3.2 VeriLogger Pro 支持的标准

VeriLogger Pro 支持多种标准, 可归纳如下:

- (1) IEEE 1076—1987 VHDL。
- (2) IEEE 1076—1993 VHDL。
- (3) IEEE Std 1364—1995 Verilog。

12.3.3 VeriLogger Pro 进行仿真的基本步骤

使用 VeriLogger Pro 进行仿真的基本步骤为:

- (1) 编写源程序文件。
- (2) 创建工程。
- (3) 向工程中添加文件。
- (4) 编译源代码。
- (5) 调试程序。
- (6) 进行仿真。

12.3.4 VeriLogger Pro 的窗口构成

如果 VeriLogger Pro 仿真器安装于 Windows 98 操作系统下, 当 PC 机引导进入操作系统后, 桌面将出现 VeriLogger Pro 的图标, 用鼠标双击该图标就使系统进入 VeriLogger Pro 仿真器的窗口, 其窗口构成如图 12.2 所示。

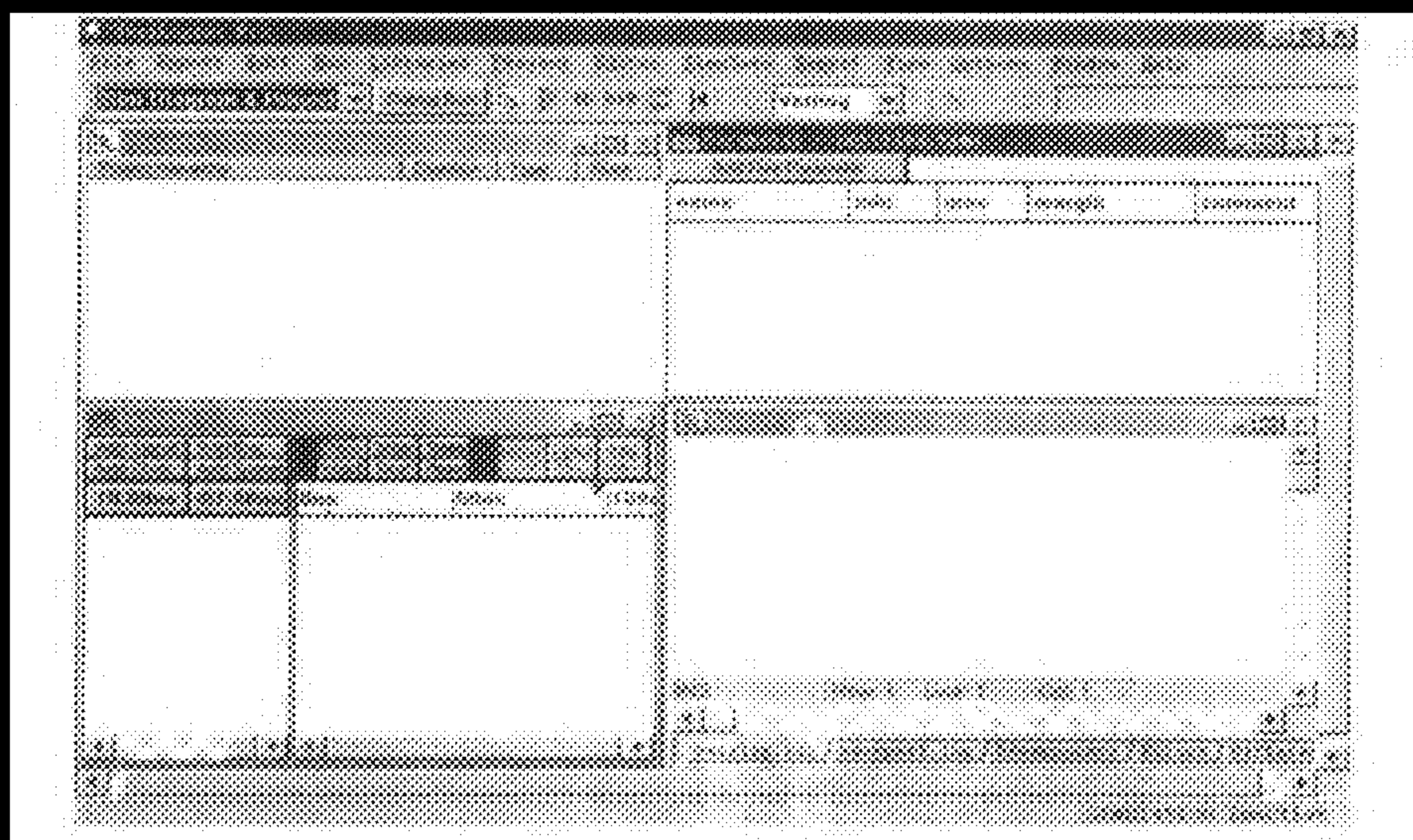


图 12.2 VeriLogger Pro 的窗口构成

从图 12.2 中可以看出, VeriLogger Pro 除了主菜单、工具栏、状态栏外, 还有 4 个子窗口, 它们分别是 Project(工程)窗口、Parameter(参数)窗口、Diagram(时序图)窗口及 Report(报告)窗口。主菜单中有 13 个选项, 各选项下还有子菜单, 某些重要的主菜单选项与工具栏中的图标相对应, 用来快速操作。

12.4 VeriLogger Pro 使用指南

通过对上一节的学习, 读者对 VeriLogger Pro 的概况已有了大致的了解。在此基础上, 本节通过一个具体的例子来介绍有关 VeriLogger Pro 仿真器的使用, 以便读者在利用 Verilog 语言进行数字电路设计时参考。



12.4.1 创建与编辑一个 Verilog 语言的文件与工程

创建与编辑一个 Verilog 语言的文件与工程，是利用 VeriLogger Pro 仿真器进行仿真的第一步。

1. 编写源程序文件

通过前面几章的介绍，读者利用 Verilog HDL 编写源程序文件已经不成问题，这里不再赘述。下面主要介绍 VeriLogger Pro 仿真器中关于源程序文件的建立、保存、打开等问题。

在屏幕上用鼠标单击 Editor 选项，此时出现如图 12.3 所示的子菜单。

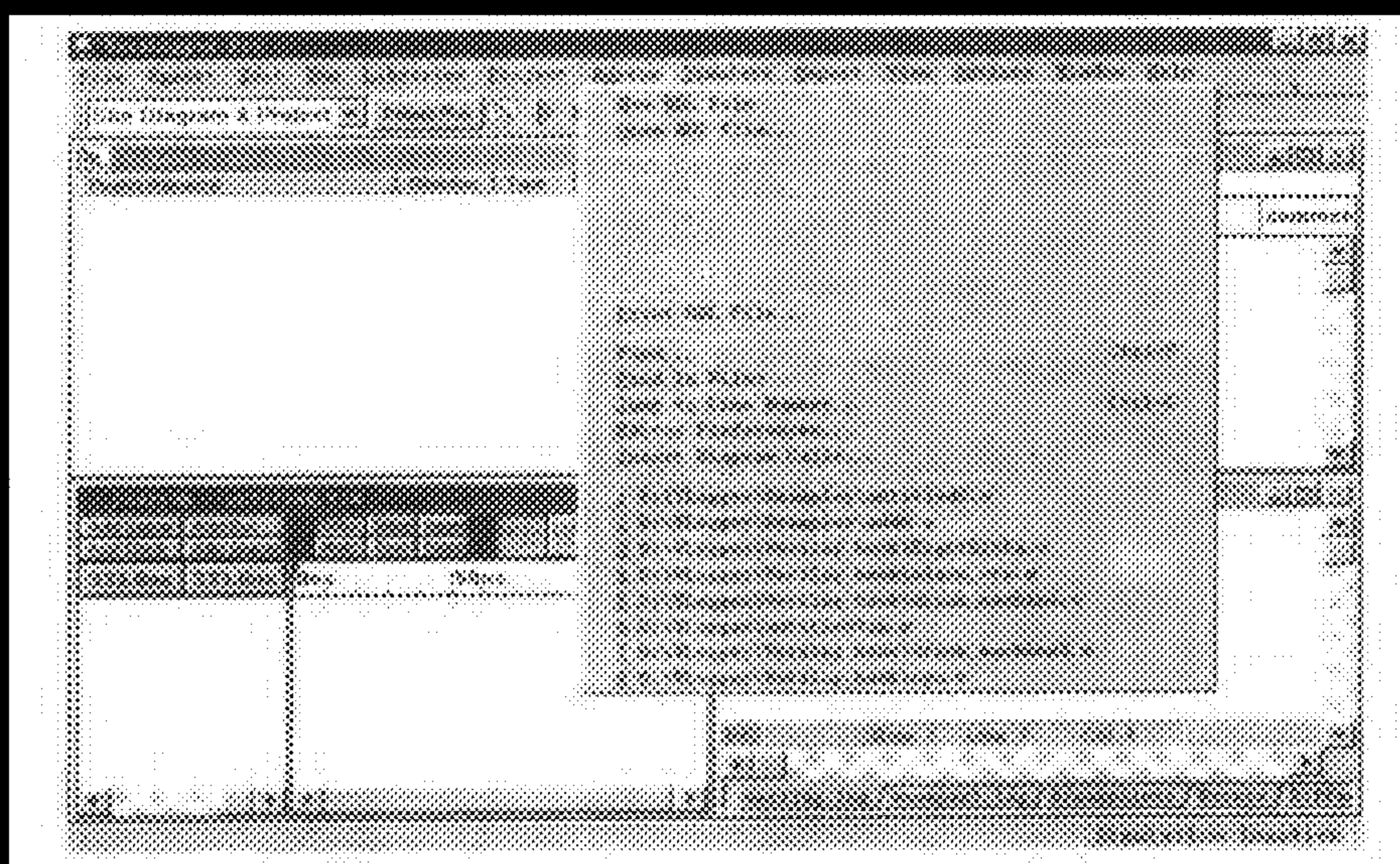


图 12.3 Editor 选项的子菜单

通过 Editor 选项的子菜单，我们可以很容易地完成源程序文件的建立、保存、打开等操作：

- (1) 建立：Editor>New HDL File。
- (2) 保存：Editor>Save HDL File。
- (3) 打开：Editor>Open HDL File。
- (4) 关闭：Editor>Close。

利用前面几章所学的知识，并结合上面所述内容，分别建立两个源程序文件：add4.v 与 add4test.v，以用来进行后面的编译与仿真操作，从而说明 VeriLogger Pro 仿真器的使用方法。

为了便于以后说明的方便，这里给出 add4.v 与 add4test.v 的源程序：

add4.v 的源程序：

```
/** Full Adder *****/  
module fulladder(sum, c_out, x, y, c_in);  
    output sum, c_out;  
    input x, y, c_in;
```




```
wire a, b, c;
xor (a, x, y);
xor (sum, a, c_in);
and (b, x, y);
and (c, a, c_in);
or (c_out, c, b);
endmodule

/** 4-Bit Adder ****~****
module FourBitAdder(sum, c_out, x, y, c_in);
    output [3:0] sum;
    output c_out;
    input  [3:0] x, y;
    input c_in;
    wire c1, c2, c3;
    fulladder fa0(sum[0], c1, x[0], y[0], c_in);
    fulladder fa1(sum[1], c2, x[1], y[1], c1);
    fulladder fa2(sum[2], c3, x[2], y[2], c2);
    fulladder fa3(sum[3], c_out, x[3], y[3], c3);
endmodule
```

add4test.v 的源程序:

```
/**~****
`timescale 1ps / 1ps
module testbed();
    reg  c_in;
    reg [3:0] y;
    reg [3:0] x;
    wire c_out;
    wire [3:0]sum;
    FourBitAdder A1(sum, c_out, x, y, c_in);
    initial
        begin
            //SIGNAL x
            x = 4'b0001;
            #25000
            x = 4'b0001;
            #25000
            x = 4'b0010;
            #25000
            x = 4'b0001;
            #25000
        end
    endmodule
```




```
        x = 4'b0001;
    #25000
        x = 4'b1000;
    #25000
        x = 4'b0001;
    #25000
        x = 4'b0001;
    #25000
        x = 4'b0010;
    #25000
        x = 4'b0011;
    #25000
    ;
end
initial
begin          //SIGNAL v
    y = 4'b0001;
    #25000
    y = 4'b0010;
    #25000
    y = 4'b0011;
    #25000
    y = 4'b1111;
    #25000
    y = 4'b1111;
    #25000
    y = 4'b1111;
    #25000
    y = 4'b0001;
    #25000
    y = 4'b0010;
    #25000
    y = 4'b0011;
    #25000
    y = 4'b1111;
    #25000;
end
initial
begin          //SIGNAL c_in
```



```
c_in = 1'b0;  
#100000  
c_in = 1'b1;  
#25000  
c_in = 1'b0;  
#25000  
c_in = 1'b1;  
#100000;  
end  
initial  
#250000 $finish;  
endmodule
```

2. 创建工程

在屏幕上用鼠标单击 **Project** 选项，此时出现如图 12.4 所示的子菜单。单击子菜单中的 **New HDL Project** 项，这样就建立了一个无名的工程。

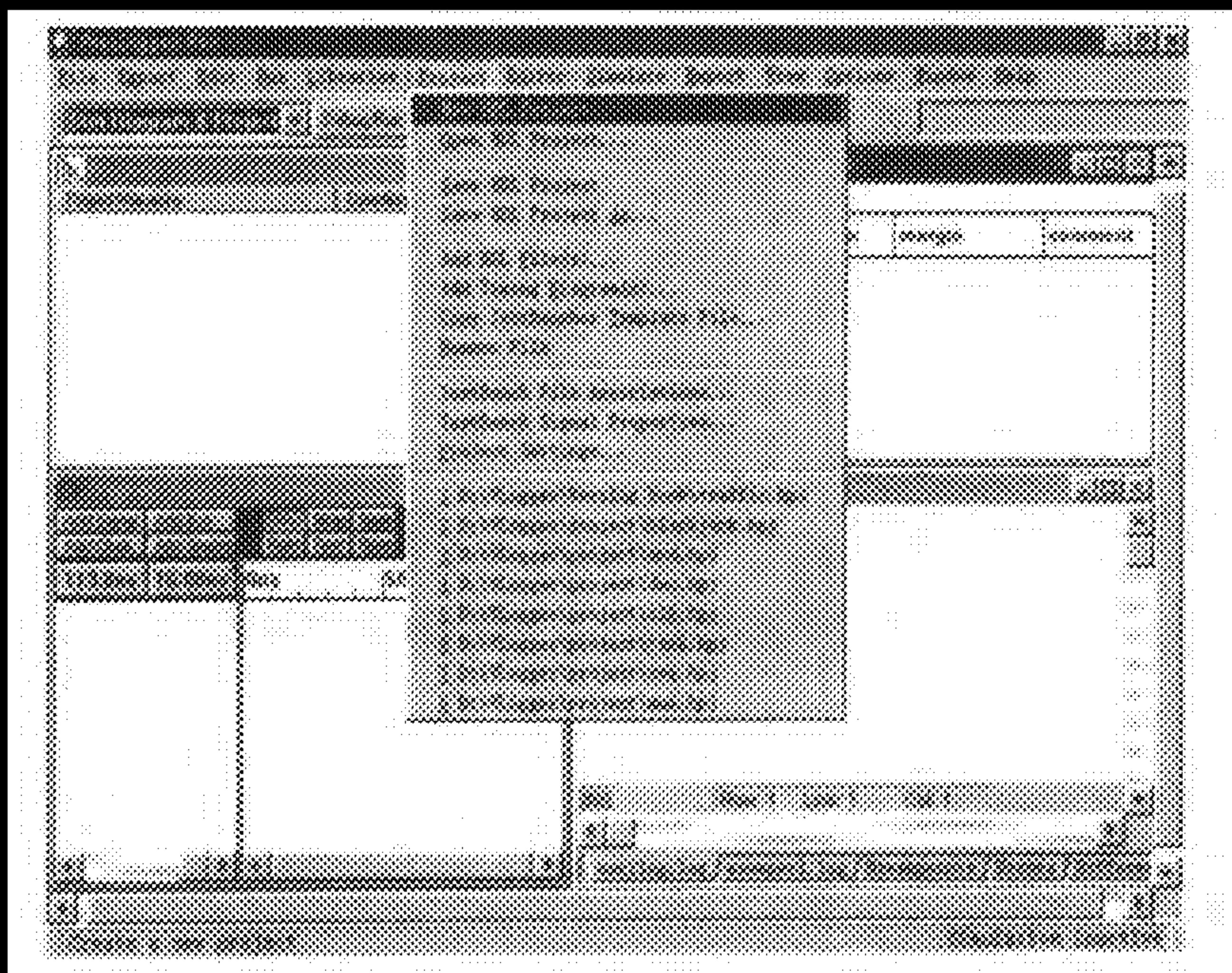


图 12.4 Project 选项的子菜单

3. 保存工程

用鼠标单击主菜单中的 **Project** 选项，选择子菜单中的 **Save HDL Project** 项，屏幕上会弹出如图 12.5 所示的对话框。

在对话框的文件名中输入 **test**，单击保存按钮，该文件名就以文件名 **test.hpj** 保存在当前目录下。此时 **Project** 子窗口的标题如图 12.6 所示。



图 12.5 保存对话框



图 12.6 Project 子窗口的标题

4. 向工程中添加文件

在建立了工程之后，必须向该工程中添加要仿真的源程序文件。在 Project 子窗口中单击鼠标右键，得到如图 12.7 所示的弹出式菜单。

选择 Add HDL File(s)...项，屏幕上就出现 Add File(s)对话框，如图 12.8 所示。

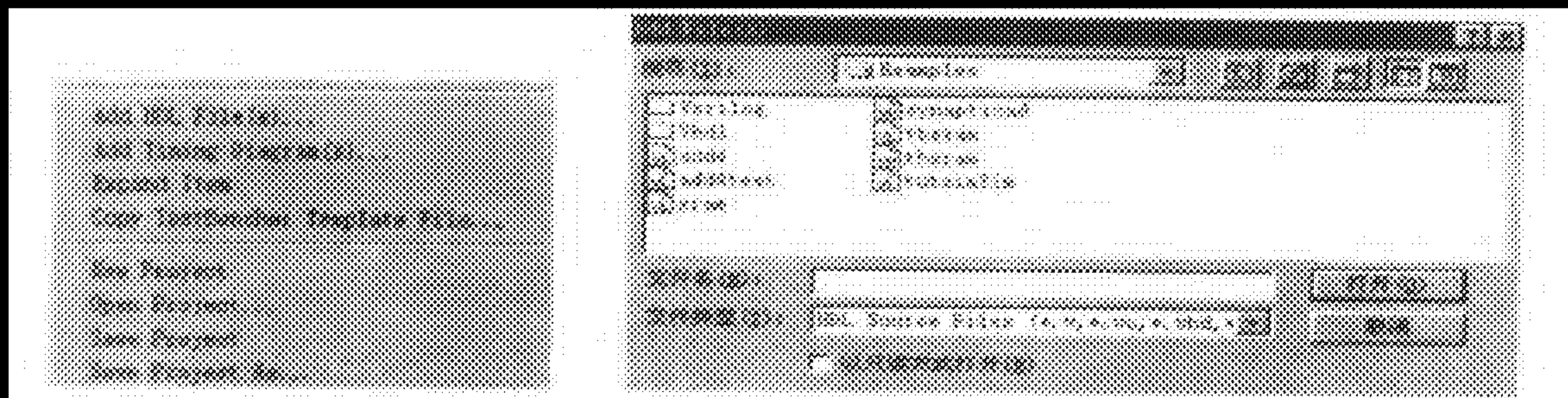


图 12.7 添加文件的弹出式菜单

图 12.8 添加文件对话框

按住 Shift 键，选择 add4.v 与 add4test.v 两个文件，并单击打开按钮，就将这两个文件添加到刚才建立的 test.hpj 工程中，此时的 Project 子窗口如图 12.9 所示。



图 12.9 添加文件后的 Project 子窗口

5. 修改文件

如果需要修改源程序文件，只需用鼠标双击 Project 子窗口中相应的文件名，即可打开相应的文件，以便查看或修改。例如需要修改文件 add4.v，则用鼠标双击 Project 子窗口中的 D:\Vlogger\Examples\add4.v 项，屏幕上就出现 add4.v 的源程序文件，如图 12.10 所示。对修改后的文件必须存盘，以保证更新原来的文件。

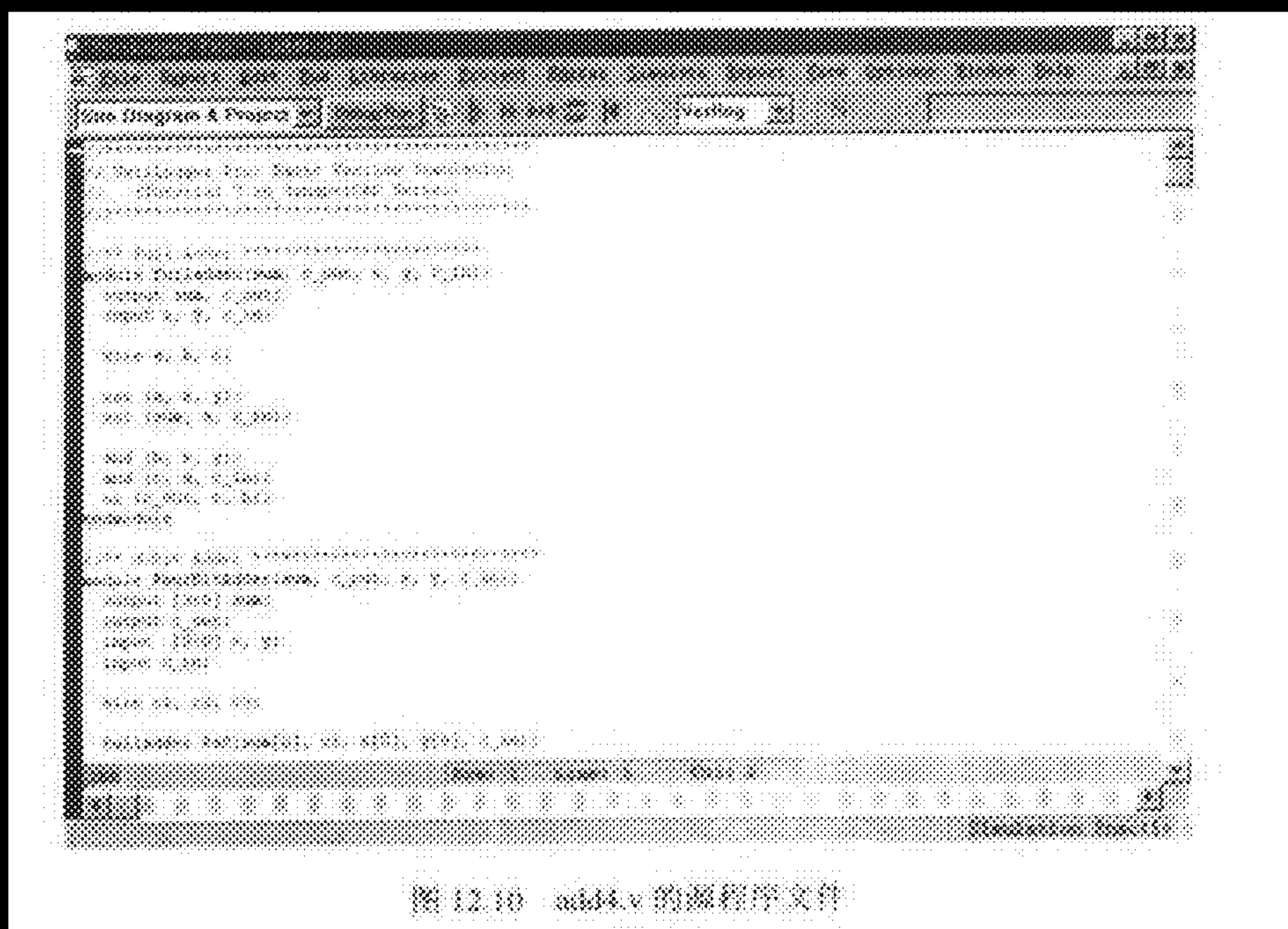


图 12.10 add4.v 的源程序文件

12.4.2 Verilog 语言工程的编译

在建立了新的工程并添加文件后，Project 子窗口如图 12.9 所示，从图中只能看到文件名，而并不能看出 Verilog HDL 源程序中各模块的层次，这可通过对工程进行编译来实现。

1. 工程编译方法

Verilog 语言工程的编译方法有以下三种：

- (1) 利用工具栏上的  按钮。
- (2) 利用 Simulate 选项下子菜单中的 Build 项。
- (3) 利用快捷键 F7。

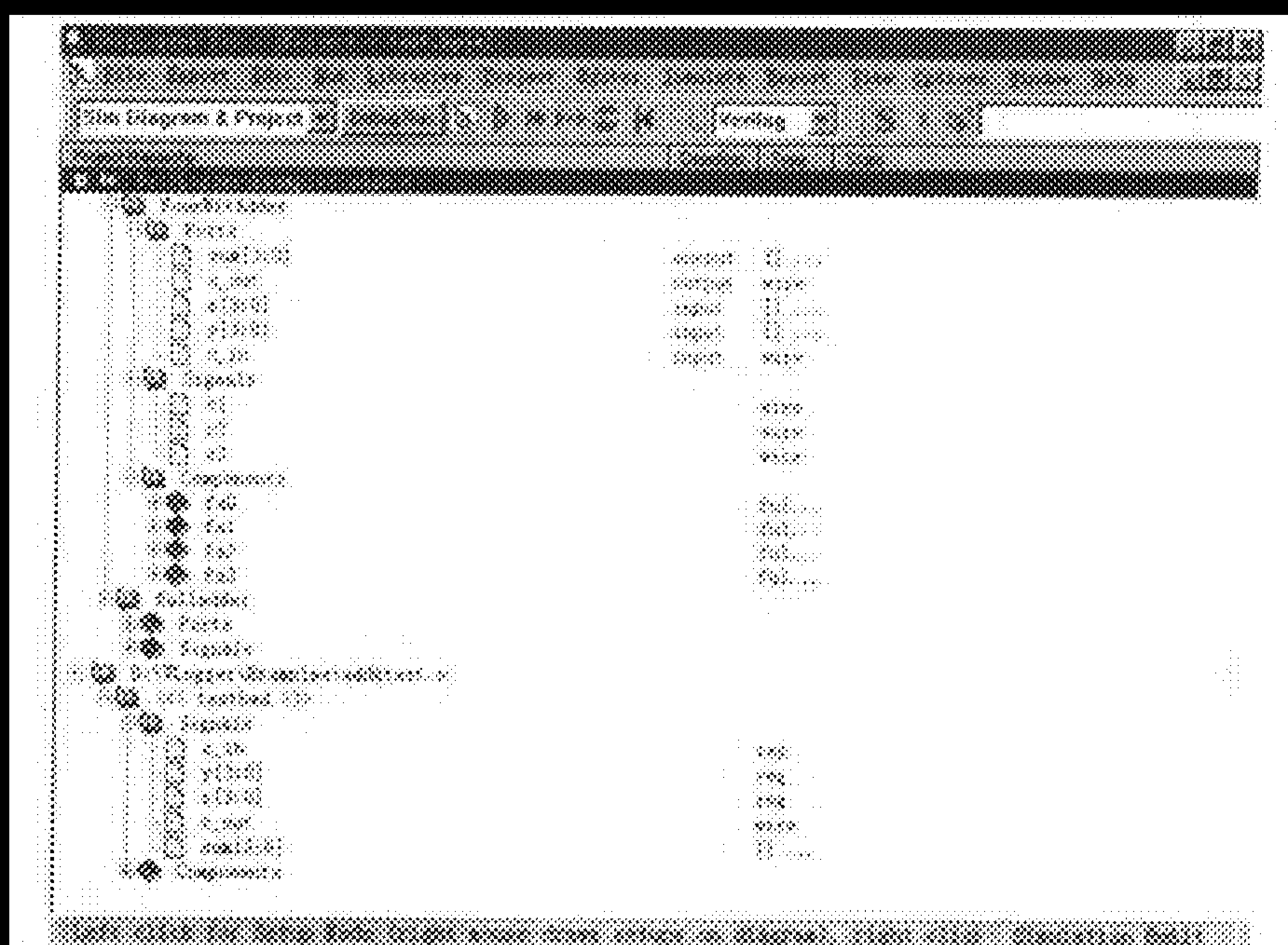
以上三种方法是等效的。

2. 编译工程

利用上述三种编译方法中的任一种对工程 test.hpj 进行编译，编译后各子窗口的内容发生了以下变化。

Project 子窗口不再是如图 12.9 所示的只有两个添加到工程中的文件名，而是经过编译后各源文件中模块的分级树状结构，如图 12.11 所示。从图中可以明显看出，每一个源程序文件由几个模块构成，模块包括的端口、信号与元件，以及端口与信号的类型。

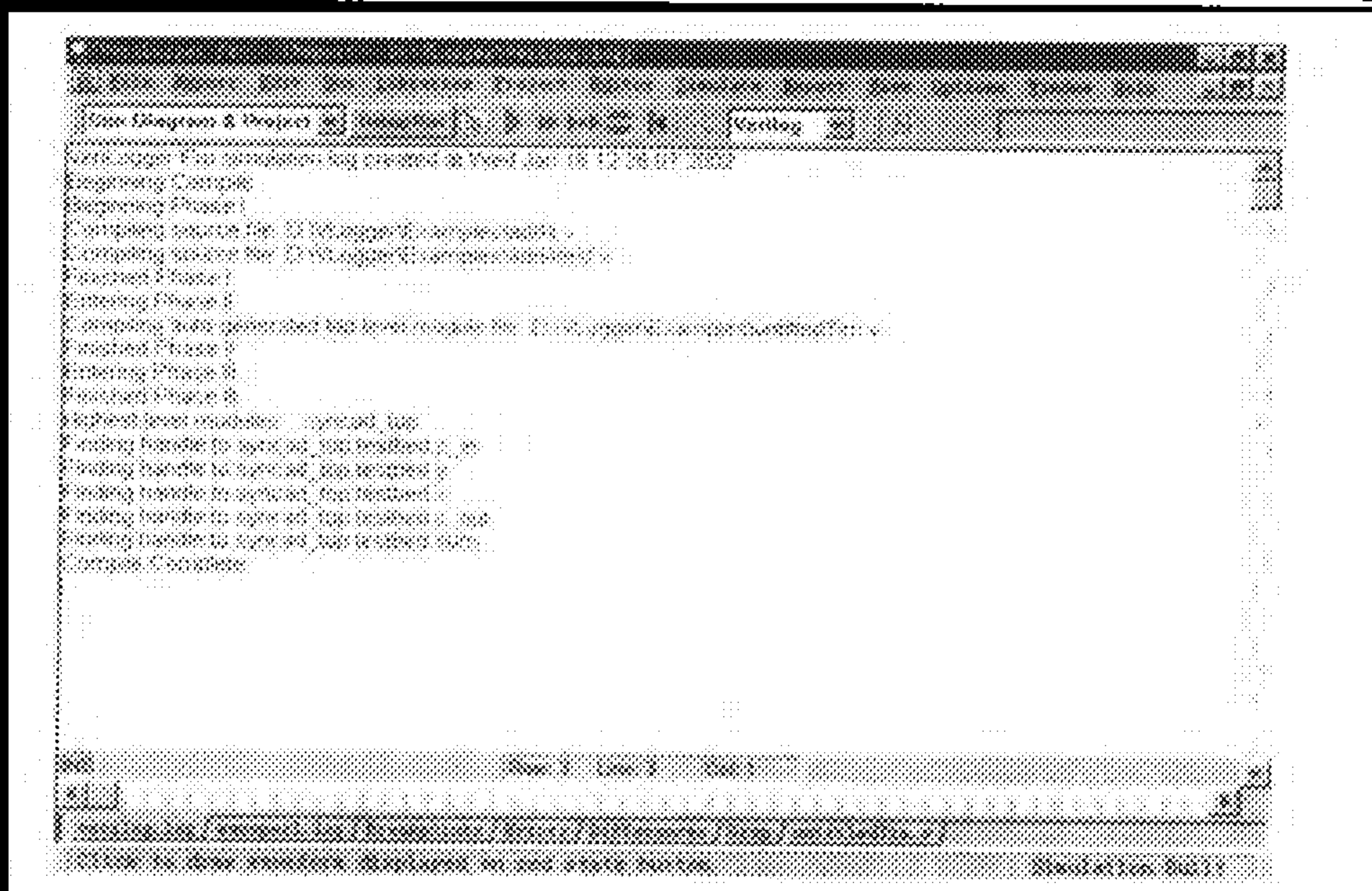
编译后的 Diagram 子窗口如图 12.12 所示，测试源程序文件 add4test.v 中顶层模块的信号均自动添加到该窗口中，而源程序文件 add4.v 中模块的端口、信号及元件并不能添加到该窗口中，这一点需要读者注意。这时信号并没有任何波形显示，因为还没有对工程中的源程序文件进行仿真操作，信号并未被赋值。





由此可以看到，工程编译分为以下三个步骤进行：

(1) 读入源程序文件，并将其转换成内部数据结构，然后检查语法错误与语义错误，包括未定义的变量、端口及变量的非法使用。



(2) 建立模块的分级结构, 连接模块端口, 给变量分配内存。如果某模块被多次调用, 则它的结构也被复制多次。该步对模块的调用、模块参数的传递及内存分配等问题进行错误检查。

12.4.3 Verilog 语言工程的调试

例如，将源程序文件 `add4.v` 稍作改动，然后重新编译，便得到如图 12.16 与图 12.17 所示的出错信息。

```

/** Full Adder *****/
module fulladder(sum, c_out, x, y, c_in);
    output sum, c_out;
    //input x, y, c_in; (第一处改动：未定义输入端口)
    wire a, b, c;
    xor(a, x, y);
    xor(sum, a, c_in);

```




```
and (b, x, y);
and (c, a, c_in);
or (c_out, c, b);
endmodule

/** 4-Bit Adder *****/
module FourBitAdder(sum, c_out, x, y, c_in);
    output [3:0] sum;
    output c_out;
    input [3:0] x, y;
    input c_in;
    wire c1, c2, c3;
    fulladder fa0(sum[0], c1, x[0], y[0], c_in); (第二处改动: fulladder 错写为 fullader)
    fulladder fa1(sum[1], c2, x[1], y[1], c1);
    fulladder (sum[2], c3, x[2], y[2], c2); (第三处改动: 漏掉模块调用名)
    fulladder fa3(sum[3], c_out, x[3], y[3], c3);
endmodule
```

根据图 12.16 中 verilog.log 提供的出错信息, 可知错误发生在编译过程的第一步, 且图 12.16 与图 12.17 均说明了源程序中包含端口未定义的错误, 修改第一处错误, 然后重新编译, 便可得到如图 12.18 与图 12.19 所示的出错信息。

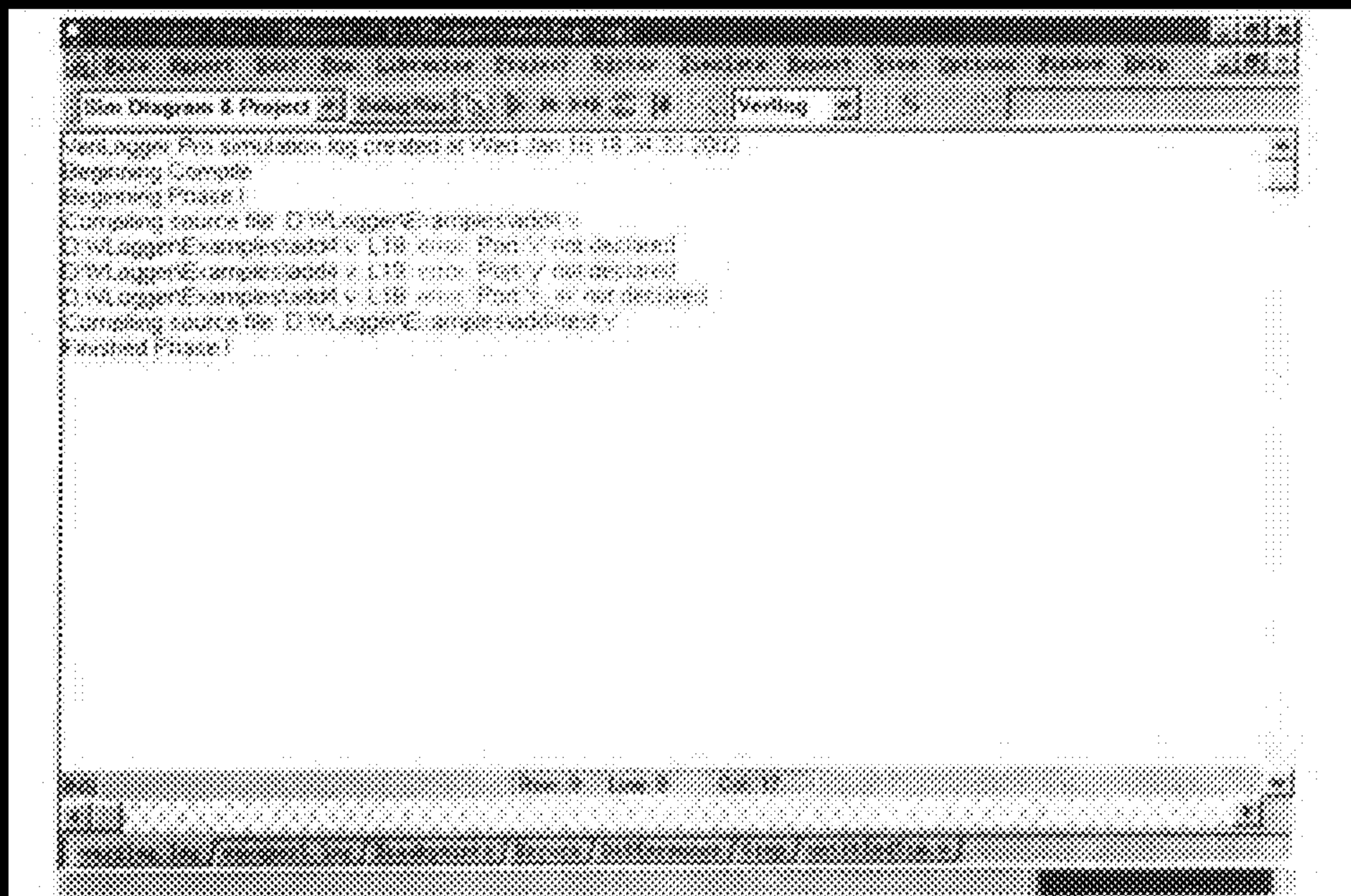


图 12.16 改动后第一次编译 verilog.log 提供的出错信息

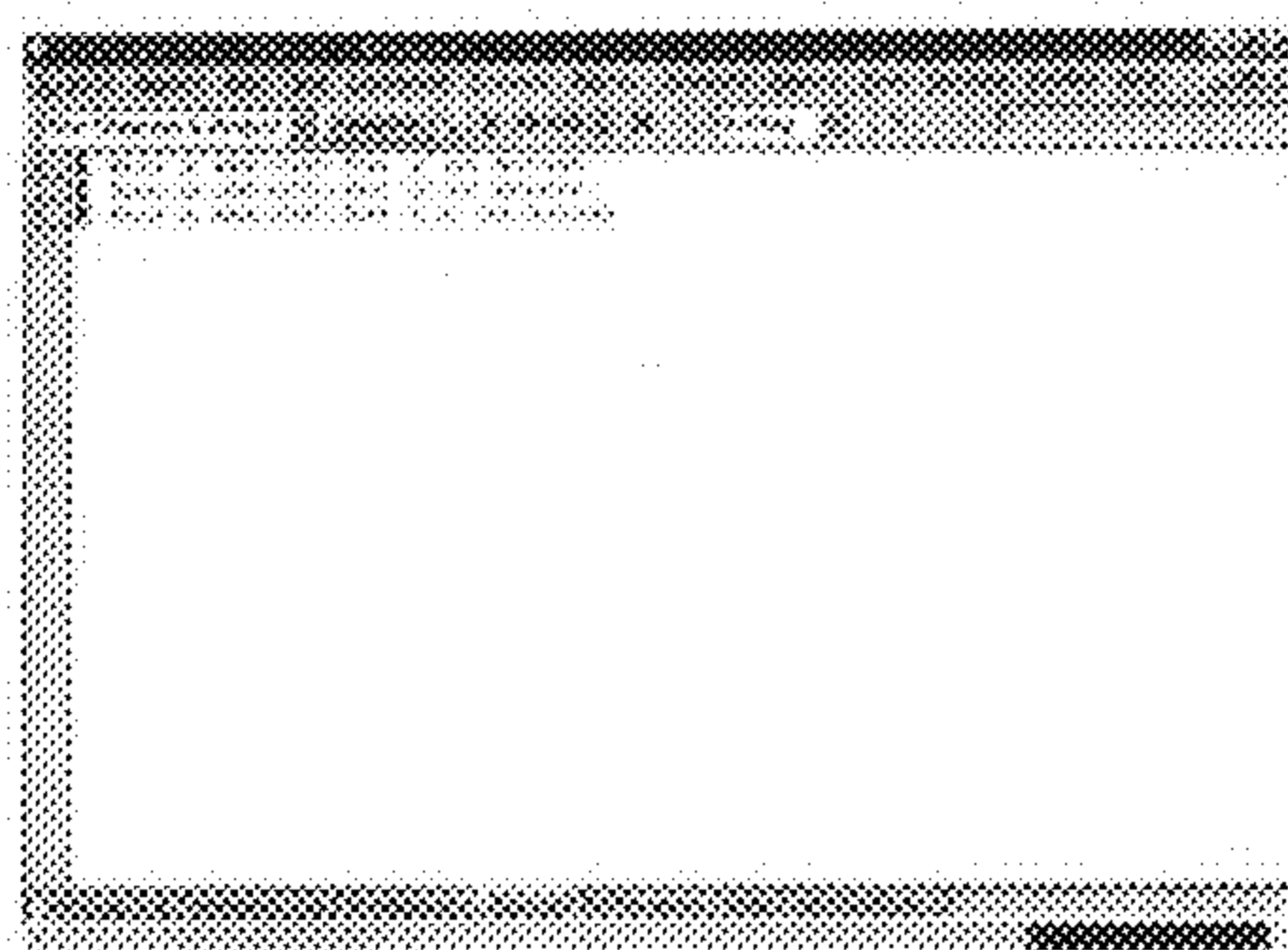


图 12.17 图 12.17 图 12.17 图 12.17



图 12.18 图 12.18 图 12.18 图 12.18

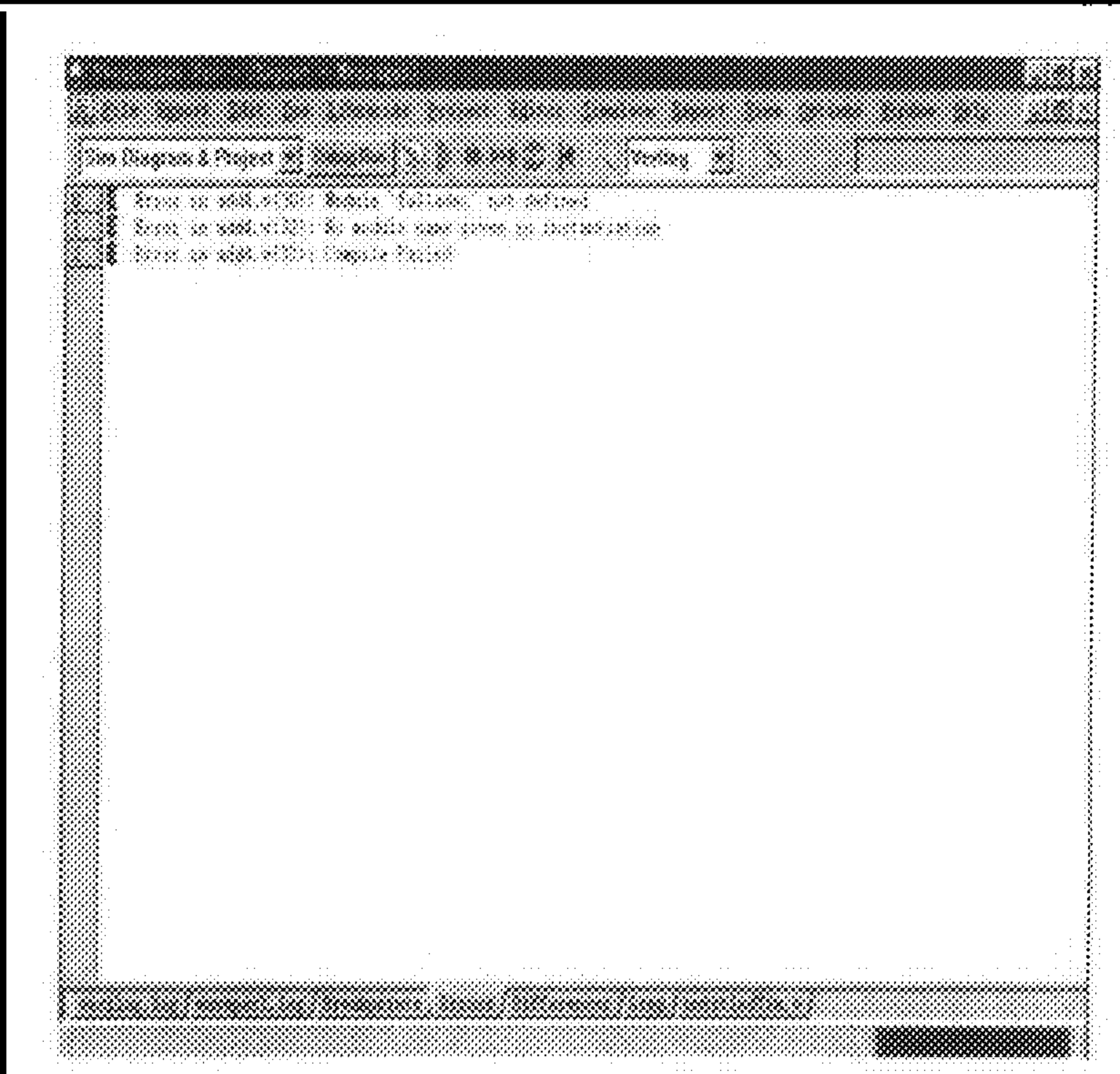


图 12.19 改动后第二次编译 Errors 提供的出错信息

根据图 12.18 提供的出错信息, 可知错误发生在编译过程的第二步, 且图 12.18 与图 12.19 均说明了源程序中包含模块未定义与缺少模块调用名的错误。利用图 12.19 中的出错信息找到源程序中相应的出错位置, 根据提示信息进行修改即可。将第二处与第三处的错误修改后, 重新进行编译, 便得到如图 12.15 所示的编译成功信息。

12.4.4 Verilog 语言工程的仿真

1. 工程仿真方法

VeriLogger Pro 仿真器共有四种仿真方法:

- (1) 利用工具栏上的  按钮。
- (2) 利用工具栏上的  按钮, 进行单步仿真。
- (3) 利用 Simulate 选项的子菜单中的 Run 项。
- (4) 利用快捷键 F5。

以上四种仿真方法作用相同, 且均在 Debug Run 仿真模式下使用。

2. 工程仿真模式

VeriLogger Pro 仿真器支持两种仿真模式: Auto Run 和 Debug Run。

两种仿真模式之间的变换通过工具栏上的  按钮来实现。如果当前为 Debug Run 仿真模式, 只需用鼠标单击  按钮, 就可转换成 Auto Run 仿真模式。当处于 Debug Run 仿真模式时, 只有用户按上述三种仿真方法中的任一种操作时, 仿真才执行; 而 Auto Run 仿真模式则不同, 只要输入信号波形被添加或改变, 仿真器会自动进行仿真, 不需用



户干预。

3. 工程仿真状态

VeriLogger Pro 仿真器有两种仿真状态：Sim Diagram & Project 和 Simulate Project。

当使用 Sim Diagram & Project 仿真状态时，波形图与源程序文件代码一起被仿真；使用 Simulate Project 仿真状态时，Verilog 源代码与波形图分别进行仿真。Debug Run 仿真模式只对 Verilog 源代码进行仿真，波形图的仿真由 Diagram 子窗口中的  按钮来实现。

4. 仿真

当使用 Sim Diagram & Project 仿真状态，且处于 Debug Run 仿真模式时，通过上述四种仿真方法中的任一种对 test.hpj 工程进行仿真。经过仿真后，得到的仿真波形显示于如图 12.20 所示的 Diagram 子窗口中。

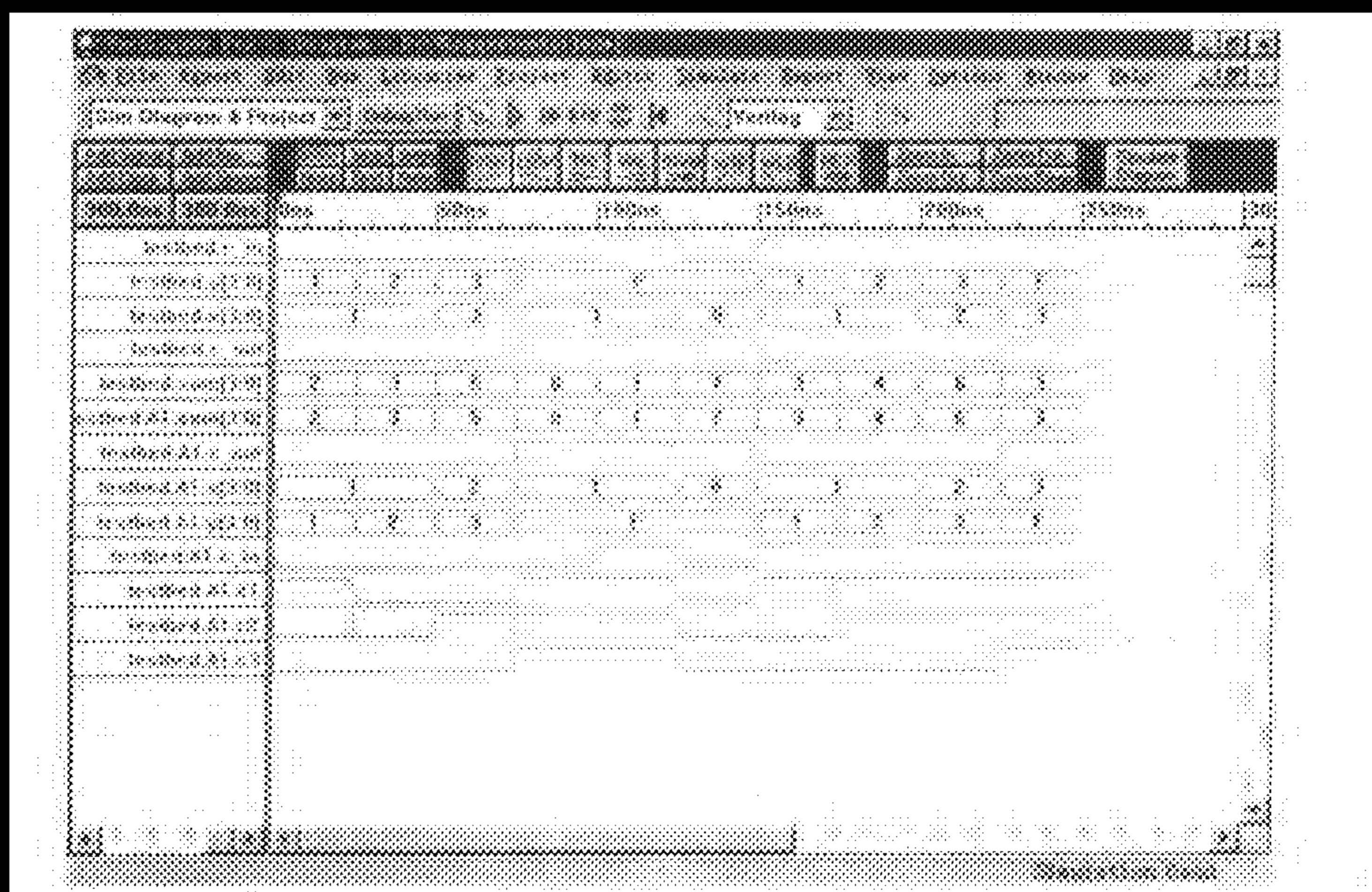


图 12.20 仿真后的 Diagram 子窗口

为了便于观察信号值，可以利用 Diagram 子窗口中的下面四个按钮来改变仿真结果的显示格式：

(1) Zoom Full: 显示整个仿真时间中信号的变化，单击该按钮可得到如图 12.21 所示的显示结果。

(2) Zoom In: 单击一次，将未单击该按钮前的显示范围缩小一倍，图 12.22 为连续单击两次该按钮所得的显示结果。

(3) Zoom Out: 单击一次，将未单击该按钮前的显示范围放大一倍，显示结果如图 12.23 所示。

(4) Zoom Range: 可以设定显示时间的起始时刻与结束时刻，例如设起始时刻为 50 个时间单位，结束时刻为 200 个时间单位，则得到图 12.24 所示的显示结果。

[illegible]

.....



The image shows a blank ledger page. At the top, there is a header section with a grid of 10 columns and 1 row. Below this, there is a table with 10 columns and 10 rows. The columns are labeled as follows: 1. Date, 2. Description, 3. Amount, 4. Balance, 5. Interest, 6. Dividend, 7. Total, 8. Profit, 9. Loss, 10. Remarks. The table is currently empty.



Figure 10.24: A table with 10 columns and 10 rows



附录 Verilog HDL 形式化语法

1. 形式化语法的格式说明

Verilog HDL 的形式化语法格式说明如表 1 所示。

表 1 形式化语法及其格式说明

形式化语法	格式说明
white space	separates lexical tokens
name	a keyword
<name> in upper case	a syntax construct definition
<name> in lower case	a syntax construct item
<name>?	an optional item
<name>*	zero, one, or more items
<name>+	one or more items
<port> <, <port>>*	a comma-separated list of items
::=	gives a syntax definition to an item

2. 源文本

<source_text>

::= <description>*

<description>

::= <module>

<module>

::= module <name_of_module> <list_of_ports>? ;

<module_item>*

endmodule

<name_of_module>

::= <IDENTIFIER>

<list_of_ports>

::= (<port> <,<port>>*)

||= ()

<port>

::= <port_expression>?

||= . <name_of_port> (<port_expression>?)

<port_expression>



```

    ::= <port_reference>
    ||= { <port_reference> <, <port_reference>>* }

<port_reference>
    ::= <name_of_variable>
    ||= <name_of_variable> [ <expression> ]
    ||= <name_of_variable> [ <expression> : <expression> ]

<name_of_port>
    ::= <IDENTIFIER>
    <name_of_variable>
    ::= <IDENTIFIER>

<module_item>
    ::= <parameter_declaration>
    ||= <input_declaration>
    ||= <output_declaration>
    ||= <inout_declaration>
    ||= <net_declaration>
    ||= <reg_declaration>
    ||= <integer_declaration>
    ||= <gate_instantiation>
    ||= <module_instantiation>
    ||= <continuous_assign>
    ||= <function>

<UDP>
    ::= primitive <name_of_UDP> ( <name_of_variable>
        <,<name_of_variable>>* ) ;
        <UDP_declaration>+
        <UDP_initial_statement>?
        <table_definition>
        endprimitive

<name_of_UDP>
    ::= <IDENTIFIER>

<UDP_declaration>
    ::= <output_declaration>
    ||= <reg_declaration>
    ||= <input_declaration>

<UDP_initial_statement>
    ::= initial <output_terminal_name> = <init_val> ;

<init_val>
    ::= 1'b0

```




||= 1'b1

||= 1'bx

||= 1'bX

||= 1'B0

||= 1'B1

||= 1'Bx

||= 1'BX

||= 1

||= 0

<output_terminal_name>

::= <name_of_variable>

<table_definition>

::= table <table_entries> endtable

<table_entries>

::= <combinational_entry>+

||= <sequential_entry>+

<combinational_entry>

::= <level_input_list> : <OUTPUT_SYMBOL> ;

<sequential_entry>

::= <input_list> : <state> : <next_state> ;

<input_list>

::= <level_input_list>

||= <edge_input_list>

<level_input_list>

::= <LEVEL_SYMBOL>+

<edge_input_list>

::= <LEVEL_SYMBOL>* <edge> <LEVEL_SYMBOL>*

<edge>

::= (<LEVEL_SYMBOL> <LEVEL_SYMBOL>)

||= <EDGE_SYMBOL>

<state>

::= <LEVEL_SYMBOL>

<next_state>

::= <OUTPUT_SYMBOL>

||= - (This is a literal hyphen, see Chapter 5 for details).

<OUTPUT_SYMBOL> is one of the following characters:

0 1 x X

<LEVEL_SYMBOL> is one of the following characters:

0 1 x X ? b B



<EDGE_SYMBOL> is one of the following characters:

r R f F p P n N *

<task>

```
::= task <name_of_task> ;
    <tf_declaration>*
    <statement_or_null>
    endtask
```

<name_of_task>

```
::= <IDENTIFIER>
```

<function>

```
::= function <range>? <name_of_function> ;
    <func_declaration>*
    <statement_or_null>
    endfunction
```

<name_of_function>

```
::= <IDENTIFIER>
```

<func_declaration>

```
::= <parameter_declaration>
    ||= <input_declaration>
    ||= <reg_declaration>
    ||= <integer_declaration>
```

<always>

```
::= always @ ( <identifier> or <identifier> )
    ||= always @ ( posedge <identifier> )
    ||= always @ ( negedge <identifier> )
    ||= always @ ( <edge> or <edge> or ... )
```

<edge>

```
::= posedge <identifier>
```

```
||= negedge <identifier>
```

3. 声明

<parameter_declaration>

```
::= parameter <range>? <list_of_assignments> ;
```

<input_declaration>

```
::= input <range>? <list_of_variables> ;
```

<output_declaration>

```
::= output <range>? <list_of_variables> ;
```

<inout_declaration>

```
::= inout <range>? <list_of_variables> ;
```


**<net_declaration>**

::= <NETTYPE> <charge_strength>? <expandrange>? <delay>?

<list_of_variables> ;

||= <NETTYPE> <drive_strength>? <expandrange>? <delay>?

<list_of_assignments> ;

<NETTYPE>

::= wire

||= wor

||= wand

||= tri

<expandrange>

::= <range>

||= scaled <range>

||= vectored <range>

<reg_declaration>

::= reg <range>? <list_of_register_variables> ;

<integer_declaration>

::= integer <list_of_integer_variables> ;

<continuous_assign>

::= assign <drive_strength>? <delay>?

<list_of_assignments>;

<list_of_variables>

::= <name_of_variable> <,<name_of_variable>>*

<name_of_variable>

::= <IDENTIFIER>

<list_of_register_variables>

::= <register_variable> <,<register_variable>>*

<register_variable>

::= <IDENTIFIER>

<list_of_integer_variables>

::= <integer_variable> <,<integer_variable>>*

<integer_variable>

::= <IDENTIFIER>

<charge_strength>

::= (small)

||= (medium)

||= (large)

<drive_strength>

::= (<STRENGTH0> , <STRENGTH1>)



```

||= ( <STRENGTH1> , <STRENGTH0> )
<STRENGTH0>
    ::= supply0
    ||= strong0
    ||= pull0
    ||= weak0
    ||= highz0
<STRENGTH1>
    ::= supply1
    ||= strong1
    ||= pull1
    ||= weak1
    ||= highz1
<range>
    ::= [ <expression> : <expression> ]
<list_of_assignments>
    ::= <assignment> < , <assignment> > *

```

4. 例示

```

<gate_instantiation>
    ::= <GATETYPE> <drive_strength>? <delay>?
        <gate_instance> < , <gate_instance> > * ;
<GATETYPE>
    ::= and
    ||= nand
    ||= or
    ||= nor
    ||= xor
    ||= xnor
    ||= buf
    ||= not
<gate_instance>
    ::= <name_of_gate_instance>? ( <terminal>
        < , <terminal> > * )
<name_of_gate_instance>
    ::= <IDENTIFIER>
<terminal>
    ::= <identifier>
    ||= <expression>

```


**<module_instantiation>**

::= <name_of_module> <parameter_value_assignment>?

<module_instance> <, <module_instance>>* ;

<name_of_module>

::= <IDENTIFIER>

<parameter_value_assignment>

::= #(<expression> <,<expression>>*)

<module_instance>

::= <name_of_module_instance>

(<list_of_module_terminals>?)

<name_of_module_instance>

::= <IDENTIFIER>

<list_of_module_terminals>

::= <module_terminal>? <,<module_terminal>>*

||= <named_port_connection> <,<named_port_connection>>*

<module_terminal>

::= <identifier>

||= <expression>

<named_port_connection>

::= . IDENTIFIER (<identifier>)

||= . IDENTIFIER (<expression>)

5. 行为语句

<statement>

::= <assignment>

||= if (<expression>)

<statement_or_null>

||= if (<expression>)

<statement_or_null>

else

<statement_or_null>

||= case (<expression>)

<case_item>+

endcase

||= casex (<expression>)

<case_item>+

endcase

||= casez (<expression>)

<case_item>+



```

    endcase
    ::= for ( <assignment> ; <expression> ; <assignment> )
        <statement>
    ::= <seq_block>
    ::= disable <IDENTIFIER> ;
    ::= forever <statement>
    ::= while ( <expression> ) <statement>
<statement_or_null>
    ::= statement
    ::= ;
<assignment>
    ::= <lvalue> = <expression>
<case_item>
    ::= <expression> <,<expression>>* ;
        <statement_or_null>
    ::= default : <statement_or_null>
    ::= default <statement_or_null>
<seq_block>
    ::= begin
        <statement>*
    end
    ::= begin : <name_of_block>
        <block_declaration>*
        <statement>*
    end
<name_of_block>
    ::= <IDENTIFIER>
<block_declaration>
    ::= <parameter_declaration>
    ::= <reg_declaration>
    ::= <integer_declaration>

```

6. Specify 部分

```

<specify_block>
    ::= specify <specify_item>* endspecify
<specify_item>
    ::= <specparam_declaration>
    ::= <path_declaration>
    ::= <level_sensitive_path_declaration>

```




```

    ::= <edge_sensitive_path_declaration>
    ::= <system_timing_check>
    ::= <sdpcd>

<specparam_declaration>
    ::= specparam <list_of_param_assignments> ;

<list_of_param_assignments>
    ::= <param_assignment> <,<param_assignment>> *

<param_assignment>
    ::= <<identifier> = <constant_expression>>

<path_declaration>
    ::= <path_description> = <path_delay_value> ;

<path_description>
    ::= ( <specify_input_terminal_descriptor> => <specify_output_terminal_descriptor> )
    ::= ( <list_of_path_inputs> * > <list_of_path_outputs> )

<list_of_path_inputs>
    ::= <specify_input_terminal_descriptor> <,<specify_input_terminal_descriptor>> *

<list_of_path_outputs>
    ::= <specify_output_terminal_descriptor> <,<specify_output_terminal_descriptor>> *

<specify_input_terminal_descriptor>
    ::= <input_identifier>
    ::= <input_identifier> [ <constant_expression> ]
    ::= <input_identifier> [ <constant_expression> : <constant_expression> ]

<specify_output_terminal_descriptor>
    ::= <output_identifier>
    ::= <output_identifier> [ <constant_expression> ]
    ::= <output_identifier> [ <constant_expression> : <constant_expression> ]

<input_identifier>
    ::= the <IDENTIFIER> of a module input or inout terminal

<output_identifier>
    ::= the <IDENTIFIER> of a module output or inout terminal.

<path_delay_value>
    ::= <path_delay_expression>
    ::= ( <path_delay_expression>, <path_delay_expression> )
    ::= ( <path_delay_expression>, <path_delay_expression>,
        <path_delay_expression> )
    ::= ( <path_delay_expression>, <path_delay_expression>,
        <path_delay_expression>, <path_delay_expression>,
        <path_delay_expression>, <path_delay_expression> )
    ::= ( <path_delay_expression>, <path_delay_expression>,
        <path_delay_expression>, <path_delay_expression>,
        <path_delay_expression>, <path_delay_expression>,
        <path_delay_expression> )
```




```

    <path_delay_expression>, <path_delay_expression>,
    <path_delay_expression>, <path_delay_expression>,
    <path_delay_expression>, <path_delay_expression>,
    <path_delay_expression>, <path_delay_expression>,
    <path_delay_expression>, <path_delay_expression> )

```

<path_delay_expression>

```
 ::= <mintypmax_expression>
```

<system_timing_check>

```
 ::= $setup( <timing_check_event>, <timing_check_event>,
    <timing_check_limit>
    <,<notify_register>>? ) ;
```

```
 ::= $hold( <timing_check_event>, <timing_check_event>,
    <timing_check_limit>
    <,<notify_register>>? ) ;
```

```
 ::= $period( <controlled_timing_check_event>, <timing_check_limit>
    <,<notify_register>>? ) ;
```

```
 ::= $width( <controlled_timing_check_event>, <timing_check_limit>
    <,<constant_expression>,<notify_register>>? ) ;
```

```
 ::= $skew( <timing_check_event>, <timing_check_event>,
    <timing_check_limit>
    <,<notify_register>>? ) ;
```

```
 ::= $recovery( <controlled_timing_check_event>,
    <timing_check_event>,
    <timing_check_limit> <,<notify_register>>? ) ;
```

```
 ::= $setuphold( <timing_check_event>, <timing_check_event>,
    <timing_check_limit>, <timing_check_limit> <,<notify_register>>? ) ;
```

<timing_check_event>

```
 ::= <timing_check_event_control>? <specify_terminal_descriptor>
    <&&&& <timing_check_condition>>?
```

<specify_terminal_descriptor>

```
 ::= <specify_input_terminal_descriptor>
```

```
 ::= <specify_output_terminal_descriptor>
```

<controlled_timing_check_event>

```
 ::= <timing_check_event_control> <specify_terminal_descriptor>
    <&&&& <timing_check_condition>>?
```

<timing_check_event_control>

```
 ::= posedge
```

```
 ::= negedge
```

```
 ::= <edge_control_specifier>
```


**<edge_control_specifier>**

::= edge [<edge_descriptor><, <edge_descriptor>>*]

<edge_descriptor>

::= 0i

||= 10

||= 0x

||= x1

||= 1x

||= x0

<timing_check_condition>

::= <scalar_timing_check_condition>

||= (<scalar_timing_check_condition>)

<scalar_timing_check_condition>

::= <scalar_expression>

||= ~<scalar_expression>

||= <scalar_expression> == <scalar_constant>

||= <scalar_expression> === <scalar_constant>

||= <scalar_expression> != <scalar_constant>

||= <scalar_expression> !== <scalar_constant>

<scalar_expression>

A scalar expression is a one bit net or a bit-select of an expanded vector net.

<timing_check_limit>

::= <expression>

<scalar_constant>

::= 1'b0

||= 1'b1

||= 1'B0

||= 1'B1

||= 'b0

||= 'b1

||= 'B0

||= 'B1

||= 1

||= 0

<notify_register>

::= <identifier>

<level_sensitive_path_declaration>

::= if (<conditional_port_expression>

(<specify_input_terminal_descriptor> <polarity_operator>? =>



```

    <specify_output_terminal_descriptor>) = <path_delay_value>;
||= if (<conditional_port_expression>
    (<list_of_path_inputs> <polarity_operator>? *>
    <list_of_path_outputs>) = <path_delay_value>;
(Note: The following two symbols are literal symbols, not syntax description conventions:)
*>    =>

```

<conditional_port_expression>

```

::= <port_reference>
||= <UNARY_OPERATOR><port_reference>
||= <port_reference><BINARY_OPERATOR><port_reference>

```

<polarity_operator>

```

::= +
||= -

```

<edge_sensitive_path_declaration>

```

::= <if (<expression>)>? (<edge_identifier>?
    <specify_input_terminal_descriptor> =>
    (<specify_output_terminal_descriptor> <polarity_operator>?
    : <data_source_expression>)) = <path_delay_value>;
||= <if (<expression>)>? (<edge_identifier>?
    <specify_input_terminal_descriptor> *>
    (<list_of_path_outputs> <polarity_operator>?
    : <data_source_expression>)) = <path_delay_value>;

```

<data_source_expression>

Any expression, including constants and lists. Its width must be one bit or equal to the destination's width. If the destination is a list, the data source must be as wide as the sum of the bits of the members.

<edge_identifier>

```

::= posedge
||= negedge

```

<sdpd>

```

::=if(<sdpd_conditional_expression>)<path_description>=<path_delay_value>;

```

<sdpd_conditional_expresssion>

```

::=<expression><BINARY_OPERATOR><expression>
||=<UNARY_OPERATOR><expression>

```

7. 表达式

<lvalue>

```

::= <IDENTIFIER>
||= <IDENTIFIER> [ <expression> ]

```




||= <concatenation>

<expression>

::= <primary>

||= <UNARY_OPERATOR> <primary>

||= <expression> <BINARY_OPERATOR>

||= <expression> ? <expression> : <expression>

<UNARY_OPERATOR>

::= !

||= ~

||= &

||= ~&

||= |

||= ~|

||= ^

||= ~^

||= --

||= +

<BINARY_OPERATOR>

::= +

||= -

||= *

||= /

||= %

||= ==

||= !=

||= &&

||= ||

||= <

||= <=

||= >

||= >=

||= &

||= |

||= <<

||= >>

<primary>

::= <number>

||= <identifier>

||= <identifier> [<expression>]



||= <identifier> [<expression> : <expression>]

||= <concatenation>

||= <multiple_concatenation>

||= <function_call>

||= (<expression>)

<number>

::= <NUMBER>

||= <BASE> <NUMBER>

||= <SIZE> <BASE> <NUMBER>

<NUMBER>

A number can have any of these characters:

0 1 2 3 4 5 6 7 8 9 a b c d e f x z A B C D E F X Z.

<SIZE>

::= 'b

||= 'B

||= 'o

||= 'O

||= 'd

||= 'D

||= 'h

||= 'H

A size can have any number of these digits: 0 1 2 3 4 5 6 7 8 9

<concatenation>

::= { <expression> <,<expression>>* }

<multiple_concatenation>

::= { <expression> { <expression> <,<expression>>* } }

<function_call>

::= <name_of_function> (<expression> <,<expression>>*)

<name_of_function>

::= <IDENTIFIER>

8. 通用格式

<comment>

::= <short_comment>

||= <long_comment>

<short_comment>

::= // <comment_text> <END-OF-LINE>

<long_comment>

::= /* <comment_text> */

<comment_text>

::= The comment text is zero or more ASCII characters

<identifier>

::= <IDENTIFIER>.<IDENTIFIER>*

(Note: the period may not be preceded or followed by a space.)

<IDENTIFIER>

An identifier is any sequence of letters, digits, and the underscore character (_), where the first character is a letter or an underscore. Uppercase and lowercase letters are treated as different characters. Identifiers can be any size, and all characters are significant. Escaped identifiers start with the backslash character (\) and end with a space. The leading backslash character (\) is not part of the identifier. Use escaped identifiers to include any printable ASCII characters in an identifier.

<delay>

::= # <NUMBER>

||= # <identifier>

||= # (<expression> <, <expression> > *)

<delay_control>

::= # <number>

||= # <identifier>

||= # (<mintypmax_expression>)

<event_control>

::= @ <identifier>

||= @ (<event_expression>)

<event_expression>

::= <expression>

||= posedge <scalar_event_expression>

||= negedge <scalar_event_expression>

||= <event_expression> or <event_expression>

<scalar_event_expression>

Scalar event expression is an expression that resolves to a one bit value.



参 考 文 献

- [1] The Verilog Language Reference Manual. 1995
- [2] The IEEE Verilog 1364-2000 Standard. 2000
- [3] On-line Verilog HDL Quick Reference Guide. Sutherland HDL, Inc., 1997
- [4] CSCI 320 Computer Architecture Handbook on Verilog HDL. 1997
- [5] Gerard M Blair's Verilog Course
- [6] Verilog Pro Manual. 1999
- [7] Sternheim, Singh, Madhavan. Digital Design and Synthesis with Verilog HDL. San Jose: Automata Publishing Company, 1993
- [8] 夏宇闻. 复杂数字电路与系统的 Verilog HDL 设计技术. 北京: 北京航空航天大学出版社, 1998
- [9] 张明. Verilog HDL 实用教程. 成都: 电子科技大学出版社, 1999